2

A camada física

Neste capítulo, analisaremos a camada mais baixa da hierarquia em nosso modelo de referência, a camada física. Ela define as interfaces elétrica, de sincronização e outras, pelas quais os bits são enviados como sinais pelos canais. A camada física é o alicerce sobre o qual a rede é construída. Como as propriedades dos diferentes tipos de canais físicos determinam o desempenho (p. ex., troughput, latência e taxa de erros), este é um bom lugar para começar nossa jornada até a "terra das redes".

Inicialmente, discutiremos três meios de transmissão: guiado ou cabeado (p. ex., fio de cobre, cabo coaxial, fibra óptica), sem fio (rádio terrestre) e satélite. Cada uma dessas tecnologias tem diferentes propriedades que afetam o projeto e o desempenho das redes que as utilizam. Esse material fornecerá informações fundamentais sobre as principais tecnologias de transmissão usadas em redes modernas.

Depois, faremos uma análise teórica da transmissão de dados, apenas para descobrir que a mãe natureza impõe limites sobre o que pode ser enviado por um canal (ou seja, um meio de transmissão físico usado para enviar bits). Em seguida, abordaremos a modulação digital, que trata de como os sinais analógicos são convertidos em bits digitais e em sinais novamente. A seguir, examinaremos os esquemas de multiplexação, explorando como várias conversas podem ser feitas no mesmo meio de transmissão ao mesmo tempo, sem interferir umas com as outras.

Por fim, veremos três exemplos de sistemas de comunicação usados na prática nas redes de computadores a longas distâncias: o sistema de telefonia (fixa), o sistema de telefonia móvel (ou celular) e o sistema de televisão a cabo. Como os três são muito importantes na prática, dedicaremos uma boa quantidade de espaço a cada um.

2.1 MEIOS DE TRANSMISSÃO GUIADOS

O objetivo da camada física é transmitir um fluxo bruto de bits de uma máquina para outra. Vários meios físicos podem ser usados para realizar a transmissão real. Quase sempre, o meio de transmissão que ocorre por um fio ou cabo físico é chamado de **meio de transmissão guiado**, pois as transmissões de sinal são guiadas ao longo de um caminho por um fio ou cabo. Os meios de transmissão guiados mais comuns são o cabo de cobre (na forma de cabo coaxial ou par trançado) e a fibra óptica. Cada tipo de meio de transmissão guiada tem seu próprio conjunto de desvantagens em termos de frequência, largura de banda, atraso, custo e facilidade de instalação e manutenção. A largura de banda é uma métrica da capacidade de transporte de um meio. Ela é medida em **Hz** (ou MHz ou GHz) e seu nome é uma homenagem ao físico alemão Heinrich Hertz. Isso será discutido em detalhes neste capítulo.

2.1.1 Armazenamento persistente

Uma das formas mais comuns de transportar dados de um dispositivo para outro é gravá-los em armazenamento persistente, como o armazenamento magnético ou em estado sólido (p. ex., DVDs graváveis) e transportar fisicamente a fita ou os discos para a máquina de destino, onde eles finalmente serão lidos. Apesar de não ser tão sofisticado quanto a utilização de um satélite de comunicação geossíncrono, esse método costuma ser muito mais econômico, em especial nas aplicações em que a alta largura de banda ou o custo por bit transportado é o fator mais importante.

Um cálculo simples esclarecerá essa questão. Uma fita Ultrium de padrão industrial pode armazenar 30 terabytes. Uma caixa de 60 × 60 × 60 cm pode conter cerca de 1.000 fitas desse tipo, perfazendo uma capacidade total de 800 terabytes, ou 6.400 terabits (6,4 petabits). Uma caixa de fitas pode ser entregue em qualquer parte dos Estados Unidos em 24 horas pelo serviço de Sedex dos Correios, pela Federal Express e por outras transportadoras. A largura de banda efetiva dessa transmissão é de 6.400 terabits/86.400 s, ou um pouco mais de 70 Gbps. Se o destino estiver a apenas uma hora de distância, a largura de banda será ampliada para mais de 1.700 Gbps. Nenhuma rede de computadores consegue nem mesmo se aproximar desse desempenho. Logicamente, as redes estão ficando mais rápidas, mas as densidades das fitas também estão aumentando.

Se considerarmos o custo, obteremos um quadro semelhante. O custo de uma fita Ultrium é de aproximadamente US$ 40 quando a compra é feita no atacado. Uma fita pode ser reutilizada pelo menos 10 vezes. Portanto, o custo das fitas passa a ser US$ 4.000 por caixa, para cada utilização. Adicione a esse montante mais US$ 1.000 pelo frete (provavelmente muito menos) e teremos um custo final aproximado de US$ 5.000 para transportar 800 TB. Consequentemente, para transportar 1 gigabyte, gastaremos pouco mais de meio centavo de dólar. Nenhuma rede pode competir com esses valores. Moral da história:

> *Nunca subestime a largura de banda de uma caminhonete cheia de fitas "voando" na estrada.*

Para mover quantidades de dados *muito* grandes, geralmente essa é a melhor solução. A Amazon tem o que chama de "Snowmobile", que é um grande caminhão cheio de milhares de discos rígidos, todos conectados a uma rede de alta velocidade dentro do caminhão. A capacidade total do caminhão é de 100 PB (100.000 TB ou 100 milhões de GB). Quando uma empresa tem uma grande quantidade de dados para mover, ela pode levar o caminhão às suas instalações e se conectar à rede de fibra óptica da empresa e, em seguida, sugar todos os dados para o caminhão. Feito isso, o caminhão se dirige para outro local e despeja todos os dados. Por exemplo, uma empresa que deseja substituir seu próprio centro de dados completo pela nuvem da Amazon pode estar interessada nesse serviço. Para enormes volumes de dados, nenhum outro método de transporte de dados pode se aproximar disso.

2.1.2 Pares trançados

Embora as características de largura de banda do armazenamento persistente sejam excelentes, os aspectos de atraso são ruins. O tempo de transmissão é medido em horas ou dias, e não em milissegundos. Muitas aplicações, incluindo a Web, videoconferência e jogos on-line, contam com a transmissão de dados com pouco atraso. Um dos meios de transmissão mais antigos e ainda mais comuns é o **par trançado**, que consiste em dois fios de cobre encapados, em geral com cerca de 1 mm de espessura. Os fios são enrolados de forma helicoidal, assim como uma molécula de DNA. O trançado dos fios é feito porque dois fios paralelos formam uma antena simples. Quando os fios são trançados, as ondas de diferentes partes dos fios se cancelam, o que significa menor interferência. Um sinal normalmente é transportado a partir da diferença das tensões terminais (diferença de potencial – ddp) entre os dois fios no par. Essa transmissão do sinal como a diferença entre os dois níveis de tensão, em vez de uma tensão absoluta, oferece melhor imunidade ao ruído externo, pois o ruído tende a afetar a tensão trafegando pelos dois fios da mesma forma, mantendo a ddp relativamente inalterada.

A aplicação mais comum do par trançado é o sistema telefônico. Quase todos os telefones estão conectados à estação central da companhia telefônica por um par trançado. Tanto as chamadas telefônicas quanto o acesso à Internet por ADSL utilizam essas linhas. Os pares trançados podem se estender por diversos quilômetros sem amplificação, mas, quando se trata de distâncias mais longas, o sinal é atenuado e existe a necessidade de repetidores. Quando muitos pares trançados percorrem paralelamente uma distância muito grande, como acontece na ligação entre um prédio e a estação central da companhia telefônica, eles são reunidos e envolvidos por uma capa protetora. Se não estivessem trançados, esses pares provocariam muitas interferências. Em locais onde as linhas telefônicas são instaladas em postes, é comum vermos cabos de pares trançados com vários centímetros de diâmetro.

Os pares trançados podem ser usados na transmissão de sinais analógicos ou digitais. A largura de banda depende da espessura do fio e da distância percorrida, mas, em muitos casos, é possível alcançar diversos megabits/s por alguns quilômetros, e mais do que isso quando são usados alguns truques. Em virtude do desempenho adequado, da grande disponibilidade e do baixo custo, os pares trançados são usados em larga escala e é provável que permaneçam assim nos próximos anos.

O cabeamento de par trançado pode ser de vários tipos. A variedade mais comum empregada em muitos prédios é chamada cabeamento de **Categoria 5e**, ou "Cat 5e". Um par trançado de Categoria 5e consiste em dois fios isolados e levemente trançados. Quatro pares desse tipo normalmente são agrupados em uma capa plástica para proteger os fios e mantê-los juntos. Esse arranjo pode ser visto na Figura 2.1.

Diferentes padrões de LAN podem usar os pares trançados de formas distintas. Por exemplo, a Ethernet de 100 Mbps usa dois (dos quatro) pares, um para cada direção. Para alcançar velocidades mais altas, a Ethernet de 1 Gbps usa todos os quatro pares nas duas direções

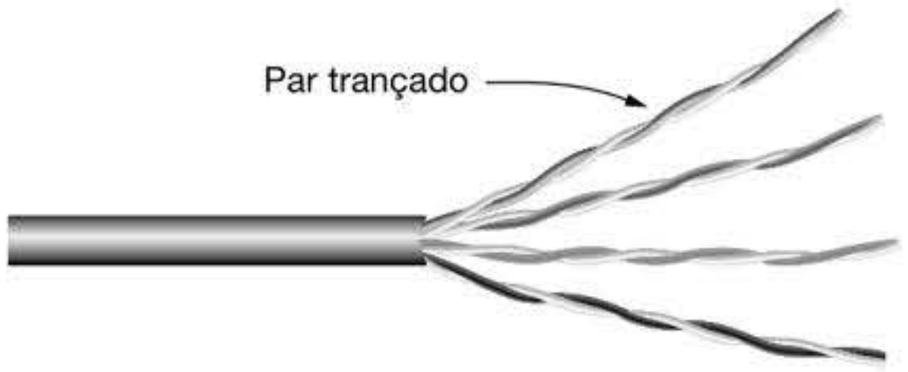


Figura 2.1 Cabo UTP Categoria 5e com quatro pares trançados. Estes cabos podem ser usados para redes locais (LANs).

simultaneamente; isso requer que o receptor decomponha o sinal que é transmitido localmente.

Neste ponto, devemos explicar alguma terminologia geral. Os enlaces que podem ser usados nos dois sentidos ao mesmo tempo, como uma estrada de mão dupla, são chamados enlaces **full-duplex**. Ao contrário, os que são usados em qualquer sentido, mas apenas um de cada vez, como uma linha férrea de trilho único, são chamados enlaces **half-duplex**. Uma terceira categoria consiste em enlaces que permitem o tráfego em apenas uma direção, como uma rua de mão única, chamados enlaces **simplex**.

Retornando ao par trançado, os cabos Cat 5 substituíram os cabos **Categoria 3** mais antigos com um cabo semelhante que usa o mesmo conector, porém com mais voltas por metro. Mais voltas resultam em menos interferências e em um sinal de melhor qualidade por distâncias maiores, tornando os cabos mais adequados para a comunicação de computador de alta velocidade, especialmente LANs Ethernet de 100 Mbps e 1 Gbps.

Os fios mais novos provavelmente serão de **Categoria 6** ou mesmo de **Categoria 7**, as quais têm especificações mais rígidas para lidar com sinais de larguras de banda maiores. Alguns cabos na Categoria 6 e superiores podem aceitar os enlaces de 10 Gbps que agora são comuns em muitas redes, como em novos prédios comerciais. Os fios de **Categoria 8** trabalham com velocidades maiores que as categorias inferiores, mas operam somente a curtas distâncias, em torno de 30 m, e portanto são adequados apenas dentro dos centros de dados. O padrão de Categoria 8 tem duas opções: Classe I, que é compatível com a Categoria 6A, e Classe II, que é compatível com a Categoria 7A.

Até a Categoria 6, esses tipos de fios são conhecidos como **par trançado não blindado**, ou **UTP** (**Unshielded Twisted Pair**), pois consistem simplesmente em fios e isolamento. Ao contrário, os cabos de Categoria 7 possuem uma blindagem nos pares de fios individuais e também ao redor do cabo inteiro (mas dentro da capa plástica protetora). A blindagem reduz a suscetibilidade à interferência externa e linha cruzada com outros cabos vizinhos, atendendo às especificações de desempenho exigidas. Os cabos são reminiscências dos cabos de pares trançados blindados de alta qualidade, porém grossos e caros, que a IBM introduziu no início da década de 1980, mas que não se tornaram populares fora das instalações da empresa. Evidentemente, é hora de tentar novamente.

2.1.3 Cabo coaxial

Outro meio de transmissão comum é o **cabo coaxial** (conhecido por muitos apenas como "coax"). Ele tem melhor blindagem e maior largura de banda que os pares trançados e, assim, pode se estender por distâncias mais longas em velocidades mais altas. Dois tipos de cabo coaxial são amplamente utilizados. Um deles, o cabo de 50 ohms, é comumente empregado nas transmissões digitais. O outro tipo, o cabo de 75 ohms, é usado com frequência nas transmissões analógicas e de televisão a cabo. Essa distinção se baseia mais em fatores históricos do que técnicos (p. ex., as primeiras antenas dipolo tinham uma impedância de 300 ohms e era fácil desenvolver transformadores de casamento de impedância de 4:1). Começando em meados da década de 1990, as operadoras de TV a cabo começaram a oferecer acesso à Internet por cabo, o que tornou o cabo de 75 ohms mais importante para a comunicação de dados.

Um cabo coaxial consiste em um fio de cobre esticado na parte central, protegido por um material isolante. O isolante é envolvido por um condutor cilíndrico, geralmente como uma malha sólida entrelaçada. O condutor externo é coberto por uma camada plástica protetora. A Figura 2.2 apresenta uma vista de corte de um cabo coaxial.

A construção e a blindagem do cabo coaxial proporcionam a ele uma boa combinação de alta largura de banda e excelente imunidade ao ruído (p. ex., contra controles de porta de garagem e fornos de micro-ondas). A largura de banda possível depende da qualidade e do tamanho do

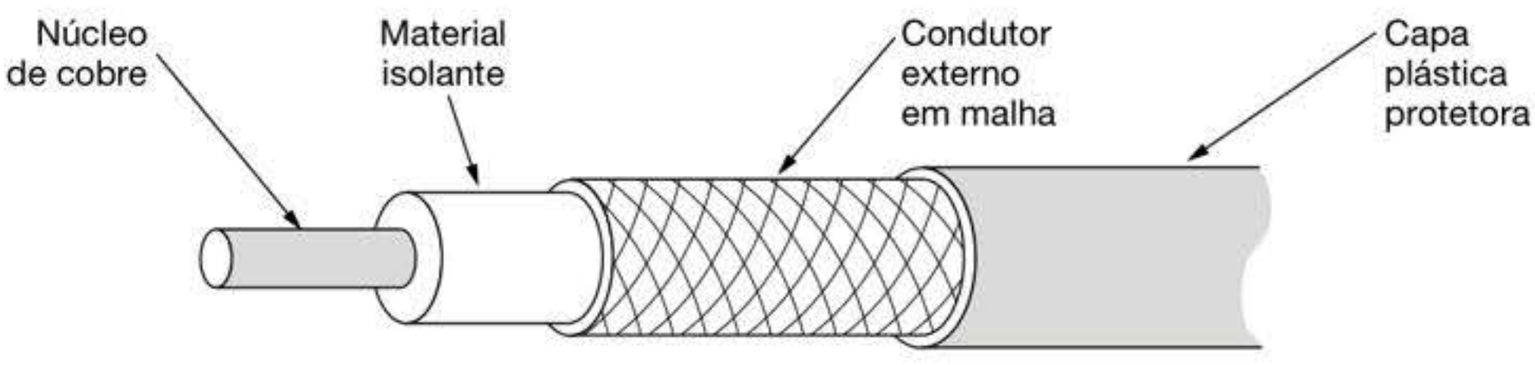


Figura 2.2 Um cabo coaxial.

cabo. O cabo coaxial possui largura de banda extremamente alta; os cabos modernos têm uma largura de banda de até 6 GHz, permitindo, assim, que muitas conversas sejam transmitidas simultaneamente por um único cabo coaxial (um único programa de televisão poderia ocupar aproximadamente 3,5 MHz). Os cabos coaxiais eram muito usados no sistema telefônico para linhas de longa distância, mas agora estão sendo substituídos por fibras ópticas nessas rotas. Porém, eles ainda são usados em larga escala pelas redes de televisão a cabo e em redes metropolitanas, além da conectividade com a Internet de alta velocidade para residências em muitas partes do mundo.

2.1.4 Linhas de energia elétrica

As redes de telefonia e de televisão a cabo não são as únicas fontes de fiação que podem ser reutilizadas para a comunicação de dados. Há um outro tipo de fiação ainda mais comum: as linhas de energia elétrica. Estas oferecem energia elétrica às casas, e a fiação elétrica dentro das casas distribui a energia às tomadas elétricas.

O uso das linhas de energia elétrica para comunicação de dados é uma ideia antiga. Elas têm sido usadas pelas companhias de eletricidade para a comunicação de baixo nível há muitos anos, como para fazer medição remota e controlar dispositivos em casa (p. ex., o padrão X10). Nos últimos anos, tem havido um interesse renovado na comunicação de alto nível por essas linhas, tanto dentro de casa, como uma LAN, quanto fora dela, para o acesso de banda larga à Internet. Vamos nos concentrar no cenário mais comum: usar fios elétricos dentro de casa.

A conveniência de usar linhas de energia para a rede de dados deve ser clara. Basta conectar uma TV e um receptor na parede, o que você precisa fazer de qualquer forma, pois ele precisa de energia, e ele poderá enviar e receber filmes pela fiação elétrica. Essa configuração pode ser vista na Figura 2.3. Não há outro conector ou rádio. O sinal de dados é sobreposto ao sinal de baixa frequência (ou no fio ativo, ou "quente"), enquanto os dois sinais usam a fiação ao mesmo tempo.

A dificuldade em usar a fiação elétrica domiciliar como uma rede é que ela foi projetada para distribuir energia elétrica. Essa tarefa é muito diferente de distribuir sinais de dados, algo para o qual a fiação doméstica é pouco eficiente. Os sinais elétricos são enviados a 50-60 Hz e a fiação atenua os sinais de frequência muito mais alta (MHz) necessários para a comunicação de dados de alto nível. As propriedades elétricas da fiação variam de uma casa para outra e mudam à medida que os aparelhos são ligados e desligados, fazendo os sinais de dados oscilarem pela fiação. As correntes transitórias quando os aparelhos são ligados e desligados criam ruído por uma grande faixa de frequências. Sem o trançado cuidadoso dos pares trançados, a fiação elétrica atua como uma boa antena, apanhando sinais externos e emitindo sinais próprios. Esse comportamento significa que, para atender aos requisitos da regulamentação, o sinal de dados precisa excluir frequências licenciadas, como as faixas de radioamador.

Apesar dessas dificuldades, é possível enviar pelo menos 500 Mbps pela fiação elétrica doméstica usando esquemas de comunicação que resistem às frequências comprometidas e às sucessões de erros. Muitos produtos usam diversos padrões próprios para as redes de energia elétrica, de modo que há padrões em desenvolvimento ativo.

2.1.5 Fibra óptica

Muitas pessoas na indústria de informática se orgulham da rapidez com que a tecnologia usada nos computadores vem melhorando, conforme a lei de Moore, que prevê a duplicação do número de transistores por chip a cada dois anos aproximadamente (Kuszyk e Hammoudeh, 2018). O IBM PC original (de 1981) funcionava com uma velocidade de clock de 4,77 MHz; 40 anos depois, os PCs podiam usar uma CPU de quatro núcleos a 3 GHz. Esse aumento é um ganho de fator em torno de 2.500. Impressionante.

No mesmo período, enlaces de comunicação remotos passaram de 45 Mbps (uma linha T3 no sistema telefônico) para 100 Gbps (uma linha moderna de longa distância). Esse ganho também é impressionante, um fator de mais de 2.000, enquanto, no mesmo período, a taxa de erros passou de 10^{-5} por bit para quase zero. Na última década, as CPUs isoladas se aproximaram dos limites físicos, motivo pelo qual agora é o número de CPUs que está sendo aumentado por chip. Por sua vez, a largura de banda alcançável com a tecnologia de fibra óptica pode ultrapassar a casa dos 50.000 Gbps (50 Tbps) e nem estamos perto de alcançar esses limites. O limite prático atual é de cerca de 100 Gbps,

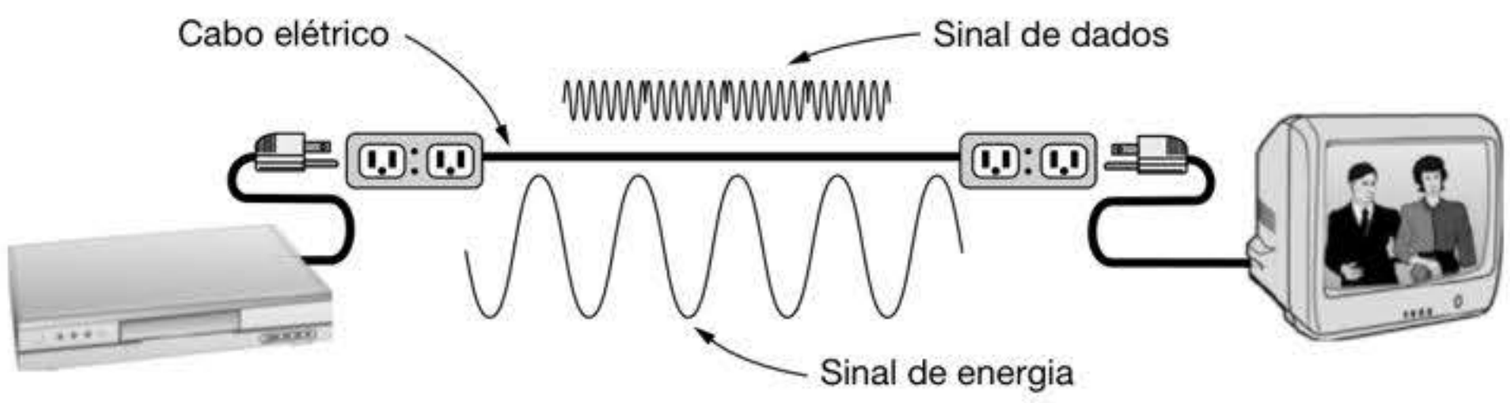


Figura 2.3 Rede de dados que usa a fiação elétrica domiciliar.

em razão de nossa incapacidade de realizar a conversão entre sinais elétricos e ópticos em uma velocidade maior. Para criar enlaces de maior capacidade, muitos canais simplesmente correm em paralelo por uma única fibra.

Nesta seção, estudaremos a fibra óptica para descobrir como funciona essa tecnologia de transmissão. Na corrida entre informática e comunicação, esta pode ganhar em virtude das redes de fibra óptica. As implicações reais disso seriam essencialmente largura de banda infinita e uma nova premissa de que os computadores são terrivelmente lentos e, por essa razão, as redes deveriam tentar evitar a computação a todo custo, independentemente do desperdício de largura de banda que isso signifique. Essa mudança levará algum tempo para entrar na cabeça de uma geração de cientistas da computação e engenheiros ensinados a pensar em termos dos baixos limites de transmissão impostos pelos fios de cobre.

Naturalmente, esse cenário não diz tudo, pois não inclui custos. O custo para instalar fibra até chegar aos consumidores, evitando a baixa largura de banda dos fios e a disponibilidade limitada de espectro, é muito alto. A energia para a movimentação de bits também custa mais que a energia para sua computação. Sempre podemos ter ilhas de injustiça, onde a computação ou a comunicação é basicamente gratuita. Por exemplo, na borda da Internet, usamos computação e armazenamento para o problema de compactação e caching de conteúdo, tudo para fazer melhor uso dos enlaces de acesso à Internet. Dentro da Internet, podemos fazer o contrário, com empresas como o Google movendo grandes quantidades de dados pela rede até onde for mais barato armazená-los ou computá-los.

A fibra óptica é usada para transmissão por longa distância nos backbones da rede, LANs de alta velocidade (embora, até aqui, o cobre sempre tenha conseguido acompanhar) e acesso à Internet em alta velocidade, como **FTTH** (**Fiber to the Home**). Um sistema de transmissão óptico tem três componentes-chave: a fonte de luz, o meio de transmissão e o detector. Convencionalmente, um pulso de luz indica um bit 1 e a ausência de luz indica um bit 0. O meio de transmissão é uma fibra de vidro ultrafina. O detector gera um pulso elétrico quando a luz incide sobre ele. Conectando uma fonte de luz em uma ponta de uma fibra óptica e um detector na outra, temos um sistema de transmissão de dados unidirecional (i.e., simplex) que aceita um sinal elétrico, o converte e o transmite por pulsos de luz, para depois novamente converter a saída para um sinal elétrico na ponta receptora.

Esse sistema de transmissão perderia luz para o meio e seria inútil na prática se não fosse por um princípio interessante da física. Quando um raio de luz passa de um meio para outro – por exemplo, de sílica fundida para o ar –, o raio é refratado (inclinado) no limite sílica/ar, como mostra a Figura 2.4(a). Aqui, vemos um raio de luz incidindo no limite em um ângulo α_1 emergindo em um ângulo β_1. A quantidade de refração depende das propriedades dos dois meios (em particular, seus índices de refração). Para ângulos de incidência acima de um certo valor crítico, a luz é refratada de volta para a sílica; nada escapa para o ar. Assim, um raio de luz incidente em um ângulo crítico ou acima é interceptado dentro da fibra, como mostra a Figura 2.4(b), e pode se propagar por muitos quilômetros praticamente sem perdas.

O exemplo da Figura 2.4(b) mostra apenas um raio interceptado, mas, como qualquer raio de luz incidente no limite acima do ângulo crítico será refletido internamente, muitos raios distintos estarão ricocheteando em diferentes ângulos. Dizemos que cada raio tem um modo específico; assim, uma fibra que apresenta essa propriedade é chamada de **fibra multimodo**. Se o diâmetro da fibra for reduzido a alguns comprimentos de onda de luz (menos de 10 micra, ao contrário de mais de 50 micra para a fibra multimodo), a fibra agirá como um guia de onda e a luz só poderá se propagar em linha reta, sem ricochetear, produzindo, assim, uma **fibra de modo único** ou **fibra monomodo**. As fibras de modo único são mais caras, mas são amplamente utilizadas em distâncias mais longas; elas podem transmitir sinais para distâncias aproximadamente 50 vezes maiores que as fibras multimodo. As fibras de modo único disponíveis no momento podem transmitir dados a 100 Gbps por 100 km sem amplificação. Foram obtidas taxas de dados ainda mais altas em laboratório, para distâncias mais curtas. A decisão entre o uso de fibra monomodo ou multimodo depende da aplicação. A fibra multimodo pode ser usada

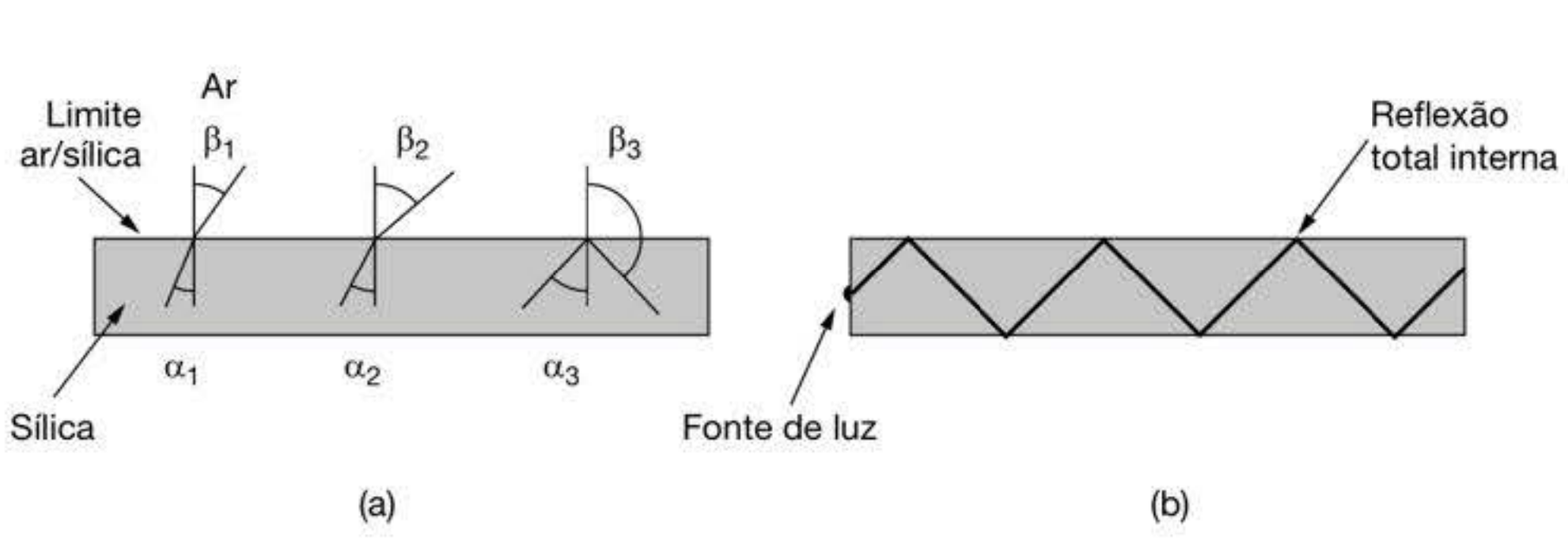


Figura 2.4 (a) Três exemplos de um raio de luz dentro de uma fibra de sílica incidindo no limite fronteira ar/sílica em diferentes ângulos. (b) A luz interceptada pela reflexão total interna.

para transmissões de até cerca de 15 km e permitir o uso de equipamento de fibra óptica relativamente mais barato. Contudo, a largura de banda da fibra multimodo torna-se mais limitada com o aumento da distância.

Transmissão de luz na fibra

As fibras ópticas são feitas de vidro, que, por sua vez, é produzido a partir da areia, uma matéria-prima abundante e de baixo custo. Os antigos egípcios já dominavam a manufatura do vidro, mas o material produzido por eles não podia ter mais de 1 mm de espessura para que a luz pudesse atravessá-lo. O vidro transparente usado nas janelas foi desenvolvido durante a Renascença. O vidro usado nas modernas fibras ópticas é tão transparente que se, em vez de água, os oceanos fossem cheios desse tipo de material, seria possível ver o fundo do mar da superfície, da mesma forma que é possível ver o solo quando voamos de avião em um dia claro.

A atenuação de luz através do vidro depende do comprimento de onda da luz (bem como de algumas propriedades físicas do vidro). Ela é definida como a razão da potência do sinal de entrada e saída. Para o tipo de vidro usado nas fibras, a atenuação é mostrada na Figura 2.5, em decibéis (dB) por quilômetro linear de fibra. Por exemplo, quando o fator de potência é igual a dois, obtemos uma atenuação de $10 \log_{10} 2 = 3$ dB. Discutiremos sobre decibéis mais adiante. Resumindo, essa é uma forma logarítmica de medir as taxas de potência, com 3 dB significando um fator de potência de dois. A figura mostra a parte do infravermelho do espectro que, na prática, é a utilizada. A luz visível tem comprimentos de onda ligeiramente mais curtos, que variam de 0,4 a 0,7 mícron (1 mícron é igual a 10^{-6} m). Na verdade, esses comprimentos de onda seriam de 400 nm a 700 nm, mas manteremos a nomenclatura tradicional.

A comunicação óptica comumente utiliza três bandas de comprimentos de onda. Elas são centralizadas em 0,85, 1,30 e 1,55 micra, respectivamente. As três bandas têm entre 25.000 e 30.000 GHz de largura. A banda de 0,85 mícron foi usada primeiro. Ela tem maior atenuação e, por isso, é usada para distâncias mais curtas, mas, nesse comprimento de onda, os lasers e os circuitos eletrônicos podem ser produzidos a partir do mesmo material (arseneto de gálio). As duas últimas bandas têm boas propriedades de atenuação (uma perda inferior a 5% por quilômetro). A banda de 1,55 mícron agora é muito utilizada com amplificadores dopados com érbio, que atuam diretamente no domínio óptico.

Os pulsos de luz enviados através de uma fibra se expandem à medida que se propagam. Essa expansão é chamada **dispersão cromática**. O volume da dispersão depende do comprimento de onda. Uma forma de impedir que esses pulsos dispersos se sobreponham é aumentar a distância entre eles, mas isso só pode ser feito reduzindo-se a taxa de sinalização. Felizmente, descobriu-se que, quando os pulsos são produzidos em uma forma especial relacionada ao recíproco do cosseno hiperbólico, praticamente todos os efeitos de dispersão são cancelados e é possível enviar pulsos por milhares de quilômetros sem que haja uma distorção significativa. Esses pulsos são chamados **sólitons**. Eles estão começando a ser muito utilizados na prática.

Cabos de fibra

Os cabos de fibra óptica são semelhantes aos cabos coaxiais, exceto por não terem a malha metálica. A Figura 2.6(a) mostra a vista lateral de uma única fibra. No centro fica o núcleo de vidro através do qual a luz se propaga. Nas fibras multimodo, o núcleo normalmente tem 50 micra de diâmetro, o que corresponde à espessura de um fio de

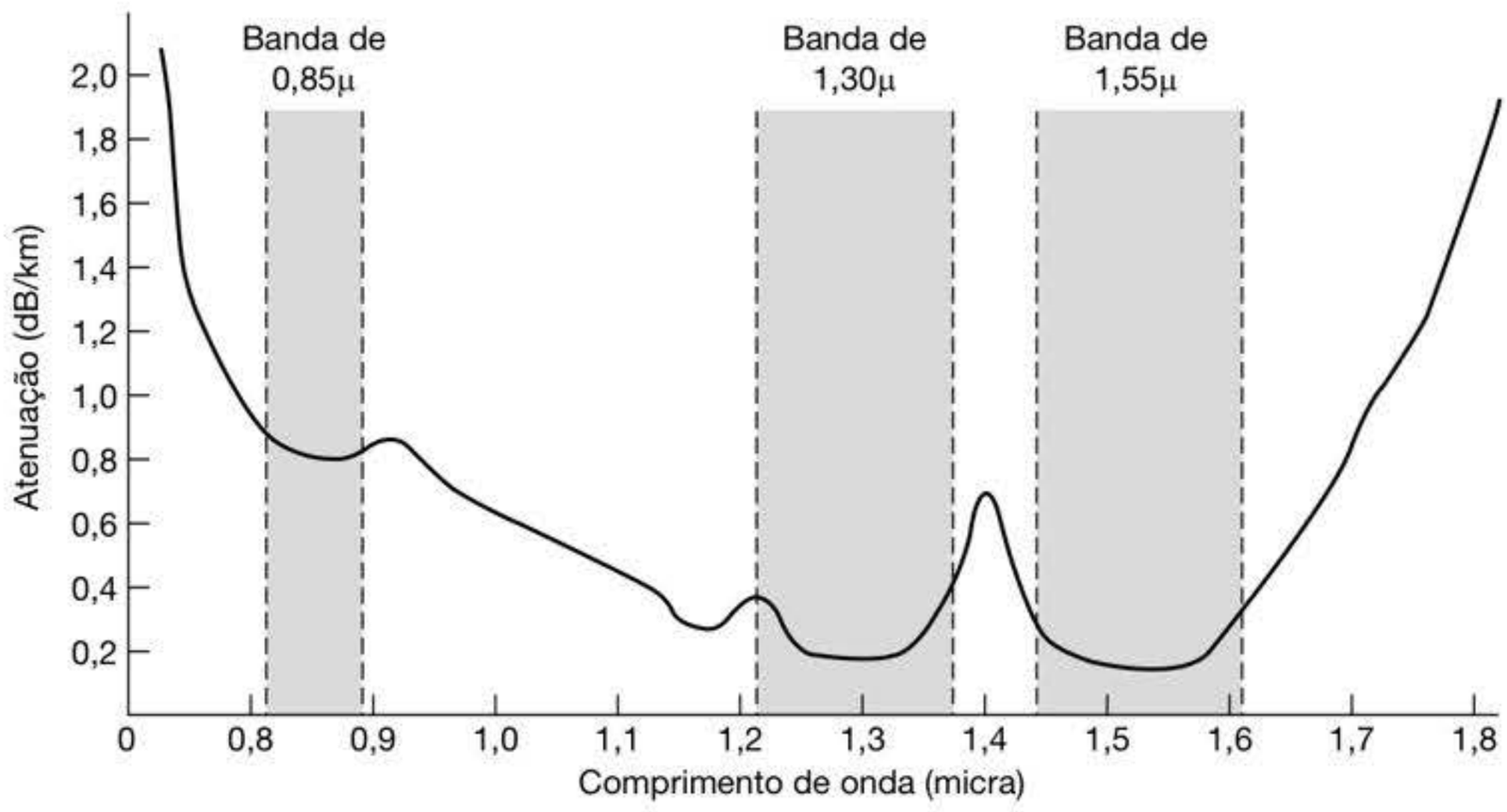


Figura 2.5 Atenuação da luz na fibra, na região do infravermelho.

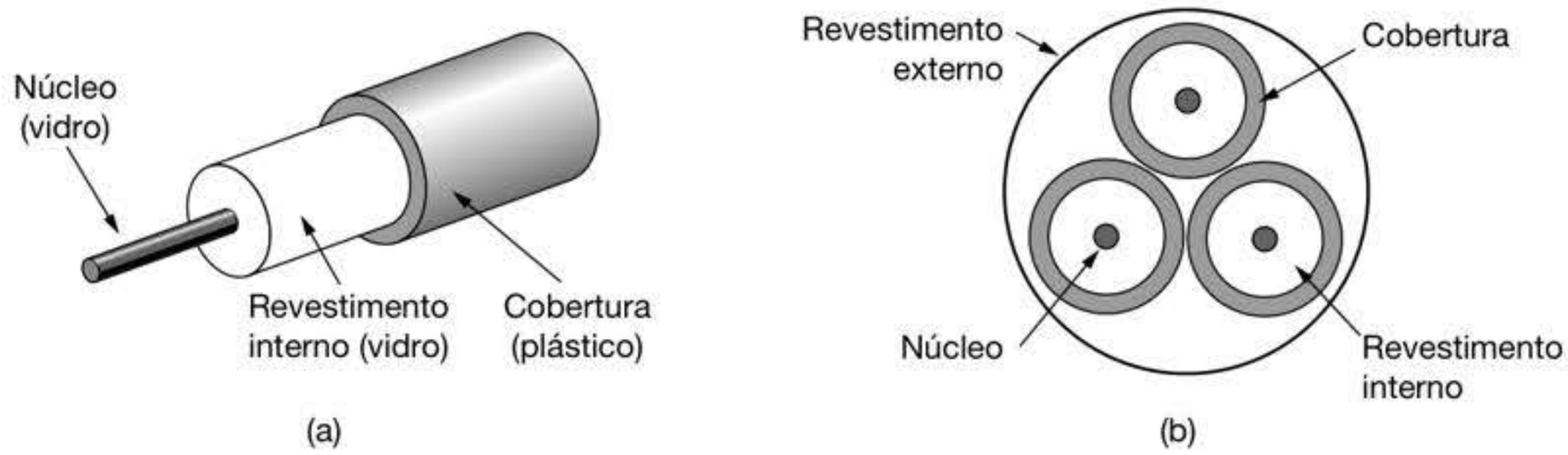


Figura 2.6 (a) Vista lateral de uma única fibra. (b) Vista da extremidade de um cabo com três fibras.

cabelo humano. Nas fibras de modo único, o núcleo tem entre 8 e 10 micra.

O núcleo é envolvido por um revestimento de vidro com um índice de refração inferior ao do núcleo, para manter toda a luz nele. Em seguida, há uma cobertura de plástico fino para proteger o revestimento interno. Geralmente, as fibras são agrupadas em feixes, protegidas por um revestimento externo. A Figura 2.6(b) mostra um cabo com três fibras.

Normalmente, os cabos de fibra terrestres são colocados no solo a um metro da superfície, onde ocasionalmente são atacados por retroescavadeiras ou roedores. Próximo ao litoral, cabos de fibra transoceânicos são enterrados em trincheiras por uma espécie de arado marítimo. Em águas profundas, eles são depositados no fundo, onde podem ser arrastados por redes de pesca ou comidos por lulas gigantes.

As fibras podem estar conectadas de três maneiras diferentes. Primeiro, elas podem ter conectores em suas extremidades e serem plugadas em soquetes de fibra. Os conectores perdem de 10 a 20% da luz, mas facilitam a reconfiguração dos sistemas. Segundo, elas podem ser unidas mecanicamente. Nesse caso, as duas extremidades são cuidadosamente colocadas uma perto da outra em uma luva especial e fixadas no lugar. O alinhamento pode ser melhorado fazendo-se a luz passar pela junção e, em seguida, realizando-se pequenos ajustes cuja finalidade é maximizar o sinal. As junções mecânicas são realizadas em cerca de 5 minutos por uma equipe treinada e resultam em uma perda de 10% da luz. Terceiro, duas peças de fibra podem ser fundidas de modo a formar uma conexão sólida. A união por fusão é quase tão boa quanto uma fibra sem emendas; no entanto, mesmo nesse caso, há uma pequena atenuação. Nos três tipos de uniões podem ocorrer reflexões no ponto de junção, e a energia refletida pode interferir no sinal.

Dois tipos de fontes de luz geralmente são usados para fazer a sinalização: os diodos emissores de luz (Light Emitting Diodes, ou LEDs) e os lasers semicondutores. Eles têm diferentes propriedades, como mostra a Figura 2.7. O comprimento de onda desses elementos pode ser ajustado pela inserção de interferômetros de Fabry-Perot ou Mach-Zehnder entre a fonte e a fibra. Os interferômetros de Fabry-Perot são cavidades ressonantes simples que consistem em dois espelhos paralelos. A luz incide perpendicularmente aos espelhos. O comprimento da cavidade filtra os comprimentos de onda que cabem em um número inteiro de períodos. Os interferômetros de Mach-Zehnder separam a luz em dois feixes, os quais percorrem distâncias ligeiramente diferentes e são recombinados no destino, ficando em fase para certos comprimentos de onda.

A extremidade de recepção de uma fibra óptica consiste em um fotodiodo, que emite um pulso elétrico ao ser atingido pela luz. O tempo de resposta de um fotodiodo, que converte o sinal do domínio óptico para o elétrico, limita as taxas de dados a cerca de 100 Gbps. O ruído térmico também é importante, de modo que um pulso de luz deve conduzir energia suficiente para ser detectado. Com pulsos de potência suficiente, a taxa de erros pode se tornar muito pequena.

Item	LED	Laser semicondutor
Taxa de dados	Baixa	Alta
Tipo de fibra	Multimodo	Multimodo ou modo único
Distância	Curta	Longa
Vida útil	Longa	Curta
Sensibilidade à temperatura	Insignificante	Substancial
Custo	Baixo	Alto

Figura 2.7 Uma comparação entre laser semicondutor e LEDs utilizados como fontes de luz.

Comparação entre fibras ópticas e fios de cobre

É instrutivo comparar a fibra com o cobre. A fibra tem muitas vantagens. Para começar, ela pode gerenciar larguras de banda muito mais altas do que o cobre. Essa característica sozinha já justificaria seu uso nas redes de última geração. Em razão da baixa atenuação, os repetidores só são necessários a cada 50 km de distância em linhas longas, em comparação com a distância de 5 km no caso do cobre, resultando em uma economia de custo significativa. A fibra também tem a vantagem de não ser afetada por picos de tensão, interferência eletromagnética ou quedas no fornecimento de energia. Ela também está imune à ação corrosiva de alguns elementos químicos que pairam no ar, o que é importante em ambientes industriais desfavoráveis.

Por mais estranho que possa parecer, as empresas telefônicas gostam da fibra por outra razão: ela é fina e leve. Muitos dos dutos de cabos atuais estão completamente lotados, de modo que não há espaço para aumentar sua capacidade. Além da remoção e subsequente substituição de todo o cobre por fibras esvaziar os dutos, o cobre tem um excelente valor de revenda para as refinarias especializadas, pois trata-se de um minério de altíssima qualidade. Além disso, a fibra é muito mais leve que o cobre. Mil pares trançados com 1 km de comprimento pesam 8 toneladas. Duas fibras têm mais capacidade e pesam apenas 100 kg, reduzindo de maneira significativa a necessidade de sistemas mecânicos de suporte, que exigem mais manutenção. Nas novas rotas, as fibras são preferidas por terem um custo de instalação muito mais baixo. Por fim, elas não desperdiçam luz e dificilmente são interceptadas. Por essas razões, a fibra é uma alternativa com excelente nível de segurança contra possíveis escutas telefônicas.

No entanto, a fibra tem a desvantagem de ser uma tecnologia menos familiar, exigindo conhecimentos que nem todos os engenheiros têm e, além disso, as fibras podem ser danificadas com facilidade, se forem encurvadas demais. Como a transmissão óptica é basicamente unidirecional, a comunicação bidirecional exige duas fibras ou duas bandas de frequência em uma única fibra. Por fim, as interfaces de fibra são mais caras que as interfaces elétricas. Apesar disso, o futuro de toda a comunicação fixa de dados para distâncias superiores a alguns metros depende claramente da fibra. Para obter mais informações sobre todos os aspectos das fibras ópticas e de suas redes, consulte Pearson (2015).

2.2 TRANSMISSÃO SEM FIO

Muitas pessoas agora possuem conectividade sem fio para muitos dispositivos, desde notebooks e smartphones até relógios e refrigeradores inteligentes. Todos esses dispositivos contam com a comunicação sem fio para transmitir informações para outros dispositivos e extremidades da rede.

Nas próximas seções, examinaremos os conceitos básicos da comunicação sem fio em geral, pois ela tem muitas outras aplicações importantes além de oferecer conectividade aos usuários que desejam navegar na Web enquanto estão na praia. Existem algumas outras circunstâncias em que a comunicação sem fio apresenta vantagens até mesmo para dispositivos fixos. Por exemplo, quando há dificuldades para instalar cabos de fibra óptica em um prédio, por causa de acidentes geográficos (montanhas, florestas, pântanos, etc.), a tecnologia de transmissão sem fio pode ser mais apropriada. Vale a pena observar que a moderna comunicação digital sem fio teve início como um projeto de pesquisa do professor Norman Abramson, da University of Hawaii, na década de 1970, no qual os usuários estavam separados de seu centro de computação pelo Oceano Pacífico e o sistema de telefonia era inadequado. Discutiremos a respeito desse sistema, ALOHA, no Capítulo 4.

2.2.1 O espectro eletromagnético

Quando se movem, os elétrons criam ondas eletromagnéticas que podem se propagar pelo espaço livre (até mesmo no vácuo). Essas ondas foram previstas pelo físico inglês James Clerk Maxwell em 1865 e foram observadas pela primeira vez pelo físico alemão Heinrich Hertz em 1887. O número de oscilações de uma onda eletromagnética por segundo é chamado **frequência**, f, e é medido em Hz (em homenagem a Heinrich Hertz). A distância entre dois pontos máximos (ou mínimos) consecutivos é chamada **comprimento de onda**, designada universalmente pela letra grega λ (lambda).

Quando se instala uma antena de tamanho apropriado em um circuito elétrico, as ondas eletromagnéticas podem ser transmitidas e recebidas com eficiência por um receptor localizado a uma distância bastante razoável. Toda a comunicação sem fio é baseada nesse princípio.

No vácuo, todas as ondas eletromagnéticas viajam à mesma velocidade, independentemente de sua frequência. Essa velocidade, geralmente chamada **velocidade da luz**, c, é aproximadamente igual a 3×10^8 m/s, ou cerca de 30 cm por nanossegundo. No cobre ou na fibra, a velocidade cai para cerca de 2/3 desse valor e se torna ligeiramente dependente da frequência. A velocidade da luz é o limite máximo que se pode alcançar no universo. Nenhum objeto ou sinal pode se mover com maior rapidez do que ela.

A relação fundamental entre f, λ e c (no vácuo) é:

$$\lambda f = c \tag{2.1}$$

Como c é uma constante, se conhecermos f, chegaremos a λ e vice-versa. Via de regra, quando λ é medido em metros e f em MHz, $\lambda f \approx 300$. Por exemplo, ondas de 100 MHz têm cerca de 3 m de comprimento, ondas de 1.000 MHz têm 0,3 m e ondas com 0,1 metro têm uma frequência igual a 3.000 MHz.

O espectro eletromagnético é mostrado na Figura 2.8. As faixas de rádio, micro-ondas, infravermelho e luz visível do espectro podem ser usadas na transmissão de informações, por meio de modulação da amplitude, da frequência ou da fase das ondas. A luz ultravioleta, os raios X e os raios gama representariam opções ainda melhores, por terem frequências mais altas, mas são difíceis de produzir e modular, além de não se propagarem bem através dos prédios e de serem perigosos para os seres vivos.

As bandas (ou faixas) de frequências listadas na parte inferior da Figura 2.8 são os nomes oficiais definidos pela ITU (International Telecommunication Union) e se baseiam nos comprimentos de onda; portanto, a banda LF vai de 1 a 10 km (aproximadamente, de 30 kHz a 300 kHz). Os termos LF, MF e HF são as abreviaturas, em inglês, de baixa, média e alta frequências, respectivamente. É claro que, quando esses nomes foram criados, ninguém esperava ultrapassar 10 MHz, de forma que foram atribuídos os seguintes nomes às bandas mais altas surgidas posteriormente: Very, Ultra, Super, Extremely e Tremendously High Frequency. Além desses não há outros nomes, mas Incredibly, Astonishingly e Prodigiously High Frequency (IHF, AHF e PHF) também serviriam muito bem. Acima de 10^{12} Hz, entramos no espectro do infravermelho, onde a comparação normalmente é com a luz, e não com o rádio.

A base teórica para a comunicação, que veremos mais adiante neste capítulo, afirma que o volume de informações que um sinal como uma onda eletromagnética é capaz de transportar depende da potência recebida e é proporcional à sua largura de banda. Observando a Figura 2.8, é possível entender com clareza por que as pessoas ligadas a redes gostam tanto das fibras ópticas. Muitos GHz de largura de banda estão disponíveis para a transmissão de dados na banda de micro-ondas, e ainda mais na fibra, pois está mais à direita em nossa escala logarítmica. Como exemplo, considere a banda de 1,30 mícron da Figura 2.5, que tem uma largura de 0,17 micra. Se usarmos a Equação 2.1 para encontrar a faixa de frequência dos comprimentos de onda inicial e final, descobrimos que ela é de aproximadamente 30.000 GHz. Com uma razoável relação sinal/ruído de 10 dB, teremos 300 Tbps.

A maioria das transmissões utiliza uma banda de frequência relativamente estreita (ou seja, $\Delta f/f << 1$). Elas concentram seus sinais nessa banda estreita para usar o espectro com mais eficiência e obter taxas de dados razoáveis transmitindo com potência suficiente. O restante desta seção descreve três tipos diferentes de transmissão que utilizam bandas de frequência mais largas.

2.2.2 Espectro de dispersão por salto de frequência

No **espectro por salto de frequência**, o transmissor salta de uma frequência para outra centenas de vezes por segundo. Essa técnica é muito usada em comunicações militares, pois dificulta a detecção das transmissões e é praticamente impossível obstruí-las. Ela também oferece boa resistência ao enfraquecimento devido a sinais tomando múltiplos caminhos da origem ao destino e interferindo após a recombinação. Também oferece resistência à interferência de banda estreita, porque o receptor não fica restrito a uma frequência, quando impossibilitada, por tempo suficiente para encerrar a comunicação. Essa robustez a torna útil para as partes mais sobrecarregadas do espectro, como as bandas ISM que descreveremos em breve. Essa técnica também é

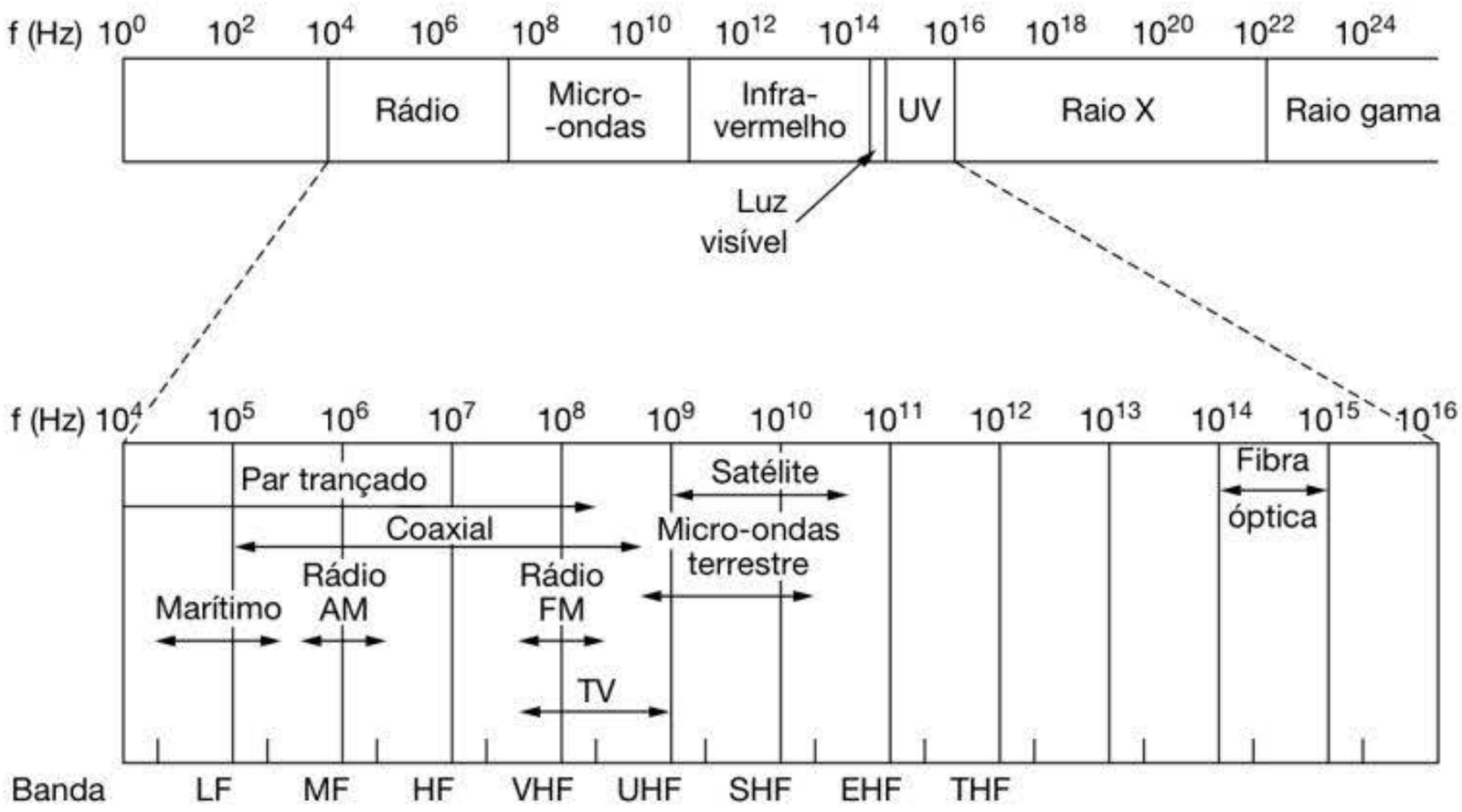


Figura 2.8 O espectro eletromagnético e a maneira como ele é usado na comunicação.

aplicada comercialmente, por exemplo, no Bluetooth e nas versões mais antigas das redes 802.11.

Como curiosidade, vale a pena mencionar que uma das pessoas que criaram essa técnica foi a atriz de cinema austríaca Hedy Lamarr, famosa por atuar em filmes europeus na década de 1930 com seu nome real Hedwig (Hedy) Kiesler. Seu primeiro marido era um rico fabricante de armamentos e mostrou a ela como era fácil bloquear os sinais de rádio então empregados para controlar torpedos. Quando descobriu que ele estava vendendo armas a Hitler, ela ficou horrorizada, se disfarçou de criada para escapar dele e fugiu para Hollywood, a fim de continuar sua carreira como atriz de cinema. Em seu tempo livre, Hedy inventou o salto de frequência para ajudar no esforço de guerra dos Aliados.

Seu esquema utilizava 88 frequências, o número de teclas (e frequências) do piano. Por sua invenção, ela e seu amigo, o compositor George Antheil, receberam a patente 2.292.387 dos Estados Unidos. Porém, eles não conseguiram convencer a Marinha americana de que sua invenção tinha alguma utilidade prática, e nunca receberam royalties por ela. Somente anos depois de expirar a patente, a técnica foi redescoberta e usada em dispositivos eletrônicos móveis, em vez de bloquear sinais para torpedos na época da guerra.

2.2.3 Espectro de dispersão de sequência direta

O **espectro de dispersão de sequência direta** usa uma sequência de código para dispersar o sinal de dados por uma banda de frequência mais ampla. É bastante usado comercialmente, como um meio eficiente na utilização do espectro, para permitir que vários sinais compartilhem a mesma banda de frequência. Esses sinais podem receber diferentes códigos, um método chamado acesso múltiplo por divisão de código, ao qual retornaremos mais adiante neste capítulo. Tal método aparece em contraste com o salto de frequência na Figura 2.9. Ele forma a base das redes de telefonia móvel 3G e também é usado no GPS (Global Positioning System). Mesmo sem códigos diferentes, o espectro de dispersão de sequência direta, assim como o espectro de dispersão por salto de frequência, pode tolerar interferência e enfraquecimento, pois apenas uma fração do sinal desejado é perdida. Ele é usado nesse papel nas versões mais antigas do protocolo de LANs sem fio 802.11b. Para obter informações mais detalhadas e fascinantes sobre a história da comunicação por espectro de dispersão, consulte Walters (2013).

2.2.4 Comunicação de banda ultralarga

A comunicação de banda ultralarga, ou **UWB (Ultra-Wide-Band)** envia uma série de pulsos rápidos, de baixa energia, variando suas frequências de portadora para trocar informações. As rápidas transições levam a um sinal que se espalha estreitamente por uma banda de frequência muito larga. Esse método é definido como sinais que têm uma largura de banda de pelo menos 500 MHz ou pelo menos 20% da frequência central de sua banda de frequência. A UWB também aparece na Figura 2.9. Com tanta largura de banda, a UWB tem o potencial de se comunicar a várias centenas de megabits por segundo. Por se espalhar por uma ampla banda de frequência, ela pode tolerar uma grande quantidade de interferência relativamente forte de outros sinais de banda estreita. Tão importante quanto isso, como a UWB tem pouquíssima energia em qualquer frequência quando usada para transmissão em curta distância, ela não causa interferência prejudicial para outros sinais de rádio de banda estreita. Ao contrário da transmissão por espectro de dispersão, a comunicação UWB transmite de formas que não interferem com os sinais da portadora na mesma banda de frequência. Ela também pode ser usada para formar imagens através de objetos sólidos (solo, paredes e corpos), ou como parte de sistemas de localização precisos. A tecnologia é popular para aplicações internas a curta distância, além de imagens precisas de radar e tecnologias de rastreamento de local.

2.3 TRANSMISSÃO ATRAVÉS DO ESPECTRO

A seguir, mostraremos como as diversas partes do espectro eletromagnético da Figura 2.8 são usadas, começando pelo rádio. Partiremos da premissa de que todas as transmissões utilizam uma banda de frequência estreita, a menos que seja dito de outra forma.

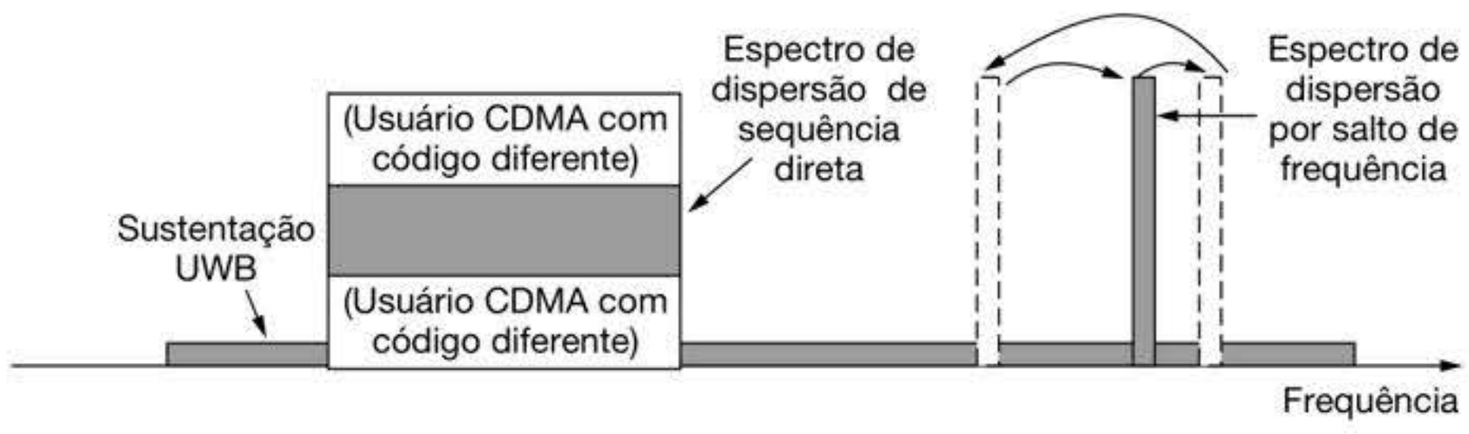


Figura 2.9 Comunicação por espectro de dispersão e banda ultralarga (UWB).

2.3.1 Transmissão de rádio

As ondas de rádio são fáceis de gerar, podem percorrer longas distâncias e penetrar facilmente em prédios, portanto, são muito utilizadas para comunicação, seja em ambientes fechados, seja em locais abertos. As ondas de rádio também são omnidirecionais, o que significa que elas viajam em todas as direções a partir da origem. Desse modo, o transmissor e o receptor não precisam estar cuidadosa e fisicamente alinhados.

Vale lembrar que o rádio omnidirecional nem sempre é bom. Na década de 1970, a General Motors decidiu equipar todos os seus novos Cadillacs com freios controlados por computador, que impediam o travamento das rodas. Quando o motorista pisava no pedal de freio, o computador prendia e soltava os freios, em vez de travá-los de verdade. Um belo dia, um policial rodoviário de Ohio começou a usar seu novo rádio móvel para falar com a central de polícia e, de repente, o Cadillac próximo a ele passou a se comportar como um cavalo selvagem. Depois de ser abordado pelo patrulheiro, o motorista disse que não tinha feito nada e que o carro tinha ficado louco de uma hora para outra.

Com o tempo, começou a surgir um padrão: às vezes, os Cadillacs enlouqueciam, mas somente quando trafegavam pelas principais estradas de Ohio, particularmente quando estavam sendo observados por um policial rodoviário. A General Motors demorou a entender o motivo pelo qual os Cadillacs funcionavam sem problemas nos outros estados e também em rodovias secundárias de Ohio. Só depois de muita pesquisa, eles descobriram que a fiação do Cadillac formava uma ótima antena que captava a frequência usada pelo novo sistema de rádio da Patrulha Rodoviária de Ohio.

As propriedades das ondas de rádio dependem da frequência. Em baixas frequências, as ondas de rádio atravessam bem os obstáculos, mas a potência cai bruscamente à medida que a distância da origem aumenta – pelo menos cerca de $1/r^2$ no ar –, pois a energia do sinal se espalha de forma mais estreita por uma superfície maior. Essa atenuação é chamada de **perda no caminho**. Em altas frequências, as ondas de rádio tendem a viajar em linha reta e a ricochetear nos obstáculos. A perda no caminho ainda reduz a potência, embora o sinal recebido também possa depender muito das reflexões. Ondas de rádio de alta frequência também são absorvidas pela chuva e outros obstáculos, até certo ponto, mais do que as frequências baixas. Em todas as frequências, as ondas de rádio estão sujeitas à interferência de motores e outros equipamentos elétricos.

É interessante comparar a atenuação das ondas de rádio com a dos sinais nos meios guiados. Com fibra, cabo coaxial e par trançado, o sinal cai pela mesma fração por distância unitária, por exemplo, 20 dB por 100 m para o par trançado. Com o rádio, o sinal cai pela mesma fração enquanto a distância dobra, por exemplo, 6 dB pelo dobro do espaço livre. Isso significa que as ondas de rádio podem percorrer longas distâncias, e a interferência entre os usuários é um problema. Por essa razão, todos os governos exercem um rígido controle sobre o licenciamento do uso de transmissores de rádio, com apenas algumas exceções, descritas mais adiante neste capítulo.

Nas bandas VLF, LF e MF, as ondas de rádio se propagam perto do solo, como mostra a Figura 2.10(a). Elas podem ser detectadas dentro de um raio de, talvez, mil quilômetros nas frequências mais baixas; nas frequências mais altas, esse raio é menor. A radiodifusão em frequências AM utiliza a banda MF, razão pela qual as ondas de rádio produzidas pelas estações de rádio AM de Boston não podem ser captadas facilmente em Nova York. As ondas de rádio nessas bandas atravessam os prédios com facilidade; esse é o motivo por que os rádios portáteis funcionam em ambientes fechados. O principal problema relacionado à utilização dessas bandas para comunicação de dados diz respeito à baixa largura de banda que oferecem.

Nas bandas HF e VHF, as ondas terrestres tendem a ser absorvidas pela Terra. No entanto, as ondas que alcançam a ionosfera, uma camada de partículas carregadas situadas em torno da Terra a uma altura de 100 a 500 km, são refratadas por ela e enviadas de volta à Terra, como mostra a Figura 2.10(b). Em determinadas condições atmosféricas, os sinais podem ricochetear diversas vezes. Os operadores de radioamador utilizam essas bandas em comunicações de longa distância. Os militares também se comunicam nas bandas HF e VHF.

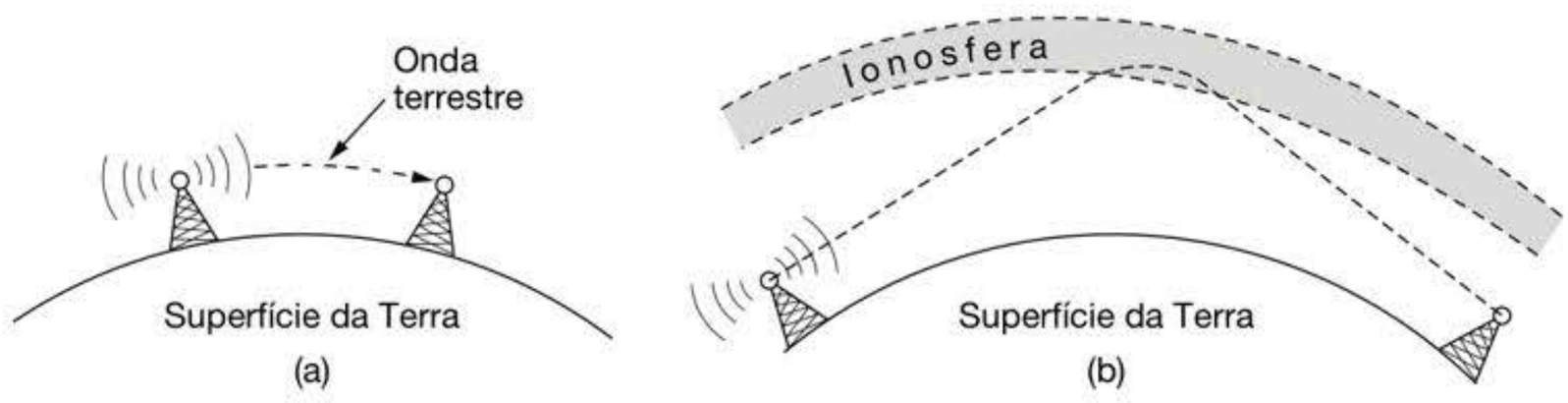


Figura 2.10 (a) Nas bandas VLF, LF e MF, as ondas de rádio obedecem à curvatura da Terra. (b) Na banda HF, elas ricocheteiam na ionosfera.

2.3.2 Transmissão de micro-ondas

Acima de 100 MHz, as ondas trafegam praticamente em linha reta e, portanto, podem ser concentradas em uma faixa estreita. A concentração de toda a energia em um pequeno feixe através de uma antena parabólica (como a conhecida antena de TV por satélite) oferece uma relação sinal/ruído muito mais alta, mas as antenas de transmissão e recepção devem estar alinhadas com o máximo de precisão. Além disso, essa direcionalidade permite o alinhamento de vários transmissores em uma única fileira, fazendo eles se comunicarem com vários receptores também alinhados sem que haja interferência, desde que sejam observadas algumas regras mínimas de espaçamento. Antes da fibra óptica, durante décadas essas micro-ondas formaram o núcleo do sistema de transmissão telefônica de longa distância. Na verdade, a MCI, uma das primeiras concorrentes da AT&T após sua desregulamentação, construiu todo o seu sistema com comunicações de micro-ondas que percorriam dezenas de quilômetros entre uma torre e outra. Até mesmo o nome da empresa refletia isso (MCI significava Microwave Communications, Inc.). Há muito tempo, a MCI passou a utilizar as fibras ópticas e, por meio de uma longa série de fusões corporativas e falências no setor de telecomunicações, passou a fazer parte da Verizon.

Tendo em vista que as micro-ondas trafegam em linha reta, se as torres estiverem muito afastadas, a Terra acabará ficando entre elas (como acontece no caso de um enlace entre Seattle e Amsterdã). Assim, é preciso instalar repetidores em intervalos periódicos. Quanto mais altas as torres, mais distantes elas podem estar umas das outras. A distância entre os repetidores aumenta de acordo com a raiz quadrada da altura da torre. Torres com 100 m de altura devem ter repetidores a cada 80 km.

Ao contrário das ondas de rádio nas frequências mais baixas, as micro-ondas não atravessam muito bem as paredes dos prédios. Além disso, embora o feixe possa estar bem concentrado no transmissor, ainda há alguma divergência no espaço. Algumas ondas podem ser refratadas nas camadas atmosféricas mais baixas e, consequentemente, sua chegada pode ser mais demorada que a das ondas diretas. As ondas atrasadas podem chegar fora de fase em relação à onda direta, e assim cancelar o sinal. Esse efeito é chamado de **enfraquecimento por múltiplos caminhos** (**multipath fading**) e em geral é um problema sério. Ele depende das condições atmosféricas e da frequência. Algumas operadoras mantêm 10% de seus canais ociosos como sobressalentes, para serem usados quando o enfraquecimento por múltiplos caminhos eliminar temporariamente alguma banda de frequência.

A demanda por taxas de dados mais altas incentiva as operadoras a usarem transmissões com frequências cada vez mais altas. As bandas de até 10 GHz agora são de uso rotineiro, mas a partir de 4 GHz surge um novo problema: a absorção pela água. Essas ondas têm apenas alguns centímetros de comprimento e são absorvidas pela chuva. Esse efeito não causaria qualquer problema se estivéssemos planejando construir um gigantesco forno de micro-ondas para ser usado a céu aberto mas, no caso das comunicações, trata-se de um grave problema. Assim como acontece com o enfraquecimento por múltiplos caminhos, a única solução é desligar os enlaces que estão sendo afetados pela chuva e criar uma nova rota que os contorne.

Em resumo, a comunicação por micro-ondas é tão utilizada na telefonia de longa distância, em telefones celulares, na distribuição de sinais de televisão e em outros usos que foi preciso desenvolver uma severa diminuição do espectro. Ela tem uma série de vantagens significativas sobre a fibra. A mais importante delas é que as micro-ondas dispensam a necessidade de se ter direitos sobre um percurso. Além do mais, quando se compra um pequeno lote de terra a cada 50 km e nele é instalada uma torre de micro-ondas, é possível ignorar o sistema telefônico e se comunicar diretamente. Foi por essa razão que a MCI passou a trabalhar tão rapidamente como uma companhia telefônica de longa distância. (A Sprint, outro antigo concorrente da AT&T, trilhou um caminho bem diferente: ela se formou a partir da Southern Pacific Railroad, que já detinha um grande número de concessões de direitos de percurso, e simplesmente enterrava os cabos de fibra ao lado das ferrovias.)

O uso de micro-ondas também é relativamente econômico. A instalação de duas torres simples (com alguns postes com quatro esteios) e a colocação de antenas em cada uma delas pode ser menos dispendiosa que enterrar 50 km de fibra em uma área urbana congestionada ou em uma região montanhosa, e talvez seja mais econômica que arrendar a rede de fibra da companhia telefônica, especialmente se esta ainda não tiver coberto totalmente os custos da retirada do cobre quando os cabos de fibra foram instalados.

2.3.3 Transmissão de infravermelho

As ondas de infravermelho não guiadas são bastante utilizadas na comunicação de curto alcance. Todos os dispositivos de controle remoto utilizados nos aparelhos de televisão, ar condicionado e aparelhos de som empregam a comunicação por infravermelho. Elas são relativamente direcionais, baratas e fáceis de montar, mas têm uma desvantagem importante: não atravessam objetos sólidos (para provar essa afirmação, posicione-se entre o controle remoto e o televisor e veja se ele funciona). Em geral, quando nos deslocamos do rádio de onda longa em direção à luz visível, as ondas assumem um comportamento cada vez mais parecido com o da luz, perdendo pouco a pouco as características de ondas de rádio.

No entanto, o fato de as ondas de infravermelho não atravessarem paredes sólidas pode ser visto como uma qualidade. É por essa razão que um sistema infravermelho

instalado em um ambiente fechado não interfere em um sistema semelhante instalado nas salas ou nos prédios adjacentes: não é possível controlar o aparelho de televisão do vizinho com o seu controle remoto. Além disso, a segurança dos sistemas de infravermelho contra espionagem é melhor que a dos sistemas de rádio, exatamente por essa razão. Portanto, não é necessária nenhuma licença do governo para operar um sistema de infravermelho, ao contrário dos sistemas de rádio, que devem ser licenciados fora das bandas ISM. A comunicação por infravermelho tem uso limitado em escritórios, por exemplo, para conectar notebooks e impressoras com o padrão **IrDA** (**Infrared Data Association**), mas não tem um papel importante no jogo das comunicações.

2.3.4 Transmissão via luz

A transmissão óptica não guiada, ou **óptica do espaço livre**, vem sendo utilizada há séculos. Paul Revere usou a sinalização óptica binária a partir da Old North Church imediatamente antes de sua famosa corrida. Uma aplicação mais moderna consiste em conectar as LANs em dois prédios por meio de lasers instalados em seus telhados. Por sua própria natureza, a transmissão óptica usando raios laser é unidirecional; assim, cada prédio precisa de seu próprio raio laser e de seu próprio fotodetector. Esse esquema oferece uma largura de banda muito alta, tem baixo custo e é relativamente seguro, pois é difícil interceptar um raio laser estreito. Ele também é relativamente fácil de ser instalado e, ao contrário das micro-ondas, não precisa de uma licença da **FCC** (**Federal Communications Commission**) nos Estados Unidos e de outras agências governamentais em outros países.

A potência do laser, um feixe muito estreito, também é uma desvantagem aqui. Apontar um raio laser de 1 mm para um alvo do tamanho de uma cabeça de alfinete a 500 m de distância exige a mira de um pistoleiro dos filmes de faroeste. Normalmente, são colocadas lentes no sistema para tirar um pouco do foco do raio. Para aumentar a dificuldade, mudanças no vento e na temperatura podem distorcer o raio, e os feixes de raios laser de fato não podem atravessar chuva ou neblina espessa, mas normalmente funcionam bem em dias ensolarados. Contudo, muitos desses fatores não são problemas quando o uso é para conectar duas naves espaciais.

Um dos autores deste livro (AST) certa vez participou de uma conferência em um moderno hotel europeu cujos organizadores tiveram a felicidade de oferecer uma sala repleta de terminais para que os participantes pudessem ler seus e-mails durante as apresentações menos interessantes. Como a companhia telefônica local não havia se disposto a instalar um grande número de linhas telefônicas por apenas três dias, os organizadores colocaram um raio laser no telhado e o apontaram na direção do prédio de ciência da computação da universidade onde trabalhavam, situada a alguns quilômetros dali. Eles testaram o sistema na noite anterior à conferência e ele havia funcionado perfeitamente. Às 9h da manhã seguinte, em um belo dia de sol, o sistema entrou em pane e ficou fora do ar durante todo o dia. Nos dois dias seguintes, o problema se repetiu. Após a conferência, os organizadores descobriram o problema. O calor do sol fez as correntes de convecção do telhado do prédio emanarem, como mostra a Figura 2.11. Esse ar turbulento desviou o feixe e fez ele dançar em torno do detector, algo parecido com uma estrada que parece cintilar em um dia muito quente. A lição aqui é que, para funcionar bem em condições difíceis e também em boas condições, os enlaces ópticos não guiados precisam ser elaborados com uma margem de erro suficiente.

Atualmente, a comunicação óptica não guiada pode parecer uma tecnologia de rede exótica, mas logo poderá se tornar mais predominante. Em muitos lugares, estamos cercados de câmeras (que sentem a luz) e telas (que emitem luz usando LEDs ou outra tecnologia). A comunicação de dados pode ser disposta em cima dessas telas codificando informações no padrão em que os LEDs se acendem e apagam, abaixo do limiar da percepção humana. A comunicação via luz visível dessa maneira é inerentemente segura e cria uma rede de baixa velocidade na vizinhança imediata da tela. Isso poderia permitir todo o tipo de cenário de computação ubíqua. As luzes piscando nos veículos de emergência podem alertar os sinais de trânsito e veículos mais próximos para ajudar a liberar um caminho. Os sinais de informação poderiam transmitir mapas por difusão. As lâmpadas em ocasiões festivas poderiam até mesmo transmitir canções sincronizadas com sua exibição.

2.4 DE FORMAS DE ONDA A BITS

Nesta seção, descrevemos como os sinais são transmitidos através do meio físico que já discutimos. Vamos começar com uma discussão da base teórica para a comunicação de dados, seguindo com uma discussão sobre modulação (o processo de conversão de formas analógicas de onda em bits) e multiplexação (que permite que um único meio físico carregue várias transmissões simultâneas).

2.4.1 A base teórica da comunicação de dados

As informações podem ser transmitidas por fios, fazendo-se variar alguma propriedade física, como tensão ou corrente. Representando o valor dessa tensão ou corrente como uma função do tempo com um valor único, $f(t)$, podemos criar um modelo para o comportamento do sinal e

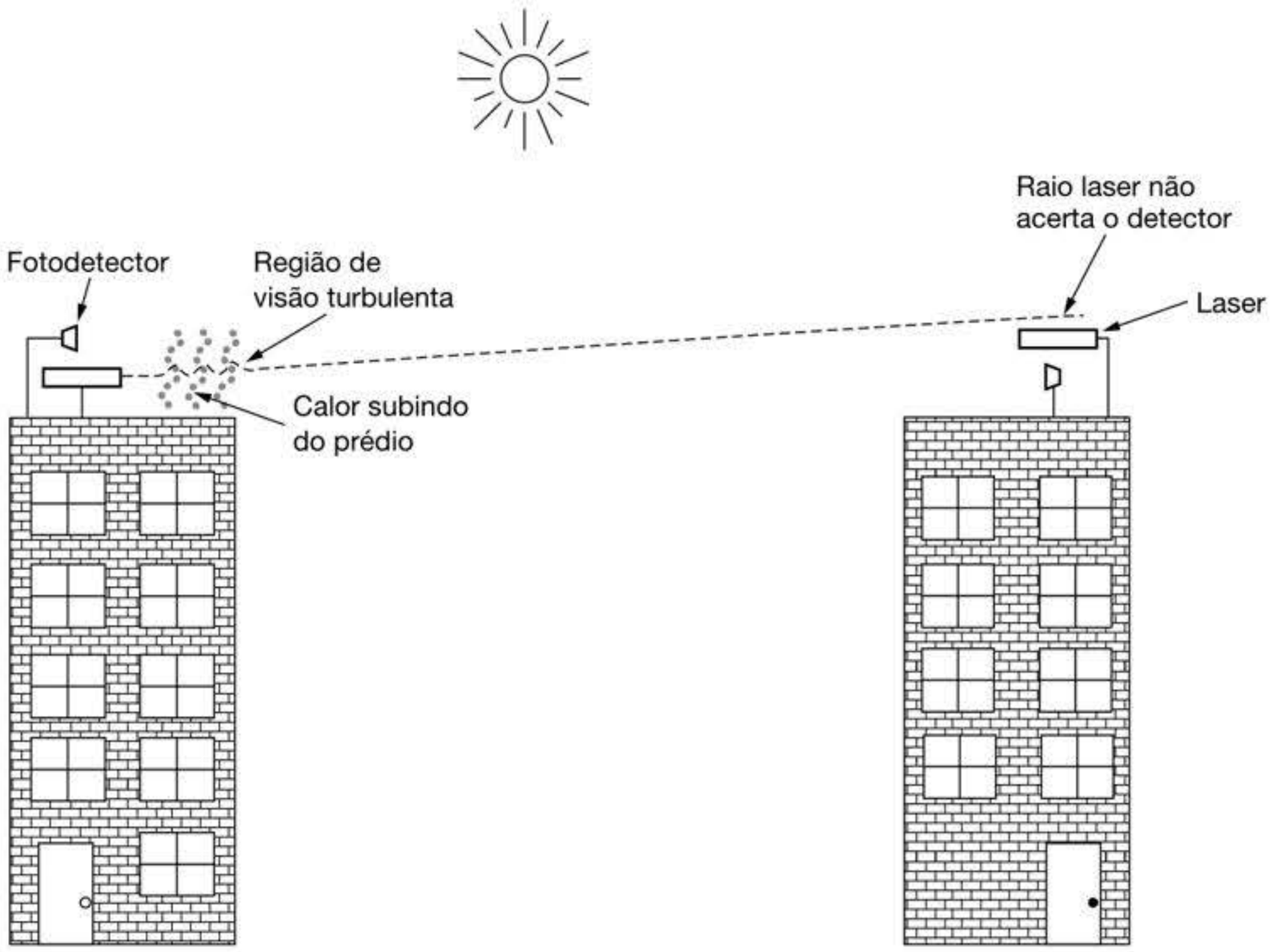


Figura 2.11 Correntes de convecção podem interferir nos sistemas de comunicação a laser. A figura mostra um sistema bidirecional com dois lasers.

analisá-lo matematicamente. Essa análise será o assunto das próximas seções.

Análise de Fourier

No início do século XIX, o matemático francês Jean-Baptiste Fourier provou que qualquer função periódica razoavelmente estável, $g(t)$, com período T, pode ser construída como a soma de um número (possivelmente infinito) de senos e cossenos:

$$g(t) = \frac{1}{2}c + \sum_{n=1}^{\infty} a_n \operatorname{sen}(2\pi nft) + \sum_{n=1}^{\infty} b_n \cos(2\pi nft) \quad (2.2)$$

onde $f = 1/T$ é a frequência fundamental, a_n e b_n são as amplitudes do seno e do cosseno dos n-ésimos **harmônicos** (termos) e c é uma constante que determina o valor médio da função. Essa decomposição é chamada **série de Fourier**. A partir dela, a função pode ser reconstruída, ou seja, se o período T for conhecido e as amplitudes forem dadas, a função original no tempo poderá ser encontrada efetuando-se as somas da Equação 2.2.

Um sinal de dados com uma duração finita (como acontece com todos eles) pode ser tratado apenas com base na premissa de que ele repete o mesmo padrão indefinidamente (ou seja, o intervalo de T a $2T$ é igual ao de 0 a T, etc.).

As a_n amplitudes podem ser calculadas para qualquer $g(t)$ dada, multiplicando-se ambos os lados da Equação 2.2 por $\operatorname{sen}(2\pi kft)$ e, em seguida, integrando-se de 0 a T. Como

$$\int_0^T \operatorname{sen}(2\pi kft)\operatorname{sen}(2\pi nft)\,dt = \begin{cases} 0 \text{ para } k \neq n \\ T/2 \text{ para } k = n \end{cases}$$

apenas um termo do somatório permanece: a_n. O somatório b_n desaparece completamente. Da mesma forma, multiplicando a Equação 2.2 por $\cos(2\pi kft)$ e integrando entre 0 e T, podemos derivar b_n. Simplesmente integrando ambos os lados da equação tal como ela se encontra, podemos achar c. Os resultados dessas operações são:

$$a_n = \frac{2}{T}\int_0^T g(t)\operatorname{sen}(2\pi nft)\,dt$$

$$b_n = \frac{2}{T}\int_0^T g(t)\cos(2\pi nft)\,dt \qquad c = \frac{2}{T}\int_0^T g(t)\,dt$$

Sinais limitados pela largura de banda

A importância de tudo isso para a comunicação de dados é que os canais reais afetam diferentes sinais de frequência de formas distintas. Vamos analisar um exemplo específico: a transmissão do caractere ASCII "b" codificado como um

byte de 8 bits. O padrão de bits que deve ser transmitido é 01100010. A parte esquerda da Figura 2.12(a) mostra a saída de tensão do computador transmissor. A análise de Fourier desse sinal produz os seguintes coeficientes:

$$a_n = \frac{1}{\pi n}[\cos(\pi n/4) - \cos(3\pi n/4) + \cos(6\pi n/4) - \cos(7\pi n/4)]$$

$$b_n = \frac{1}{\pi n}[\text{sen}(3\pi n/4) - \text{sen}(\pi n/4) + \text{sen}(7\pi n/4) - \text{sen}(6\pi n/4)]$$

$c = 3/4$.

A raiz quadrada média das amplitudes, $\sqrt{a_n^2 + b_n^2}$ para os primeiros termos são mostradas no lado direito da Figura 2.12(a). Esses valores são importantes porque seus quadrados são proporcionais à energia transmitida na frequência correspondente.

Nenhum recurso de transmissão é capaz de transmitir sinais sem perder parte da energia no processo. Se todos os coeficientes da série de Fourier fossem igualmente reduzidos, o sinal resultante seria reduzido em amplitude, mas não distorcido (i.e., ele teria a mesma forma quadrada mostrada na Figura 2.12(a)). Infelizmente, todos os meios de transmissão reduzem diferentes componentes de Fourier

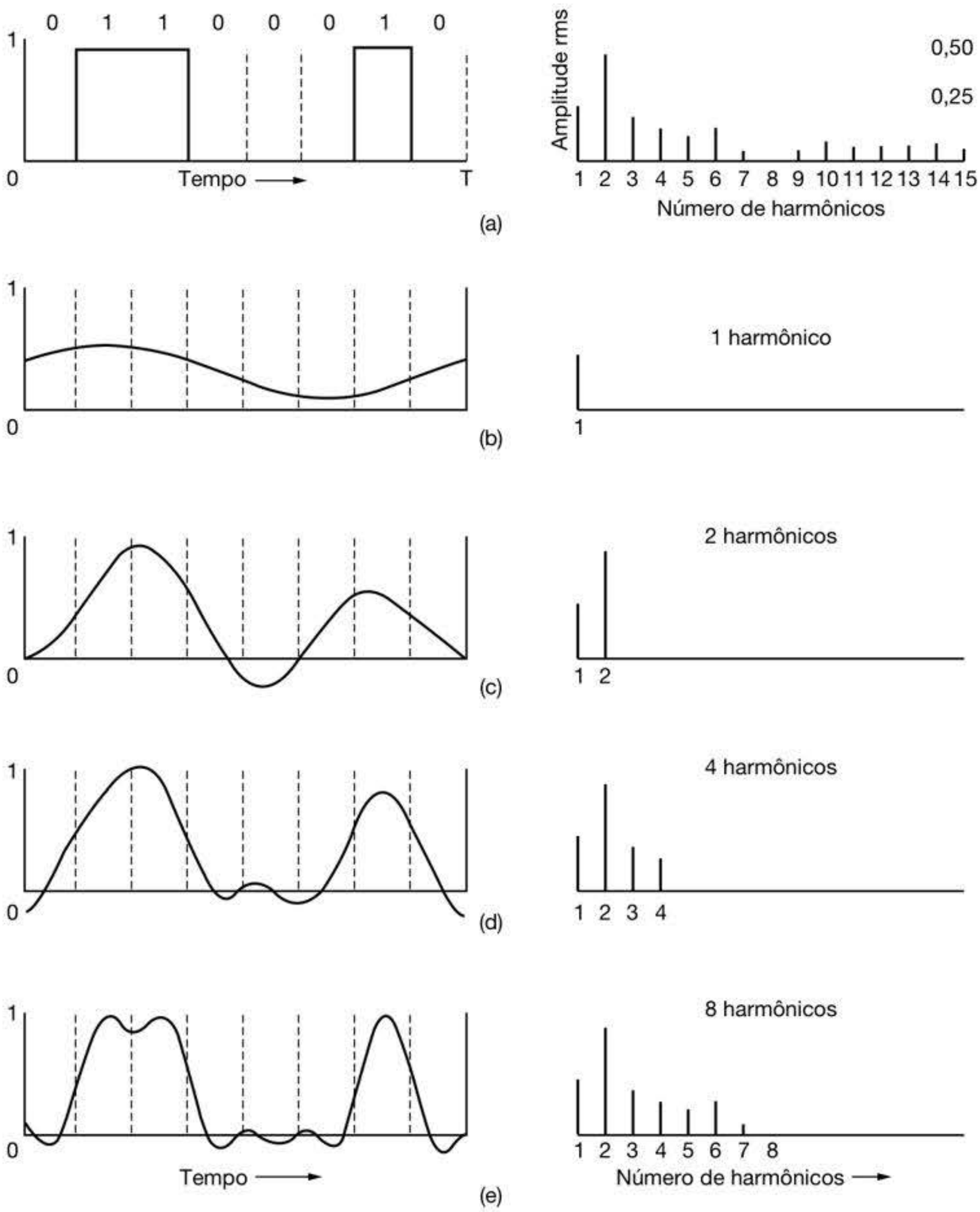


Figura 2.12 (a) Um sinal binário e a raiz quadrada média das amplitudes de Fourier. (b)-(e) Aproximações sucessivas do sinal original.

por diferentes valores e, em consequência disso, introduzem distorção. Em geral, para um fio, as amplitudes são transmitidas sem redução, de 0 até alguma frequência f_c (medida em ciclos/s ou Hz), com todas as frequências acima dessa frequência de corte (*cutoff*) sendo atenuadas. A faixa de frequências transmitidas sem serem fortemente atenuadas denomina-se **largura de banda**. Na prática, o corte não é nítido; assim, muitas vezes a largura de banda varia de 0 até a frequência em que a potência recebida caiu para a metade.

A largura de banda é uma propriedade física do meio de transmissão, que depende, por exemplo, da construção, da espessura e do comprimento do meio (fio ou fibra). Os filtros normalmente são usados para limitar ainda mais o volume de largura de banda de um sinal. Os canais sem fio 802.11 têm permissão para usar até aproximadamente 20 MHz, por exemplo, de modo que rádios 802.11 filtram a largura de banda do sinal para essa faixa (embora, em alguns casos, seja usada uma banda de 80 MHz).

Como outro exemplo, os canais de televisão tradicionais (analógicos) ocupam 6 MHz cada, por um fio ou pelo ar. Essa filtragem permite que mais sinais compartilhem determinada região do espectro, o que melhora a eficiência geral do sistema. Isso significa que a faixa de frequência para alguns sinais não começará em zero, mas em algum número maior. Porém, isso não importa. A largura de banda ainda é a largura da banda de frequências que são passadas, e a informação que pode ser transportada depende apenas dessa largura, e não das frequências inicial e final. Os sinais que vão de 0 para cima, até uma frequência máxima, são chamados sinais de **banda base**. Sinais que são deslocados para ocupar uma faixa de frequências mais alta, como acontece para todas as transmissões sem fio, são chamados sinais de **banda passante**.

Vejamos agora como seria a forma do sinal da Figura 2.12(a) se a largura de banda fosse tão estreita que apenas as frequências mais baixas fossem transmitidas (ou seja, se a função estivesse sendo aproximada pelos primeiros termos da Equação 2.2). A Figura 2.12(b) mostra o sinal resultante de um canal pelo qual apenas o primeiro harmônico (o fundamental, f) pode passar. Da mesma forma, a Figura 2.12(c)-(e) mostra os espectros e as funções reconstruídas para canais com uma largura de banda mais alta. Para a transmissão digital, o objetivo é receber um sinal com fidelidade suficiente para reconstruir a sequência de bits que foi enviada. Já podemos fazer isso com facilidade na Figura 2.12(e), de modo que é um desperdício usar mais harmônicos para receber uma réplica mais precisa.

Dada uma taxa de bits de b bits/s, o tempo necessário para o envio de 8 bits (em nosso exemplo, um bit de cada vez) é $8/b$ s; assim, a frequência do primeiro harmônico desse sinal é $b/8$ Hz. Uma linha telefônica comum, frequentemente chamada de **linha de qualidade de voz**, tem uma frequência de corte artificialmente introduzida, pouco acima de 3.000 Hz. A presença dessa restrição significa que o número do harmônico mais alto transmitido é aproximadamente $3.000/(b/8)$ ou $24.000/b$ (o corte não é preciso).

Para algumas taxas de dados, os números funcionam de acordo com o padrão mostrado na Figura 2.13. Esses números deixam claro que, quando se tenta fazer uma transmissão a 9.600 bps usando uma linha telefônica de qualidade de voz, o modelo sugerido na Figura 2.12(a) assume a forma mostrada na Figura 2.12(c), tornando complicada a recepção precisa do fluxo original de bits. Podemos perceber também que, em taxas de dados bem mais altas que 38,4 kbps, não existe a menor possibilidade de que todos os sinais sejam *binários*, mesmo quando não há o menor ruído no meio de transmissão. Em outras palavras, limitando-se a largura de banda, limita-se a taxa de dados, mesmo nos canais perfeitos (sem ruídos). No entanto, sofisticados esquemas de codificação que usam diversos níveis de tensão possibilitam a existência e a utilização de taxas de dados mais altas. Vamos discutir essa questão ao longo deste capítulo.

Há muita confusão sobre largura de banda, pois ela significa coisas diferentes para engenheiros elétricos e para cientistas da computação. Para os engenheiros elétricos, a largura de banda (analógica) é, como descrevemos, uma quantidade medida em Hz. Para os cientistas da computação, a largura de banda (digital) é a taxa de dados máxima de um canal, uma quantidade medida em bits/s. Essa taxa de dados é o resultado final do uso da largura de banda analógica de um canal físico para a transmissão digital, e as duas estão relacionadas, conforme veremos a seguir. Neste livro, o contexto esclarecerá se queremos dizer largura de banda analógica (Hz) ou largura de banda digital (bits/s).

Bps	T (ms)	Primeiro harmônico (Hz)	Número de harmônicos enviados
300	26,67	37,5	80
600	13,33	75	40
1.200	6,67	150	20
2.400	3,33	300	10
4.800	1,67	600	5
9.600	0,83	1.200	2
19.200	0,42	2.400	1
38.400	0,21	4.800	0

Figura 2.13 Relação entre as taxas de dados e os harmônicos em nosso exemplo muito simples.

2.4.2 A taxa de dados máxima de um canal

Em 1924, Henry Nyquist, um engenheiro da AT&T, percebeu que até mesmo um canal perfeito tem uma capacidade de transmissão finita. Ele derivou uma equação expressando a taxa máxima de dados de um canal sem ruído com largura de banda finita. Em 1948, Claude Shannon aprofundou o trabalho de Nyquist e o estendeu ao caso de um canal sujeito a ruído aleatório (i.e., ruído termodinâmico) (Shannon, 1948). Esse artigo é o mais importante de toda a teoria da informação. Veja a seguir um resumo dos resultados, agora clássicos.

Nyquist provou que, se um sinal arbitrário atravessar um filtro passa-baixa de largura de banda B, o sinal filtrado poderá ser completamente reconstruído a partir de apenas $2B$ amostras (exatas) por segundo. Fazer uma amostragem da linha com uma rapidez maior que $2B$ vezes por segundo seria inútil, pois os componentes de frequência mais alta que essa amostragem poderia recuperar já teriam sido filtrados. Se o sinal consistir em V níveis discretos, o teorema de Nyquist afirma que:

$$\text{taxa máxima de dados} = 2B \log_2 V \text{ bits/s} \qquad (2.3)$$

Por exemplo, um canal de 3 kHz sem ruído não pode transmitir sinais binários (ou seja, de dois níveis) a uma taxa superior a 6.000 bps.

Até agora, só mencionamos os canais sem ruído. Se houver ruído aleatório, a situação se deteriorará com rapidez. Além disso, sempre há ruído aleatório (térmico), em virtude do movimento das moléculas no sistema. O volume de ruído térmico presente é medido pela relação entre a potência do sinal e a do ruído, chamada relação sinal/ruído, ou **SNR (Signal-to-Noise Ratio)**. Se representarmos a potência do sinal por S e a potência do ruído por N, a relação sinal/ruído será S/N. Em geral, a relação é expressa em uma escala logarítmica como a quantidade $10 \log_{10} S/N$, pois ela pode variar por uma faixa muito grande. As unidades da escala logarítmica são chamadas de **decibéis (dB)**, com "deci" significando 10 e "bel" escolhido em homenagem a Alexander Graham Bell, que patenteou o telefone. Uma relação S/N igual a 10 corresponde a 10 dB, uma relação igual a 100 equivale a 20 dB, uma relação igual a 1.000 equivale a 30 dB, e assim por diante. Com frequência, os fabricantes de amplificadores estereofônicos caracterizam a largura de banda (faixa de frequência) na qual seu produto é linear oferecendo a frequência de 3 dB em cada extremidade. Esses são os pontos em que o fator de amplificação foi dividido aproximadamente ao meio (porque $10 \log_{10} 0{,}5 \approx -3$).

O principal resultado de Shannon é que a taxa máxima de dados ou **capacidade** de um canal com ruídos cuja largura de banda é B Hz, e cuja relação sinal/ruído é S/N, é dada por:

$$\text{Taxa máxima de dados} = B \log_2 (1 + S/N) \textit{ bits/s} \qquad (2.4)$$

Esta equação nos diz as melhores capacidades que os canais reais podem ter. Por exemplo, ADSL (Asymmetric Digital Subscriber Line), que oferece acesso à Internet por linhas telefônicas normais, usa uma largura de banda com cerca de 1 MHz. A SNR depende muito da distância da central telefônica até a casa, e uma SNR com cerca de 40 dB para linhas curtas de 1 a 2 km é muito boa. Com essas características, o canal nunca pode transmitir muito mais do que 13 Mbps, não importa quantos níveis de sinal sejam usados e não importa com que frequência as amostras são tomadas. Na prática, a ADSL é especificada para até 12 Mbps, embora os usuários normalmente vejam taxas mais baixas. Essa taxa de dados é realmente muito boa, com mais de 60 anos de técnicas de comunicação tendo reduzido bastante a lacuna entre a capacidade de Shannon e a capacidade dos sistemas reais.

O resultado de Shannon utilizou os argumentos da teoria da informação e se aplica a qualquer canal sujeito a ruído térmico. Os exemplos contrários devem ser tratados na mesma categoria das máquinas de movimento contínuo (ou moto-perpétuo). Para a ADSL ultrapassar os 12 Mbps, ela deve melhorar a SNR (p. ex., inserindo repetidores digitais nas linhas mais próximas do cliente) ou usar mais largura de banda, como é feito com a evolução para ADSL2+.

2.4.3 Modulação digital

Agora que já estudamos as propriedades dos canais com e sem fio, voltamos nossa atenção para o problema de enviar informações digitais. Os canais com fio e sem fio transportam sinais analógicos, como a tensão variando continuamente, a intensidade de luz ou a intensidade de som. Para enviar informações digitais, temos de criar sinais digitais para representar os bits. O processo de conversão entre bits e sinais que os representam é chamado de **modulação digital**.

Começaremos com esquemas que convertem diretamente os bits em um sinal. Eles resultam em **transmissão de banda base**, em que o sinal ocupa frequências de zero até um máximo que depende da taxa de sinalização. Ele é comum para fios. Depois, vamos considerar os esquemas que regulam a amplitude, a fase ou a frequência de um sinal da portadora para transportar bits. Esses esquemas resultam em **transmissão de banda passante**, em que o sinal ocupa uma banda de frequências em torno da frequência do sinal da portadora. Isso é comum para canais sem fio e ópticos, para os quais os sinais devem residir em uma determinada banda de frequência.

Os canais normalmente são compartilhados por vários sinais. Afinal, é muito mais conveniente usar um único fio para transportar vários sinais do que instalar um fio para cada sinal. Esse tipo de compartilhamento é chamado de **multiplexação** e pode ser realizado de diversas maneiras.

Apresentaremos métodos para multiplexação por divisão de tempo, frequência e código.

As técnicas de modulação e multiplexação que descrevemos nesta seção são todas bastante usadas para canais com fio, de fibra, terrestres sem fio e por satélite.

Transmissão de banda base

A forma mais simples de modulação digital é usar uma tensão positiva para representar um bit 1 e uma tensão negativa para representar um bit 0, como pode ser visto na Figura 2.14(a). Para uma fibra óptica, a presença de luz poderia representar 1 e a ausência de luz, 0. Esse esquema é chamado **NRZ (Non-Return-to-Zero)**. O nome esquisito tem motivos históricos, e significa simplesmente que o sinal acompanha os dados. Um exemplo aparece na Figura 2.14(b).

Uma vez enviado, o sinal NRZ se propaga pelo fio. Na outra ponta, o receptor o converte para bits fazendo a amostragem do sinal em intervalos de tempo regulares. Esse sinal não ficará exatamente como o enviado, e será atenuado e distorcido pelo canal e pelo ruído no receptor. Para decodificar os bits, o receptor mapeia as amostras de sinal para os símbolos mais próximos. Para o NRZ, uma tensão positiva será considerada para indicar que foi enviado um bit 1, e uma tensão negativa será considerada para indicar que foi enviado um 0.

O NRZ é um bom ponto de partida para nossos estudos, pois é simples, mas na prática raramente é usado por si só. Esquemas mais complexos podem converter bits em sinais, os quais atendem melhor às considerações da engenharia. Esses esquemas são chamados de **códigos de linha**. A seguir, descreveremos os códigos de linha que ajudam na eficiência da largura de banda, recuperação de clock e equilíbrio na componente CC.

Eficiência da largura de banda

Com o NRZ, o sinal pode alternar entre os níveis positivo e negativo a cada 2 bits (no caso de 1s e 0s alternados). Isso significa que precisamos de uma largura de banda de pelo menos $B/2$ Hz quando a taxa de bits é B bits/s. Essa relação vem da taxa de Nyquist (Equação 2.3). Por ser um limite fundamental, não podemos usar o NRZ mais rápido sem usar mais largura de banda, sendo normalmente um recurso limitado, até mesmo para canais com fio. Sinais de maior frequência são cada vez mais atenuados, tornando-os menos úteis, e sinais de maior frequência também exigem circuitos eletrônicos mais rápidos.

Uma estratégia para usar largura de banda limitada com mais eficiência é usar mais de dois níveis de sinalização. Usando quatro níveis de tensão, por exemplo, podemos enviar 2 bits ao mesmo tempo como um único **símbolo**. Esse projeto funcionará desde que o sinal no receptor seja suficientemente forte para distinguir os quatro níveis. A taxa em que o sinal muda é, então, metade da taxa de bits, de modo que a largura de banda necessária foi reduzida.

Chamamos a taxa em que o sinal muda de **taxa de símbolos**, para distingui-la da **taxa de bits** (que é a taxa de símbolos multiplicada pelo número de bits por símbolo). Um nome mais antigo para a taxa de símbolos, principalmente no contexto de dispositivos chamados modems de telefone, que transmitem dados digitais pelas linhas telefônicas, é **taxa baud**. Na literatura, "taxa de bits" e "taxa baud" normalmente são usadas de modo incorreto.

Observe que o número de níveis de sinal não precisa ser uma potência de dois. Normalmente ele não é, com alguns dos níveis usados para proteção contra erros e simplificação do projeto do receptor.

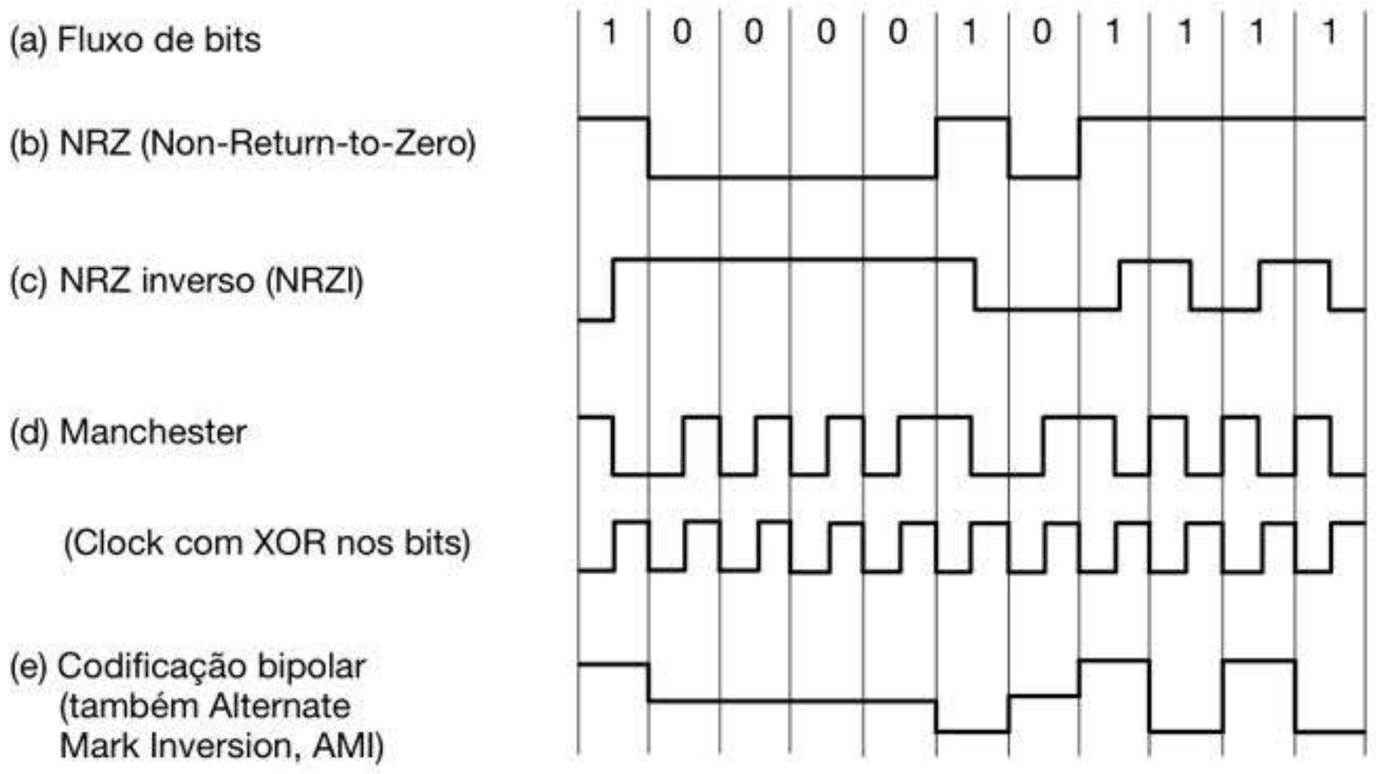


Figura 2.14 Códigos de linha: (a) Bits, (b) NRZ, (c) NRZI, (d) Manchester, (e) Bipolar ou AMI.

Recuperação de clock

Para todos os esquemas que codificam bits em símbolos, o receptor precisa saber quando um símbolo termina e o próximo começa, para decodificar os bits corretamente. Com o NRZ, em que os símbolos são simplesmente níveis de tensão, uma longa sequência de 0s ou 1s deixa o sinal inalterado. Após um tempo, é difícil distinguir os bits, pois 15 zeros se parecem muito com 16 zeros, a menos que você tenha um clock muito preciso.

Os clocks precisos ajudariam com esse problema, mas eles são uma solução dispendiosa para equipamentos básicos. Lembre-se de que estamos temporizando bits em links que trabalham em muitos megabits/s, de modo que o clock teria de fluir por menos de uma fração de microssegundo pela maior sequência permitida. Isso pode ser razoável para enlaces lentos ou mensagens curtas, mas essa não é uma solução geral.

Uma estratégia é enviar um sinal de clock separado para o receptor. Outra linha de clock não é grande coisa para barramentos de computador ou cabos curtos, em que existem muitas linhas paralelas, mas é um desperdício para a maioria dos enlaces de rede, pois, se tivéssemos outra linha para enviar um sinal, poderíamos usá-la para enviar dados. Aqui, um truque inteligente é misturar o sinal de clock com o sinal de dados efetuando a operação XOR por ambos, de modo que nenhuma linha extra seja necessária. Os resultados aparecem na Figura 2.14(d). O clock faz uma transição de clock em cada tempo de bit, de modo que ele funciona no dobro da taxa de bits. Quando ele é XORado com o nível 0, faz a transição de baixo para alto, que é simplesmente o clock. Essa transição é o 0 lógico. Quando ele é XORado com o nível 1, é invertido e faz uma transição de alto para baixo. Essa transição é o 1 lógico. Esse esquema é conhecido como **codificação Manchester**, e foi usado para a Ethernet clássica.

A desvantagem da codificação Manchester é que ela exige duas vezes mais largura de banda que a NRZ, em virtude do clock, e aprendemos que a largura de banda normalmente importa. Uma estratégia diferente é baseada na ideia de que devemos codificar os dados para garantir que haja transições suficientes no sinal. Considere que o NRZ terá problemas de recuperação de clock somente para longas sequências de 0s e 1s. Se houver transições frequentes, será fácil para o receptor permanecer sincronizado com o fluxo de símbolos de chegada.

Como um passo na direção certa, podemos simplificar a situação codificando um 1 como uma transição e 0 como nenhuma transição, ou vice-versa. Essa codificação é chamada **NRZI** (**Non-Return-to-Zero Inverted**), uma variação da NRZ. Um exemplo pode ser visto na Figura 2.14(c). O popular padrão **USB** (**Universal Serial Bus**) para a conexão de periféricos de computador usa NRZI. Com ele, longas sequências de 1s não causam problema.

Naturalmente, longas sequências de 0s ainda causam um problema que precisamos consertar. Se fôssemos a companhia telefônica, poderíamos simplesmente solicitar que o transmissor não transmitisse muitos 0s. Para que funcionassem corretamente, as linhas telefônicas digitais mais antigas nos Estados Unidos, chamadas **linhas T1** (discutidas mais adiante), realmente exigiam que não mais do que 15 zeros consecutivos fossem enviados. Para resolver realmente o problema, podemos dividir sequências de 0s mapeando pequenos grupos de bits para serem transmitidos de modo que os grupos com 0s sucessivos sejam mapeados para padrões ligeiramente maiores, que não têm muitos 0s consecutivos.

Um código bem conhecido para fazer isso é chamado **4B/5B**. Cada 4 bits são mapeados para um padrão de 5 bits com uma tabela de tradução fixa. Os padrões de 5 bits são escolhidos de tal forma que nunca haverá uma sequência de mais de três 0s consecutivos. O mapeamento aparece na Figura 2.15. Esse esquema acrescenta 20% de overhead, que é melhor do que os 100% de overhead da codificação Manchester. Como existem 16 combinações de entrada e 32 combinações de saída, algumas das combinações de saída não são usadas. Deixando de lado as combinações com muitos 0s sucessivos, ainda existirão alguns códigos. Como um bônus, podemos usar esses códigos não de dados para representar sinais de controle da camada física. Por exemplo, em alguns casos, "11111" representa a linha ociosa, e "11000" representa o início de um quadro.

Uma alternativa é fazer os dados parecerem aleatórios, o que é conhecido como embaralhamento (ou scrambling). Nesse caso, provavelmente haverá transições frequentes. Um **embaralhador** funciona realizando o XOR dos dados com uma sequência pseudoaleatória antes de serem transmitidos. Esse embaralhamento tornará os dados tão aleatórios quanto a sequência pseudoaleatória (supondo que eles sejam independentes da sequência pseudoaleatória). O receptor, então, realiza o XOR dos bits de entrada com

Dados (4B)	Código (5B)	Dados (4B)	Código (5B)
0000	11110	1000	10010
0001	01001	1001	10011
0010	10100	1010	10110
0011	10101	1011	10111
0100	01010	1100	11010
0101	01011	1101	11011
0110	01110	1110	11100
0111	01111	1111	11101

Figura 2.15 Mapeamento 4B/5B.

a mesma sequência pseudoaleatória para registrar os dados reais. Para que isso seja prático, a sequência pseudoaleatória precisa ser fácil de criar. Em geral, ela é dada como uma semente para um gerador de números aleatórios simples.

O embaralhamento é atraente porque não acrescenta largura de banda ou tempo. De fato, normalmente é importante condicionar o sinal de modo que ele não mantenha energia em componentes de frequência dominantes (causados por padrões de dados repetitivos) que poderiam radiar interferência eletromagnética. O embaralhamento ajuda porque os sinais aleatórios tendem a ser "brancos" ou ter a energia espalhada pelos demais componentes de frequência.

Contudo, ele não garante que não haverá sequências longas. Ocasionalmente, é possível não ter sorte. Se os dados forem iguais à sequência pseudoaleatória, com o XOR, eles serão todos 0. Esse resultado geralmente não ocorre com uma sequência pseudoaleatória longa, difícil de prever. Contudo, com uma sequência curta ou previsível, é possível que usuários maliciosos enviem padrões de bits que causem longas sequências de 0s depois do embaralhamento, causando falha nos enlaces. As antigas versões dos padrões para enviar pacotes IP por enlaces SONET no sistema telefônico tinham esse defeito (Malis e Simpson, 1999). Era possível que os usuários enviassem certos "pacotes fatais", sabendo que isso causaria problemas.

Sinais balanceados

Sinais que têm tanto tensão positiva quanto negativa, mesmo que seja por curtos períodos, são chamados de **sinais balanceados**. Sua média é zero, o que significa que eles não têm o componente elétrico CC (corrente contínua). A falta desse componente é uma vantagem, pois alguns canais, como o cabo coaxial ou as linhas com transformadores, atenuam bastante um componente CC, em virtude de suas propriedades físicas. Além disso, o método de conexão do receptor ao canal de comunicação, chamado de **acoplamento capacitivo**, deixa passar apenas a parte CA (corrente alternada) de um sinal. De qualquer modo, se enviarmos um sinal cuja média não seja zero, desperdiçamos energia, pois o componente CC será eliminado.

O balanceamento ajuda a oferecer transições para a recuperação de clock, pois existe uma mistura de tensões positiva e negativa. Ele também oferece um modo simples de calibrar receptores, pois a média do sinal pode ser medida e usada como um patamar de decisão para decodificar símbolos. Com sinais não balanceados, a média pode flutuar do nível de decisão verdadeiro, em razão de uma alta taxa de "1s", por exemplo, que faria com que mais símbolos fossem decodificados com erros.

Um modo simples de construir um código balanceado é usar dois níveis de tensão para representar um "1" lógico e um 0 lógico. Por exemplo, +1 V para um bit 1 e –1 V para um bit 0. Para enviar 1, o transmissor alterna entre os níveis +1 V e –1 V, de modo que eles sempre resultem na média. Esse esquema é chamado de **codificação bipolar**. Nas redes de telefone, ele é chamado **AMI** (**Alternate Mark Inversion**), baseado em uma terminologia antiga, em que 1 é chamado "marca" e 0 é chamado "espaço". Um exemplo pode ser visto na Figura 2.14(e).

A codificação bipolar acrescenta um nível de tensão para alcançar o equilíbrio. Como alternativa, podemos usar um mapeamento como 4B/5B para conseguir o equilíbrio (bem como as transições para a recuperação de clock). Um exemplo desse tipo de código balanceado é o código de linha **8B/10B**. Ele mapeia 8 bits de entrada para 10 bits de saída, de modo que é 80% eficiente, assim como o código de linha 4B/5B. Os 8 bits são divididos em um grupo de 5 bits, que é mapeado para 6 bits, e um grupo de 3 bits, que é mapeado para 4 bits. Os símbolos de 6 e 4 bits são, então, concatenados. Em cada grupo, alguns padrões de entrada podem ser mapeados para padrões de saída balanceados, com o mesmo número de 0s e 1s. Por exemplo, "001" é mapeado para "1001", que é balanceado. Mas não existem combinações suficientes para todos os padrões de saída serem balanceados. Para esses casos, cada padrão de entrada é mapeado para dois padrões de saída. Um terá um 1 extra e a alternativa terá um 0 extra. Por exemplo, "000" é mapeado tanto para "1011" quanto para seu complemento "0100". À medida que os bits de entrada são mapeados para bits de saída, o codificador recorda a **disparidade** do símbolo anterior. A disparidade é o número total de 0s ou 1s pelos quais o sinal está fora de equilíbrio. O codificador, então, seleciona um padrão de saída ou seu padrão alternativo, para reduzir a disparidade. Com 8B/10B, a disparidade será de no máximo 2 bits. Assim, o sinal nunca estará longe de ser balanceado. Nunca haverá mais do que cinco 1s ou 0s consecutivos, para ajudar com a recuperação do clock.

Transmissão de banda passante

A comunicação por frequências de banda base é mais adequada para transmissões com fio, como par trançado, cabo coaxial ou fibra óptica. Em outras circunstâncias, particularmente aquelas envolvendo redes sem fio e transmissões de rádio, precisamos usar uma faixa de frequências que não começa com zero para enviar informações por um canal. Especificamente, para canais sem fio, não é prático enviar sinais de frequência muito baixa, pois o tamanho da antena precisa ser uma fração do comprimento de onda do sinal, que se torna grande em altas frequências de transmissão. De qualquer forma, restrições da regulamentação e a necessidade de evitar interferência normalmente ditam a escolha de frequências. Até mesmo para fios, colocar um sinal em determinada banda de frequência é útil para permitir que diferentes tipos de sinais coexistam no canal. Esse tipo de transmissão é chamado transmissão de banda

passante, pois uma banda de frequência arbitrária é usada para passar o sinal.

Felizmente, nossos resultados fundamentais, mostrados anteriormente, são todos em termos de largura de banda, ou da *largura* da banda de frequência. Os valores de frequência absoluta não importam para a capacidade. Isso significa que podemos apanhar um sinal de **banda base**, que ocupa de 0 a B Hz, e deslocá-lo para cima para ocupar uma banda passante de S a $S + B$ Hz sem mudar a quantidade de informação que ele pode transportar, embora o sinal tenha aparência diferente. Para processar um sinal no receptor, podemos deslocá-lo de volta para a banda base, onde é mais conveniente detectar símbolos.

A modulação digital é realizada com a transmissão da banda passante regulando ou modulando um sinal de portadora sobreposto à banda passante. Podemos modular a amplitude, a frequência ou a fase do sinal da portadora. Cada um desses métodos tem um nome correspondente. No **ASK** (**Amplitude Shift Keying**), duas amplitudes diferentes são usadas para representar 0 e 1. Um exemplo com um nível diferente de zero e zero aparece na Figura 2.16(b). Mais de dois níveis podem ser usados para codificar vários bits por símbolo.

De modo semelhante, com o chaveamento por deslocamento de frequência, ou **FSK** (**Frequency Shift Keying**), dois ou mais tons diferentes são usados. O exemplo na Figura 2.16(c) usa apenas duas frequências. Na forma mais simples do chaveamento por deslocamento de fase, ou **PSK** (**Phase Shift Keying**), a onda da portadora é sistematicamente deslocada em 0 ou 180 graus em cada período de símbolo. Como existem duas fases, ela é chamada de **chaveamento binário por deslocamento de fase**, ou **BPSK** (**Binary Phase Shift Keying**). O chaveamento "binário", nesse caso, refere-se aos dois símbolos, não significa que eles representem 2 bits. Um exemplo aparece na Figura 2.16(d). Um esquema melhor, que usa a largura de banda do canal de modo mais eficiente, é usar quatro deslocamentos, por exemplo, 45, 135, 225 ou 315 graus, para transmitir 2 bits de informação por símbolo. Essa versão é chamada **chaveamento por deslocamento de fase em quadratura**, ou **QPSK** (**Quadrature Phase Shift Keying**).

Podemos combinar esses esquemas e usar mais níveis para transmitir mais bits por símbolo. Somente a frequência ou a fase podem ser moduladas em determinado momento, pois elas estão relacionadas, normalmente sendo a taxa de mudança de fase com o tempo. Em geral, amplitude e fase são moduladas em combinação. Três exemplos aparecem na Figura 2.17 e, em cada um, os pontos dão as combinações válidas de amplitude e de fase de cada símbolo. Na Figura 2.17(a), vemos pontos equidistantes em 45, 135, 225 e 315 graus. A fase de um ponto é indicada pelo ângulo que uma linha a partir dele até a origem faz com o eixo x

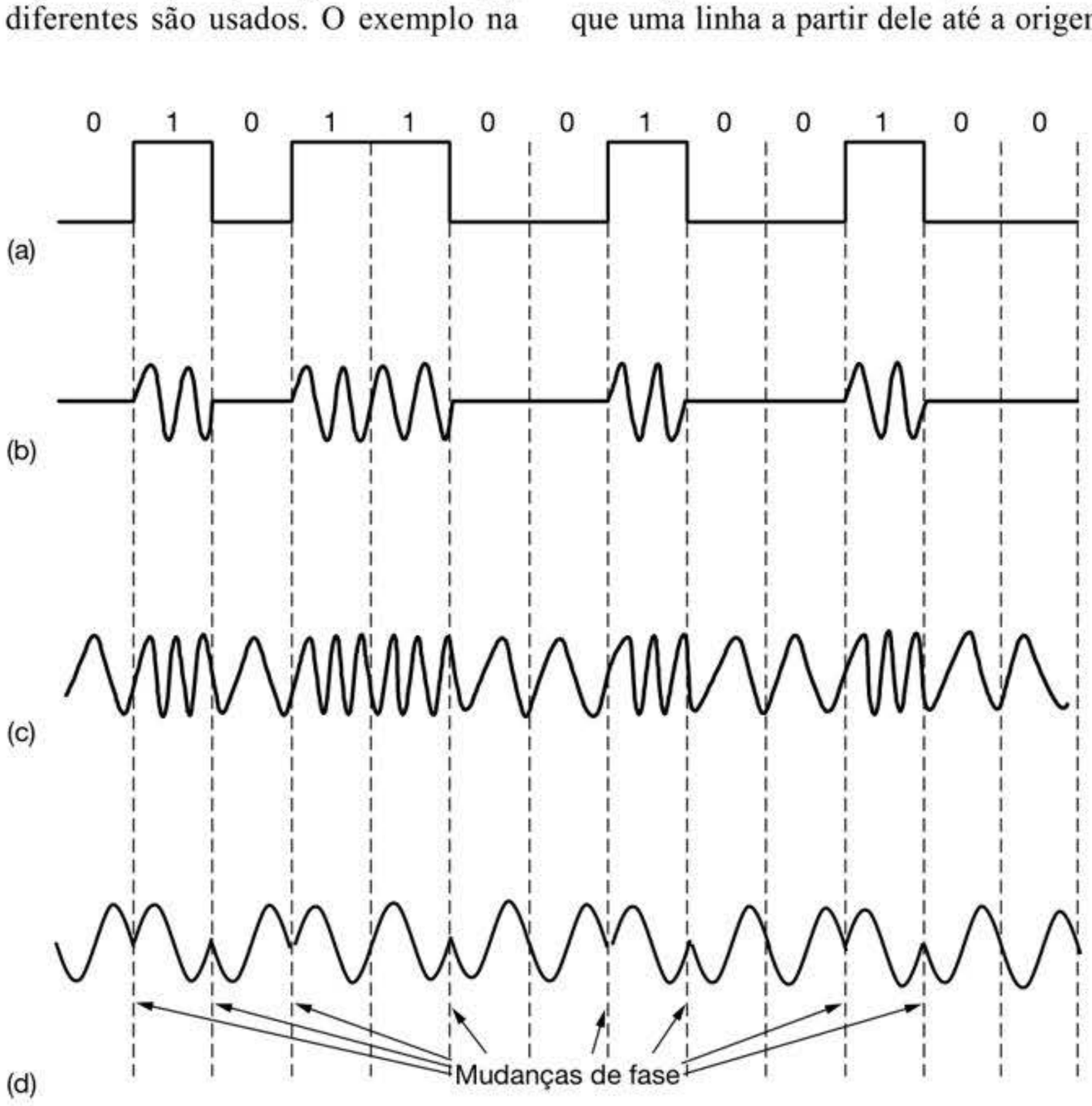


Figura 2.16 (a) Um sinal binário. (b) Chaveamento por deslocamento de amplitude. (c) Chaveamento por deslocamento de frequência. (d) Chaveamento por deslocamento de fase.

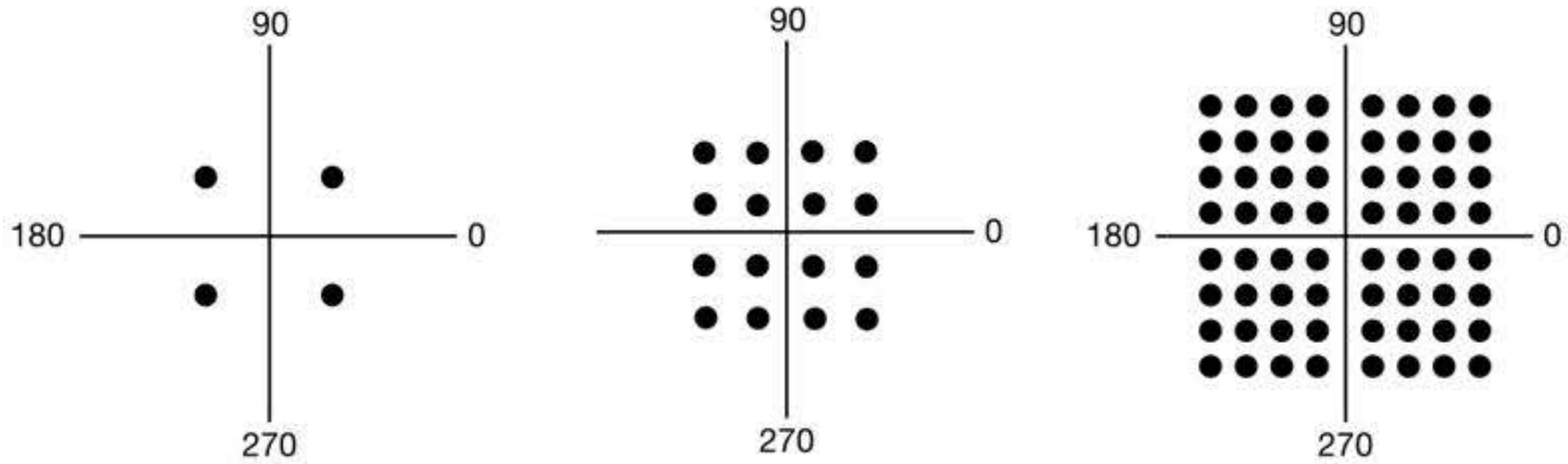


Figura 2.17 (a) QPSK. (b) QAM-16. (c) QAM-64.

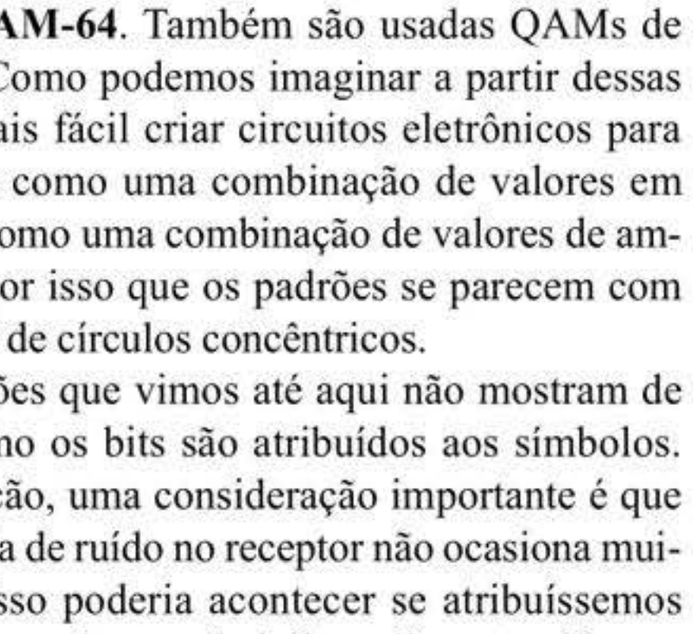
positivo. A amplitude de um ponto é a distância a partir da origem. Essa figura é uma representação do QPSK.

Esse tipo de diagrama é chamado de **diagrama de constelação**. Na Figura 2.17(b), vemos um esquema de modulação com um diagrama de constelação mais denso. Como são utilizadas 16 combinações de amplitudes e fase, o esquema de modulação pode ser usado para transmitir 4 bits por símbolo. Ele é chamado **QAM-16**, em que QAM significa **Quadrature Amplitude Modulation** (**modulação por amplitude de quadratura**). A Figura 2.17(c) é outro esquema de modulação com 64 combinações diferentes, de forma que podem ser transmitidos 6 bits por símbolo. Ele é chamado **QAM-64**. Também são usadas QAMs de ordem mais alta. Como podemos imaginar a partir dessas constelações, é mais fácil criar circuitos eletrônicos para produzir símbolos como uma combinação de valores em cada eixo do que como uma combinação de valores de amplitude e fase. É por isso que os padrões se parecem com quadrados, em vez de círculos concêntricos.

As constelações que vimos até aqui não mostram de forma alguma como os bits são atribuídos aos símbolos. Ao fazer a atribuição, uma consideração importante é que uma pequena rajada de ruído no receptor não ocasiona muitos erros de bit. Isso poderia acontecer se atribuíssemos valores de bit consecutivos a símbolos adjacentes. Com a QAM-16, por exemplo, se um símbolo designasse 0111 e o símbolo vizinho designasse 1000, se o receptor por engano apanhasse o símbolo adjacente, ele faria com que todos os bits ficassem errados. Uma solução melhor é mapear entre bits e símbolos, de modo que os símbolos adjacentes difiram em apenas 1 posição de bit. Esse mapeamento é chamado de **código Gray**. A Figura 2.18 mostra um diagrama de constelação QAM-16 que foi codificada pelo código Gray. Agora, se o receptor decodificar o símbolo com erro, ele cometerá apenas um erro de único bit no caso esperado em que o símbolo codificado está próximo do símbolo transmitido.

2.4.4 Multiplexação

Os esquemas de modulação que vimos nos permitem enviar um sinal para transmitir bits por um enlace com ou sem fio, mas eles só descrevem como transmitir um fluxo de bits por vez. Na prática, as economias de escala desempenham um papel importante no modo como usamos as redes. Custa basicamente o mesmo valor instalar e manter uma linha de transmissão com uma largura de banda alta e uma linha com largura de banda baixa entre dois escritórios diferentes (em outras palavras, os custos advêm de ter de cavar a



Quando 1101 é enviado:

Ponto	Decodifica como	Erros de bit
A	1101	0
B	1100	1
C	1001	1
D	1111	1
E	0101	1

Figura 2.18 QAM-16 em código Gray.

trincheira, não do tipo de cabo ou fibra que se utiliza). Consequentemente, esquemas de multiplexação têm sido desenvolvidos para compartilhar as linhas entre muitos sinais. As três principais formas de multiplexar uma única linha física são tempo, frequência e código; há também uma técnica chamada multiplexação por divisão de comprimento de onda, que é basicamente uma forma óptica de multiplexação por divisão de frequência. Discutiremos cada uma dessas técnicas a seguir.

Multiplexação por divisão de frequência

A **multiplexação por divisão de frequência**, ou **FDM** (**Frequency Division Multiplexing**), tira proveito da transmissão de banda passante para compartilhar um canal. Ela divide o espectro em bandas de frequência, com cada usuário tendo posse exclusiva de alguma banda para enviar seu sinal. A transmissão de rádio AM ilustra a FDM. O espectro alocado é de cerca de 1 MHz, aproximadamente 500 a 1500 kHz. Diferentes frequências são alocadas a diferentes canais lógicos (estações), cada um operando em uma parte do espectro, com a separação entre canais grande o bastante para impedir interferência.

Para ver um exemplo mais detalhado, a Figura 2.19 mostra como três canais telefônicos de nível de voz são multiplexados com o uso da FDM. Os filtros limitam a largura de banda utilizável a cerca de 3.100 Hz por canal de qualidade de voz. Quando muitos canais são multiplexados ao mesmo tempo, são alocados 4.000 Hz para cada um. A largura de banda em excesso é chamada **banda de proteção**, e mantém os canais bem separados. Primeiro, os canais de voz têm sua frequência aumentada, cada qual com um valor diferente. Depois eles podem ser combinados, pois agora não há dois canais ocupando a mesma porção do espectro. Observe que, apesar de haver intervalos entre os canais graças às bandas de proteção, há certa sobreposição entre canais adjacentes, porque os filtros reais não têm limites nítidos ideais. Essa sobreposição significa que um forte pico na borda de um canal será sentido no canal adjacente como ruído não térmico.

Esse esquema tem sido usado para multiplexar chamadas no sistema telefônico há muitos anos, mas o esquema de multiplexação no tempo agora é preferido em lugar dele. Contudo, a FDM continua a ser usada nas redes telefônicas, assim como na telefonia celular, redes sem fio terrestres e redes por satélite em um nível de detalhamento mais alto.

Ao enviar dados digitais, é possível dividir o espectro de modo eficiente sem usar bandas de proteção. Na multiplexação ortogonal por divisão de frequência, ou **OFDM** (**Orthogonal Frequency Division Multiplexing**), a largura de banda do canal é dividida em muitas subportadoras que enviam dados de maneira independente (p. ex., com QAM). As subportadoras compactam bastante o domínio de frequência, assim, os sinais de cada subportadora se estendem para as adjacentes. Contudo, como vemos na Figura 2.20, a resposta em frequência de cada subportadora é projetada de modo que seja zero no centro das subportadoras adjacentes. As subportadoras podem, portanto, ser amostradas em suas frequências centrais sem interferência de seus vizinhos. Para que isso funcione, um **tempo de proteção** é necessário para repetir uma parte dos sinais de símbolo no tempo, de modo que tenham a resposta de frequência desejada. Porém, esse overhead é muito menor que o necessário para muitas bandas de proteção.

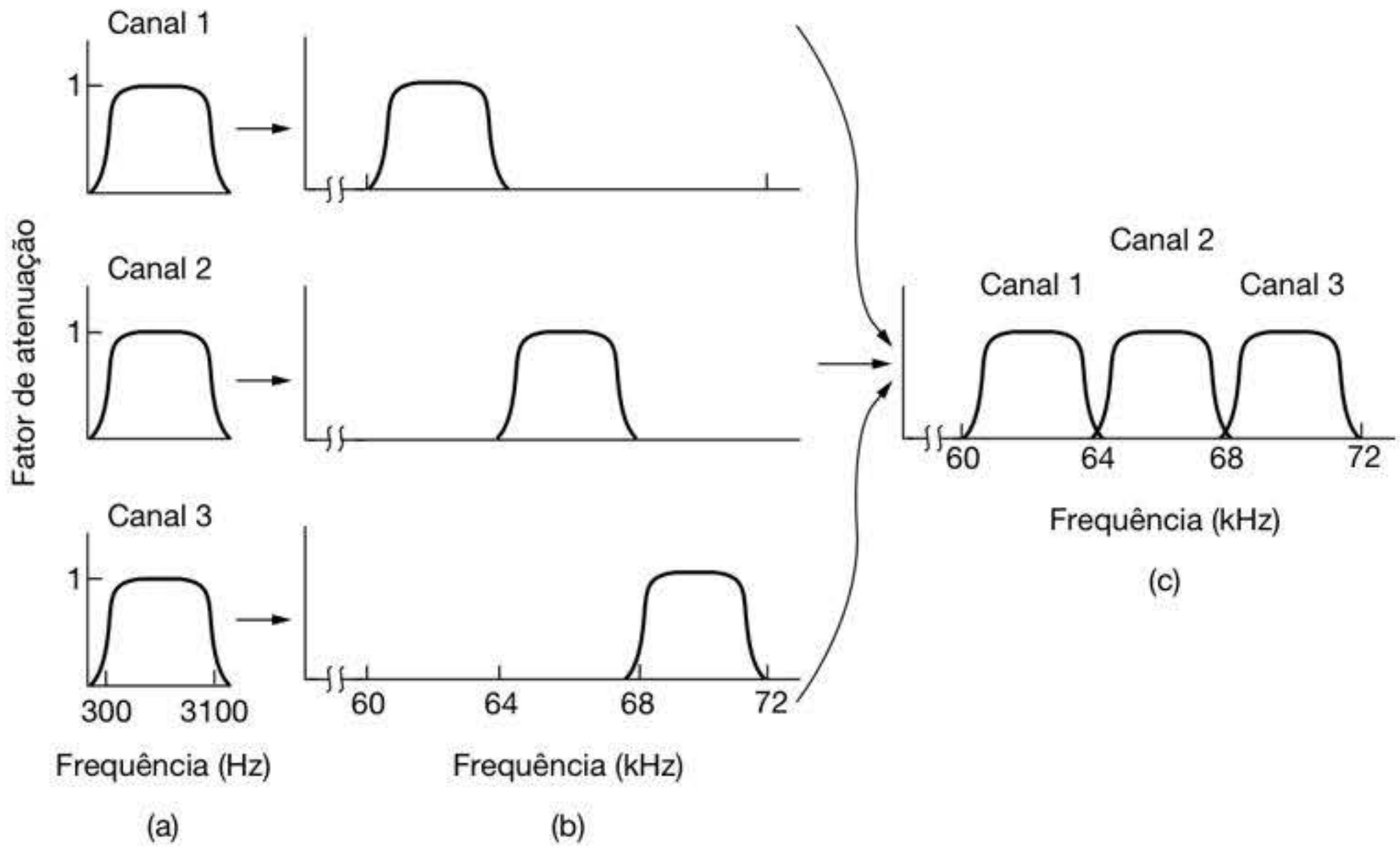


Figura 2.19 Multiplexação por divisão de frequência. (a) As larguras de banda originais. (b) As larguras de banda aumentaram em frequência. (c) O canal multiplexado.

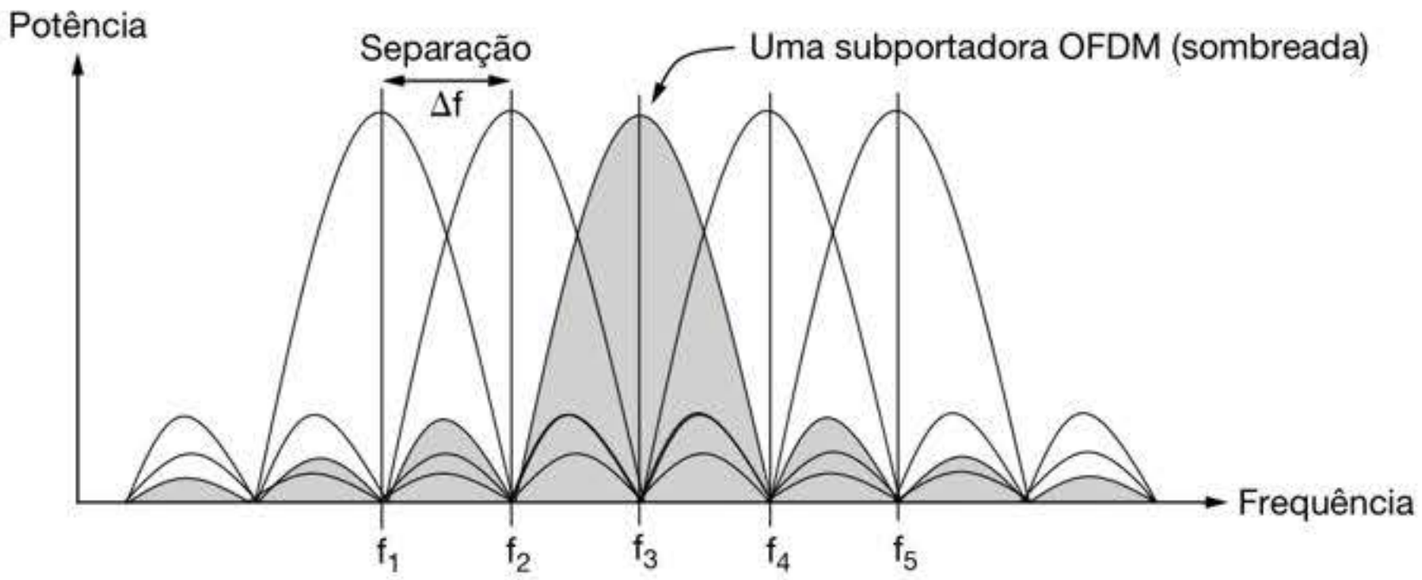


Figura 2.20 Multiplexação ortogonal por divisão de frequência (OFDM).

A ideia da OFDM já existe há muito tempo, mas somente no início da década de 2000 ela foi amplamente adotada, seguindo a observação de que é possível implementá-la de modo eficiente em termos de uma transformada de Fourier dos dados digitais por todas as subportadoras (em vez de modular separadamente cada subportadora). A OFDM é usada em 802.11, redes a cabo, redes por linhas de energia elétrica e sistemas celulares de quarta geração (4G). Normalmente, o fluxo de informações digitais de alta velocidade é dividido em muitos fluxos de baixa velocidade, os quais são transmitidos nas subportadoras em paralelo. Essa divisão é valiosa porque é mais fácil lidar com as degradações do canal no nível da subportadora; algumas subportadoras podem ser muito degradadas e excluídas em favor de subportadoras que são bem recebidas.

Multiplexação por divisão de tempo

Uma alternativa à FDM é a **multiplexação por divisão de tempo**, ou **TDM (Time Division Multiplexing)**. Aqui, os usuários se alternam (em um padrão de rodízio), cada um periodicamente usando a largura de banda inteira por um pequeno período. Um exemplo de três fluxos sendo multiplexados com TDM aparece na Figura 2.21. Os bits de cada fluxo de entrada são apanhados em um **slot de tempo** fixo e enviados para o fluxo agregado. Esse fluxo trabalha em uma velocidade que é a soma dos fluxos individuais. Para que isso funcione, os fluxos precisam estar sincronizados no tempo. Pequenos intervalos de tempo de proteção (semelhantes à banda de proteção de frequência) podem ser acrescentados para acomodar pequenas variações de sincronização.

A TDM é bastante usada como parte das redes de telefone e celular. Para evitar um ponto de confusão, vamos esclarecer que isso é muito diferente da **multiplexação estatística por divisão de tempo**, ou **STDM (Statistical Time Division Multiplexing)**. O termo "estatística" é acrescentado para indicar que os fluxos individuais contribuem para o fluxo multiplexado, e *não* sobre uma programação fixa, mas de acordo com a estatística de sua demanda. A STDM é a comutação de pacotes com outro nome.

Multiplexação por divisão de código

Existe um terceiro tipo de multiplexação, que funciona de modo completamente diferente da FDM e da TDM. A **multiplexação por divisão de código**, ou **CDM (Code Division Multiplexing)** é uma forma de comunicação por **dispersão espectral**, na qual um sinal de banda estreita é espalhado por uma banda de frequência mais larga. Isso pode torná-la ainda mais tolerante às interferências, além de permitir que vários sinais de diferentes usuários compartilhem a mesma banda de frequência. Como a CDM é usada principalmente para essa última finalidade, ela normalmente é chamada de **acesso múltiplo por divisão de código**, ou **CDMA (Code Division Multiple Access)**.

O CDMA permite que cada estação transmita por todo o espectro de frequência o tempo todo. Várias transmissões simultâneas são separadas usando a teoria da codificação. Antes de entrarmos no algoritmo, vamos considerar uma analogia: o saguão de um aeroporto com muitos pares de pessoas conversando. Com a TDM, todas as pessoas estariam no meio do saguão, mas conversariam por turnos, um par de pessoas de cada vez. Com a FDM, as pessoas formariam grupos bem separados, cada

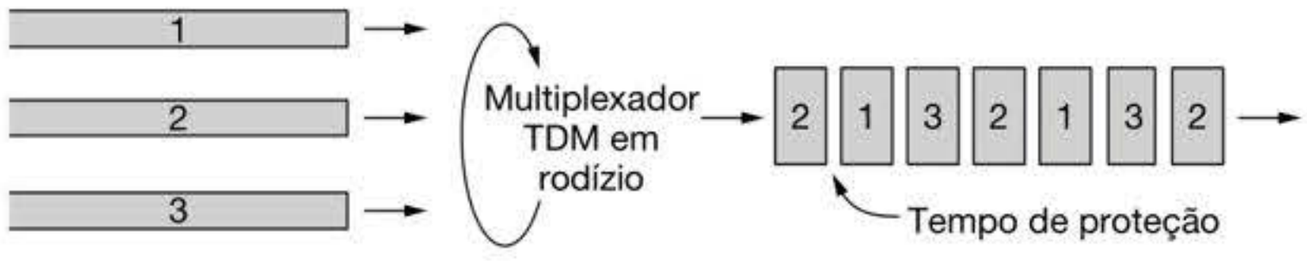


Figura 2.21 Multiplexação por divisão de tempo (TDM).

um mantendo sua própria conversação ao mesmo tempo, alguns com uma altura maior e outros com uma altura menor, de modo que cada par pudesse manter sua própria conversa ao mesmo tempo, mas ainda independentemente dos outros grupos. Com o CDMA, todas as pessoas estariam no meio do saguão falando ao mesmo tempo, mas cada par de pessoas conversando em um idioma diferente. O par que estivesse falando em francês só reconheceria esse idioma, rejeitando tudo que não fosse francês como ruído. Desse modo, a chave do CDMA é a capacidade de extrair o sinal desejado e rejeitar todos os outros como ruído aleatório. A seguir, veremos uma descrição um pouco simplificada do sistema.

No CDMA, cada tempo de duração de um bit é subdividido em m intervalos curtos, denominados **chips**, que são multiplicados pela sequência de dados original (os chips são uma sequência de bits, mas são denominados chips para que não sejam confundidos com os bits da própria mensagem). Normalmente, existem 64 ou 128 chips por bit, mas, no exemplo apresentado a seguir, usaremos 8 chips/bit, para simplificar. A cada estação é atribuído um código de m bits exclusivo, chamado **sequência de chips**. Para fins pedagógicos, é mais conveniente escrever esses códigos como sequências de –1 e +1. Mostraremos as sequências de chips entre parênteses.

Para transmitir um bit 1, uma estação envia sua sequência de chips. Para transmitir um bit 0, envia a negação de sua sequência de chips. Não são permitidos quaisquer outros padrões. Assim, para $m = 8$, se a estação A receber a atribuição da sequência de chips (−1 −1 −1 +1 +1 −1 +1 +1), ela transmitirá um bit 1 ao enviar a sequência de chips, e 0 transmitindo seu complemento: (+1 +1 +1 −1 −1 +1 −1 −1). Na realidade, são enviados sinais com esses níveis de tensão, mas isso é suficiente para pensarmos em termos das sequências.

O aumento do volume de informações a serem enviadas de b bits/s para mb chips/s só poderá ocorrer se a largura de banda disponível for m vezes maior que a largura de banda necessária para uma estação que não usa CDMA (se não houver mudanças nas técnicas de modulação ou codificação). Se tivéssemos uma banda de 1 MHz disponível para 100 estações, com FDM, cada uma teria 10 kHz e poderia transmitir a uma velocidade de 10 kbps (supondo-se 1 bit por Hz). No CDMA, cada estação utiliza 1 MHz inteiro e, portanto, a taxa de chips é de 100 chips por bit, para espalhar a taxa de bits da estação de 10 kbps pelo canal.

Na Figura 2.22(a) e (b), mostramos as sequências de chips binárias atribuídas a quatro exemplos de estações e os sinais que elas representam. Cada estação tem sua própria sequência exclusiva de chips. Vamos usar o símbolo **S** para indicar o vetor de m chips correspondente à estação S, e $\overline{\mathbf{S}}$ para sua negação. Todas as sequências de chips são **ortogonais** par a par – isso significa que o produto interno normalizado de duas sequências de chips distintas, **S** e **T** (indicado como **S•T**), é 0. Sabemos como gerar tal sequência ortogonal de chips usando um método conhecido como **códigos de Walsh**. Em termos matemáticos, a ortogonalidade das sequências de chips pode ser expressa por:

$$\mathbf{S \cdot T} \equiv \frac{1}{m} \sum_{i=1}^{m} S_i T_i = 0 \qquad (2.5)$$

Em linguagem comum, o número de pares iguais é igual ao número de pares diferentes. Essa propriedade da ortogonalidade será essencial mais adiante. Observe que, se **S•T** = 0, então **S•**$\overline{\mathbf{T}}$ também será 0. O produto interno normalizado de qualquer sequência de chips por ela mesma é igual a 1:

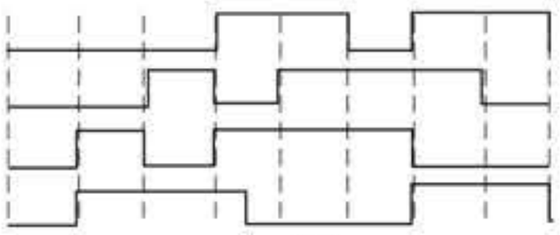
$$\mathbf{S \cdot S} = \frac{1}{m} \sum_{i=1}^{m} S_i S_i = \frac{1}{m} \sum_{i=1}^{m} S_i^2 = \frac{1}{m} \sum_{i=1}^{m} (\pm 1)^2 = 1$$

Isso ocorre porque cada um dos m termos do produto interno é 1 e, portanto, a soma é m. Observe também que **S•**$\overline{\mathbf{S}}$ = –1.

Durante cada intervalo com duração de um bit, uma estação pode transmitir um bit 1 (enviando sua sequência de chips), pode transmitir um bit 0 (enviando a negativa

A = (–1 –1 –1 +1 +1 –1 +1 +1)
B = (–1 –1 +1 –1 +1 +1 +1 –1)
C = (–1 +1 –1 +1 +1 +1 –1 –1)
D = (–1 +1 –1 –1 –1 –1 +1 –1)

(a)



(b)

$S_1 = C = (-1\ +1\ -1\ +1\ +1\ +1\ -1\ -1)$
$S_2 = B+C = (-2\ \ 0\ \ 0\ \ 0\ +2\ +2\ \ 0\ -2)$
$S_3 = A+\overline{B} = (\ 0\ \ 0\ -2\ +2\ \ 0\ -2\ \ 0\ +2)$
$S_4 = A+\overline{B}+C = (-1\ +1\ -3\ +3\ +1\ -1\ -1\ +1)$
$S_5 = A+B+C+D = (-4\ \ 0\ -2\ \ 0\ +2\ \ 0\ +2\ -2)$
$S_6 = A+B+\overline{C}+D = (-2\ -2\ \ 0\ -2\ \ 0\ -2\ +4\ \ 0)$

(c)

$S_1 \cdot C = [1+1+1+1+1+1+1+1]/8 = 1$
$S_2 \cdot C = [2+0+0+0+2+2+0+2]/8 = 1$
$S_3 \cdot C = [0+0+2+2+0-2+0-2]/8 = 0$
$S_4 \cdot C = [1+1+3+3+1-1+1-1]/8 = 1$
$S_5 \cdot C = [4+0+2+0+2+0-2+2]/8 = 1$
$S_6 \cdot C = [2-2+0-2+0-2-4+0]/8 = -1$

(d)

Figura 2.22 (a) Sequências de chips binárias para quatro estações. (b) Sinais que as sequências representam. (c) Seis exemplos de transmissões. (d) Recuperação do sinal da estação *C*.

de sua sequência de chips), ou pode ficar inativa e não realizar nenhuma transmissão. Por enquanto, supomos que todas as estações estão sincronizadas e que a transmissão de todas as sequências de chips começa no mesmo instante. Quando duas ou mais estações transmitem simultaneamente, suas sequências bipolares somam-se linearmente. Por exemplo, se, durante um período de um chip, três estações transmitirem +1 e uma estação transmitir –1, o resultado +2 será recebido. Isso pode ser considerado como a soma de tensões sobrepostas no canal: se três estações transmitirem + 1 volt e uma estação transmitir –1 volt como saída, o resultado será 2 volts. A Figura 2.22(c) apresenta seis exemplos em que uma ou mais estações transmitem um bit 1 ao mesmo tempo. No primeiro exemplo, *C* transmite um bit 1 e, assim, simplesmente obtemos a sequência de chips de *C*. No segundo, *B* e *C* transmitem bits 1 e obtemos a soma de suas sequências bipolares de chips da seguinte forma:

$$(-1\ -1\ +1\ -1\ +1\ +1\ +1\ -1) + (-1\ +1\ -1\ +1\ +1\ +1\ -1\ -1) = (-2\ 0\ 0\ 0\ +2\ +2\ 0\ -2)$$

Para recuperar o fluxo de bits de uma estação individual, o receptor precisa conhecer com antecedência a sequência de chips da estação transmissora. Ele executa a recuperação calculando o produto interno normalizado da sequência de chips recebida e a sequência de chips da estação cujo fluxo de bits está tentando recuperar. Se a sequência de chips recebida for **S** e o receptor estiver tentando ouvir uma estação cuja sequência de chips é **C**, ele apenas calcula o produto interno normalizado, **S•C**.

Para entender por que esse procedimento funciona, imagine que as duas estações, *A* e *C*, transmitem um bit 1 ao mesmo tempo que *B* transmite um bit 0, como no caso do terceiro exemplo. O receptor percebe a soma, $\mathbf{S} = \mathbf{A} + \overline{\mathbf{B}} + \mathbf{C}$, e calcula:

$$\mathbf{S}\bullet\mathbf{C} = (\mathbf{A} + \overline{\mathbf{B}} + \mathbf{C})\bullet\mathbf{C} = \mathbf{A}\bullet\mathbf{C} + \overline{\mathbf{B}}\bullet\mathbf{C} + \mathbf{C}\bullet\mathbf{C} = 0 + 0 + 1 = 1$$

Os dois primeiros termos desaparecem porque todos os pares de sequências de chips foram cuidadosamente escolhidos para serem ortogonais, como mostra a Equação 2.5. Agora já deve estar claro por que essa propriedade precisa ser imposta às sequências de chips.

Para tornar mais concreto o processo de decodificação, vamos considerar novamente os seis exemplos da Figura 2.22(d). Suponha que o receptor esteja interessado em extrair o bit enviado pela estação *C* de cada uma das seis somas de S_1 a S_6. Ele calcula o bit somando aos pares os produtos da **S** recebida com o vetor **C** da Figura 2.22(a), extraindo depois 1/8 do resultado (pois, neste caso $m = 8$). Os exemplos incluem casos em que *C* é silencioso, envia um bit 1 e um bit 0, individualmente e em combinação com outras transmissões. Como vemos, o bit correto é decodificado a cada vez. É como falar francês.

Em princípio, dada uma capacidade de computação suficiente, o receptor poderá escutar todos os transmissores ao mesmo tempo, executando o algoritmo de decodificação correspondente a cada um deles em paralelo. Na prática, é mais fácil falar do que fazer, e é útil saber quais transmissores poderiam estar transmitindo.

O ideal é que, no sistema CDMA sem ruído que estudamos aqui, o número de estações que enviam simultaneamente possa ser arbitrariamente grande, usando sequências de chips maiores. Para 2^n estações, os códigos de Walsh podem oferecer 2^n sequências de chips ortogonais de tamanho 2^n. Contudo, uma limitação significativa é o fato de considerarmos que todos os chips são sincronizados no tempo no receptor. Essa sincronização nem mesmo é aproximadamente verdadeira em algumas aplicações, como em redes de celular (nas quais o CDMA foi bastante utilizado a partir da década de 1990). Isso leva a diferentes projetos.

Assim como as redes de celular, o CDMA é usado por satélites e redes a cabo. Nesta rápida introdução, passamos brevemente pelos muitos fatores complicadores. Os engenheiros que desejam obter um conhecimento profundo do CDMA deverão ler Viterbi (1995) e Harte et al. (2012). Entretanto, essas referências exigem muita base em engenharia de comunicação.

Multiplexação por divisão de comprimento de onda

A **multiplexação por divisão de comprimento de onda**, ou **WDM (Wavelength Division Multiplexing)** é uma forma de FDM que multiplexa diversos sinais em uma fibra óptica usando diferentes comprimentos de onda da luz. Na Figura 2.23, quatro fibras chegam juntas a um colimador óptico, cada uma com sua energia presente em um comprimento de onda distinto. Os quatro feixes são combinados em uma única fibra compartilhada para transmissão a um destino remoto. Na extremidade remota, o feixe é dividido no mesmo número de fibras que havia no lado da entrada. Cada fibra de saída contém um núcleo curto, especialmente construído, que filtra todos os comprimentos de onda, com exceção de um. Os sinais resultantes podem ser roteados até seu destino ou recombinados de diferentes maneiras para transporte ou multiplexação adicional.

Realmente não há nada de novo aqui. Trata-se apenas da FDM em frequências muito altas, com o termo WDM devendo-se à descrição dos canais de fibra óptica em função de seu comprimento de onda ou "cor" em vez da frequência propriamente dita. Desde que cada canal tenha sua própria frequência dedicada (i.e., de comprimentos de onda) e todas as faixas sejam disjuntas, elas poderão ser multiplexadas na fibra de longa distância. A única diferença em relação à FDM elétrica é que um sistema óptico que utilize uma rede de difração será completamente passivo e, portanto, altamente confiável.

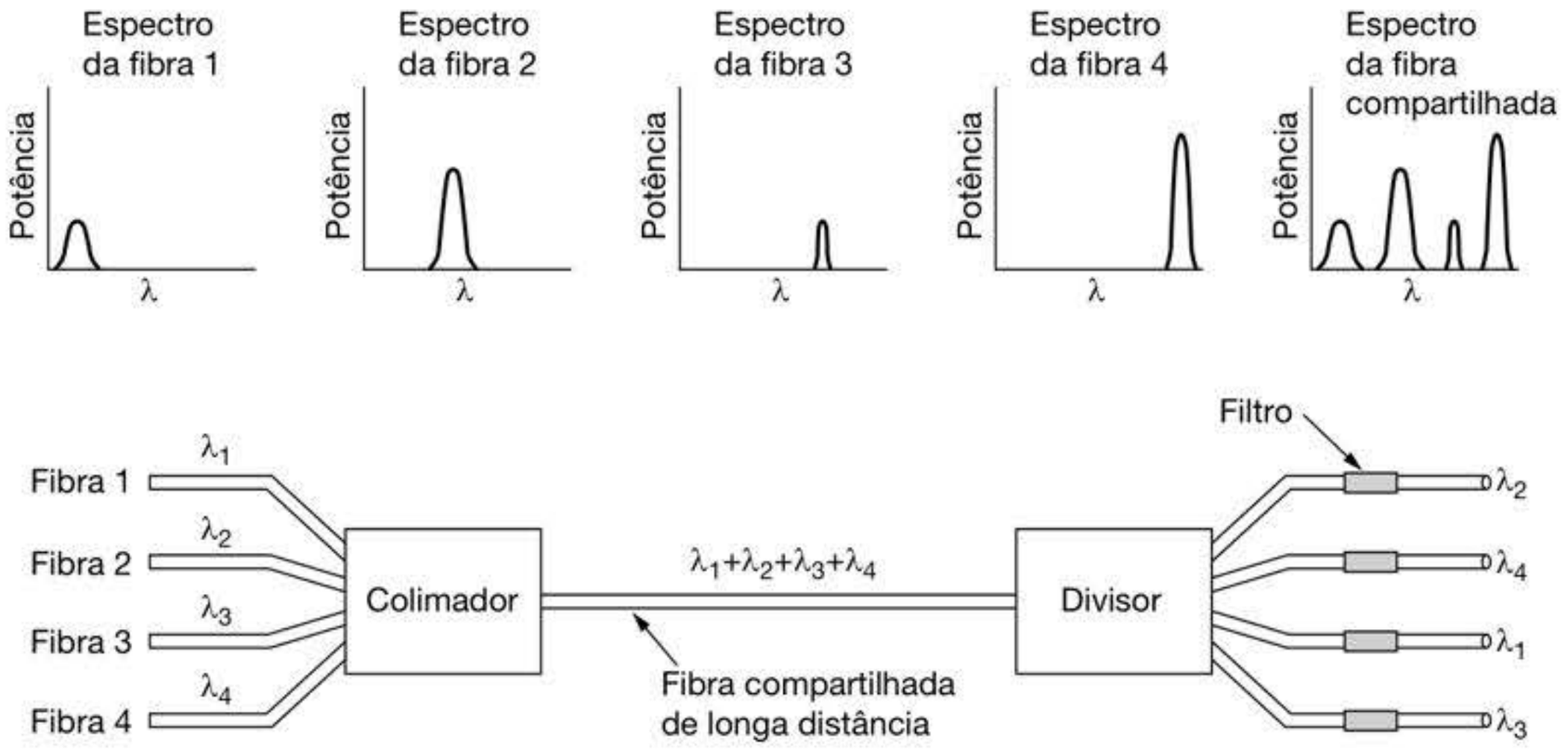


Figura 2.23 Multiplexação por divisão de comprimento de onda.

A razão para a WDM ser popular é o fato de a energia em um único canal normalmente ter apenas alguns gigahertz de largura porque, no momento, é impossível realizar a conversão entre sinais elétricos e ópticos com rapidez maior que essa. Utilizando-se muitos canais em paralelo com diferentes comprimentos de onda, a largura de banda agregada aumenta de forma linear com o número de canais. Como a largura de banda de uma única banda de fibra é de aproximadamente 25.000 GHz (veja a Figura 2.5), teoricamente existe espaço para 2.500 canais de 10 Gbps, mesmo a 1 bit/Hz (e também são possíveis taxas mais altas).

A tecnologia WDM tem progredido a uma taxa que faria a tecnologia computacional se envergonhar. O sistema WDM foi criado por volta de 1990, com os primeiros sistemas comerciais tendo 8 canais de 2,5 Gbps por canal. Em 1998, havia sistemas com 40 canais de 2,5 Gbps no mercado. Em 2006, já eram utilizados produtos com 192 canais de 10 Gbps e 64 canais de 40 Gbps, capazes de mover até 2,56 Tbps. Em 2019, havia sistemas que podiam lidar com 160 canais, aceitando mais de 16 Tbps por um único par de fibra. Isso é 800 vezes mais capacidade que os sistemas de 1990. Os canais também estão bastante compactados na fibra, com 200, 100 ou mesmo 50 GHz de separação.

Estreitando o espaçamento para 12,5 GHz, é possível usar 320 canais em uma única fibra, aumentando ainda mais a capacidade de transmissão. Tais sistemas, com um grande número de canais pouco espaçados, são conhecidos como **DWDM** (**Dense WDM**). Sistemas DWDM costumam ser mais caros, pois precisam manter comprimentos de onda e frequências estáveis, devido ao pouco espaçamento de cada canal. Como resultado, esses sistemas regulam de perto sua temperatura, a fim de garantir que as frequências sejam precisas.

Um dos impulsionadores da tecnologia WDM é o desenvolvimento de amplificadores totalmente ópticos. Antes, a cada 100 km era necessário dividir todos os canais e converter cada um deles a um sinal elétrico para amplificação separada, antes de convertê-los novamente em sinais ópticos e combiná-los. Hoje, os amplificadores totalmente ópticos podem regenerar o sinal inteiro uma única vez a cada 1.000 km, sem a necessidade de várias conversões ópticas/elétricas.

No exemplo da Figura 2.23, temos um sistema de comprimento de onda fixo. Bits da fibra de entrada 1 vão para a fibra de saída 3, bits da fibra de entrada 2 vão para a fibra de saída 1, etc. Porém, também é possível criar sistemas WDM comutados no domínio óptico. Em dispositivos como esses, os filtros de saída são ajustáveis com o uso de interferômetros de Fabry-Perot ou de Mach-Zehnder. Esses dispositivos permitem que as frequências selecionadas sejam trocadas dinamicamente por um computador de controle. Essa capacidade oferece uma grande flexibilidade para a provisão de muitos caminhos de comprimento de onda diferentes através da rede telefônica a partir de um conjunto fixo de fibras. Para obter mais informações sobre redes ópticas e WDM, consulte Grobe e Eiselt (2013).

2.5 A REDE PÚBLICA DE TELEFONIA COMUTADA

Quando dois computadores, instalados perto um do outro, precisam se comunicar, geralmente é mais fácil conectá-los por meio de um cabo. As LANs funcionam dessa forma. No entanto, quando as distâncias começam a ficar grandes ou muitos computadores e cabos têm de atravessar uma estrada ou outra passagem pública, os custos de instalação de cabos privados costumam ser proibitivos. Além disso, em quase todos os países do mundo, também é ilegal estender linhas de transmissão privadas em (ou sob) propriedades públicas. Consequentemente, os projetistas de rede devem

confiar nos recursos de telecomunicações existentes, como a rede de telefonia, a rede celular ou a rede de televisão a cabo.

O fator limitante para a rede de dados tem sido a "última milha" sobre a qual os clientes se conectam, que pode depender de qualquer uma dessas tecnologias físicas, ao contrário da chamada infraestrutura de "backbone" para o restante da rede de acesso. Na última década, essa situação mudou drasticamente, com velocidades de 1 Gbps para casa se tornando cada vez mais comuns. Embora uma contribuição para velocidades de último quilômetro mais rápidas seja a implantação contínua de fibra na extremidade da rede, talvez um fator ainda mais significativo em alguns países seja a engenharia sofisticada das redes de telefone e cabo *existentes* para espremer cada vez mais largura de banda da infraestrutura. Acontece que trabalhar com a infraestrutura física existente para aumentar as velocidades de transmissão é muito mais barato do que colocar novos cabos (de fibra) no solo. Agora, vamos explorar as arquiteturas e características de cada uma dessas infraestruturas físicas de comunicação.

Esses recursos existentes, em particular a **PSTN** (**Public Switched Telephone Network**), foram projetados há muitos anos, tendo em vista um objetivo completamente diferente: a transmissão da voz humana de uma forma mais ou menos reconhecível. Um cabo instalado entre dois computadores pode transferir dados a 10 Gbps ou mais; a rede telefônica, portanto, tem seu trabalho reduzido em termos de transmissão de bits em altas velocidades. As primeiras tecnologias DSL (Digital Subscriber Line) só podiam transmitir dados em velocidades de alguns Mbps; agora, as versões mais modernas da DSL podem alcançar velocidades que se aproximam de 1 Gbps. Nas próximas seções, descreveremos o sistema telefônico e mostraremos como ele funciona. Para obter informações adicionais sobre o funcionamento interno do sistema telefônico, consulte Laino (2017).

2.5.1 Estrutura do sistema telefônico

Logo depois que Alexander Graham Bell patenteou a invenção do telefone, em 1876 (apenas algumas horas antes de seu concorrente, Elisha Gray), houve uma grande demanda por essa nova invenção. Inicialmente, o mercado estava voltado para a venda de telefones, que eram comercializados aos pares. Era o usuário quem tinha de conectar os dois aparelhos usando um fio. Se o proprietário de um telefone quisesse usar o aparelho para conversar com n outros proprietários de telefone, tinha de conectar fios em todas as n residências. Em um ano, as cidades ficaram tomadas por fios que passavam pelas casas e pelas árvores, em uma selva de fios. Logo ficou óbvio que o modelo de conexão de um telefone a outro, como é mostrado na Figura 2.24(a), não funcionaria.

Bell percebeu essa situação e criou a Bell Telephone Company, que abriu sua primeira estação de comutação (em New Haven, Connecticut) em 1878. A empresa ligava um fio até a casa ou o escritório de cada usuário. Para fazer uma chamada, o usuário girava a manivela, o que emitia um som na companhia telefônica e chamava a atenção de um operador. Este, por sua vez, conectava manualmente o emissor da chamada ao receptor usando um cabo curto. Observe, na Figura 2.24(b), o modelo de uma única estação de comutação.

Não demorou muito tempo para as estações de comutação da Bell System se espalharem por todos os locais, e logo as pessoas passaram a querer fazer chamadas interurbanas. Por isso, a Bell System passou a conectar uma estação de comutação a outra. Contudo, o problema original veio à tona mais uma vez: conectar cada estação de comutação a outra através de um fio logo se tornou inviável. Então, foram inventadas as estações de comutação de segundo nível. Depois de algum tempo, tornaram-se necessárias várias estações de segundo nível, como mostra a Figura 2.24(c). Mais tarde, a hierarquia cresceu até alcançar cinco níveis.

Em 1890, era possível notar a presença das três principais partes do sistema telefônico: as estações de comutação, os fios que ligavam os usuários a essas estações (agora já operando com cabos de pares trançados, isolados e balanceados em vez de cabos abertos com retorno por terra) e as conexões de longa distância existentes entre

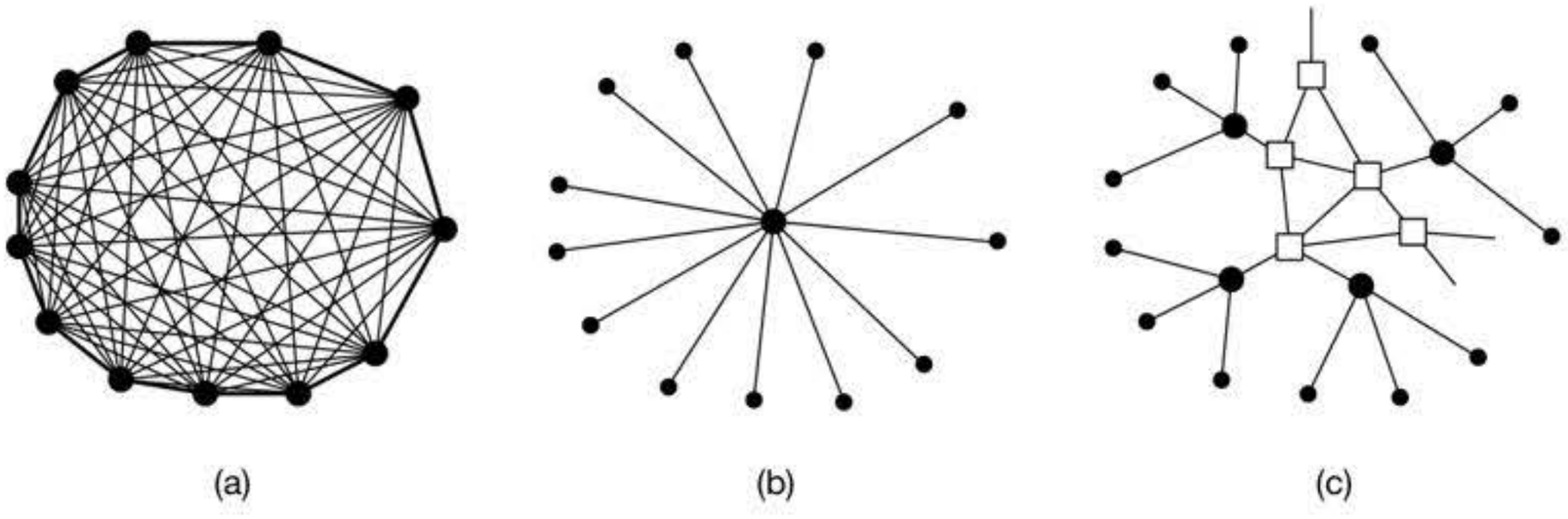


Figura 2.24 (a) Rede totalmente interconectada. (b) Switch centralizado. (c) Hierarquia de dois níveis.

as estações de comutação. Para obter breves informações técnicas sobre o sistema telefônico e sua história, consulte Hawley (1991).

Embora tenha havido melhorias em todas as três áreas desde então, o modelo básico da Bell System continuou praticamente intacto por mais de 100 anos. Embora seja bastante simplificada, a descrição a seguir apresenta a ideia básica do sistema telefônico. Cada telefone contém dois fios de cobre que saem do aparelho e se conectam diretamente à **estação final** mais próxima da companhia telefônica (também denominada **estação central local**). Em geral, a distância varia de 1 a 10 km, sendo menor nas cidades que nas regiões rurais. Só nos Estados Unidos, existem cerca de 22 mil estações finais. As conexões através de dois fios entre o telefone de cada assinante e a estação final são conhecidas no mercado como **circuito terminal**. Se todos os circuitos terminais existentes no mundo fossem esticados de uma extremidade a outra, seu comprimento equivaleria a mil vezes a distância da Terra à Lua e de volta à Terra.

Houve uma época em que 80% do capital da AT&T estava no cobre dos circuitos terminais, o que a tornava a maior mina de cobre do mundo. Felizmente, essa informação não foi muito difundida na comunidade financeira. Se tivessem conhecimento desse fato, alguns empresários poderiam comprar a AT&T, encerrar todos os serviços de telefonia dos Estados Unidos, descascar toda a fiação e vender os fios a uma refinaria de cobre para ter um retorno rápido do capital.

Se um assinante conectado a uma determinada estação final ligar para outro assinante da mesma estação, o mecanismo de comutação dentro da estação configurará uma conexão elétrica direta entre os dois circuitos terminais. Essa conexão permanecerá intacta durante a chamada.

Se o telefone chamado estiver conectado a outra estação final, outro procedimento terá de ser usado. Cada estação final contém uma série de linhas de saída para um ou mais centros de comutação vizinhos, denominados **estações interurbanas** (ou, se estiverem na mesma área, **estações Tandem**). Essas linhas são denominadas **troncos de conexão interurbana**. O número de diferentes tipos de centros de comutação e sua topologia variam de um país para outro, dependendo da densidade da telefonia do país.

Se as estações finais do transmissor e do receptor tiverem um tronco de conexão interurbana ligado à mesma estação interurbana (uma situação bastante provável caso estejam geograficamente próximos), a conexão poderá ser estabelecida dentro da estação interurbana. Observe, na Figura 2.24(c), uma rede telefônica formada apenas por telefones (os pontos pequenos), estações finais (os pontos maiores) e estações interurbanas (os quadrados).

Se o transmissor e o receptor não compartilharem a mesma estação interurbana, terá de ser estabelecido um caminho entre duas estações interurbanas, as quais se comunicam entre si por meio de **troncos interurbanos** de alta largura de banda (também denominados **troncos entre estações**). Antes da dissolução da AT&T, em 1984, o sistema telefônico dos Estados Unidos usava o roteamento hierárquico para encontrar um caminho, indo para níveis mais altos da hierarquia até que houvesse uma estação de comutação em comum. Isso foi substituído pelo roteamento não hierárquico, mais flexível. A Figura 2.25 mostra como uma conexão de média distância pode ser roteada.

Nas telecomunicações, são usados vários meios de transmissão. Diferentemente dos prédios de escritórios modernos, nos quais a fiação normalmente é de Categoria 5 ou 6, os circuitos terminais são formados por cabos de pares trançados de Categoria 3, com a fibra apenas começando a aparecer. Entre as estações de comutação, o uso de cabos coaxiais, micro-ondas e, principalmente, de fibras ópticas, é bastante frequente.

No passado, a transmissão em todo o sistema telefônico era analógica, com o sinal de voz real sendo transmitido como uma tensão elétrica da origem até o destino. Com o advento da fibra óptica, da eletrônica digital e dos computadores, todos os troncos e switches são agora digitais, deixando o circuito terminal como o último fragmento de tecnologia analógica no sistema. A transmissão digital é preferida porque, em uma chamada longa, não é necessário reproduzir com precisão uma forma de onda analógica depois de ter passado por muitos amplificadores.

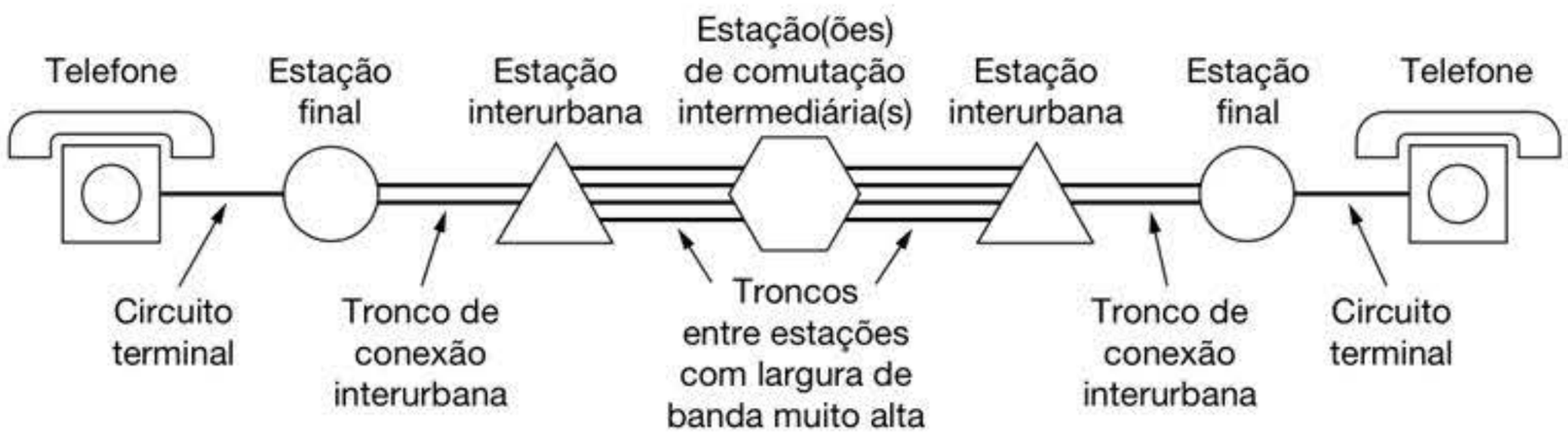


Figura 2.25 Rota de um circuito típico para uma chamada de longa distância.

Ser capaz de distinguir corretamente 0 de 1 já é suficiente. Essa propriedade torna a transmissão digital mais confiável que a analógica, sendo também mais econômica e de manutenção mais fácil.

Em suma, o sistema telefônico é formado por três componentes principais:

1. Circuitos terminais (pares trançados analógicos indo para as residências e para as empresas).
2. Troncos (fibra óptica digital de altíssima largura de banda conectando as estações de comutação).
3. Estações de comutação (onde as chamadas são transferidas de um tronco para outro, elétrica ou opticamente).

Os circuitos terminais oferecem acesso ao sistema inteiro para todas as pessoas; assim, eles são críticos. Infelizmente, eles também constituem o elo mais fraco no sistema. Para os troncos de longa distância, a principal questão é reunir várias chamadas e transmiti-las ao mesmo tempo, pela mesma fibra, o que é feito usando a WDM. Por último, existem duas formas fundamentalmente distintas de executar a comutação: comutação de circuitos e comutação de pacotes. Portanto, analisaremos ambas.

2.5.2 O circuito terminal: modems, ADSL e fibra óptica

Nesta seção, estudaremos o circuito terminal, tanto o novo quanto o antigo. Veremos os modems de telefone, ADSL e FTTH. Em alguns lugares, o circuito terminal foi modernizado instalando-se fibra óptica até a residência (ou muito próximo dela). Essas instalações admitem redes de computadores desde o início, com o circuito terminal tendo ampla largura de banda para os serviços de dados. Infelizmente, o custo para levar a fibra até as residências é muito grande. Às vezes, isso é feito quando as ruas são escavadas para outras finalidades; alguns municípios, especialmente em áreas urbanas densamente povoadas, possuem circuitos terminais de fibra. Porém, em sua maioria, eles são a exceção, mas certamente farão parte do futuro.

Modems de telefone

A maioria das pessoas está familiarizada com o circuito terminal de dois fios, que vem da estação final de uma companhia telefônica até as residências. Com frequência, o circuito terminal também é chamado "o último quilômetro", embora o comprimento real possa chegar a vários quilômetros. Muito esforço tem sido dedicado para espremer a rede de dados pelos circuitos terminais de cobre que já estão instalados. Os modems de telefone enviam dados digitais entre computadores pelo canal estreito que a rede telefônica oferece para uma chamada de voz. Eles já foram muito utilizados, mas foram substituídos em grande parte por tecnologias de banda larga, como ADSL, que reutilizam o circuito terminal para enviar dados digitais de um cliente para a estação final, onde são enviados para a Internet. Os modems e a ADSL precisam lidar com as limitações dos antigos circuitos terminais: largura de banda relativamente estreita, atenuação e distorção dos sinais, e suscetibilidade ao ruído elétrico, como linha cruzada.

Para enviar bits pelo circuito terminal, ou qualquer outro canal físico pelo mesmo motivo, eles precisam ser convertidos para sinais analógicos que podem ser transmitidos pelo canal. Essa conversão é realizada usando os métodos para modulação digital que estudamos na seção anterior. Na outra ponta do canal, o sinal analógico é convertido de volta para bits.

Um dispositivo que converte entre um fluxo de bits digitais e um sinal analógico que representa os bits é chamado **modem**, que é uma abreviação de "*mo*dulador-*demo*dulador". Os modems vêm em muitas variedades: modems de telefone, DSL, a cabo e sem fio. No caso de um modem a cabo ou DSL, o dispositivo geralmente é um hardware separado que fica entre a linha física entrando na casa e o restante da rede dentro da casa. Os dispositivos sem fio geralmente possuem seus próprios modems embutidos. Logicamente, o modem é inserido entre o computador (digital) e o sistema telefônico (analógico), como vemos na Figura 2.26.

Os modems de telefone são usados para enviar bits entre dois computadores por uma linha telefônica com

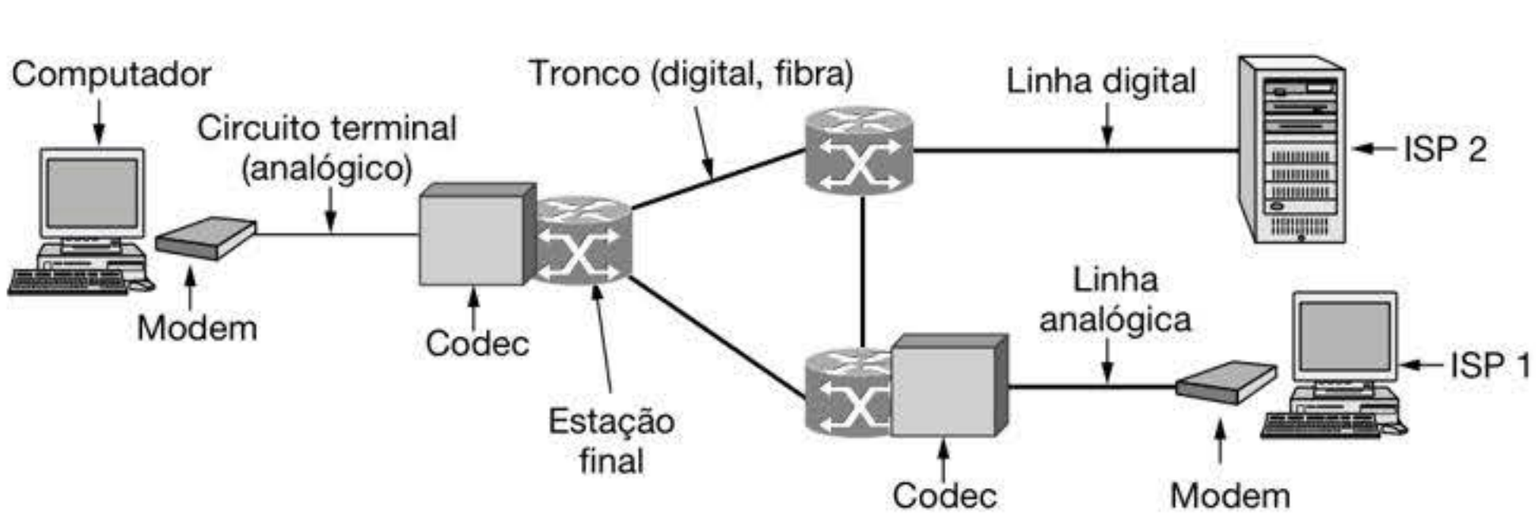


Figura 2.26 O uso de transmissão analógica e digital para uma chamada de computador para computador. A conversão é feita por modems e codecs.

qualidade de voz, em lugar da conversação que normalmente preenche a linha. A principal dificuldade ao fazer isso é que uma linha de telefone para a voz humana é limitada a 3.100 Hz, o suficiente para transportar uma conversa. Essa largura de banda é mais de quatro ordens de grandeza menor que a largura de banda usada para Ethernet ou rede 802.11 (WiFi). Não é surpresa que as taxas de dados dos modems de telefone também sejam quatro ordens de grandeza menores que as da Ethernet e da rede 802.11.

Vamos analisar os números para ver por que isso acontece. O teorema de Nyquist nos diz que, mesmo com uma linha perfeita de 3.000 Hz (que uma linha telefônica decididamente não é), não há sentido em enviar símbolos em uma taxa mais rápida que 6.000 bauds. Por exemplo, vamos considerar um modem mais antigo enviando a uma taxa de 2.400 símbolos/s, ou 2.400 bauds, que captura vários bits por símbolo, enquanto permite o tráfego nas duas direções ao mesmo tempo (usando diferentes frequências para diferentes direções).

O humilde modem de 2.400 bps usa 0 volt para um valor lógico 0 e 1 volt para um valor 1, com 1 bit por símbolo. No próximo passo, ele pode usar quatro símbolos diferentes, como nas quatro fases do QPSK, de modo que, com 2 bits/símbolo, ele pode alcançar uma taxa de dados de 4.800 bps.

Uma longa progressão de taxas mais altas tem sido alcançada à medida que a tecnologia melhora. Taxas mais altas exigem um conjunto maior de símbolos (ver Figura 2.17). Com muitos símbolos, até mesmo uma pequena quantidade de ruído na amplitude ou fase detectada pode resultar em um erro. Para reduzir a chance de erros, os padrões para modems de velocidade mais alta utilizam alguns dos símbolos para correção de erros. Os esquemas são conhecidos como **modulação codificada por treliças**, ou **TCM (Trellis Coded Modulation)**. A Figura 2.27 mostra alguns padrões de modem comuns.

Por que parar em 33.600 bps? A razão para isso é que o limite de Shannon para o sistema telefônico é de aproximadamente 35 kbps, com base no comprimento médio dos circuitos terminais e na qualidade dessas linhas. Uma transmissão mais rápida que isso violaria as leis da física (departamento da termodinâmica) ou exigiria novos circuitos terminais (o que gradualmente está sendo feito).

Padrão de modem	Baud	Bits/símbolo	Bps
V.32	2.400	4	9.600
V.32 bis	2.400	6	14.400
V.34	2.400	12	28.800
V.34 bis	2.400	14	33.600

Figura 2.27 Alguns padrões de modem e suas taxas de bits.

Contudo, existe um meio de mudar a situação. Na estação final da companhia telefônica, os dados são convertidos para a forma digital para transmissão dentro da rede telefônica (o núcleo da rede telefônica convertia de analógico para digital há muito tempo). O limite de 35 kbps é para a situação em que existem dois circuitos terminais, um em cada ponta. Cada um deles acrescenta ruído ao sinal. Se pudéssemos nos livrar de um desses circuitos terminais, aumentaríamos a SNR e a taxa máxima seria dobrada.

É com essa técnica que os modems de 56 kbps funcionam. Uma extremidade, normalmente um ISP, recebe uma entrada digital de alta qualidade da estação final mais próxima. Desse modo, quando uma extremidade da conexão é um sinal de alta qualidade, como ocorre com a maioria dos ISPs atuais, a taxa máxima de dados pode chegar a 70 kbps. Entre dois usuários domésticos com modems e linhas analógicas, o máximo ainda é 33,6 kbps.

A razão para o uso de modems de 56 kbps (em vez de modems de 70 kbps) está relacionada ao teorema de Nyquist. Um canal de telefonia é transportado dentro do sistema telefônico como amostras digitais. Cada canal telefônico tem cerca de 4.000 Hz de largura, incluindo as bandas de proteção. O número máximo de amostras independentes por segundo é, portanto, 8.000. O número de bits por amostra na América do Norte é 8, um dos quais pode ser usado para fins de controle, permitindo 56.000 bits/s de dados do usuário. Na Europa, todos os 8 bits estão disponíveis para os usuários, e assim poderiam ser usados modems de 64.000 bits/s; porém, para chegar a um acordo internacional sobre um padrão, o valor de 56.000 foi escolhido.

O resultado final são os modems padrão **V.90** e **V.92**. Eles são capazes de transmitir 56 kbps no canal downstream (ISP ao usuário) e 33,6 kbps e 48 kbps no canal upstream (usuário para ISP), respectivamente. A assimetria ocorre porque normalmente existem mais dados transportados do ISP para o usuário do que o contrário. Isso também significa que uma parte maior da largura de banda limitada pode ser alocada ao canal downstream, para aumentar as chances de que ele realmente funcione a 56 kbps.

Digital Subscriber Lines (DSL)

Quando a indústria de telefonia finalmente conseguiu alcançar 56 kbps, ela se congratulou pelo serviço bem-feito. Enquanto isso, a indústria de TV a cabo estava oferecendo velocidades de até 10 Mbps sobre cabos compartilhados. À medida que o acesso à Internet se tornou uma parte cada vez mais importante de seus negócios, as companhias telefônicas locais começaram a perceber que precisavam de um produto mais competitivo. Sua resposta foi começar a oferecer novos serviços digitais ao circuito terminal do assinante.

Inicialmente, havia muitas ofertas sobrepostas, todas sob o nome genérico de **linha digital do assinante**, ou **xDSL (Digital Subscriber Line)**, para diversos *x*.

Os serviços com mais largura de banda do que o serviço telefônico padrão às vezes são chamados de **banda larga** (**broadband**), embora o termo seja mais um conceito de marketing do que um conceito técnico específico. Mais adiante, vamos nos concentrar principalmente naquele que provavelmente se tornou o mais popular desses serviços, a **ADSL** (**Asymmetric DSL**). Também usaremos o termo DSL ou xDSL como uma abreviação geral.

A razão para os modems serem tão lentos é que os telefones foram inventados para transportar a voz humana, e o sistema inteiro foi cuidadosamente otimizado para esse propósito. Os dados sempre estiveram em segundo plano. No ponto em que cada circuito terminal chega à estação final, o fio passa por um filtro que atenua todas as frequências abaixo de 300 Hz e acima de 3.400 Hz. O corte não é nítido – 300 Hz e 3.400 Hz são os pontos de 3 dB – assim, a largura de banda normalmente é mencionada como 4.000 Hz, embora a distância entre os pontos de 3 dB seja de 3.100 Hz. Portanto, os dados no fio também estão restritos a essa banda estreita.

O artifício que faz o xDSL funcionar é o fato de, quando um cliente se inscreve nele, a linha de entrada ser conectada a um tipo diferente de switch, que não tem esse filtro, tornando disponível toda a capacidade do circuito terminal. Então, o fator limitador passa a ser a constituição física do circuito terminal, que aceita aproximadamente 1 MHz, e não a largura de banda artificial de 3.100 Hz criada pelo filtro.

Infelizmente, a capacidade do circuito terminal cai rapidamente com a distância da estação final, pois o sinal é degradado cada vez mais ao longo do fio. Ela também depende da espessura e da qualidade geral do par trançado. A Figura 2.28 mostra um esboço da largura de banda potencial como uma função da distância, pressupondo que todos os outros fatores estão otimizados (novos fios, pacotes de serviços, etc.).

A implicação dessa figura cria um problema para a companhia telefônica. Quando escolhe uma velocidade para oferecer, ela está ao mesmo tempo escolhendo um raio a partir de suas estações finais, além do qual o serviço não poderá ser oferecido. Isso significa que, quando clientes distantes tentarem assinar o serviço, eles receberão a seguinte mensagem: "Muito obrigado por seu interesse, mas você está 100 m além da distância máxima da central mais próxima que poderia lhe oferecer o serviço. Você não gostaria de se mudar?". Quanto mais baixa a velocidade escolhida, maior o raio e maior o número de clientes cobertos. Porém, quanto mais baixa a velocidade, menos atraente será o serviço e menor o número de pessoas que estarão dispostas a pagar por ele. É aqui que os negócios encontram a tecnologia.

Todos os serviços xDSL foram criados visando certos objetivos. Primeiro, os serviços devem funcionar nos circuitos terminais de pares trançados da Categoria 3 existente. Segundo, eles não devem afetar os telefones e os aparelhos de fax atuais dos clientes. Terceiro, eles devem ser muito mais rápidos que 56 kbps. Quarto, eles devem estar sempre ativos, apenas com uma tarifa mensal e nenhuma tarifa por minuto.

Para cumprir os objetivos técnicos, o espectro de 1,1 MHz disponível no circuito terminal é dividido em 256 canais independentes de 4.312,5 Hz cada um. Esse arranjo pode ser visto na Figura 2.29. O esquema OFDM, que vimos na seção anterior, é usado para enviar dados por esses canais, embora normalmente seja chamado **DMT** (**Discrete MultiTone**) no contexto da ADSL. O canal 0 é usado para o **POTS** (**Plain Old Telephone Service**), e os canais de 1 a 5 não são usados, a fim de impedir que o sinal de voz e os sinais de dados interfiram uns com os outros. Dos 250 canais restantes, um é utilizado para o controle upstream e outro é empregado para o controle downstream. Os demais estão disponíveis para dados do usuário.

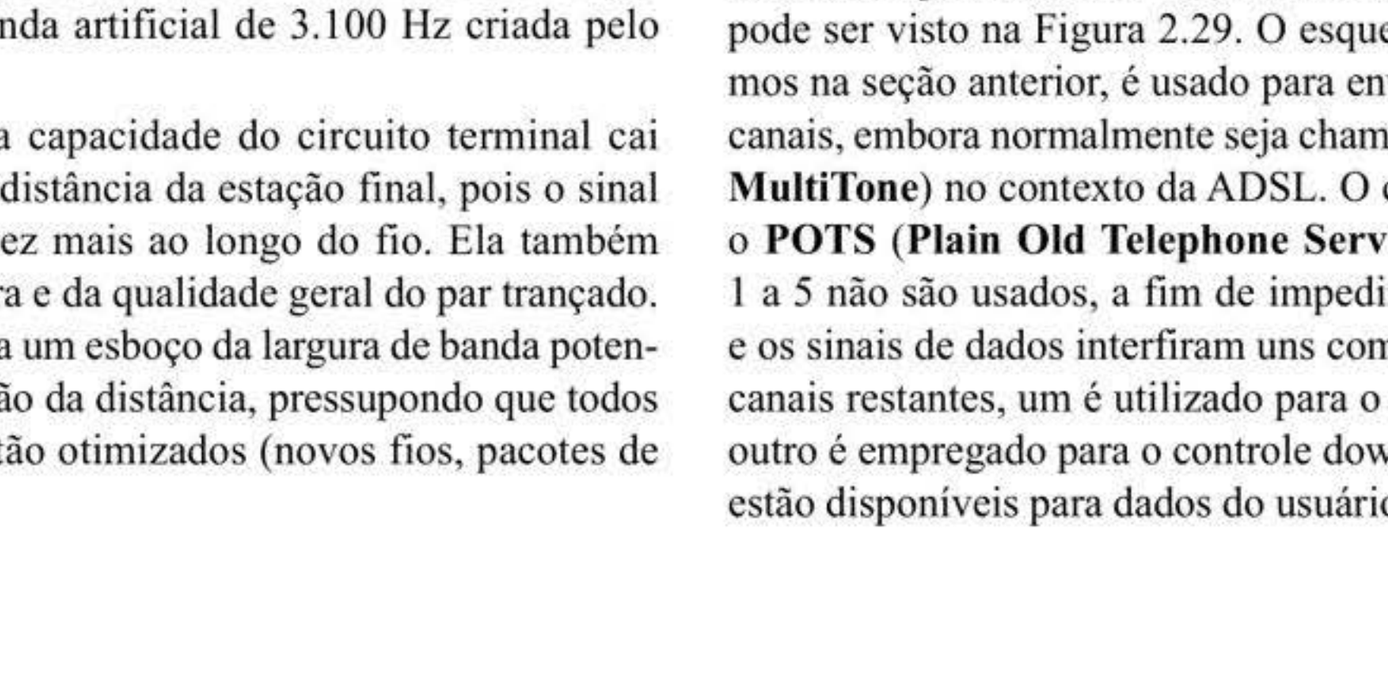


Figura 2.28 Variação da largura de banda *versus* a distância sobre o UTP da categoria 3 para DSL.

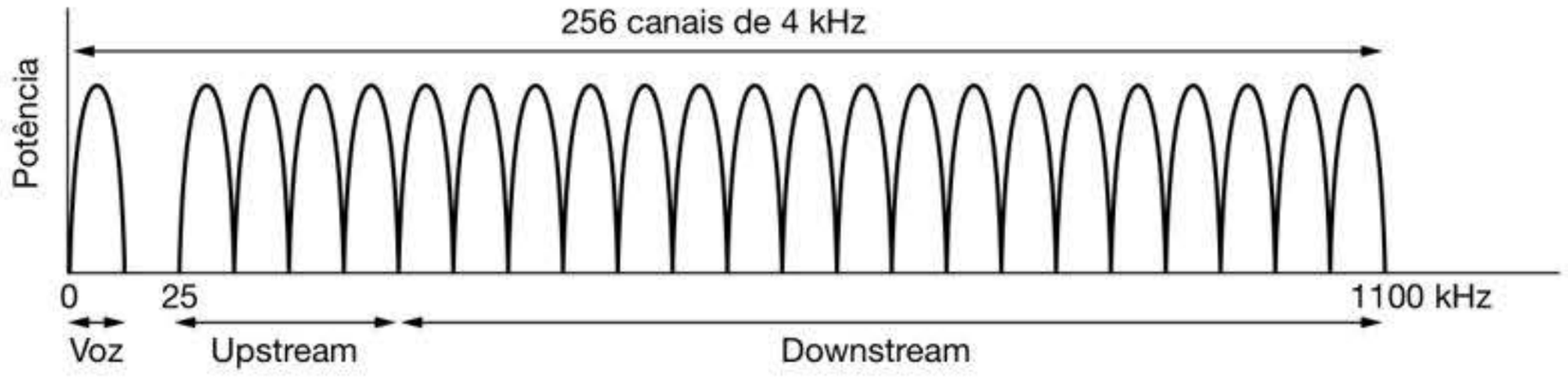


Figura 2.29 Operação ADSL usando modelagem discreta por multitom.

Em princípio, cada um dos canais restantes pode ser usado em um fluxo de dados full-duplex; porém, harmônicos, linhas cruzadas e outros efeitos mantêm a utilização de sistemas práticos bem abaixo do limite teórico. Cabe ao provedor definir quantos canais serão usados para upstream e quantos serão usados para downstream. Uma mistura de 50% para cada um é tecnicamente possível, mas a maioria dos provedores aloca algo como 80 a 90% da largura de banda ao canal downstream, pois a maioria dos usuários faz mais download do que upload de dados. Essa escolha deu origem à letra "A" (assimétrica) no acrônimo ADSL. Uma divisão comum reserva 32 canais para upstream e os restantes para downstream. Também é possível tornar bidirecionais alguns dos canais upstream mais altos para aumentar a largura de banda, embora essa otimização exija o uso de um circuito especial para cancelamento de ecos.

O padrão ADSL internacional, conhecido como **G.dmt**, foi aprovado em 1999 e permite velocidades de até 8 Mbps downstream e 1 Mbps upstream. Ele foi substituído por uma segunda geração em 2002, chamada ADSL2, com diversas melhorias para permitir velocidades de até 12 Mbps downstream e 1 Mbps upstream. ADSL2+ dobra a velocidade downstream para 24 Mbps ao dobrar a largura de banda para usar 2,2 MHz pelo par trançado.

Em 2006 ocorreu a melhoria seguinte, o **VDSL**, que empurrou a taxa de dados nos circuitos terminais mais curtos para 52 Mbps downstream e 3 Mbps upstream. Então, de 2007 a 2011, uma série de novos padrões, sob o nome de **VDSL2**, sobre circuitos terminais de alta qualidade, conseguiram usar largura de banda de 12 MHz e atingir taxas de dados de 200 Mbps downstream e 100 Mbps upstream. Em 2015, o **Vplus** foi proposto para circuitos terminais menores que 250 m. Em princípio, ele pode atingir 300 Mbps downstream e 100 Mbps upstream, mas não é fácil fazê-lo funcionar na prática. Podemos estar perto do fim da linha aqui para a fiação Categoria 3 existente, exceto talvez para distâncias ainda mais curtas.

Dentro de cada canal, a modulação QAM é usada a uma taxa de aproximadamente 4.000 símbolos/s. A qualidade da linha em cada canal é monitorada constantemente, e a taxa de dados é ajustada usando um diagrama de constelação maior ou menor, como os da Figura 2.17. Diferentes canais podem ter diferentes taxas de dados, com até 15 bits por símbolo enviado em um canal com uma SNR alta, e descendo até 2, 1 ou nenhum bit por símbolo enviado em um canal com uma SNR baixa, dependendo do padrão.

A Figura 2.30 mostra a organização de uma ADSL típica. Nesse esquema, um técnico da companhia telefônica deve instalar um **dispositivo de interface de rede**, ou **NID** (**Network Interface Device**) no local do cliente. Essa pequena caixa plástica marca o fim da propriedade da companhia telefônica e o início da propriedade do cliente. Próximo ao NID (ou às vezes combinado a ele) há um **divisor**, um filtro analógico que separa a banda de 0 a 4.000 Hz utilizada pelo POTS dos dados. O sinal do POTS é roteado até o telefone ou o equipamento de fax existente, e o sinal de dados é roteado até um modem ADSL, que usa o processamento do sinal digital para implementar OFDM. Como a maioria dos modems ADSL atuais é externa, o computador deve estar conectado ao modem em alta velocidade. Normalmente, isso é feito usando Ethernet, cabo USB ou rede 802.11.

Na outra extremidade do fio, ao lado da estação final, está instalado um divisor correspondente. Aqui, a porção de voz do sinal é filtrada e enviada ao switch de voz normal. O sinal acima de 26 kHz é roteado para um novo tipo de dispositivo, chamado **multiplexador de acesso à linha digital do assinante**, ou **DSLAM** (**Digital Subscriber Line Access Multiplexer**), que contém a mesma espécie de processador de sinal digital que o modem ADSL. O DSLAM converte o sinal em bits e envia pacotes para a rede de dados do provedor de serviços da Internet.

Essa separação completa entre o sistema de voz e a ADSL torna relativamente fácil para uma companhia telefônica distribuir esse serviço. Basta adquirir um DSLAM e um divisor, e conectar os assinantes da ADSL ao divisor. Outros serviços de alta largura de banda entregues pela rede telefônica (p. ex., ISDN) exigem mudanças muito maiores no equipamento de comutação existente.

A próxima fronteira para a implantação da DSL é atingir velocidades de transmissão de 1 Gbps ou mais. Esses esforços estão se concentrando em uma série de técnicas complementares, incluindo uma técnica chamada **bonding** (ou **união**), que cria uma única conexão DSL virtual combinando duas ou mais conexões DSL físicas. Obviamente, se combinarmos dois pares trançados, será possível dobrar a largura de banda. Em alguns lugares, os fios telefônicos que entram nas casas usam um cabo que, na verdade, tem dois

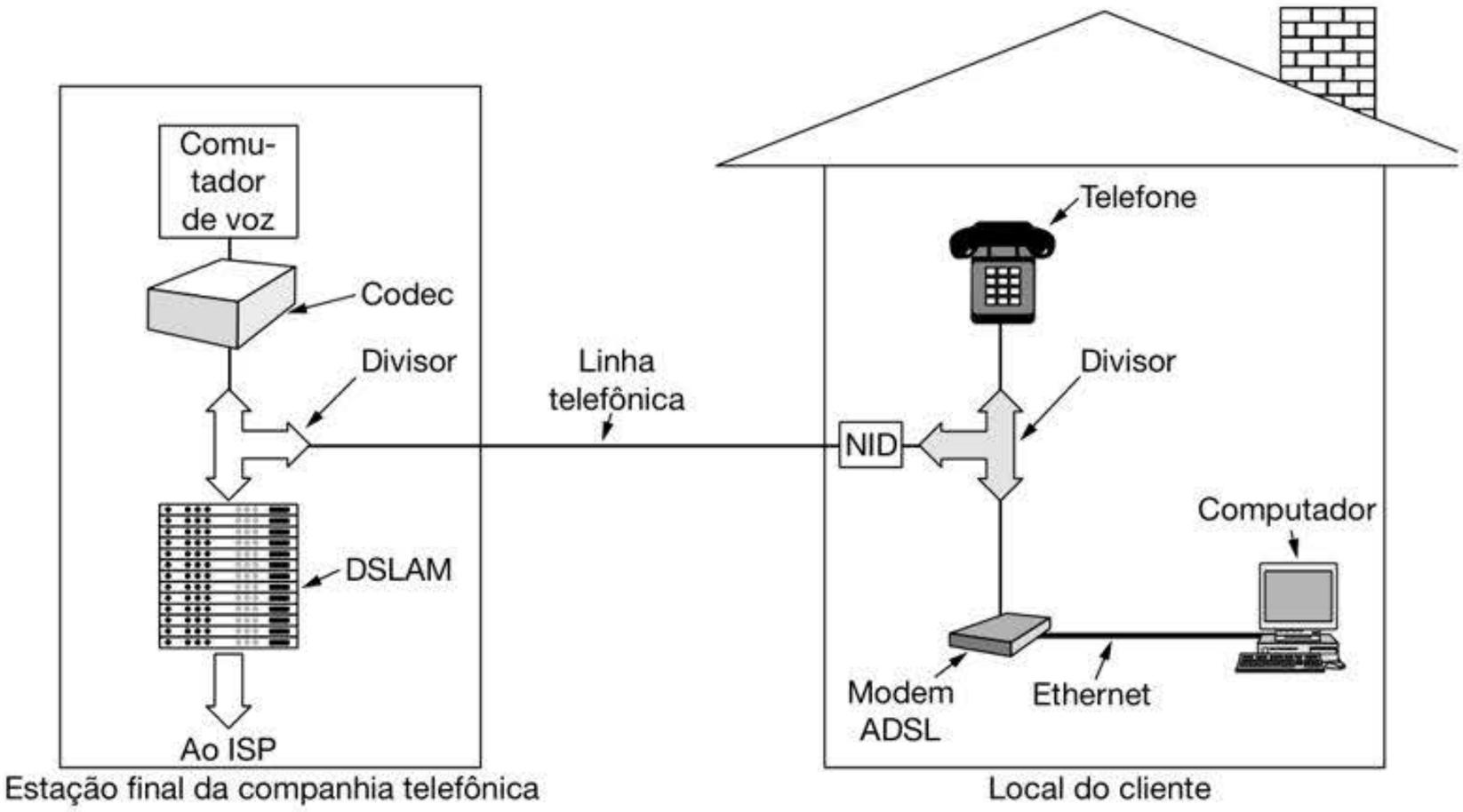


Figura 2.30 Configuração típica de equipamento ADSL.

pares trançados. A ideia original era permitir duas linhas e números de telefone separados na casa, mas usando a união dos pares, uma única conexão de Internet de alta velocidade pode ser alcançada. Um número cada vez maior de ISPs na Europa, Austrália, Canadá e Estados Unidos já está implantando uma tecnologia chamada **G.fast**, que usa união de pares. Tal como acontece com outras formas de DSL, o desempenho do G.fast depende da distância da transmissão; testes recentes viram velocidades simétricas se aproximando de 1 Gbps a distâncias de 100 m, quando acoplado a uma implantação de fibra conhecida como **FTTdp** (**Fiber to the Distribution Point**), que leva a fibra a um ponto de distribuição de várias centenas de assinantes e usa cobre para transmitir dados pelo restante do caminho para a casa (em VDSL2, isso pode chegar a 1 km, embora a velocidades mais baixas). FTTdp é apenas um tipo de implantação de fibra que leva a fibra do núcleo da rede a algum ponto próximo à extremidade da rede. A próxima seção descreve diversos modos de implantação de fibra.

Fiber To The X (FTTX)

A velocidade dos circuitos terminais normalmente é restrita pelos cabos de cobre instalados na rede de telefonia convencional, que não podem transmitir dados em altas velocidades por uma distância tão grande quanto a fibra. Assim, um objetivo final, onde for economicamente viável, é levar a fibra até a casa do cliente, algo chamado de **FTTH** (**Fiber To The Home**). As companhias telefônicas continuam a tentar melhorar o desempenho do circuito terminal, geralmente instalando a fibra o mais próximo possível da casa. Se não for até ela, eles podem oferecer **FTTN** (**Fiber to the Node**), onde a fibra termina em um gabinete em uma rua, às vezes a alguns quilômetros da casa do cliente. Fiber to the Distribution Point (FTTdp), como já dissemos, leva a fibra um passo a mais para a casa do cliente, geralmente levando-a para alguns metros do local desejado. Entre essas duas opções há o **FTTC** (**Fiber to the Curb**). Todos esses projetos **FTTX** (**Fiber to the X**) às vezes são chamados de "fibra no terminal", pois é usada uma certa quantidade de fibra no circuito terminal.

Existem diversas variações na forma "FTTX" (onde *X* significa porão, calçada ou vizinhança). Elas são usadas para indicar que a implantação da fibra pode chegar perto da casa. Nesse caso, o cobre (par trançado ou cabo coaxial) oferece velocidades rápidas o suficiente pela última distância curta. A escolha da extensão em que a fibra é disposta é uma questão econômica, balanceando custo com receita esperada. De qualquer forma, o importante é que a fibra óptica atravessou a barreira tradicional do "último quilômetro". Focaremos o FTTH em nossa discussão.

Assim como os fios de cobre antes dele, o circuito terminal de fibra é passivo. Isso significa que nenhum equipamento energizado é necessário para amplificar ou processar os sinais de alguma outra forma. A fibra simplesmente transporta sinais entre a casa e a estação final. Isso, por sua vez, reduz o custo e melhora a confiabilidade. Normalmente, as fibras das casas são reunidas de modo que apenas uma fibra alcance a estação final por grupo de até 100 casas. Na direção downstream, os divisores ópticos dividem o sinal da estação final, de modo que alcance todas as casas. A criptografia é necessária por segurança se apenas uma casa puder ser capaz de decodificar o sinal. Na direção upstream, colimadores ópticos mesclam os sinais das casas para um único sinal, que é recebido na estação final.

Essa arquitetura é chamada **rede óptica passiva**, ou **PON** (**Passive Optical Network**), mostrada na Figura 2.31. É comum usar um comprimento de onda compartilhado

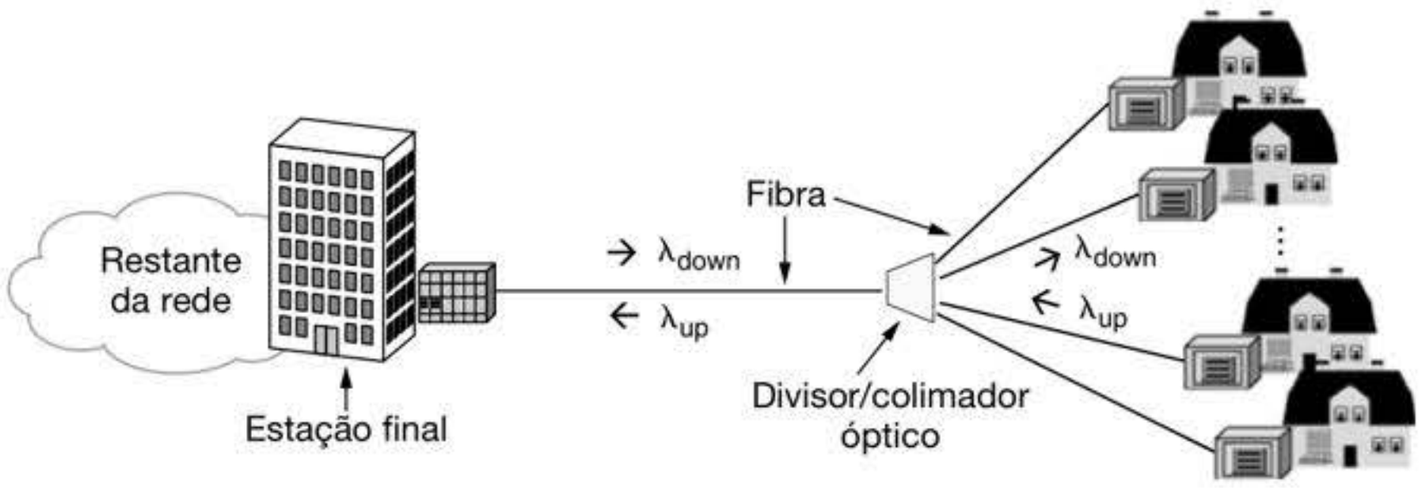


Figura 2.31 Rede óptica passiva para Fiber To The Home.

entre todas as casas para a transmissão downstream, e outro comprimento de onda para a transmissão upstream.

Mesmo com a divisão, uma enorme largura de banda e a baixa atenuação da fibra significam que as PONs podem oferecer altas velocidades aos usuários por distâncias de até 20 km. As taxas de dados reais e outros detalhes dependem do tipo de PON. Dois tipos são comuns: as **GPONs (Gigabit-capable PONs)** vêm do mundo das telecomunicações e são definidas por um padrão ITU; as **EPONs (Ethernet PONs)** estão mais ligadas ao mundo das redes e são definidas por um padrão IEEE. Ambas trabalham em torno de um gigabit e podem transportar tráfego para diferentes serviços, incluindo Internet, vídeo e voz. Por exemplo, as GPONs oferecem 2,4 Gbps downstream e 1,2 ou 2,4 Gbps upstream.

Para compartilhar a capacidade de uma única fibra na estação final entre diferentes casas, é preciso que haja protocolos adicionais. A direção downstream é fácil, pois a estação final pode enviar mensagens a cada casa diferente na ordem que desejar. Na direção upstream, porém, as mensagens de diferentes casas não podem ser enviadas ao mesmo tempo, ou diferentes sinais colidiriam. As casas também não podem escutar as transmissões umas das outras, de modo que não podem escutar antes de transmitir. A solução é que o equipamento nas casas solicite e receba fatias de tempo para utilizar o equipamento na estação final. Para que isso funcione, existe um processo de localização para ajustar os tempos de transmissão a partir das casas, de modo que todos os sinais recebidos na estação final sejam sincronizados. O projeto é semelhante aos modems a cabo, que explicaremos mais adiante neste capítulo. Para obter mais informações sobre PONs, veja Grobe e Elbers (2008) ou Andrade et al. (2014).

2.5.3 Troncos e multiplexação

Na rede telefônica, os troncos não são apenas muito mais rápidos do que os circuitos terminais; eles são diferentes em dois outros aspectos. O núcleo da rede telefônica transporta informações digitais, não informações analógicas, ou seja, bits e não voz. Isso necessita de uma conversão na estação final para a forma digital, para a transmissão por troncos de longa distância. Os troncos transportam milhares, ou mesmo milhões, de chamadas simultaneamente. Esse compartilhamento é importante para economizar escalas, pois custa basicamente a mesma coisa instalar e manter um tronco de alta largura de banda e um de baixa largura de banda entre duas estações de comutação. Isso é realizado com as versões de multiplexação TDM e FDM.

A seguir, examinaremos rapidamente como os sinais de voz são digitalizados de modo que possam ser transportados pela rede telefônica. Depois, veremos como a TDM é usada para transportar bits nos troncos, incluindo o sistema TDM usado para fibra óptica (SONET). Em seguida, passaremos para FDM conforme se aplica à fibra óptica, chamada de WDM.

Sinais de voz digitalizados

Desde cedo no desenvolvimento da rede telefônica, o núcleo lidava com chamadas de voz como informação analógica. As técnicas de FDM foram usadas por muitos anos para multiplexar canais de voz de 4.000 Hz (compostos de 3.100 Hz mais bandas de proteção) em unidades cada vez maiores. Por exemplo, 12 chamadas na banda de 60 a 108 kHz são conhecidas como um **grupo**, cinco grupos (um total de 60 chamadas) são conhecidos como um **supergrupo**, e assim por diante. Esses métodos de FDM ainda são usados em alguns canais de fio de cobre e micro-ondas. Contudo, a FDM requer circuitos analógicos e isso não é adequado para um computador digital. Ao contrário, a TDM pode ser tratada inteiramente pela eletrônica digital, de modo que se tornou muito mais usada nos últimos anos. Como a TDM só pode ser usada para dados digitais e os circuitos terminais produzem sinais analógicos, é preciso que haja uma conversão de analógico para digital na estação final, onde todos os circuitos terminais individuais se juntam para serem combinados nos troncos de saída.

Os sinais analógicos são digitalizados na estação final por um dispositivo chamado **codec** (abreviação de "*co*dificador-*dec*odificador") usando uma técnica chamada **PCM (Pulse Code Modulation)**, que forma o coração do sistema telefônico moderno. O codec cria 8.000 amostras por segundo (125 μs/amostra), pois o teorema de Nyquist diz que isso é suficiente para capturar todas as informações de largura de banda do canal telefônico de 4 kHz. Em uma taxa de amostragem mais baixa, as informações seriam perdidas; a uma

taxa mais alta, nenhuma informação extra seria obtida. Cada amostra da amplitude do sinal é quantizada para um número de 8 bits. Quase todos os intervalos de tempo no sistema telefônico são múltiplos de 125 μs. A taxa de dados não compactada padrão para uma chamada telefônica com qualidade de voz é, portanto, de 8 bits a cada 125 μs, ou 64 kbps.

Cada amostra da amplitude do sinal é quantizada para um número de 8 bits. Para reduzir o erro em virtude da quantização, seus níveis são espaçados de forma desigual. Uma escala logarítmica é usada, gerando relativamente mais bits para menores amplitudes de sinal e relativamente menos bits para grandes amplitudes de sinal. Desse modo, o erro é proporcional à amplitude do sinal. Duas versões de quantização são bastante utilizadas: **μ-law**, usada na América do Norte e no Japão, e **A-law**, usada na Europa e no restante do mundo. As duas versões são especificadas no padrão ITU G.711. Um modo equivalente de pensar nesse processo é imaginar que a faixa dinâmica do sinal (ou a razão entre os maiores e menores valores possíveis) é compactada antes de ser quantizada (uniformemente), e então expandida quando o sinal analógico é recriado. Por esse motivo, ela é chamada **compactação/expansão**, ou **companding**. Também é possível compactar as amostras depois que elas são digitalizadas, de modo que exigem muito menos do que 64 kbps. Contudo, deixaremos esse assunto para quando explorarmos aplicações de áudio como VoIP.

Na outra extremidade da chamada, um sinal analógico é recriado a partir das amostras quantizadas, reproduzindo-as (e suavizando-as) ao longo do tempo. Não será exatamente igual ao sinal analógico original, embora o tenhamos amostrado na taxa de Nyquist, pois as amostras foram quantizadas.

T-Carrier: multiplexação de sinais digitais pela rede de telefonia

T-Carrier é uma especificação para a transmissão de vários canais TDM por um único circuito. A TDM baseada na PCM é usada para transportar várias chamadas de voz por troncos, enviando uma amostra de cada chamada a cada 125 μs. Quando a transmissão digital começou a surgir como uma tecnologia viável, a ITU (então chamada CCITT) foi incapaz de chegar a um acordo sobre um padrão internacional para a PCM. Consequentemente, diversos esquemas incompatíveis agora estão em uso em diferentes países.

O método usado na América do Norte e no Japão é a portadora **T1**, representada na Figura 2.32. (Tecnicamente falando, o formato é chamado DS1 e a portadora é chamada T1, mas, seguindo a tradição da indústria, aqui não faremos essa distinção sutil.) A portadora T1, introduzida em 1962, consiste em 24 canais de voz multiplexados. Por sua vez, cada um dos 24 canais consegue inserir 8 bits no fluxo de saída.

Um quadro consiste em 24 × 8 = 192 bits, mais um bit extra para fins de controle, produzindo 193 bits a cada 125 μs. Isso resulta em uma taxa de dados bruta de 1,544 Mbps, dos quais 8 kbps são para sinalização. O 193º bit é usado para sincronização e sinalização de quadros. Em uma variação, o 193º bit é usado por um grupo de 24 quadros, chamado **superquadro estendido**. Seis dos bits, na 4ª, 8ª, 12ª, 16ª, 20ª e 24ª posições, utilizam o padrão 001011... Normalmente, o receptor continua a conferir esse bit para garantir que não perdeu a sincronização. Seis outros bits são usados para enviar um código de verificação de erro, para ajudar o receptor a confirmar se está sincronizado. Se sair de sincronismo, o receptor poderá procurar por esse padrão e validar o código de verificação de erro para se ressincronizar. Os 12 bits restantes são usados para informação de controle, para operar e manter a rede, como o relatório de desempenho da extremidade remota.

O formato T1 tem diversas variações. As versões mais antigas enviavam informações de sinalização **na própria banda**, significando o uso do mesmo canal que os dados, usando alguns bits de dados. Esse projeto é uma

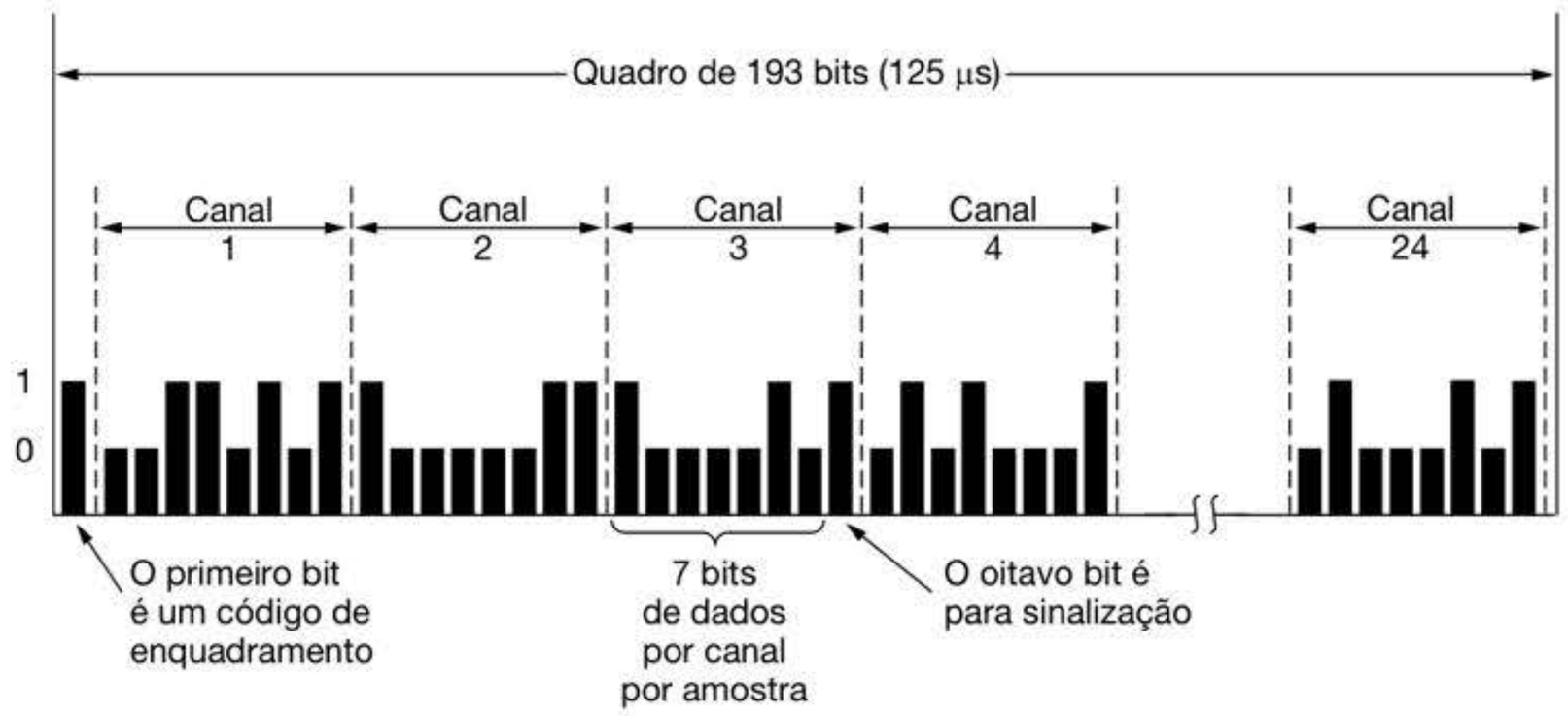


Figura 2.32 A portadora T1 (1,544 Mbps).

forma de **sinalização associada ao canal**, pois cada canal tem seu próprio subcanal de sinalização privado. Em um arranjo de bits, o bit menos significativo de uma amostra de 8 em cada canal é usado a cada seis quadros. Este recebe o nome sugestivo de **bit de sinalização roubado**. A ideia é que alguns bits roubados não influenciarão nas chamadas de voz. Ninguém notará qualquer diferença audível.

Contudo, para dados, a história é outra. Entregar os bits errados é, no mínimo, inútil. Se versões mais antigas da T1 forem usadas para transportar dados, somente 7 de 8 bits, ou 56 kbps, podem ser usados em cada um dos 24 canais. Em vez disso, versões mais novas da T1 oferecem canais limpos, em que todos os bits podem ser usados para enviar dados. Canais limpos são o que as empresas que alugam uma linha T1 desejam quando enviam dados pela rede telefônica no lugar das amostras de voz. A sinalização para quaisquer chamadas de voz é, então, tratada **fora da banda**, significando um canal próprio separado dos dados. Normalmente, a sinalização é feita com **sinalização de canal comum**, no qual existe um canal de sinalização compartilhado. Um dos 24 canais pode ser usado para essa finalidade.

Fora da América do Norte e do Japão, a portadora **E1** de 2,048 Mbps é usada no lugar da T1. Essa portadora tem 32 amostras de dados de 8 bits compactadas no quadro básico de 125 μs. Trinta dos canais são usados para informações e até dois são usados para sinalização. Cada grupo de quatro quadros oferece 64 bits de sinalização, metade deles usada para sinalização (associada ao canal ou ao canal comum) e metade usada para sincronização de quadros ou reservada para cada país usar como preferir.

A TDM permite que várias portadoras T1 sejam multiplexadas em portadoras de ordem mais alta. A Figura 2.33 mostra como isso pode ser feito. À esquerda, vemos quatro canais T1 sendo multiplexados em um canal T2. A multiplexação em T2 e acima é feita bit a bit, e não byte a byte, com os 24 canais de voz que compõem um quadro T1. Quatro fluxos T1 a uma velocidade de 1,544 Mbps deveriam gerar 6,176 Mbps, mas T2, na verdade, tem 6,312 Mbps. Os bits extras são usados para enquadramento e recuperação, no caso de a portadora apresentar alguma falha.

No nível seguinte, sete fluxos T2 são combinados bit a bit para formar um fluxo T3. Depois, seis fluxos T3 são unidos para formar um fluxo T4. Em cada etapa, um pequeno volume de overhead é adicionado para fins de enquadramento e recuperação, no caso de a sincronização entre transmissor e receptor ser perdida. T1 e T3 são extensamente utilizados pelos clientes, enquanto T2 e T4 são usados apenas dentro do sistema de telefonia propriamente dito e, portanto, não são bem conhecidos.

Da mesma forma que existe pouco consenso quanto à portadora básica entre os Estados Unidos e o restante do mundo, há igualmente pouco consenso sobre como ela será multiplexada em portadoras de largura de banda mais alta. O esquema norte-americano de avançar por 4, 7 e 6 não foi adotado por mais ninguém; assim, o padrão ITU requer multiplexação de quatro fluxos em um fluxo a cada nível. Além disso, o enquadramento e a recuperação de dados são diferentes entre os padrões dos Estados Unidos e da ITU. A hierarquia ITU para 32, 128, 512, 2.048 e 8.192 canais funciona em velocidades de 2,048, 8,848, 34,304, 139,264 e 565,148 Mbps.

Multiplexação de redes ópticas: SONET/SDH

Nos primórdios da fibra óptica, cada companhia telefônica tinha seu próprio sistema óptico TDM patenteado. Depois que o governo dos Estados Unidos desmembrou a AT&T em 1984, as companhias telefônicas locais tiveram de se conectar a diversas concessionárias de comunicações de longa distância, todas com sistemas ópticos TDM de diversos vendedores e fornecedores, o que tornou óbvia a necessidade de padronização. Em 1985, a Bellcore, a unidade de pesquisa das RBOCs (Regional Bell Operating Companies), começou a trabalhar em um padrão, denominado **rede óptica síncrona**, ou **SONET** (**Synchronous Optical NETwork**).

Mais tarde, a ITU também começou a participar desse trabalho, o que resultou em um padrão SONET e em um conjunto de recomendações paralelas (G.707, G.708 e G.709) em 1989. As recomendações da ITU são chamadas de **hierarquia digital síncrona**, ou **SDH** (**Synchronous Digital Hierarchy**), mas diferem da SONET apenas em pequenos detalhes. Praticamente todo o tráfego telefônico

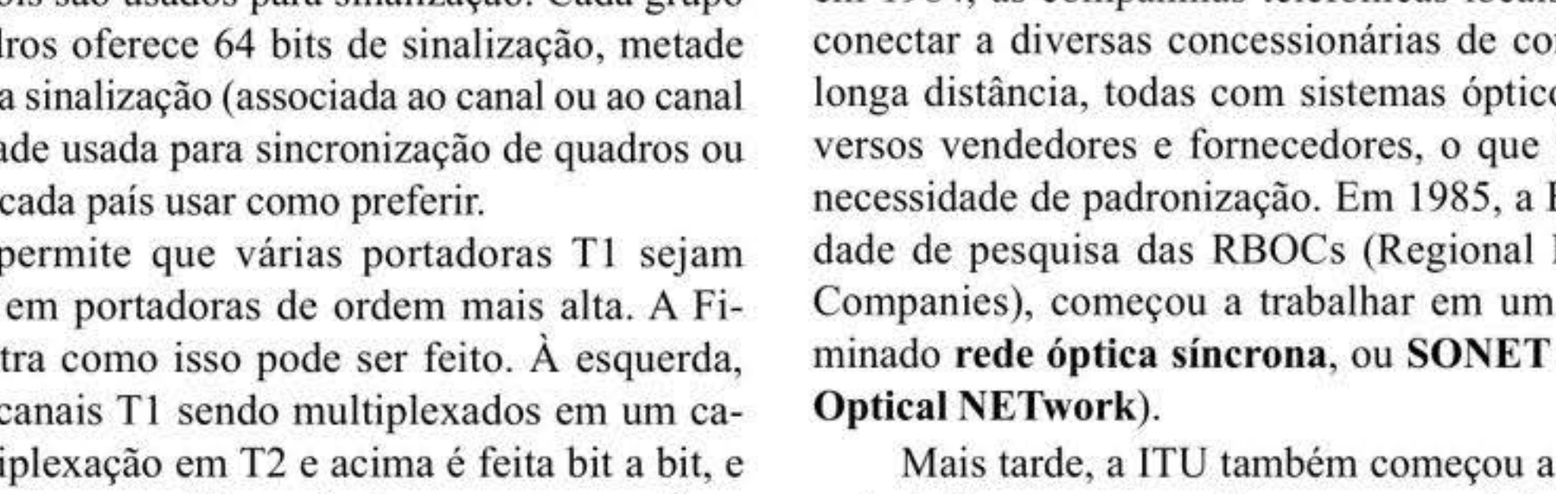


Figura 2.33 Multiplexação de fluxos T1 em portadoras de velocidade mais alta.

de longa distância nos Estados Unidos e grande parte dele em outros lugares atualmente utiliza troncos que executam a SONET na camada física. Para obter informações adicionais sobre a SONET, consulte Perros (2005).

O projeto SONET tem quatro objetivos principais.

1. Interoperabilidade de portadora: a SONET tinha de tornar possível a rede interligada para diferentes concessionárias. A concretização desse objetivo exigia a definição de um padrão de sinalização comum, relacionado a comprimento de onda, sincronização, estrutura de enquadramento e outras questões.
2. Unificação pelas regiões: foram necessários alguns meios para unificar os sistemas digitais dos Estados Unidos, Europa e Japão, todos baseados em canais PCM de 64 kbps, mas combinados de formas diferentes (e incompatíveis).
3. Multiplexação de canais digitais: a SONET teve de proporcionar um modo de multiplexar vários canais digitais. No momento em que ela surgiu, a portadora digital de velocidade mais alta usada em todo o território dos Estados Unidos era a T3, a 44,736 Mbps. A T4 já havia sido definida, mas não era muito usada, e nada que ultrapassasse a velocidade da T4 havia sido definido. Parte da missão da SONET era dar continuidade à hierarquia até gigabits/s e proporcionar velocidades ainda maiores. Também era necessária uma forma padrão de multiplexar canais mais lentos em um canal SONET.
4. Suporte para gerenciamento: a SONET tinha de oferecer recursos de operação, administração e manutenção necessários para gerenciar a rede. Os sistemas anteriores não faziam isso muito bem.

Uma decisão inicial foi tornar a SONET um sistema TDM tradicional, com toda a largura de banda da fibra dedicada a um único canal contendo slots de tempo para os diversos subcanais. Portanto, a SONET é um sistema síncrono. Cada transmissor e receptor é ligado a um clock comum. O relógio mestre, que controla o sistema, tem uma precisão de aproximadamente uma parte em 10^9. Os bits em uma linha SONET são transmitidos a intervalos extremamente precisos, controlados pelo clock mestre.

O quadro básico da SONET é um bloco de 810 bytes, transmitido a cada 125 μs. Tendo em vista que a SONET é síncrona, os quadros são emitidos independentemente de haver ou não dados úteis a enviar. A taxa de 8.000 quadros/s corresponde exatamente à taxa de amostragem dos canais PCM utilizados em todos os sistemas de telefonia digital.

Os quadros de 810 bytes da SONET são mais bem descritos como um retângulo de bytes, com 90 colunas de largura por 9 linhas de altura. Desse modo, $8 \times 810 = 6.480$ bits são transmitidos 8 mil vezes por segundo, o que resulta em uma taxa de dados bruta de 51,84 Mbps. Esse é o canal básico da SONET, chamado **STS-1 (Synchronous Transport Signal-1**). Todos os troncos SONET são múltiplos do STS-1.

As três primeiras colunas de cada quadro são reservadas para as informações de gerenciamento do sistema, conforme ilustra a Figura 2.34. Nesse bloco, as três primeiras linhas contêm o overhead de seção, e as seis linhas seguintes contêm o overhead de linha. O overhead de seção é gerado e verificado no início e no fim de cada seção, enquanto o overhead de linha é gerado e verificado no início e no fim de cada linha.

Um transmissor SONET transmite quadros de 810 bytes em sequência, sem intervalos entre eles, mesmo quando não existem dados (e, nesse caso, ele transmite dados fictícios). Do ponto de vista do receptor, tudo o que ele vê é um fluxo de bits contínuo; assim, como saber onde começa cada quadro? A resposta é que os dois

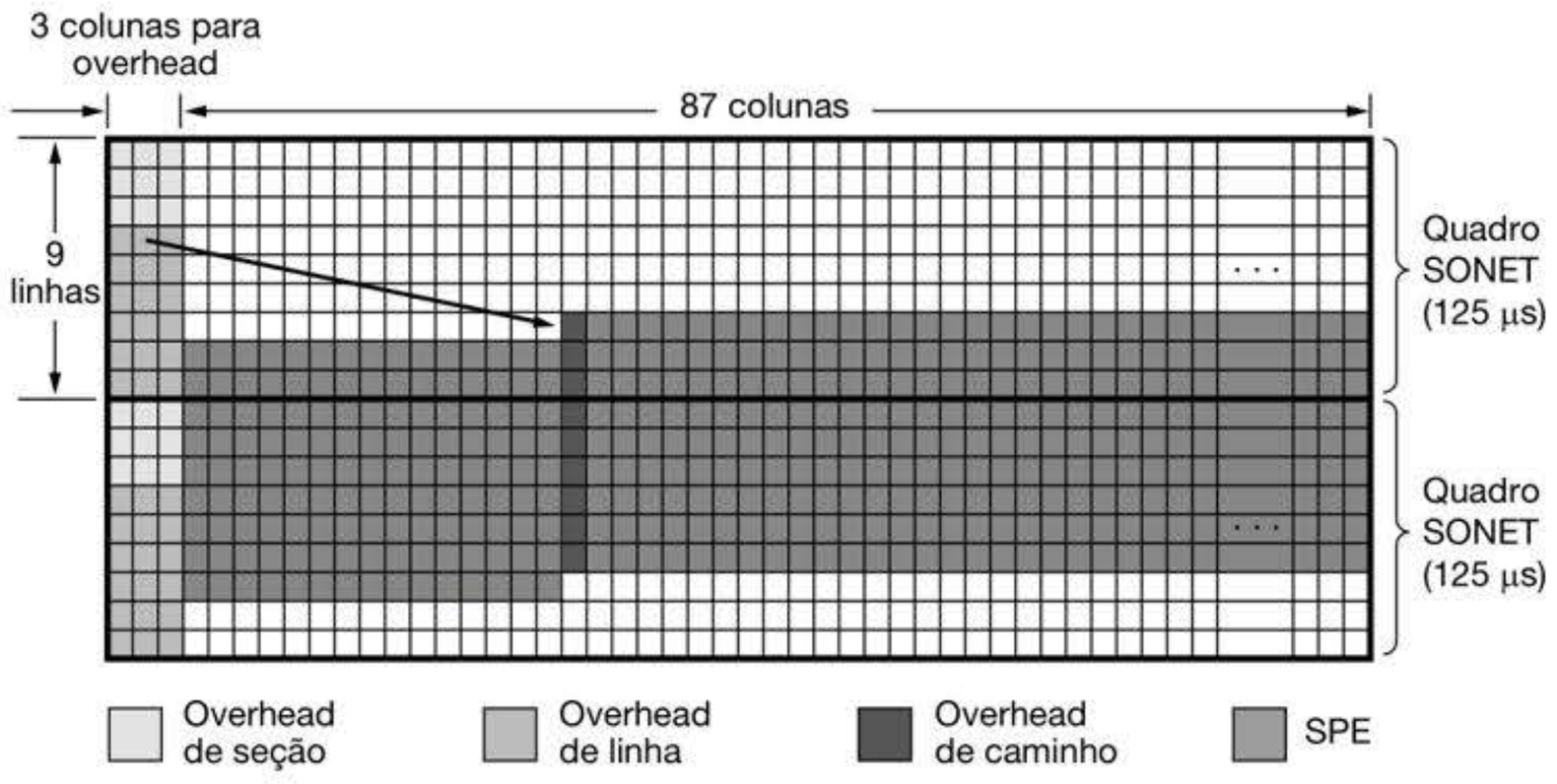


Figura 2.34 Dois quadros duplos na rede SONET.

primeiros bytes de cada quadro contêm um padrão fixo que o receptor procura. Se encontra esse padrão no mesmo lugar em um número grande de quadros consecutivos, o receptor pressupõe que está sincronizado com o transmissor. Em teoria, um usuário poderia inserir esse padrão como carga útil de uma maneira simples; porém, na prática, isso não pode ser feito em virtude da multiplexação usada por diversos usuários no mesmo quadro, além de outras razões.

As 87 colunas restantes de cada quadro contêm $87 \times 9 \times 8 \times 8.000 = 50{,}112$ Mbps de dados do usuário, os quais poderiam ser amostras de voz, T1 e outras portadoras, ou pacotes. A SONET é simplesmente um contêiner conveniente para transportar bits. O **envelope síncrono de carga útil**, ou **SPE** (**Synchronous Payload Envelope**), que transporta os dados do usuário, nem sempre começa na linha 1, coluna 4. O SPE pode começar em qualquer lugar do quadro. Um ponteiro para o primeiro byte está contido na primeira fileira do overhead de linha. A primeira coluna do SPE é o overhead de caminho (ou seja, o cabeçalho do protocolo da subcamada de caminho ponta a ponta).

A capacidade de permitir que o SPE comece em qualquer lugar dentro do quadro SONET e até mesmo se espalhe por dois quadros, como mostra a Figura 2.34, oferece flexibilidade adicional ao sistema. Por exemplo, se uma carga útil chega na origem enquanto um quadro SONET fictício está sendo construído, ele pode ser inserido no quadro atual em vez de ser mantido até o início do próximo.

A hierarquia de multiplexação da SONET/SDH é mostrada na Figura 2.35. Foram definidas taxas de STS-1 a STS-768, variando de aproximadamente uma linha T3 até 40 Gbps. Até mesmo taxas maiores certamente serão definidas com o tempo, com OC-3072 a 160 Gbps sendo o próximo na fila, se e quando isso se tornar tecnologicamente viável. A portadora óptica que corresponde a STS-*n* é chamada OC-*n*, porém sua configuração bit a bit é a mesma, exceto por uma certa reordenação de bits, necessária para sincronização. Os nomes da SDH são diferentes, começando em OC-3 porque os sistemas baseados na ITU não possuem uma taxa próxima a 51,84 Mbps. Mostramos as taxas comuns, que desenvolvem-se a partir de OC-3 em múltiplos de quatro. A taxa de dados bruta inclui todo o overhead. A taxa de dados SPE exclui o overhead de linha e o de seção. A taxa de dados do usuário exclui todos os três tipos de overhead e só considera as 86 colunas de carga útil.

Em contrapartida, quando uma portadora como a OC-3 não é multiplexada, mas transporta os dados de uma única origem, a letra *c* (significando concatenado) é acrescentada à designação; assim, OC-3 indica uma portadora de 155,52 Mbps composta por três portadoras OC-1 distintas, mas OC-3c indica um fluxo de dados de uma única origem a uma velocidade de 155,52 Mbps. Os três fluxos OC-1 contidos em um fluxo OC-3c são entrelaçados por coluna: primeiro, a coluna 1 do fluxo 1, depois a coluna 1 do fluxo 2, a coluna 1 do fluxo 3, seguida pela coluna 2 do fluxo 1, e assim por diante, resultando em um quadro com 270 colunas de largura e 9 linhas de profundidade.

2.5.4 Comutação

Do ponto de vista do engenheiro de telefonia, o sistema telefônico é dividido em duas partes principais: a planta externa (os circuitos terminais e troncos, pois eles estão localizados fisicamente fora das estações de comutação) e a planta interna (os switches, que estão dentro das estações de comutação). Acabamos de estudar a planta externa. Agora vamos examinar a planta interna.

Hoje em dia, duas técnicas de comutação diferentes são usadas pela rede: comutação de circuitos e comutação de pacotes. O sistema telefônico tradicional é baseado na comutação de circuitos, embora a tecnologia VoIP conte com a comutação de pacotes. Veremos em detalhes a comutação de circuitos, comparando-a com a comutação de pacotes. Os dois tipos de comutação são tão importantes que voltaremos a eles quando entrarmos na camada de rede.

SONET		SDH	Taxa de dados (Mbps)		
Elétrico	Óptico	Óptico	Bruto	SPE	Usuário
STS-1	OC-1		51,84	50,112	49,536
STS-3	OC-3	STM-1	155,52	150,336	148,608
STS-12	OC-12	STM-4	622,08	601,344	594,432
STS-48	OC-48	STM-16	2.488,32	2.405,376	2.377,728
STS-192	OC-192	STM-64	9.953,28	9.621,504	9.510,912
STS-768	OC-768	STM-256	39.813,12	38.486,016	38.043,648

Figura 2.35 Taxas de multiplexação da SONET e da SDH.

Comutação de circuitos

Tradicionalmente, quando você ou seu computador efetuavam uma chamada telefônica, o equipamento de comutação do sistema telefônico procurava um caminho físico desde o seu telefone até o telefone do receptor e o mantinha pela duração da chamada. Essa técnica, chamada **comutação de circuitos**, é apresentada esquematicamente na Figura 2.36(a). Cada um dos seis retângulos representa uma estação de comutação da concessionária de comunicações (estação final, estação interurbana, etc.). Nesse exemplo, cada estação tem três linhas de entrada e três linhas de saída. Quando uma chamada passa por uma estação de comutação, é estabelecida uma conexão física entre a linha que transportou a chamada e uma das linhas de saída, como mostram as linhas tracejadas.

No início da telefonia, a conexão era feita pela telefonista que conectava um cabo de ligação em ponte (jumper) aos soquetes de entrada e de saída. Na realidade, existe uma pequena história surpreendente associada à invenção do equipamento automático de comutação de circuitos. Esse dispositivo foi inventado por um agente funerário do século XIX, Almon B. Strowger. Logo depois que o telefone foi inventado, quando uma pessoa morria, alguém ligava para a telefonista do local e dizia: "Por favor, ligue-me com um agente funerário". Infelizmente para o sr. Strowger, havia dois agentes funerários em sua cidade, e a esposa do outro agente era a telefonista do local. Ele percebeu rapidamente que teria de inventar um equipamento automático de comutação telefônica ou seu negócio iria à falência. Ele escolheu a primeira opção. Por cerca de 100 anos, o equipamento de comutação de circuitos usado em todo o mundo foi conhecido como **engrenagem de Strowger**. (A história não registra se a telefonista, desempregada, conseguiu emprego como operadora de informações, respondendo a perguntas como: "Qual é o número do telefone do agente funerário?").

O modelo mostrado na Figura 2.36(a) é altamente simplificado, obviamente, porque partes do caminho físico entre os dois telefones podem de fato ser enlaces de micro-ondas ou de fibra, nos quais são multiplexadas milhares de chamadas. Entretanto, a ideia básica é válida: uma vez estabelecida uma chamada, haverá um caminho dedicado entre ambas as extremidades, e ele continuará a existir até que a chamada seja finalizada.

Uma propriedade importante da comutação de circuitos é a necessidade de se estabelecer um caminho ponta a ponta *antes* que qualquer dado possa ser enviado. O tempo decorrido entre o fim da discagem e o momento em que o telefone começa a tocar pode chegar a 10 segundos ou mais em chamadas interurbanas ou internacionais. Durante esse intervalo, o sistema telefônico procura uma conexão física, como mostra a Figura 2.37(a). Observe que, antes mesmo de se iniciar a transmissão de dados, o sinal de solicitação de chamada deve se propagar em todo o trajeto até o destino e lá ser reconhecido. Para muitas aplicações de informática (p. ex., a verificação de crédito em um ponto de venda), tempos de preparação longos são indesejáveis.

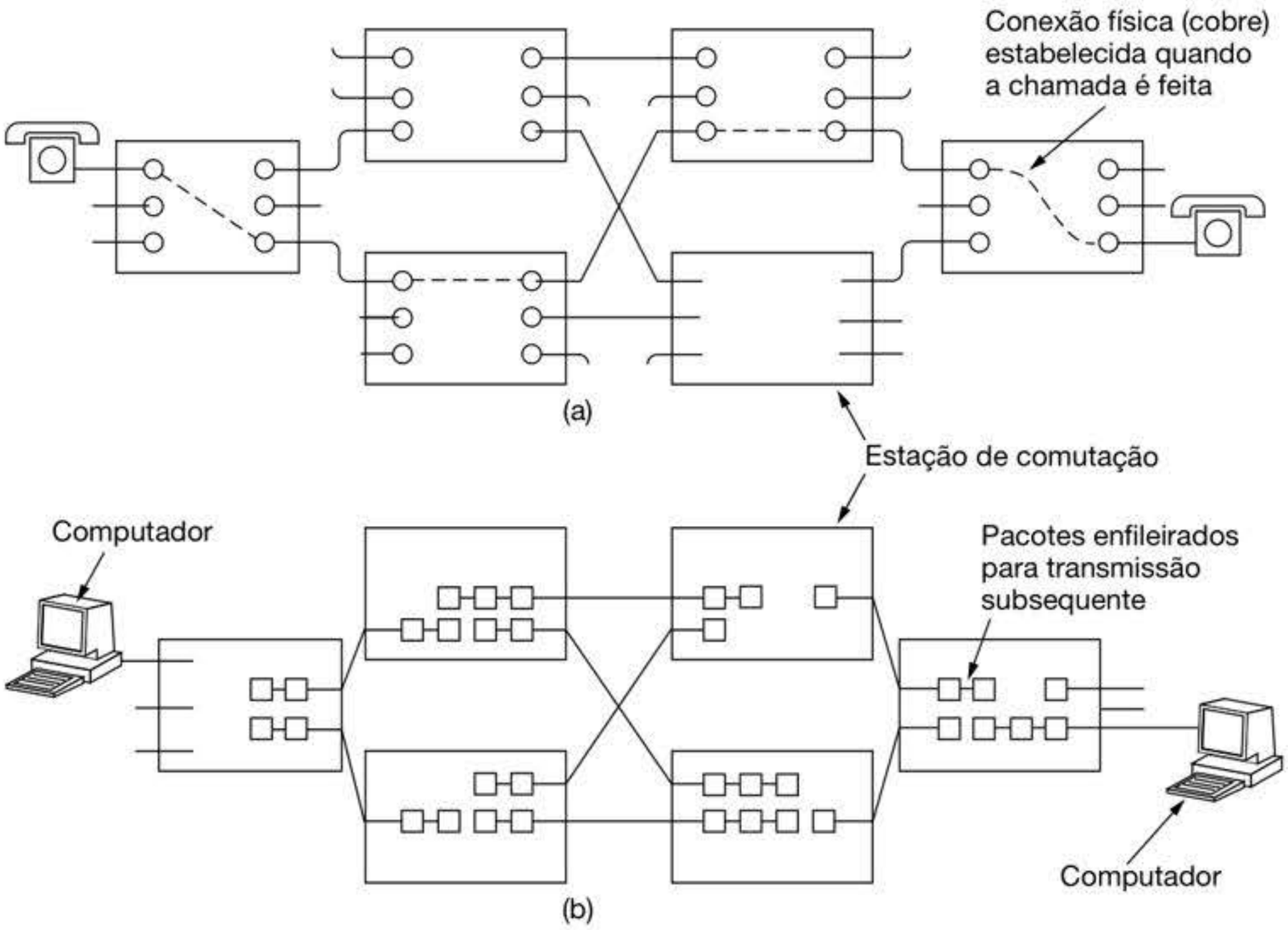


Figura 2.36 (a) Comutação de circuitos. (b) Comutação de pacotes.

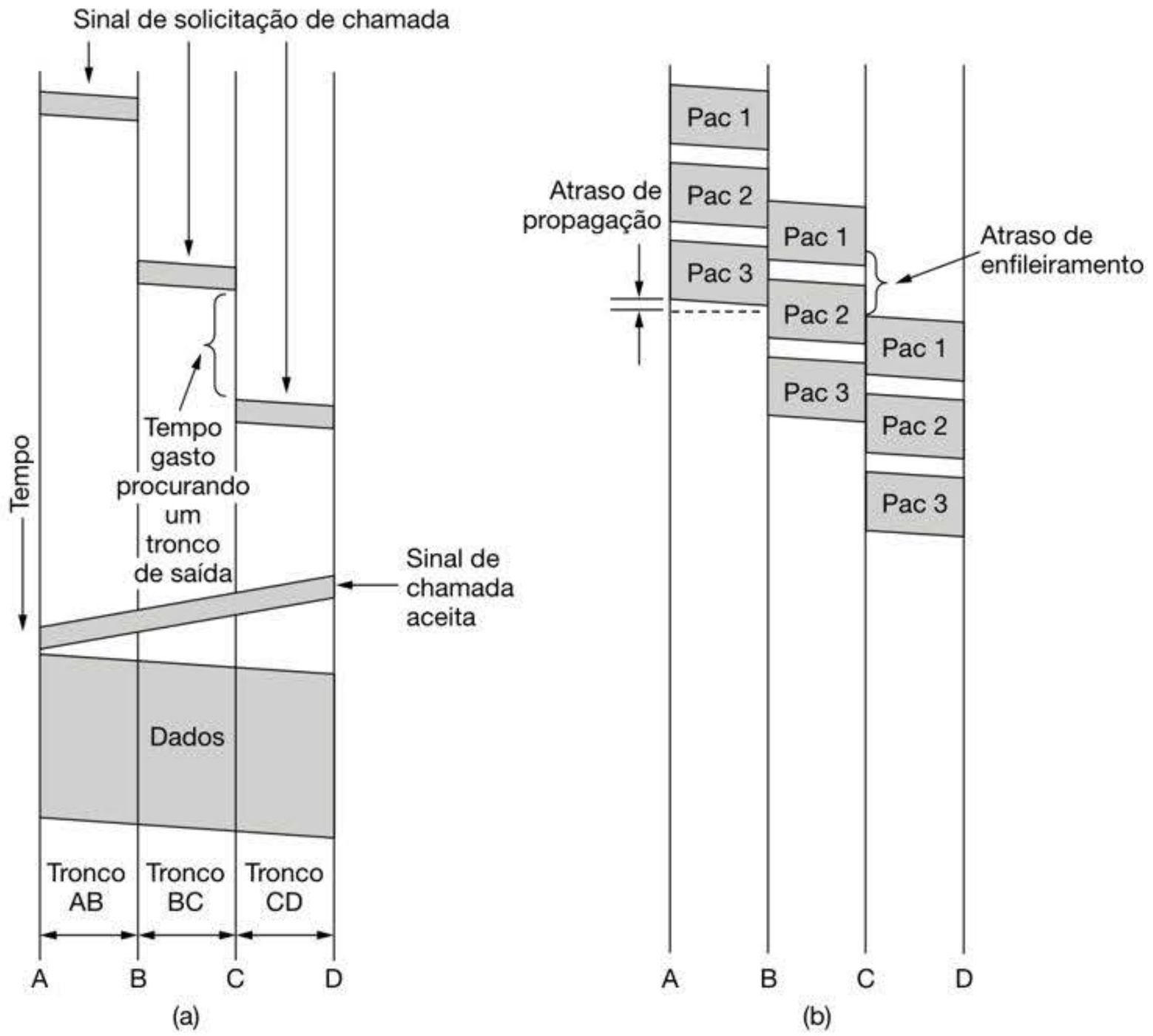


Figura 2.37 Sincronização de eventos em (a) comutação de circuitos e (b) comutação de pacotes.

Como consequência do caminho reservado entre o transmissor e o receptor da chamada, uma vez estabelecida a configuração, o único atraso para a entrega dos dados é o tempo de propagação do sinal eletromagnético, cerca de 5 ms por 1.000 km. Outra consequência do caminho estabelecido é que não há perigo de congestionamento – ou seja, quando a chamada é feita, você nunca obtém sinal de ocupado. É claro que é possível receber um sinal de ocupado antes do estabelecimento da conexão, em decorrência da falta de capacidade de comutação ou de troncos.

Comutação de pacotes

A alternativa à comutação de circuitos é a **comutação de pacotes**, mostrada na Figura 2.36(b) e descrita no Capítulo 1. Com essa tecnologia, os pacotes são enviados assim que estão disponíveis. Não é preciso estabelecer um caminho dedicado com antecedência, diferentemente da comutação de circuitos. A comutação de pacotes é semelhante a enviar uma série de cartas usando o sistema postal: cada uma segue independentemente das outras. Fica a critério dos roteadores usar a transmissão store-and-forward para enviar cada pacote em seu caminho ao destino por conta própria. Esse procedimento é diferente da comutação de circuitos, em que o resultado do estabelecimento da conexão é a reserva de largura de banda desde o transmissor até o receptor, e todos os dados no circuito seguem esse caminho. Na comutação de circuitos, fazer todos os pacotes seguirem o mesmo caminho significa que eles não poderão chegar fora de ordem. Com a comutação de pacotes, não há caminho fixo e, assim, diferentes pacotes podem seguir caminhos distintos, dependendo das condições da rede no momento em que eles são enviados. Portanto, eles podem chegar fora de ordem.

As redes de comutação de pacotes impõem um limite superior apertado sobre o tamanho dos pacotes. Isso garante que nenhum usuário poderá monopolizar qualquer linha de transmissão por muito tempo (p. ex., muitos milissegundos), de modo que as redes de comutação de pacotes podem lidar com o tráfego interativo. Isso também reduz o atraso, pois o primeiro pacote de uma mensagem longa pode ser encaminhado antes que o segundo tenha chegado por completo. Contudo, o atraso store-and-forward de acumular um pacote na memória do roteador antes que ele seja enviado para o próximo roteador excede o da comutação de circuitos. Com a comutação de circuitos, os bits simplesmente fluem pelo fio de modo contínuo. Nada é armazenado e encaminhado mais tarde.

A comutação de pacotes e de circuitos também difere de outras maneiras. Como nenhuma largura de banda é

reservada com a comutação de pacotes, estes podem ter de esperar para serem encaminhados. Se muitos pacotes forem enviados ao mesmo tempo, isso introduz o **atraso de enfileiramento** e o congestionamento. Em contrapartida, não existe perigo de obter um sinal de ocupado e não conseguir usar a rede. Assim, o congestionamento ocorre em diferentes momentos com a comutação de circuitos (no momento da configuração) e com a comutação de pacotes (quando os pacotes são enviados).

Se um circuito tiver sido reservado para determinado usuário e não houver tráfego, sua largura de banda é desperdiçada. Ela não pode ser usada para outro tráfego. A comutação de pacotes não desperdiça largura de banda e, portanto, é mais eficiente do ponto de vista do sistema. Entender essa escolha é decisivo para compreender a diferença entre comutação de circuitos e comutação de pacotes. O dilema está entre garantir serviço e desperdiçar recursos *versus* não garantir serviço e não desperdiçar recursos.

A comutação de pacotes é mais tolerante a falhas que a comutação de circuitos. De fato, é por isso que ela foi criada. Se um switch ficar inativo, todos os circuitos que o utilizam serão encerrados, e nenhum tráfego poderá mais ser transmitido em qualquer um deles. Com a comutação de pacotes, os pacotes poderão ser roteados de modo a contornar switches inativos.

Outra diferença entre a comutação de circuitos e a de pacotes é o algoritmo de tarifação. Com a comutação de circuitos (i.e., as chamadas de telefonia de voz por PSTN), a tarifação se baseava historicamente na distância e no tempo. No caso dos telefones móveis, em geral a distância não é importante, exceto para chamadas internacionais, e o tempo desempenha apenas um papel secundário (p. ex., um plano de chamadas com 2 mil minutos gratuitos custa mais que um plano com mil minutos gratuitos e, algumas vezes, chamadas noturnas ou nos finais de semana são mais econômicas que o normal). Com a comutação de pacotes, o tempo de conexão não é um problema, mas o volume de tráfego sim. Para usuários domésticos nos Estados Unidos e na Europa, os ISPs normalmente cobram uma tarifa fixa mensal, porque tal modelo é menos trabalhoso para eles e mais fácil de entender para os clientes. Em alguns países em desenvolvimento, a tarifação normalmente ainda é baseada no volume: os usuários podem comprar um "pacote de dados" de certo tamanho e usá-lo por um ciclo de cobrança. Certos horários do dia, ou mesmo certos destinos, podem não ser cobrados ou não contar para o limite de dados; esses serviços às vezes são chamados de **serviços com taxa zero**. Geralmente, os provedores de serviço de Internet da operadora no backbone da Internet cobram com base nos volumes de tráfego. Um modelo de tarifação típico é baseado no 95º percentil de amostras de 5 minutos: em um determinado enlace, um ISP medirá o volume de tráfego que passou pelo enlace nos últimos 5 minutos. Um ciclo de faturamento de 30 dias terá 8640 desses intervalos de 5 minutos e o ISP cobrará com base no 95º percentil dessas amostras. Essa técnica costuma ser chamada de **cobrança pelo percentil 95**.

As diferenças entre comutação de circuitos e comutação de pacotes estão resumidas na Figura 2.38. Tradicionalmente, as redes telefônicas têm usado a comutação de circuitos para oferecer chamadas telefônicas de alta qualidade, e as redes de computadores têm usado a comutação de pacotes por suas simplicidade e eficiência. Contudo, existem exceções dignas de nota. Algumas redes de computadores mais antigas têm sido comutadas por circuitos "por debaixo dos panos" (p. ex., X.25), e algumas redes telefônicas mais novas usam a comutação de pacotes com a tecnologia VoIP. Para os usuários, isso se parece externamente com uma chamada telefônica padrão, mas, internamente, os pacotes na rede com dados de voz são comutados. Com essa técnica, as chamadas internacionais com cartões de chamada

Item	Comutação de circuitos	Comutação de pacotes
Configuração de chamadas	Obrigatória	Não necessária
Caminho físico dedicado	Sim	Não
Cada pacote segue a mesma rota	Sim	Não
Os pacotes chegam em ordem	Sim	Não
A falha de um switch é fatal	Sim	Não
Largura de banda disponível	Fixa	Dinâmica
Momento de possível congestionamento	Durante a configuração	Em todos os pacotes
Largura de banda potencialmente desperdiçada	Sim	Não
Transmissão store-and-forward	Não	Sim
Tarifação	Por minuto	Por byte

Figura 2.38 Comparação entre redes de comutação de circuitos e redes de comutação de pacotes.

podem se tornar mais baratas, embora talvez com uma qualidade de chamada inferior à do serviço tradicional.

2.6 REDES DE TELEFONIA MÓVEL

Mesmo que um dia o sistema de telefonia convencional receba fibra multigigabit de ponta a ponta, as pessoas agora esperam efetuar chamadas telefônicas e usar seus telefones para verificar e-mail e navegar pela Web em aviões, carros, piscinas e enquanto caminham no parque. Consequentemente, há um enorme interesse (e investimento) na telefonia sem fio.

O sistema de telefonia móvel é usado para comunicação remota de voz e dados. Os **telefones móveis** (também chamados **telefones celulares**) passaram por cinco gerações distintas, normalmente chamadas 1G, 2G, 3G, 4G e 5G. As três gerações iniciais ofereciam voz analógica, voz digital e voz digital e dados (Internet, e-mail, etc.), respectivamente. A tecnologia 4G acrescenta outras capacidades, incluindo técnicas adicionais de transmissão pela camada física (p. ex., transmissões uplink por OFDM) e femtocélulas baseadas em IP (nós de celulares domésticos que são conectados à infraestrutura de Internet da linha fixa). A tecnologia 4G não admite a telefonia de comutação de circuitos, diferentemente de suas predecessoras, é baseada apenas na comutação de pacotes. A tecnologia 5G está sendo implantada agora, mas serão necessários muitos anos antes que ela substitua completamente as gerações anteriores em todos os lugares. A tecnologia 5G aceitará transmissões de até 20 Gbps, bem como implantações mais densas. Há também um foco na redução da latência da rede para dar suporte a uma maior gama de aplicações, como para jogos altamente interativos.

2.6.1 Conceitos comuns: células, handoff, paging

Em todos os sistemas de telefonia móvel, uma região geográfica é dividida em **células**, e é esse o motivo pelo qual esses dispositivos são chamados telefones celulares. Cada célula utiliza algum conjunto de frequências não utilizado por qualquer uma das células vizinhas. A ideia fundamental que dá aos sistemas celulares uma capacidade muito maior que a dos sistemas anteriores é o uso de células relativamente pequenas e a reutilização de frequências de transmissão em células próximas (mas não adjacentes). O projeto do celular aumenta a capacidade do sistema enquanto as células são menores. Além disso, células menores significam menor necessidade de energia, o que possibilita a existência de dispositivos transmissores e receptores menores e mais econômicos.

As células permitem a reutilização de frequências, que é ilustrada na Figura 2.39(a). Em geral, as células são razoavelmente circulares, porém, é mais simples representá-las como hexágonos. Na Figura 2.39(a), todas as células têm o mesmo tamanho. Elas são agrupadas em unidades de sete células. Cada letra indica um grupo de frequências. Observe que, para cada conjunto de frequências, existe um afastamento de aproximadamente duas células de extensão, no qual essa frequência não é reutilizada, o que proporciona boa separação e pouca interferência.

Em uma área em que o número de usuários cresce a ponto de o sistema ficar sobrecarregado, a potência pode ser reduzida, e as células sobrecarregadas são divididas em células menores, chamadas **microcélulas**, para permitir maior reutilização de frequências, como mostra a Figura 2.39(b). Algumas vezes, as empresas de telefonia criam microcélulas temporárias, utilizando torres portáteis com enlaces de satélite a fim de atender à demanda de eventos esportivos, shows de rock e outros eventos nos quais um

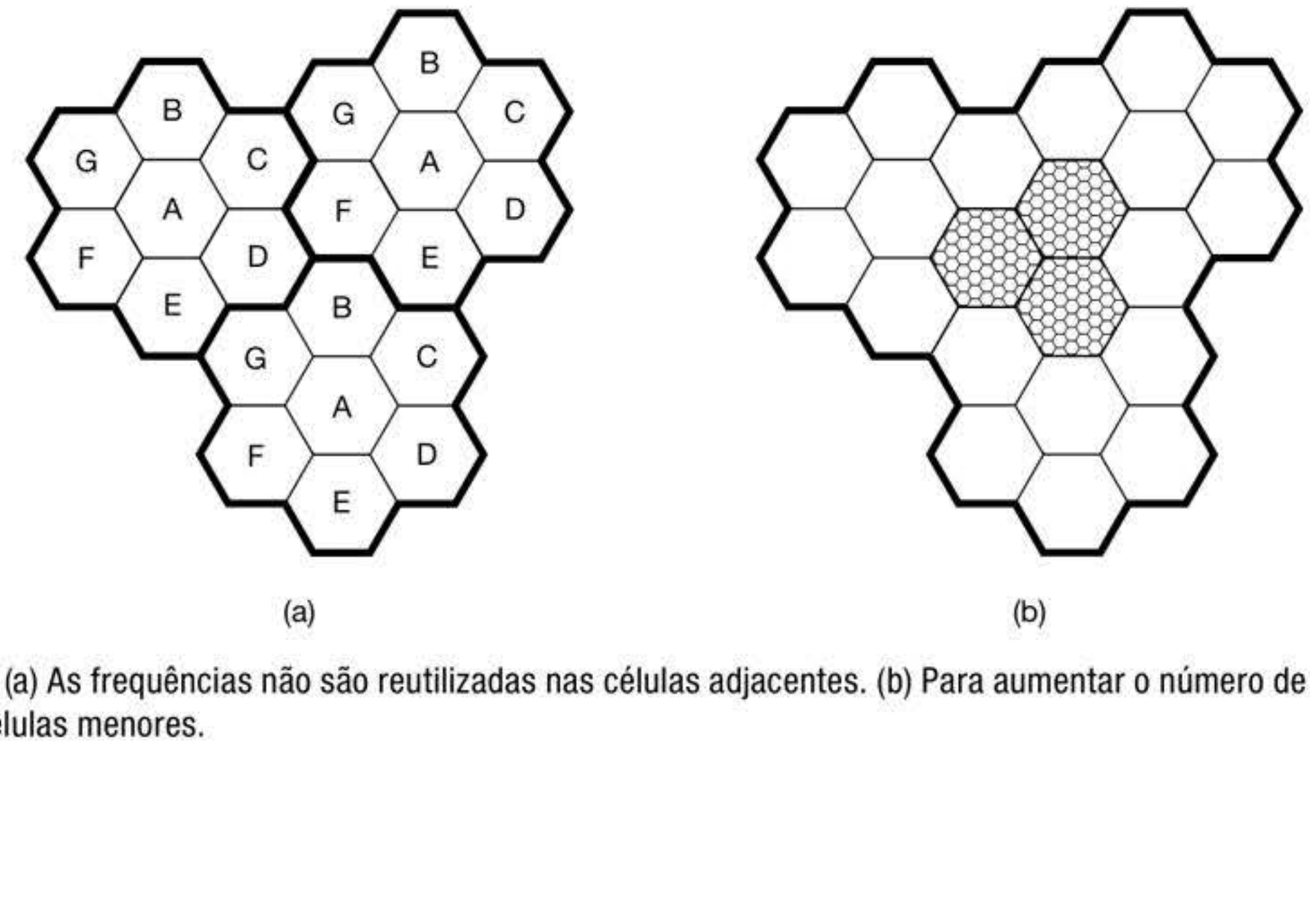


Figura 2.39 (a) As frequências não são reutilizadas nas células adjacentes. (b) Para aumentar o número de usuários, podem ser utilizadas células menores.

grande número de usuários de telefones celulares se reúne por algumas horas.

No centro de cada célula há uma estação-base que recebe as transmissões de todos os telefones presentes na célula. Essa estação consiste em um computador e um transmissor/receptor, conectados a uma antena. Em um sistema de pequeno porte, todas as estações-base estão conectadas a um único dispositivo, chamado **centro de comutação móvel**, ou **MSC** (**Mobile Switching Center**). Em um sistema maior, podem ser necessários vários MSCs, todos conectados a um MSC de segundo nível, e assim por diante. Basicamente, os MSCs são estações finais, como no sistema telefônico, e na verdade estão conectados a pelo menos uma estação final de um sistema telefônico. Os MSCs se comunicam com as estações-base, entre si e com a PSTN usando uma rede de comutação de pacotes.

Em qualquer instante, cada telefone móvel logicamente ocupa uma célula específica e está sob o controle da estação-base dela. Quando um telefone móvel deixa fisicamente uma célula, sua estação-base detecta que o sinal do telefone está enfraquecendo e questiona todas as estações-base vizinhas quanto à quantidade de energia que elas estão recebendo dele. Quando as respostas retornam, a estação-base faz a transferência para a célula que está obtendo o sinal mais forte; quase sempre, essa é a célula em que o telefone está localizado no momento. O telefone é, então, informado de quem é o seu novo chefe e, se houver uma chamada em andamento, ele será solicitado a passar para um novo canal (porque o antigo não é reutilizado em nenhuma das células adjacentes). Esse processo é chamado de **handoff** e leva cerca de 300 ms. A atribuição de canais é feita pelo MSC, o centro nervoso do sistema. Na verdade, as estações-base são apenas simples retransmissoras de rádio.

Encontrar locais altos para instalar antenas de estação-base é uma questão fundamental. Esse problema levou algumas concessionárias de telecomunicações a fazer alianças com a Igreja Católica Romana, que possui um número significativo de locais apropriados para a instalação de antenas em todo o mundo, todos convenientemente controlados por uma única administração.

As redes celulares normalmente têm quatro tipos de **canais**. **Canais de controle** (da base para a unidade móvel) são usados para gerenciar o sistema. **Canais de localização** (da base para a unidade móvel) alertam os usuários móveis a respeito de chamadas destinadas a eles. **Canais de acesso** (bidirecionais) são usados para estabelecimento de chamadas e atribuição de canais. Por fim, os **canais de dados** (bidirecionais) transportam voz, fax ou dados.

2.6.2 Tecnologia de primeira geração (1G): voz analógica

Vamos examinar a tecnologia da rede celular, começando pelo sistema mais antigo. Os radiotelefones móveis eram usados esporadicamente na comunicação militar e marítima, durante as primeiras décadas do século XX. Em 1946, foi criado em St. Louis, Estados Unidos, o primeiro sistema para telefones baseado em automóveis. O sistema utilizava um único transmissor grande no topo de um alto edifício e tinha um único canal, usado para transmissões e recepções. Para conversar, o usuário tinha de apertar um botão que ativava o transmissor e desativava o receptor. Tais sistemas, conhecidos como **sistemas "apertar para falar"** (**push-to-talk systems**), foram instalados em diversas cidades a partir da década de 1950. Táxis e carros de polícia utilizam essa tecnologia com frequência.

Na década de 1960, foi instalado o **sistema de telefonia móvel aperfeiçoado**, ou **IMTS** (**Improved Mobile Telephone System**). Ele também utilizava um transmissor de alta potência (200 watts) no topo de uma montanha, mas agora tinha duas frequências: uma para transmissão e outra para recepção. Por isso, o botão "apertar para falar" não era mais necessário. Como toda a comunicação dos telefones móveis utilizava um canal para transmissão e outro para recepção dos sinais, os usuários móveis não podiam ouvir uns aos outros (ao contrário do que acontecia com o sistema "apertar para falar" utilizado em táxis).

O IMTS admitia 23 canais espalhados pelas frequências de 150 a 450 MHz. Em virtude do pequeno número de canais, muitas vezes os usuários tinham de esperar muito tempo antes de obter um tom de discagem. Além disso, pela alta potência do transmissor, os sistemas adjacentes tinham de estar a vários quilômetros de distância uns dos outros para evitar interferência. Em suma, sua capacidade limitada tornou o sistema impraticável.

O **sistema avançado de telefonia móvel**, ou **AMPS** (**Advanced Mobile Phone System**), bastante semelhante, inventado pelo Bell Labs e instalado inicialmente nos Estados Unidos em 1983, aumentou significativamente a capacidade da rede celular. Ele também foi usado na Inglaterra, onde recebeu o nome TACS, e no Japão, onde foi chamado MCS-L1. O uso do AMPS foi encerrado formalmente em 2008, mas vamos examiná-lo para entender o contexto para os sistemas 2G e 3G, que o aperfeiçoaram. No AMPS, as células normalmente ficam afastadas por 10 a 20 km; em sistemas digitais, as células são menores. Enquanto um sistema IMTS cobrindo 100 km só pode ter uma chamada em cada frequência, um sistema AMPS poderia ter 100 células de 10 km na mesma área, podendo ter de 10 a 15 chamadas em cada frequência, em células bastante separadas.

O AMPS utiliza FDM para separar os canais e usa 832 canais full-duplex, cada um consistindo em um par de canais simplex. Esse arranjo é conhecido como **duplex por divisão de frequência**, ou **FDD** (**Frequency Division Duplex**). Os 832 canais simplex de 824 a 849 MHz são usados para transmissão do aparelho móvel à estação-base, e 832 canais simplex de 869 a 894 MHz são usados para

transmissão da estação-base ao aparelho móvel. Cada um desses canais simplex tem 30 kHz de largura.

Os 832 canais no AMPS estão divididos em quatro categorias. Como as mesmas frequências não podem ser reutilizadas em células vizinhas e 21 desses canais são reservados em cada célula para controle, o número real de canais de voz disponíveis por célula é bem menor que 832, normalmente em torno de 45.

Gerenciamento de chamadas

Cada telefone móvel tem um número de série de 32 bits e um número de telefone de dez dígitos em sua PROM (memória programável somente de leitura). O número de telefone é representado como um código de área de 3 dígitos em 10 bits e um número de assinante de 7 dígitos em 24 bits. Quando um telefone é contatado, varre uma lista pré-programada de 21 canais de controle até encontrar o sinal mais forte. Em seguida, o telefone transmite seu número de série de 32 bits e o número de telefone de 34 bits. A exemplo de todas as outras informações de controle do AMPS, esse pacote é enviado várias vezes em formato digital e com um código de correção de erros, apesar de os próprios canais de voz serem analógicos.

Quando ouve a mensagem, a estação-base avisa ao MSC, que registra a existência de seu novo cliente e também informa a sua localização atual ao MSC local. Durante a operação normal, o telefone móvel repete o registro uma vez a cada 15 minutos, aproximadamente.

Para fazer uma chamada, o usuário móvel liga o telefone, digita no teclado o número a ser chamado (pelo menos, conceitualmente) e pressiona o botão LIGAR. Em seguida, o telefone transmite o número a ser chamado e sua própria identidade no canal de acesso. Se houver uma colisão, ele tenta novamente mais tarde. Ao receber a solicitação, a estação-base informa ao MSC. Se o chamador for um cliente da empresa do MSC (ou de uma de suas parceiras), o MSC procura um canal disponível para a chamada. Se encontrar algum, o número do canal será enviado de volta no canal de controle. Em seguida, o telefone móvel se conecta automaticamente ao canal de voz selecionado e aguarda até que a parte chamada atenda ao telefone.

As chamadas recebidas funcionam de forma diferente. Para começar, todos os telefones inativos ouvem continuamente o canal de localização para detectar as mensagens destinadas a eles. Quando é feita uma chamada para um telefone móvel (a partir de um telefone fixo ou de outro telefone móvel), um pacote é enviado ao MSC local do telefone chamado, para que ele seja localizado. Em seguida, é enviado um pacote à estação-base em sua célula atual, que, então, envia um pacote de difusão no canal de localização com o formato: "Unidade 14, você esta aí?". O telefone chamado responde "Sim" no canal de acesso. Depois, a base transmite algo como: "Unidade 14, chamada para você no canal 3". Nesse momento, o telefone chamado se conecta ao canal 3 e começa a emitir sinais sonoros (ou a tocar alguma melodia que o proprietário do telefone ganhou como presente de aniversário).

2.6.3 Tecnologia de segunda geração (2G): voz digital

A primeira geração de telefones celulares era analógica; a segunda geração é digital. A troca para digital tem diversas vantagens. Ela oferece ganhos de capacidade, permitindo que os sinais de voz sejam digitalizados e compactados. Ela melhora a segurança, permitindo que sinais de voz e de controle sejam criptografados. Isso, por sua vez, impede fraude e espionagem, seja por varredura intencional, seja por ecos de outras chamadas, em virtude da propagação de ondas de rádio. Por fim, ela capacita novos serviços, como mensagens de texto.

Assim como não havia padronização internacional durante a primeira geração, também não havia padronização internacional durante a segunda. Vários sistemas diferentes foram desenvolvidos, e três foram amplamente implementados. O **sistema avançado de telefonia móvel digital**, ou **D-AMPS (Digital Advanced Mobile Phone System)** é uma versão digital do AMPS que coexiste com ele e usa TDM para fazer várias chamadas no mesmo canal de frequência. Ele é descrito no padrão internacional IS-54 e seu sucessor, o IS-136. O **sistema global para comunicações móveis**, ou **GSM (Global System for Mobile Communications)** apareceu como o sistema dominante, e, embora tenha demorado para ser aceito nos Estados Unidos, agora é usado praticamente em todo o mundo. Assim como o D-AMPS, o GSM é baseado em uma mistura de FDM e TDM. O **acesso múltiplo por divisão de código**, ou **CDMA (Code Division Multiple Access)**, descrito no **padrão internacional IS-95**, é um tipo de sistema completamente diferente, que não é baseado nem em FDM nem em TDM. Embora o CDMA não tenha se tornado o sistema 2G dominante, sua tecnologia se tornou a base para os sistemas 3G.

Além disso, o nome **serviços de comunicações pessoais**, ou **PCS (Personal Communications Services)** às vezes é usado na literatura de marketing para indicar um sistema de segunda geração (ou seja, digital). Inicialmente, isso indicava um telefone móvel usando a banda de 1.900 MHz, mas essa distinção raramente é feita nos dias atuais. O sistema 2G dominante em quase todo o mundo é o GSM, que descrevemos em seguida com detalhes.

2.6.4 GSM: Global System for Mobile Communications

O GSM surgiu na década de 1980 como um esforço para produzir um único padrão 2G europeu. A tarefa foi atribuída a um grupo de telecomunicações chamado (em

francês) Groupe Specialé Mobile. Os primeiros sistemas GSM foram implantados a partir de 1991 e experimentaram um sucesso repentino. Logo, ficou claro que o GSM seria mais do que um sucesso europeu, sendo absorvido até mesmo em países como a Austrália, de modo que o sistema foi renomeado para que tivesse um apelo mais global.

O GSM e outros sistemas de telefonia móvel que estudaremos retêm, dos sistemas 1G, um projeto baseado em células, reutilização de frequência pelas células e mobilidade com handoffs à medida que os assinantes se movem. São os detalhes que diferem. A seguir, descreveremos algumas das principais propriedades do GSM. Entretanto, o padrão GSM impresso tem mais de 5.000 [sic] páginas. Uma grande fração desse material se relaciona aos aspectos de engenharia do sistema, em especial ao projeto dos receptores para tratar da propagação de sinais por vários caminhos, e a sincronização de transmissores e receptores. Nada disso será mencionado aqui.

A Figura 2.40 mostra que a arquitetura do GSM é semelhante à arquitetura do AMPS, embora os componentes tenham nomes diferentes. O próprio aparelho móvel agora é dividido em um aparelho e em um chip removível, com informações do assinante e da conta contidas em um **cartão SIM**, uma abreviação de **Subscriber Identity Module (módulo de identidade do assinante)**. É o cartão SIM que ativa o aparelho e contém segredos que permitem que o aparelho e a rede se identifiquem e codifiquem as conversas. Um cartão SIM pode ser removido e colocado em um aparelho diferente, para que este se torne seu aparelho móvel em relação à rede.

O telefone móvel fala com as estações-base da célula por uma **interface com o ar**, que descreveremos em breve. As estações-base da célula estão conectadas a um **controlador de estação-base**, ou **BSC (Base Station Controller)**, que controla os recursos de rádio das células e cuida do handoff. O BSC, por sua vez, está conectado a um MSC (como no AMPS), que direciona as chamadas e as conecta à rede de telefonia pública comutada, ou PSTN.

Para direcionar as chamadas, o MSC precisa saber onde os aparelhos podem ser encontrados atualmente. Ele mantém um banco de dados dos aparelhos nas vizinhanças, que estão associados às células que ele controla. Esse banco de dados é chamado **registrador de local do visitante**, ou **VLR (Visitor Location Register)**. Também há um banco de dados na rede móvel que indica o último local conhecido de cada aparelho, chamado **registrador de local inicial**, ou **HLR (Home Location Register)**, usado para direcionar as chamadas que chegam para os locais corretos. Os dois bancos de dados devem ser mantidos atualizados enquanto os aparelhos passam de uma célula para outra.

Agora, vamos descrever a interface com o ar em alguns detalhes. O GSM trabalha em uma faixa de frequências internacional, incluindo 900, 1.800 e 1.900 MHz. Foi alocado um espectro maior que o AMPS, para dar suporte a um número muito maior de usuários. O GSM é um sistema celular duplex por divisão de frequência, como o AMPS. Ou seja, cada aparelho móvel transmite em uma frequência e recebe em outra mais alta (55 MHz mais alta para GSM, contra 80 MHz mais alta para AMPS). Contudo, diferentemente do AMPS, com o GSM, um único par de frequências é dividido pela TDM em slots de tempo. Desse modo, ele é compartilhado por vários aparelhos.

Para lidar com vários aparelhos, os canais GSM são muito mais largos que os canais AMPS (200 kHz contra 30 kHz). Um canal de 200 kHz é representado na Figura 2.41. Um sistema GSM operando na região de 900 MHz tem 124 pares de canais simplex. Cada canal simplex tem 200 kHz de largura e aceita oito conexões separadas nele, usando a TDM. Cada estação atualmente ativa recebe um slot de tempo em um par de canais. Teoricamente, 992 canais podem ser aceitos em cada célula, mas muitos deles não estão disponíveis, a fim de evitar conflitos de frequência com células vizinhas. Na Figura 2.41, todos os oito slots de tempo sombreados pertencem à mesma conexão, quatro deles em cada direção. A transmissão e a recepção não acontecem no mesmo slot de tempo, pois os rádios GSM não podem transmitir e receber ao mesmo tempo, e leva algum tempo para passar de um para o outro. Se a unidade móvel atribuída à faixa de 890,4/935,4 MHz e ao slot de tempo 2 quisesse transmitir algo para a estação-base, ela usaria os quatro slots sombreados inferiores (e os slots depois deles no tempo), inserindo alguns dados em cada um até que todos os dados fossem enviados.

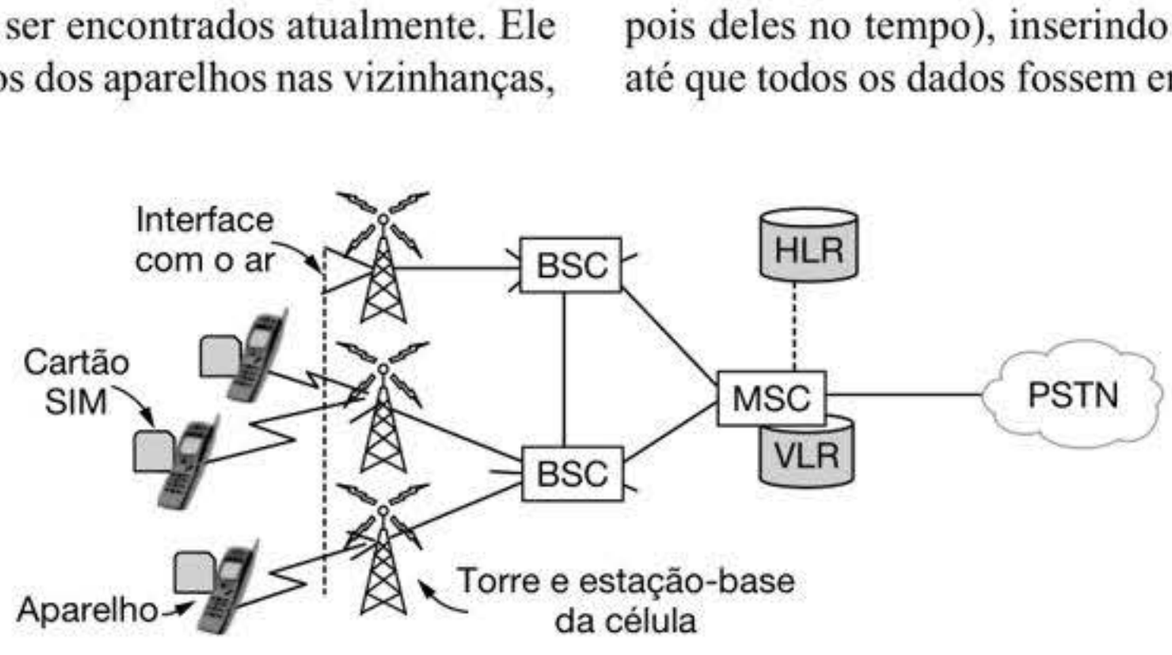


Figura 2.40 Arquitetura móvel da rede GSM.

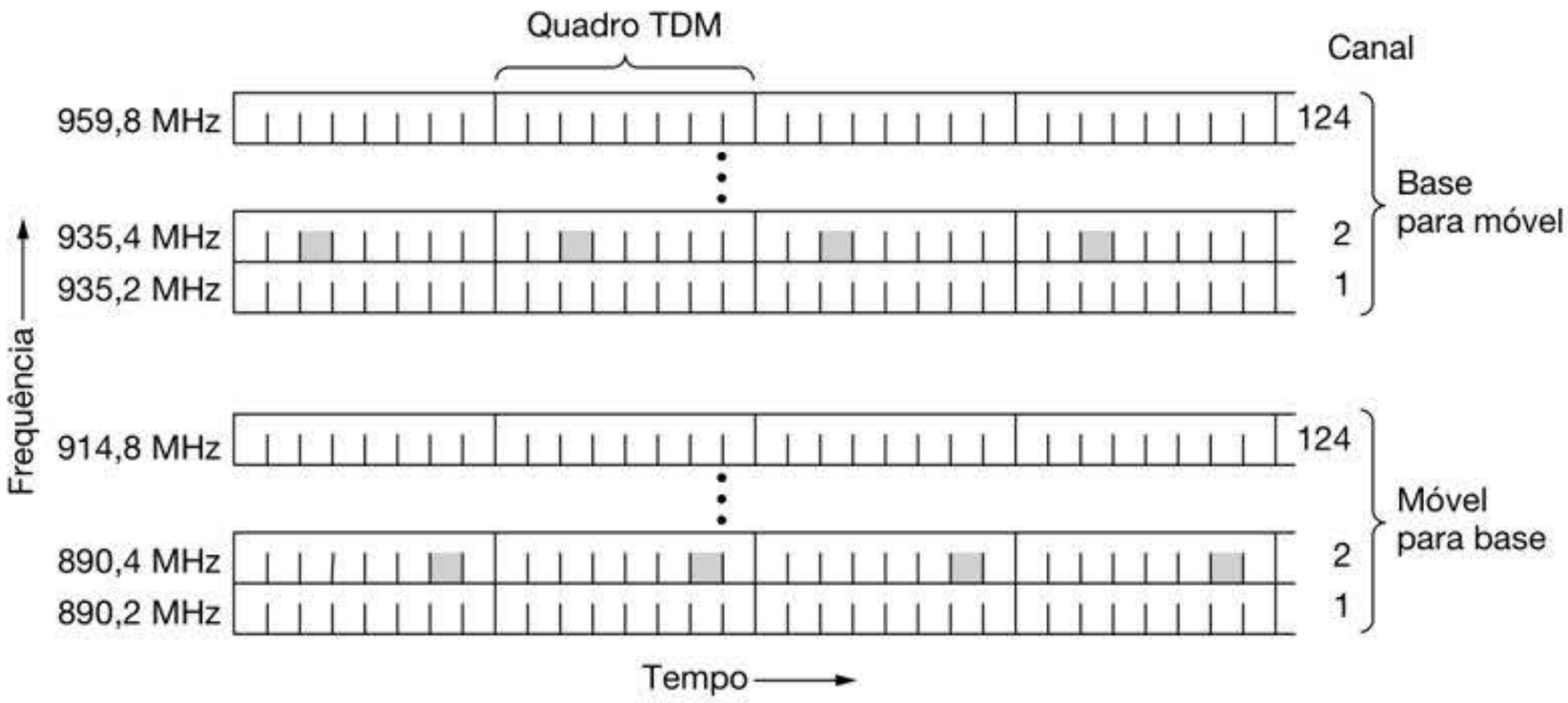


Figura 2.41 O sistema GSM utiliza 124 canais de frequência, cada um usando um sistema TDM de oito slots.

Os slots TDM mostrados na Figura 2.41 fazem parte de uma complexa hierarquia de enquadramento. Cada slot TDM tem uma estrutura específica, e grupos de slots TDM formam multiquadros, também com uma estrutura específica. Uma versão simplificada dessa hierarquia é mostrada na Figura 2.42. Aqui, podemos ver que cada slot TDM consiste em um quadro de dados de 148 bits que ocupa o canal por 577 μs (incluindo um tempo de proteção de 30 μs depois de cada slot). Cada quadro de dados começa e termina com três bits 0, para fins de delineação de quadros. Ele também contém dois campos *Informação* de 57 bits, cada um com um bit de controle que indica se o campo *Informação* seguinte se refere a voz ou a dados. Entre os campos *Informação* há um campo (de treinamento) *Sync* de 26 bits, usado pelo receptor para realizar a sincronização até os limites de quadro do transmissor.

Um quadro de dados é transmitido em 547 μs, mas um transmissor só pode enviar um quadro de dados a cada 4,615 ms, pois ele está compartilhando o canal com sete outras estações. A taxa bruta de cada canal é de 270.833 bps, dividida entre oito usuários. Porém, como ocorre com o AMPS, o overhead consome uma grande fração da largura de banda, deixando, em última análise, 24,7 kbps de carga útil por usuário antes da correção de erros. Após essa correção, restam 13 kbps para voz. Embora isso seja substancialmente menor que os 64 kbps do PCM para sinais de voz não compactados na rede de telefonia fixa, a compactação no dispositivo móvel pode alcançar esses níveis com pouca perda de qualidade.

Como podemos ver na Figura 2.42, oito quadros de dados formam um quadro TDM, e 26 quadros TDM formam um multiquadro de 120 ms. Dos 26 quadros TDM em um multiquadro, o slot 12 é usado para controle e o slot 25 é reservado para uso futuro; assim, somente 24 estão disponíveis para tráfego do usuário.

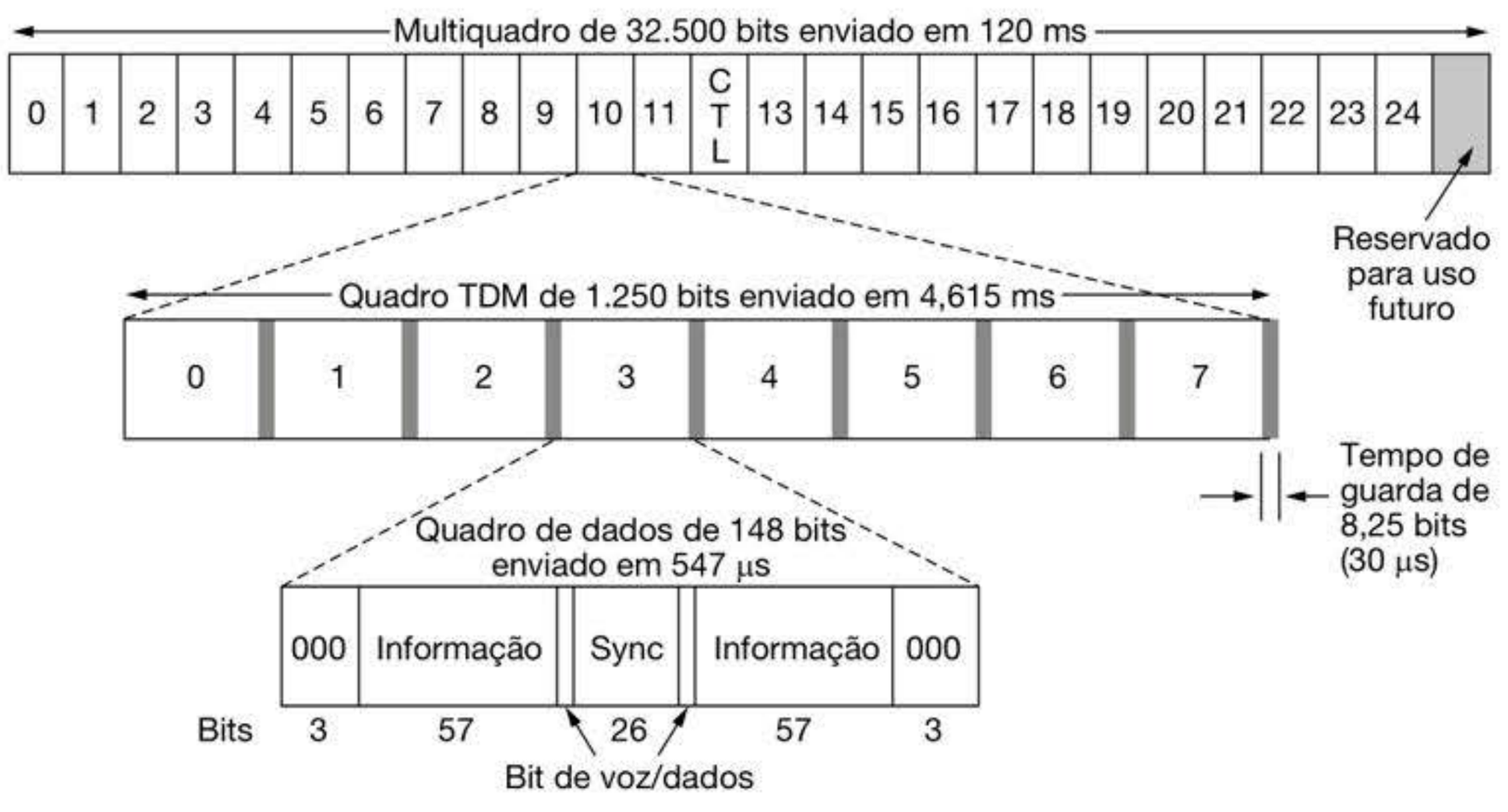


Figura 2.42 Uma parte da estrutura de enquadramento GSM.

Porém, além do multiquadro de 26 slots mostrado na Figura 2.42, também é usado um multiquadro de 51 slots (não mostrado). Alguns desses slots são empregados para guardar diversos canais de controle usados para gerenciar o sistema. O **canal de controle de broadcast** é um fluxo contínuo de saída da estação-base, contendo a identidade da estação-base e o status do canal. Todas as estações móveis monitoram a intensidade de seu sinal para verificar quando elas são transferidas para uma nova célula.

O **canal de controle dedicado** é usado para atualização de local, registro e estabelecimento de chamadas. Em particular, cada estação-base mantém o VLR, um banco de dados das estações móveis que atualmente estão sob sua jurisdição. As informações necessárias para manter o VLR são enviadas no canal de controle dedicado.

Por fim, existe o **canal de controle comum**, dividido em três subcanais lógicos. O primeiro deles é o **canal de localização**, que a estação-base utiliza para anunciar as chamadas recebidas. Cada estação móvel monitora continuamente esse canal para verificar se há chamadas a que ela deva responder. O segundo é o **canal de acesso aleatório**, que permite aos usuários solicitarem um slot no canal de controle dedicado. Se duas solicitações colidirem, elas serão adulteradas e terão de ser repetidas mais tarde. Usando o slot do canal de controle dedicado, a estação pode estabelecer uma chamada. O slot atribuído é anunciado no terceiro subcanal, o **canal de concessão de acesso**.

Por último, o GSM difere do AMPS no modo como o handoff é tratado. No AMPS, o MSC o controla totalmente, sem ajuda dos dispositivos móveis. Com os slots de tempo no GSM, na maior parte das vezes o dispositivo móvel não está nem enviando nem recebendo. Os slots ociosos são uma oportunidade para o dispositivo móvel medir a qualidade do sinal até outras estações-base nas proximidades. Ele faz isso e envia essa informação ao BSC, que pode utilizá-la para determinar quando um dispositivo móvel está saindo de uma célula e entrando em outra, de modo que possa realizar o handoff. Esse projeto é conhecido como **handoff auxiliado pela unidade móvel**, ou **MAHO (Mobile Assisted HandOff)**.

2.6.5 Tecnologia de terceira geração (3G): voz e dados digitais

A primeira geração de telefones móveis foi a voz analógica, e a segunda foi a voz digital. A terceira geração de telefones móveis, ou **3G**, como é chamada, trata de voz *e* dados digitais. Diversos fatores direcionaram o setor para a tecnologia 3G. Primeiro, na época do 3G, o tráfego de dados começou a ultrapassar o tráfego de voz na rede fixa; tendências semelhantes começaram a surgir para dispositivos móveis. Segundo, os serviços de telefone, Internet e vídeo começaram a convergir. O aumento do uso de smartphones, a partir do iPhone da Apple, que foi lançado em 2007, acelerou a mudança para dados móveis. Os volumes de dados estão subindo bastante com a popularidade dos iPhones. Quando o iPhone foi lançado, ele usava uma rede **2,5G** (basicamente uma rede 2G melhorada), que não tinha capacidade de dados suficiente. Os usuários do iPhone, ávidos por dados, impulsionaram ainda mais a transição para tecnologias 3G, para dar suporte a velocidades de transmissão de dados mais altas. Um ano depois, em 2008, a Apple lançou uma versão atualizada de seu iPhone, que podia usar a rede de dados 3G.

Inicialmente, as operadoras deram passos pequenos em direção ao 3G, chegando ao que se costuma chamar de **2,5G**. Um sistema desse tipo utiliza **taxas de dados aprimoradas para evolução do GSM**, ou **EDGE (Enhanced Data Rates for GSM Evolution)**, que é simplesmente o GSM com mais bits por símbolo. O problema é que mais bits por símbolo também significa mais erros por símbolo, e assim o EDGE tem nove esquemas diferentes para modulação e correção de erros, que se distinguem pela proporção da largura de banda dedicada à correção dos erros introduzidos pela velocidade mais alta. Trata-se de um passo em direção a um caminho evolutivo que é definido de GSM a outras tecnologias 3G que discutiremos nesta seção.

A ITU tentou ser um pouco mais específica em relação à visão do 3G em 1992. Ela apresentou um projeto para alcançá-lo, denominado **IMT-2000**, em que IMT significava **International Mobile Telecommunications (telecomunicações móveis internacionais)**. Os serviços básicos que a rede IMT-2000 deveria oferecer a seus usuários eram:

1. Transmissão de voz de alta qualidade.
2. Serviço de mensagens (substituindo e-mail, fax, SMS, bate-papo, etc.).
3. Multimídia (reprodução de música, exibição de vídeos, filmes, televisão, etc.).
4. Acesso à Internet (navegação na Web, incluindo páginas com áudio e vídeo).

Outros serviços poderiam ser: videoconferência, telepresença, jogos em grupo e m-commerce (comércio móvel, bastando utilizar seu telefone no caixa para pagar as compras feitas em uma loja). Além disso, todos esses serviços deveriam estar disponíveis em âmbito mundial (com conexão automática via satélite, quando não for possível instalar nenhuma rede terrestre), de forma instantânea (sempre ativos) e com garantias de qualidade de serviço. Em outras palavras, um sonho.

A ITU previu uma única tecnologia mundial para o IMT-2000, de forma que os fabricantes fossem capazes de construir um único dispositivo que pudesse ser vendido e utilizado em qualquer lugar do mundo. Ter uma única tecnologia também facilitaria bastante a vida dos operadores de redes e encorajaria mais pessoas a usarem os serviços.

Acontece que isso foi um pouco otimista. O número 2000 significou três coisas: (1) o ano em que o serviço

deveria ser iniciado; (2) a frequência em que ele deveria operar (em MHz); e (3) a largura de banda que o serviço deveria ter (em kbps). Nenhum desses três se concretizou. Nada foi implementado em 2000. A ITU recomendou que todos os governos reservassem o espectro em 2 GHz para que os dispositivos pudessem passar de um país para outro de forma transparente. A China reservou a largura de banda exigida, mas ninguém mais fez isso. Finalmente, reconheceu-se que 2 Mbps não são atualmente viáveis para usuários que se movimentam *muito* (em virtude da dificuldade de realizar handoffs com rapidez suficiente). O mais realista é 2 Mbps para usuários que não estão em movimento, 384 kbps para pessoas andando e 144 kbps para conexões em carros.

Apesar desses problemas iniciais, muito foi realizado desde então. Várias propostas do IMT-2000 foram feitas e, após uma seleção, elas se reduziram a duas principais: (1) o **CDMA de banda larga**, ou **WCDMA (Wideband CDMA)**, foi proposto pela Ericsson e adotado pela União Europeia, que o chamou de **sistema universal de telecomunicações móveis**, ou **UMTS (Universal Mobile Telecommunications System)**; e (2) o **CDMA2000**, proposto pela Qualcomm nos Estados Unidos.

Esses dois sistemas são mais semelhantes do que diferentes, pois são baseados no CDMA de banda larga; o WCDMA usa canais de 5 MHz e o CDMA2000 usa canais de 1,25 MHz. Se os engenheiros da Ericsson e da Qualcomm fossem confinados em uma sala e solicitados a apresentar um projeto comum, eles provavelmente conseguiriam fazê-lo rapidamente. A dificuldade é que o problema real não é de engenharia, mas político (como sempre). A Europa queria um sistema que trabalhasse junto com o GSM, enquanto os Estados Unidos queriam um sistema que fosse compatível com um sistema já amplamente desenvolvido no país (o IS-95). Cada lado (naturalmente) também apoiava sua empresa local (a Ericsson está sediada na Suécia; a Qualcomm, na Califórnia). Por fim, a Ericsson e a Qualcomm estavam envolvidas em numerosos processos relacionados a suas respectivas patentes de CDMA. Para aumentar a confusão, o UMTS se tornou um padrão 3G único com múltiplas opções incompatíveis, incluindo o CDMA2000. Essa mudança foi um esforço para unificar os vários campos, mas ele apenas encobre as diferenças técnicas e oculta o foco dos esforços em andamento. Usaremos o UMTS para indicar o WCDMA, uma forma diferente com origem no CDMA2000.

Outra melhoria em relação ao esquema CDMA básico que descrevemos anteriormente é permitir que diferentes usuários enviem dados em diferentes taxas, independentes uma da outra. Esse truque é realizado naturalmente no CDMA fixando a taxa em que os chips são transmitidos e atribuindo aos usuários sequências de chips de diferentes tamanhos. Por exemplo, no WCDMA, a taxa de chip é de 3,84 Mchips/s, e o espalhamento de códigos varia de 4 a 256 chips. Com um código de 256 chips, restam cerca de 12 kbps após a correção de erro, e essa capacidade é suficiente para uma chamada de voz. Com um código de 4 chips, a taxa de dados do usuário é próxima de 1 Mbps. Códigos de tamanho intermediário geram taxas intermediárias; para conseguir múltiplos Mbps, a unidade móvel precisa usar mais de um canal de 5 MHz ao mesmo tempo.

Vamos voltar nossa discussão para o uso do CDMA em redes celulares, por ser o fator de distinção dos dois sistemas. O CDMA não é FDM nem TDM, mas um tipo de mistura em que cada usuário envia na mesma banda de frequência ao mesmo tempo. Quando ele foi proposto inicialmente para sistemas de celular, a indústria teve aproximadamente a mesma reação que Colombo provocou na Rainha Isabel quando ele propôs alcançar as Índias navegando na direção errada. Porém, pela persistência de uma única empresa, a Qualcomm, o CDMA teve sucesso como um sistema 2G (IS-95) e amadureceu ao ponto de se tornar a base técnica para o 3G.

Para fazer o CDMA funcionar no ambiente de telefonia móvel, é preciso mais do que a técnica de CDMA básica que descrevemos na Seção 2.4. Especificamente, descrevemos um sistema chamado **CDMA síncrono**, no qual as sequências de chips são exatamente ortogonais. Esse projeto funciona quando todos os usuários estão sincronizados no momento inicial de suas sequências de chips, como no caso da estação-base transmitindo para unidades móveis. A estação-base pode transmitir as sequências de chips começando ao mesmo tempo, de modo que os sinais sejam ortogonais e capazes de ser separados. Contudo, é difícil sincronizar as transmissões de telefones móveis independentes. Sem alguns esforços especiais, suas transmissões chegariam na estação-base em momentos diferentes, sem nenhuma garantia de ortogonalidade. Para que as unidades móveis enviem para a estação-base sem sincronização, queremos codificar sequências que são ortogonais umas às outras em todos os deslocamentos possíveis, não apenas quando elas estão alinhadas no início.

Embora não seja possível encontrar sequências exatamente ortogonais para esse caso geral, longas sequências pseudoaleatórias chegam perto o suficiente. Elas têm a propriedade de que, com alta probabilidade, tenham uma baixa **correlação cruzada** entre si em todos os deslocamentos. Isso significa que, quando uma sequência é multiplicada por outra e somada para calcular o produto interno, o resultado será pequeno; ele seria zero se todas fossem ortogonais. (Intuitivamente, as sequências aleatórias sempre deverão parecer diferentes uma da outra. Multiplicá-las deverá, então, produzir um sinal aleatório, que se somará a um resultado pequeno.) Isso permite que um receptor filtre transmissões indesejadas do sinal recebido. Além disso, a **autocorrelação** de sequências pseudoaleatórias também é pequena, com alta probabilidade, exceto em um deslocamento zero. Isso significa que, quando uma sequência é multiplicada por uma cópia atrasada de si mesma e somada, o resultado será pequeno, exceto quando o atraso é

zero. (Intuitivamente, uma sequência aleatória atrasada se parece com uma sequência aleatória diferente, e voltamos ao caso da correlação cruzada.) Isso permite que um receptor intercepte o início da transmissão desejada no sinal recebido.

O uso de sequências pseudoaleatórias permite que a estação-base receba mensagens CDMA de unidades móveis não sincronizadas. Contudo, uma suposição implícita em nossa discussão do CDMA é que os níveis de potência de todas as unidades móveis são iguais no receptor. Se não forem, uma pequena correlação cruzada com um sinal poderoso poderia superar uma grande autocorrelação com um sinal fraco. Assim, a potência de transmissão nas unidades móveis deve ser controlada para reduzir a interferência entre sinais concorrentes. É essa interferência que limita a capacidade de sistemas CDMA.

Os níveis de potência recebidos em uma estação-base dependem da distância em que os transmissores se encontram e também de quanta potência eles transmitem. Pode haver muitas estações móveis em distâncias variadas da estação-base. Uma boa heurística para equalizar a potência recebida é que cada estação móvel transmita para a estação-base no inverso do nível de potência que ela recebe dessa estação-base. Em outras palavras, uma estação móvel recebendo um sinal fraco da estação-base usará mais potência do que outra obtendo um sinal forte. Para aumentar a precisão, a estação-base também oferece feedback a cada unidade móvel para aumentar, diminuir ou manter constante sua potência de transmissão. O feedback é frequente (1.500 vezes por segundo), pois o bom controle de potência é importante para reduzir a interferência.

Agora, vamos descrever as vantagens do CDMA. Primeiro, esse sistema pode melhorar a capacidade ao tirar proveito de pequenos períodos quando alguns transmissores estão silenciosos. Nas chamadas de voz, em conversas educadas, uma parte fica em silêncio enquanto a outra fala. Em média, a linha está ocupada apenas 40% do tempo. Contudo, as pausas podem ser pequenas e são difíceis de prever. Com sistemas TDM ou FDM, não é possível reatribuir slots de tempo ou canais de frequência com rapidez suficiente para se beneficiar desses pequenos silêncios. Contudo, no CDMA, um usuário reduz a interferência para outros usuários simplesmente não transmitindo, e é provável que alguma fração dos usuários não esteja transmitindo em uma célula ocupada em determinado momento. Assim, o sistema tira proveito dos silêncios esperados para permitir um número maior de chamadas simultâneas.

Em segundo lugar, com o CDMA, cada célula usa as mesmas frequências. Diferentemente de GSM e AMPS, a FDM não é necessária para separar as transmissões de diferentes usuários. Isso elimina complicadas tarefas de planejamento de frequência e melhora a capacidade, tornando mais fácil para a estação-base usar várias antenas direcionais, ou **antenas setorizadas**, em vez de uma antena omnidirecional. As antenas direcionais concentram um sinal na direção desejada e o reduzem, diminuindo, portanto, o sinal (e a interferência) em outras direções. Isso, por sua vez, aumenta a capacidade. Esquemas com três setores são comuns. A estação-base precisa rastrear a unidade móvel enquanto ela se move de um setor para outro. Esse rastreamento é fácil com o CDMA, pois todas as frequências são usadas em todos os setores.

Em terceiro lugar, o CDMA facilita o **soft handoff**, em que a unidade móvel é aceita pela nova estação-base antes de a anterior se desconectar. Desse modo, não existe perda de continuidade. O soft handoff aparece na Figura 2.43. Ele é fácil com o CDMA, pois todas as frequências são usadas em cada célula. Como opção há um **hard handoff**, em que a estação-base antiga libera a chamada antes de ela ser aceita pela nova. Se a nova não for capaz de aceitá-la (p. ex., porque não existe nenhuma frequência disponível), a chamada será desconectada de forma brusca. Os usuários tendem a notar essa interrupção, mas ela ocasionalmente é inevitável com a estrutura atual. O hard handoff é a norma nos projetos FDM para evitar o custo de fazer a unidade móvel transmitir ou receber em duas frequências simultaneamente.

2.6.6 Tecnologia de quarta geração (4G): comutação de pacotes

Em 2008, a ITU especificou um conjunto de padrões para sistemas 4G. O **4G**, que também costuma ser chamado de **IMT Advanced**, é totalmente baseado na tecnologia de rede de comutação de pacotes, incluindo seus predecessores. Seu predecessor imediato foi uma tecnologia normalmente chamada de **LTE** (**Long Term Evolution**). Outro precursor e tecnologia relacionados ao 4G foi o 3GPP

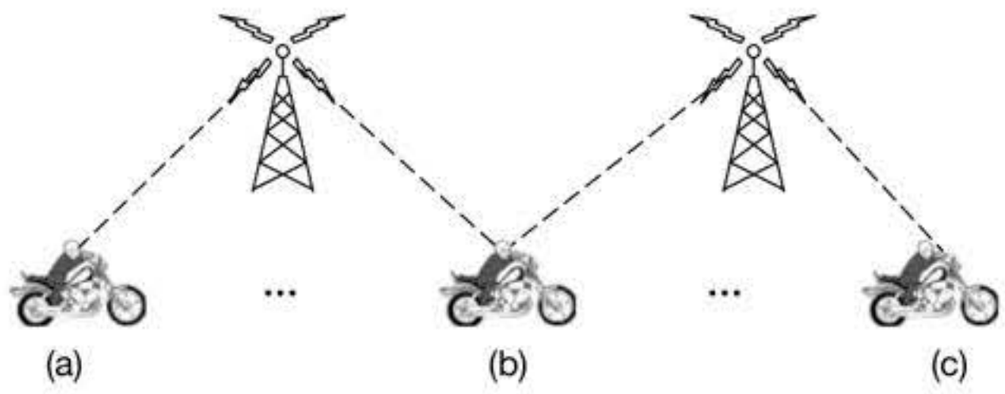


Figura 2.43 Soft handoff (a) antes, (b) durante e (c) depois.

LTE, também chamado de "4G LTE". A terminologia é um pouco confusa, pois "4G" efetivamente se refere a uma geração de comunicações móveis, em que qualquer geração pode, de fato, incluir vários padrões. Por exemplo, a ITU considera o IMT Advanced como um padrão 4G, embora também aceite LTE como um padrão 4G. Outras tecnologias, como o malfadado WiMAX (IEEE 802.16), também são consideradas 4G. Tecnicamente, LTE e "4G verdadeiro" são versões diferentes do padrão 3GPP (versões 8 e 10, respectivamente).

A principal inovação em relação aos sistemas 3G anteriores é que as redes 4G utilizam comutação de pacotes, em vez de comutação de circuitos. A inovação que permite a comutação de pacotes é chamada de **EPC** (**Evolved Packet Core**), que é basicamente uma rede IP simplificada, que separa o tráfego de voz da rede de dados. A rede EPC transporta voz e dados em pacotes IP. Assim, ela é uma rede de **voz sobre IP** (**VoIP**), com recursos alocados usando as técnicas de multiplexação estatística descritas anteriormente. Como tal, a EPC deve gerenciar os recursos de forma que a qualidade da voz permaneça alta através dos recursos de rede que são compartilhados entre muitos usuários. Os requisitos de desempenho para LTE incluem, entre outras coisas, throughput máximo para upload de 100 Mbps e download de 50 Mbps. Para alcançar essas taxas mais altas, as redes 4G usam um conjunto de frequências adicionais, incluindo 700 MHz, 850 MHz, 800 MHz e outras. Outro aspecto do padrão 4G é a "eficiência espectral", ou quantos bits podem ser transmitidos por segundo para uma determinada frequência; para tecnologias 4G, a eficiência espectral máxima deve ser 15 bps/Hz para downlink e 6,75 bps/Ghz para uplink.

A arquitetura LTE inclui os seguintes elementos como parte do EPC, conforme mostrado no Capítulo 1, na Figura 1.19.

1. **Serving Gateway** (**S-GW**). O S-GW encaminha pacotes de dados para garantir que os pacotes continuem a ser encaminhados para o dispositivo do usuário ao passar de um eNodeB para outro.
2. **Mobility Management Entity** (**MME**). O MME rastreia, pagina o dispositivo do usuário e escolhe o S-GW para um dispositivo quando ele se conecta à rede pela primeira vez, assim como durante as transferências. Ele também autentica o dispositivo do usuário.
3. **Packet Data Network Gateway** (**P-GW**). Faz a interface entre o dispositivo do usuário e uma rede de dados de pacote (ou seja, uma rede comutada por pacote) e pode executar funções como alocação de endereço para essa rede (p. ex., via DHCP), limitação de taxa, filtragem, inspeção profunda de pacotes e interceptação legal do tráfego. Os dispositivos do usuário estabelecem um serviço orientado à conexão com o gateway de pacotes usando o chamado **portador de EPS**, estabelecido quando o dispositivo do usuário se conecta à rede.
4. **Home Subscriber Server** (**HSS**). O MME consulta o HSS para determinar se o dispositivo do usuário corresponde a um assinante válido.

A rede 4G também possui uma **RAN** (**Radio Access Network**) expandida. A rede de acesso por rádio para LTE introduz um nó de acesso denominado **eNodeB**, que realiza operações na camada física (como enfocaremos neste capítulo), bem como as camadas **MAC** (**Medium Access Control**), **RLC** (**Radio Link Control**) e **PDCP** (**Packet Data Control Protocol**), muitas delas específicas para a arquitetura de rede celular. O eNodeB executa o gerenciamento de recursos, o controle de admissão, a programação e outras funções do plano de controle.

Nas redes 4G, o tráfego de voz pode ser transportado pelo EPC usando uma tecnologia chamada **VoLTE** (**Voice over LTE**), possibilitando às operadoras transmitir o tráfego de voz pela rede comutada por pacotes e tirando qualquer dependência da rede de voz legada, comutada por circuitos.

2.6.7 Tecnologia de quinta geração (5G)

Por volta de 2014, o sistema LTE atingiu a maturidade e as pessoas começaram a pensar no que viria em seguida. Obviamente, depois de 4G vem o 5G. A verdadeira questão, claro, é "o que será o 5G?", que Andrews et al. (2014) discutem bastante. Anos depois, 5G passou a significar muitas coisas diferentes, dependendo do público e de quem está usando o termo. Basicamente, a próxima geração de tecnologia de rede celular móvel se resume a dois fatores principais: taxas de dados mais altas e latência menor do que as tecnologias 4G. É claro que existem tecnologias específicas que permitem alcançar maior velocidade e menor latência, que discutiremos a seguir.

O desempenho da rede celular é geralmente medido em termos de **taxa de dados agregada** ou **capacidade de área**, que é a quantidade total de dados que a rede pode atender em bits por unidade de área. Um dos objetivos do 5G é melhorar a capacidade de área da rede em três ordens de grandeza (mais de 1000 vezes a do 4G), usando uma combinação de tecnologias:

1. Ultra-densificação e descarregamento. Uma das maneiras mais simples de melhorar a capacidade da rede é adicionar mais células por área. Enquanto os tamanhos das células 1G eram da ordem de centenas de quilômetros quadrados, o 5G visa tamanhos de células menores, incluindo **picocélulas** (com menos de 100 m de diâmetro) e até mesmo **femtocélulas** (com alcance semelhante ao WiFi, de dezenas de metros). Um dos benefícios mais importantes da redução do tamanho da célula é a capacidade de reutilizar o espectro em

uma determinada área geográfica, reduzindo assim o número de usuários que estão competindo por recursos em qualquer estação-base. É claro que a redução do tamanho da célula traz suas próprias complicações, incluindo maior dificuldade para gerenciamento de mobilidade e handoff.

2. Largura de banda aumentada com ondas milimétricas. A maior parte do espectro de tecnologias anteriores estava na faixa de várias centenas de MHz a alguns GHz, correspondendo a comprimentos de onda que variam de centímetros a cerca de um metro. Esse espectro tem se tornado cada vez mais congestionado, especialmente nos principais mercados durante os horários de pico. Existem quantidades consideráveis de espectro não utilizado na faixa de ondas milimétricas de 20 a 300 GHz, com comprimentos de onda menores que 10 mm. Até pouco tempo, esse espectro não era considerado adequado para comunicação sem fio porque comprimentos de onda mais curtos não se propagam tão bem. Uma das maneiras como os desafios de propagação estão sendo enfrentados é usando grandes matrizes de antenas direcionais, o que é uma grande mudança arquitetônica em relação às gerações anteriores de redes celulares: tudo é diferente, desde propriedades de interferência até o processo de associar um usuário a uma estação-base.
3. Maior eficiência espectral por meio de avanços na tecnologia **MIMO** (**Multiple-Input Multiple-Output**). MIMO melhora a capacidade de um enlace de rádio, usando várias antenas de transmissão e recepção para tirar vantagem da propagação de multicaminhos, através da qual o sinal de rádio transmitido chega ao receptor por meio de dois ou mais caminhos. MIMO foi introduzido na comunicação WiFi e nas tecnologias de celular 3G por volta de 2006, e possui algumas variações; os padrões celulares anteriores tiram proveito do **MU-MIMO** (**Multi-User MIMO**). Geralmente, essas tecnologias aproveitam a diversidade espacial dos usuários para cancelar a interferência que pode ocorrer em qualquer uma das extremidades da transmissão sem fio. **Massive MIMO** é um tipo de MU-MIMO que aumenta o número de antenas da estação-base para que haja muito mais antenas do que terminais. Existe até a possibilidade de usar um arranjo de antenas tridimensionais, no chamado **FD-MIMO** (**Full-Dimension MIMO**).

Outro recurso que acompanhará o 5G é o **fatiamento de rede**, que permitirá às operadoras de celular criarem várias redes virtuais na mesma infraestrutura física compartilhada, dedicando partes de sua rede a casos de uso específicos do cliente. Frações distintas da rede (e seus recursos) podem ser dedicadas a diferentes provedores de aplicações, as quais, por sua vez, podem ter diferentes requisitos. Por exemplo, aplicações que exigem alto throughput podem ser alocadas a uma fatia de rede diferente daquelas que admitem um throughput mais baixo. **SDN** (**Software-Defined Networking**) and **NFV** (**Network Functions Virtualization**) são tecnologias emergentes que ajudarão a dar suporte ao fatiamento. Discutiremos essas tecnologias em capítulos mais adiante.

2.7 REDES POR CABO

Certamente, os sistemas de telefonia fixa e móvel desempenharão um papel importante nas futuras redes, mas as redes por cabo também participarão das redes de acesso de banda larga do futuro. Hoje mesmo, muitas pessoas recebem serviços de televisão, telefone e Internet via cabo. Nas próximas seções, examinaremos com mais detalhes a televisão a cabo como uma rede e vamos compará-la com os sistemas de telefonia que acabamos de estudar. Para obter mais informações, consulte Harte (2017). O padrão DOCSIS de 2018 também tem informações úteis, principalmente em relação às modernas arquiteturas de rede por cabo.

2.7.1 História das redes por cabo: TV por antena comunitária

A televisão a cabo foi concebida no final da década de 1940 como uma forma de proporcionar melhor recepção de TV às pessoas que viviam em áreas rurais ou montanhosas. No início, o sistema consistia em uma grande antena situada no alto de uma colina para captar o sinal de televisão que se propaga pelo ar, um amplificador chamado **headend**, para reforçar o sinal, e um cabo coaxial para distribuí-lo pelas casas das pessoas, como ilustra a Figura 2.44.

Nos primeiros anos, a TV a cabo era chamada de **televisão de antena comunitária**, ou **CATV** (**Community Antenna Television**). Sua operação era muito simples – qualquer pessoa que tivesse alguma prática em eletrônica era capaz de instalar um serviço para sua cidade, e os usuários se reuniam para pagar os custos. À medida que o número de assinantes crescia, outros cabos eram conectados ao cabo original e eram acrescentados outros amplificadores conforme a necessidade. A transmissão era unidirecional, do headend para os usuários. Em 1970, havia milhares de sistemas independentes.

Em 1974, a Time Inc. lançou um novo canal, denominado Home Box Office, com novo conteúdo (filmes) e distribuído somente por cabo. Seguiram-se outros canais dedicados apenas a notícias, esportes, culinária, história, filmes, ciência, crianças e muitos outros temas. Esse desenvolvimento ocasionou duas mudanças na indústria. Primeiro, as grandes corporações começaram a adquirir os sistemas a cabo existentes e estender novos cabos para conquistar novos assinantes. Segundo, agora havia necessidade

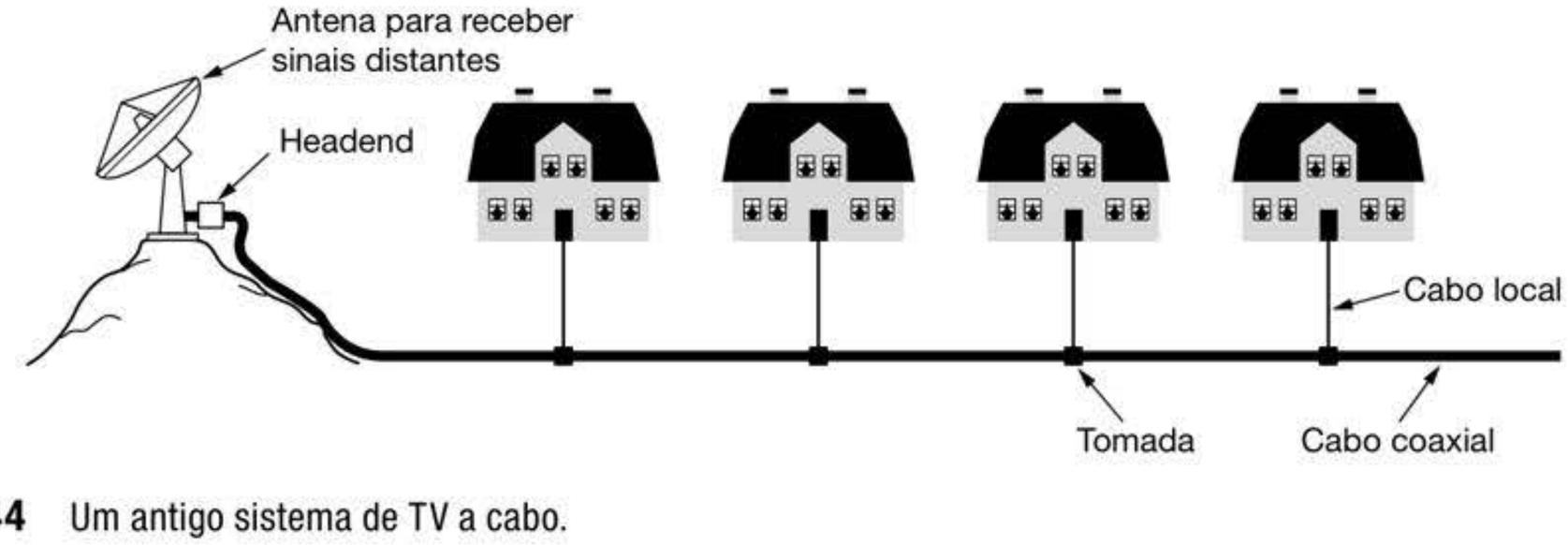


Figura 2.44 Um antigo sistema de TV a cabo.

de conectar vários sistemas, normalmente em cidades distantes, a fim de distribuir o conteúdo dos novos canais de TV a cabo. As empresas especializadas começaram a estender cabos entre as cidades para conectar todas elas em um único sistema. Esse padrão era semelhante ao que ocorreu na indústria de telefonia 80 anos antes, com a conexão de estações finais anteriormente isoladas para tornar possível a comunicação interurbana.

2.7.2 Acesso de banda larga à Internet por cabo: redes HFC

Com o passar dos anos, o sistema de TV a cabo cresceu, e os cabos entre as várias cidades foram substituídos por fibra óptica de alta largura de banda, de forma semelhante ao que ocorreu no sistema telefônico. Um sistema com fibra nas linhas principais e cabo coaxial nas ligações para as residências é chamado de **sistema híbrido de cabo coaxial e fibra**, ou **HFC** (**Hybrid Fiber Coax**), e é a arquitetura predominante nas redes a cabo atuais. A tendência de levar a fibra para mais perto do assinante doméstico continua, conforme descrevemos na seção sobre FTTX. Os conversores eletro-ópticos que constituem a interface entre as partes óptica e elétrica do sistema são chamados **nós de fibra**. Pelo fato de a largura de banda da fibra ser muito maior que a dos cabos coaxiais, um único nó de fibra pode alimentar vários cabos coaxiais. A Figura 2.45(a) mostra uma parte de um sistema HFC moderno.

No final da década de 1990, muitas operadoras de TV a cabo decidiram entrar no ramo de acesso à Internet e, muitas vezes, também no ramo de telefonia. No entanto, diferenças técnicas entre as instalações de cabo e de telefonia têm efeito sobre o que deve ser realizado para alcançar esses objetivos. Por um lado, todos os amplificadores unidirecionais no sistema tinham de ser substituídos por amplificadores bidirecionais, para dar suporte a transmissões tanto upstream quanto downstream. Enquanto isso estava acontecendo, os primeiros sistemas de Internet por cabo usavam a rede de televisão para as transmissões downstream e uma conexão discada via rede telefônica para as transmissões upstream. Essa foi uma alternativa arriscada, se é que houve uma, mas até que funcionou.

Eliminar todos os canais de TV e usar a infraestrutura de cabo estritamente para acesso à Internet provavelmente geraria um número razoável de clientes insatisfeitos (principalmente os mais antigos, pois os mais novos já haviam rompido os laços); assim, as empresas de TV a cabo hesitam em fazê-lo. Além disso, a maioria das cidades tem uma regulamentação bastante pesada sobre o que é transmitido por cabo e, portanto, as operadoras de serviços não teriam permissão para fazer isso, ainda que desejassem. Como consequência, elas precisaram encontrar um modo de fazer a televisão e a Internet coexistirem no mesmo cabo.

A solução é contar com a FDM. Os canais de TV a cabo da América do Norte ocupam a região de 54 a 550 MHz (com exceção do rádio FM, que ocupa a faixa de 88 a 108 MHz). Esses canais têm 6 MHz de largura, incluindo as bandas de proteção, e podem transportar um canal de TV analógica tradicional ou vários canais de TV digital. Na Europa, a extremidade inferior em geral é de 65 MHz, e os canais têm de 6 a 8 MHz de largura, em virtude da maior resolução exigida pelos sistemas PAL e SECAM, mas o esquema de alocação é semelhante nos outros aspectos. A parte baixa da banda não é usada. Os cabos modernos também operam bem acima de 550 MHz, chegando frequentemente a 750 MHz ou mais. A solução escolhida foi introduzir canais de upstream na banda de 5 a 42 MHz (um pouco mais alta na Europa) e usar as frequências na extremidade alta para os sinais downstream. O espectro dos serviços de cabo é ilustrado na Figura 2.46.

Como todos os sinais de televisão são downstream, é possível usar amplificadores upstream que só funcionam na região de 5 a 42 MHz e amplificadores downstream que só funcionam na frequência de 54 MHz e acima desta, como mostra a figura. Desse modo, obtemos uma assimetria nas larguras de banda upstream e downstream, porque está disponível uma parte maior do espectro acima da faixa de TV do que abaixo dela. Em contrapartida, a maior parte do tráfego provavelmente será downstream, e, assim, as operadoras de serviços a cabo não estão insatisfeitas com esse fato. Como vimos, em geral as companhias telefônicas oferecem um serviço DSL assimétrico, embora não tenham nenhuma razão técnica para fazê-lo. Além de atualizar os

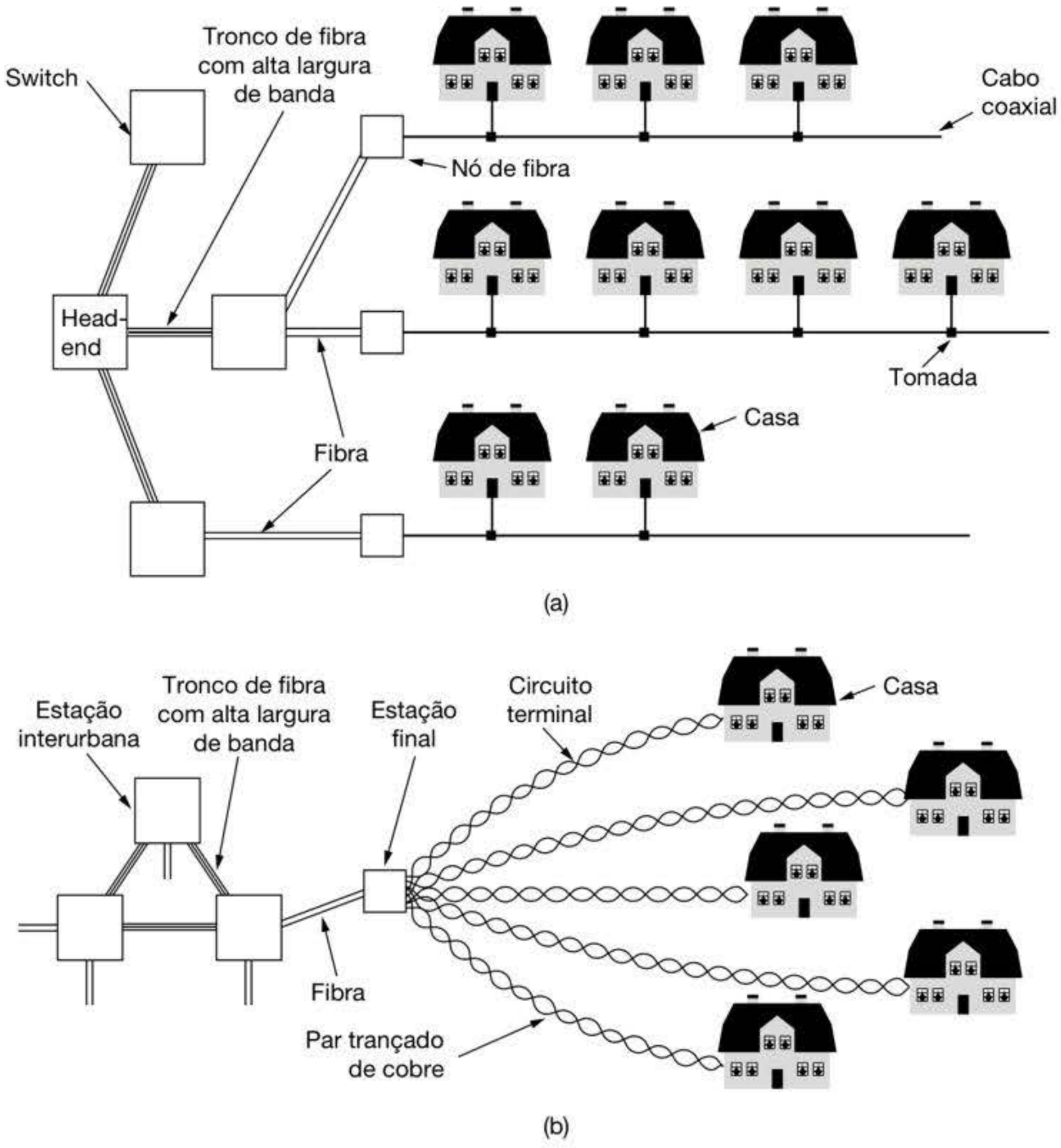


Figura 2.45 (a) Rede híbrida de cabo coaxial e fibra. (b) Sistema de telefonia fixa.

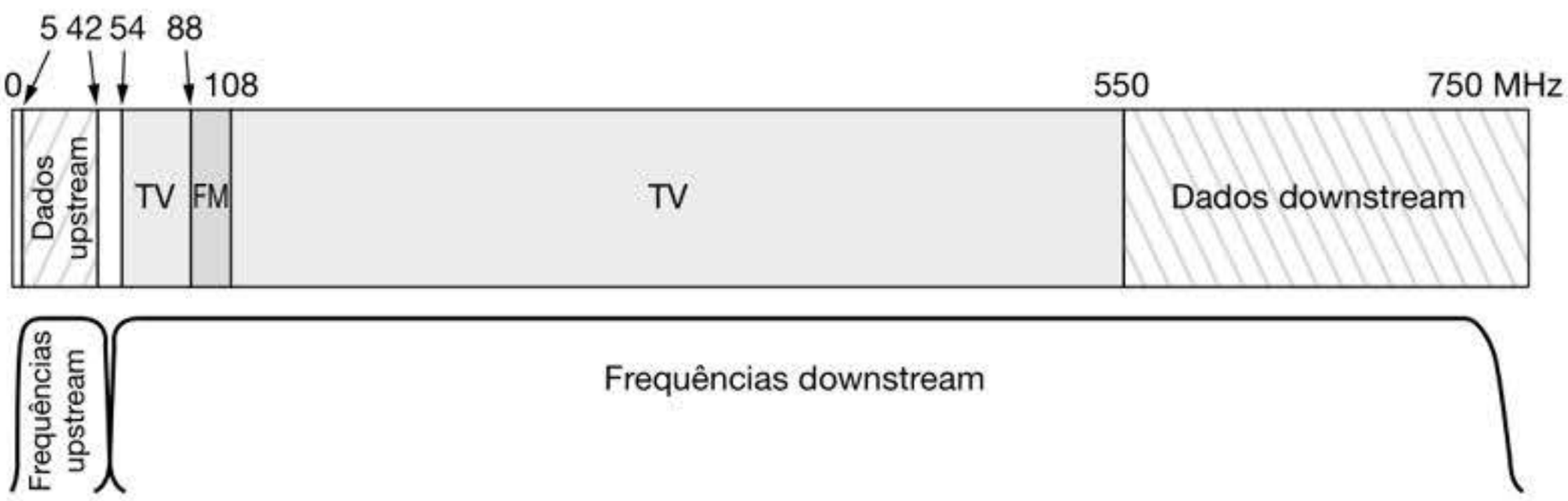


Figura 2.46 Alocação de frequências em um sistema típico de TV a cabo usado para acesso à Internet.

amplificadores, a operadora também tem de atualizar o headend, que deve passar de um amplificador não inteligente para um sistema computadorizado digital inteligente, com uma interface de fibra de alta largura de banda para um ISP. Esse headend atualizado passou a se chamar **sistema de terminação de modem a cabo**, ou **CMTS** (**Cable Modem Termination System**). CMTS e headend referem-se ao mesmo componente.

2.7.3 DOCSIS

As companhias de cabo operam redes que incluem a tecnologia da camada física HFC para a conectividade do último quilômetro, além de conexões por fibra e sem fio para o último quilômetro. A parte HFC dessas redes é bastante empregada nos Estados Unidos, Canadá, Europa e outros mercados, e usa os padrões **DOCSIS** (**Data Over Cable Service Interface Specification**) da CableLabs.

O padrão DOCSIS versão 1.0 apareceu em 1997. DOCSIS 1.0 e 1.1 tinham um limite funcional de 38 Mbps downstream e 9 Mbps upstream. Em 2001, DOCSIS 2.0 triplicou a largura de banda upstream. Mais tarde, DOCSIS 3.0 (2006) introduziu o suporte para IPv6 e permitiu a ligação de canais para as comunicações downstream e upstream, aumentando bastante a capacidade potencial para cada residência atendida para centenas de megabits por segundo. DOCSIS 3.1 (2013), que introduziu a Orthogonal Frequency Division Multiplexing (OFDM), largura de banda mais larga para os canais e maior eficiência, permitiu mais de 1 Gbps de capacidade downstream por residência. Extensões ao DOCSIS 3.1 foram acrescentadas por meio de atualizações ao padrão DOCSIS 3.1, incluindo a operação Full Duplex (2017), que permitirá capacidades downstream e upstream simétricas de vários gigabits, além do DOCSIS Low Latency (2018) e outros recursos para reduzir a latência.

Na camada de fibra coaxial híbrida, ou HFC, a rede é altamente dinâmica, com operadoras de rede a cabo realizando divisões de nó de fibra regularmente, o que empurra a fibra para mais perto da casa e reduz o número de residências atendidas por cada nó, disponibilizando mais capacidade para cada casa atendida. Em alguns casos, a HFC do último quilômetro é substituído por FTTH, e muitas novas construções também são para uso doméstico.

Os assinantes de Internet a cabo exigem um modem a cabo DOCSIS para servir como interface entre a rede doméstica e a rede do ISP. Cada modem a cabo envia dados em um canal upstream e um canal downstream, e cada um é alocado usando FDM. DOCSIS 3.0 utiliza vários canais. O esquema normal é apanhar cada canal downstream de 6 ou 8 MHz e modulá-lo com QAM-64 ou, se a qualidade do cabo for excepcionalmente boa, com QAM-256. Com um canal de 6 MHz e QAM-64, obtemos cerca de 36 Mbps. Quando o overhead da sinalização é subtraído, a largura de banda resultante é de cerca de 27 Mbps. Com QAM-256, a carga útil resultante é de cerca de 39 Mbps. Os valores europeus são 1/3 maiores devido à maior disponibilidade de largura de banda.

A interface entre o modem e a residência é simples; em geral, ela é uma conexão Ethernet. Hoje em dia, muitos usuários domésticos da Internet conectam o modem a cabo a um ponto de acesso WiFi para montar uma rede sem fio doméstica. Em alguns casos, o provedor de serviços de Internet (ISP) do usuário fornece um único dispositivo de hardware que combina o modem a cabo e o ponto de acesso sem fio. A interface entre o modem a cabo e o restante da rede ISP é mais complicada, pois envolve a coordenação do compartilhamento de recursos entre muitos assinantes de cabo que podem estar conectados ao mesmo headend. Esse compartilhamento de recursos ocorre tecnicamente na camada de enlace, não na camada física, mas iremos abordá-lo neste capítulo para garantir a continuidade.

2.7.4 Compartilhamento de recursos em redes DOCSIS: nós e minislots

Há uma diferença fundamental importante entre o sistema HFC da Figura 2.45(a) e o sistema telefônico da Figura 2.45(b). Em determinados bairros, um único cabo é compartilhado por muitas casas, ao passo que, no sistema telefônico, cada casa tem seu próprio circuito terminal privado. Quando é utilizado para difusão de televisão, esse compartilhamento é natural. Todos os programas são transmitidos no cabo e não importa se existem 10 ou 10.000 espectadores. No entanto, quando o mesmo cabo é usado para acesso à Internet, faz uma grande diferença a existência de 10 ou de 10.000 usuários. Se um usuário decidir baixar um arquivo muito grande ou fazer o stream de um filme de 8K, essa largura de banda não estará disponível a outros usuários. Quanto mais usuários compartilhando um único cabo, maior a competição pela largura de banda. O sistema de telefonia não tem essa propriedade específica: a transferência de um grande arquivo por uma linha ADSL não reduz a largura de banda do seu vizinho. Contudo, a largura de banda do cabo coaxial é muito mais alta do que a dos pares trançados. Basicamente, a largura de banda que qualquer assinante recebe em determinado momento depende muito do uso dos outros assinantes que estejam compartilhando o mesmo cabo, conforme descrevemos com mais detalhes a seguir.

A estratégia usada pelos ISPs para resolver esse problema é desmembrar cabos longos e conectar cada um deles diretamente a um nó de fibra. A largura de banda do headend até cada nó de fibra é significativa e, se não existirem muitos assinantes em cada segmento de cabo, o volume de tráfego será gerenciável. Os cabos típicos há cerca de 10 a 15 anos conectavam de 500 a 2.000 casas, embora o número de casas por nó continue a diminuir conforme a implantação até o limite continua, em um esforço para aumentar a velocidade para os assinantes. O aumento de assinantes de Internet a cabo na última década, juntamente com a crescente demanda de tráfego, criou a necessidade cada vez maior de dividir esses cabos e adicionar mais nós de fibra. Em 2019, um tamanho de nó típico era de cerca de 300-500 casas, embora em algumas áreas os ISPs estejam construindo arquiteturas HFC N+0 (também conhecidas como "Fiber Deep"), o que pode reduzir esse

3

A camada de enlace de dados

Neste capítulo, estudaremos os princípios de projeto da segunda camada em nosso modelo, a de enlace de dados. Neste estudo, trataremos de algoritmos que permitem uma comunicação eficiente e confiável de unidades de informação inteiras, chamadas quadros (em vez de bits individuais, como na camada física), entre dois computadores adjacentes. Por adjacentes queremos dizer que as duas máquinas estão fisicamente conectadas por meio de um canal de comunicação que funciona conceitualmente como um fio (p. ex., um cabo coaxial, uma linha telefônica ou um canal sem fio). A propriedade essencial de um canal que o torna semelhante a um fio é o fato de que os bits são entregues na ordem exata em que são enviados.

A princípio, você pode pensar que esse problema é tão trivial que não há nada para estudar – a máquina *A* simplesmente coloca os bits no fio e a máquina *B* os retira de lá. Infelizmente, os canais de comunicação algumas vezes produzem erros. Além disso, eles têm uma taxa de dados finita, e há um atraso de propagação diferente de zero entre o momento em que um bit é enviado e o momento em que ele é recebido. Essas limitações têm implicações importantes para a eficiência da transferência de dados. Os protocolos usados para comunicações devem levar todos esses fatores em consideração. Tais protocolos são o assunto deste capítulo.

Após uma introdução às principais questões de projeto presentes na camada de enlace de dados, começaremos nosso estudo dos protocolos verificando a natureza dos erros e como eles podem ser detectados e corrigidos. Em seguida, estudaremos uma série de protocolos de complexidade crescente e mostraremos como cada um deles resolve um número cada vez maior de problemas dessa camada. Por fim, concluiremos o capítulo com alguns exemplos de protocolos de enlace de dados.

3.1 QUESTÕES DE PROJETO DA CAMADA DE ENLACE DE DADOS

A camada de enlace de dados usa os serviços da camada física para enviar e receber bits pelos canais de comunicação (possivelmente não confiáveis), que podem perder dados. Ela tem diversas funções, entre as quais:

1. Fornecer uma interface de serviço bem definida à camada de rede (Seção 3.1.1).
2. Enquadrar sequências de bytes como segmentos autocontidos (Seção 3.1.2).
3. Detectar e corrigir erros de transmissão (Seção 3.1.3).
4. Regular o fluxo de dados de modo que receptores lentos não sejam atropelados por transmissores rápidos (Seção 3.1.4).

Para alcançar esses objetivos, a camada de enlace de dados recebe os pacotes da camada de rede e os encapsula em **quadros** para transmissão. Cada um contém um cabeçalho (header) de quadro, um campo de carga útil (payload), que conterá o pacote, e um final (trailer) de quadro, como mostra a Figura 3.1. O gerenciamento de quadros constitui o núcleo das atividades da camada de enlace de dados. Nas próximas seções, examinaremos em detalhes todas as questões mencionadas. Além disso, quando são usadas redes sem fio não confiáveis, o uso de protocolos para melhorar o enlace de dados posteriormente quase sempre melhora o desempenho.

Embora este capítulo trate explicitamente da camada de enlace de dados e de seus protocolos, muitos dos princípios que estudaremos aqui, como o controle de erros e o controle de fluxo, em algumas redes, são encontrados em

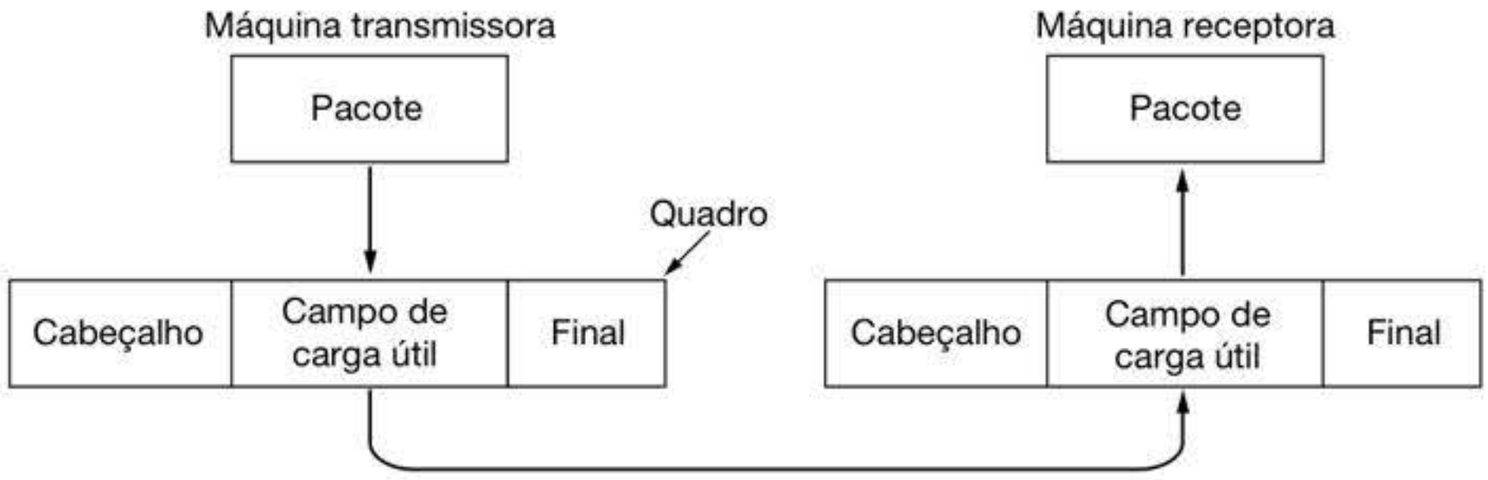


Figura 3.1 Relacionamento entre pacotes e quadros.

protocolos de transporte e também em outros protocolos. Isso ocorre porque a confiabilidade é um objetivo geral, e ela é alcançada quando todas as redes funcionam juntas. De fato, em muitas delas, essas funções são encontradas principalmente nas camadas superiores, com a camada de enlace de dados realizando a tarefa mínima, que é "suficientemente boa". Contudo, independentemente do lugar onde são encontrados, os princípios são quase idênticos. Na camada de enlace de dados, eles surgem com frequência em sua forma mais simples e mais pura, o que faz dessa camada um bom lugar para examiná-los em detalhes.

3.1.1 Serviços oferecidos à camada de rede

A função da camada de enlace de dados é fornecer serviços à camada de rede, sendo o principal transferir dados da camada de rede da máquina de origem para a camada de rede da máquina de destino. Na camada de rede da máquina de origem há uma entidade, chamada processo, que entrega alguns bits à camada de enlace de dados para transmissão ao destino. A tarefa da camada de enlace de dados é transmitir os bits à máquina de destino, de forma que eles possam ser entregues à camada de rede dessa máquina, como mostra a Figura 3.2(a). A transmissão propriamente dita segue o trajeto descrito na Figura 3.2(b); no entanto, é mais fácil pensar em termos de dois processos da camada de enlace de dados que se comunicam por intermédio de um protocolo de enlace de dados. Por essa razão, utilizaremos implicitamente o modelo da Figura 3.2(a) em todo este capítulo.

A camada de enlace de dados pode ser projetada de modo a oferecer diversos serviços. Os serviços reais oferecidos podem variar de um protocolo para outro. Três possibilidades razoáveis que consideraremos são:

1. Serviço não orientado a conexões sem confirmação.
2. Serviço não orientado a conexões com confirmação.
3. Serviço orientado a conexões com confirmação.

O serviço não orientado a conexões sem confirmação consiste em fazer a máquina de origem enviar quadros

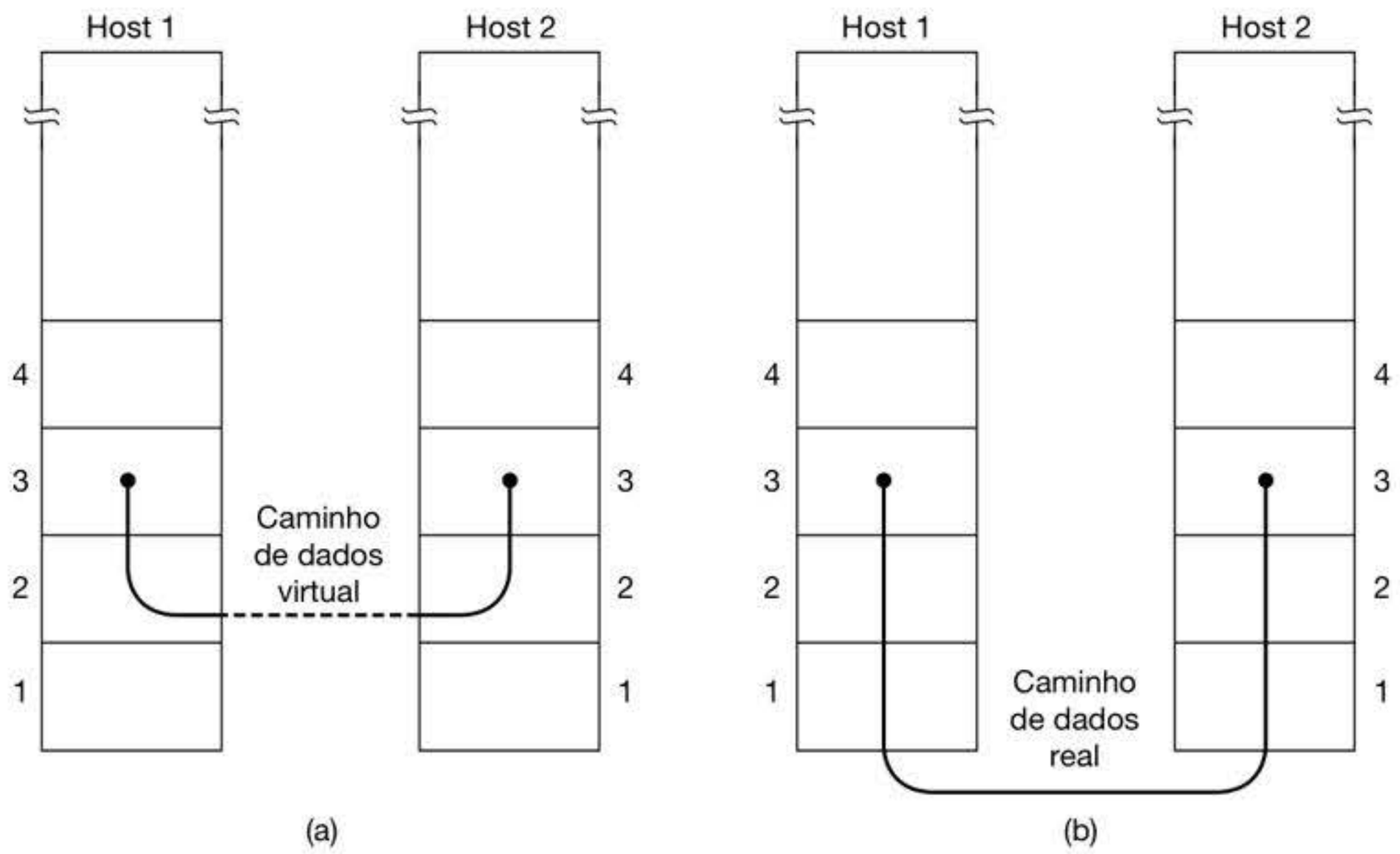


Figura 3.2 (a) Comunicação virtual. (b) Comunicação real.

independentes à máquina de destino, sem que esta confirme o recebimento deles. A Ethernet é um bom exemplo de uma camada de enlace de dados que oferece essa classe de serviço. Nenhuma conexão lógica é estabelecida antes ou liberada depois do processo. Se um quadro for perdido em decorrência de ruídos na linha, não haverá qualquer tentativa de detectar a perda ou de recuperá-lo na camada de enlace de dados. Essa classe de serviço é apropriada quando a taxa de erros é muito baixa, e a recuperação fica a cargo das camadas mais altas, bem como para o tráfego em tempo real, no qual, a exemplo da voz, os dados atrasados causam mais problemas do que os dados recebidos com falhas.

O próximo passo em termos de confiabilidade é o serviço não orientado a conexões com confirmação. Quando esse serviço é oferecido, ainda não há conexões lógicas sendo usadas, mas cada quadro enviado é confirmado individualmente. Dessa forma, o transmissor sabe se um quadro chegou corretamente ou não. Caso não tenha chegado dentro de um intervalo específico, o quadro poderá ser reenviado. Esse serviço é útil em canais não confiáveis, como os sistemas sem fio. O padrão 802.11 (WiFi) é um bom exemplo desse tipo de serviço da camada de enlace.

Talvez valha a pena destacar que oferecer recursos de confirmação no nível da camada de enlace de dados é uma questão de otimização, nunca uma exigência. A camada de rede sempre pode enviar um pacote e esperar que ele seja confirmado por seu par na máquina remota. Se a confirmação não chegar durante o intervalo do timer de retransmissão, o transmissor poderá enviar a mensagem inteira mais uma vez. O problema dessa estratégia é que ela pode ser ineficaz. Os enlaces normalmente têm um quadro com comprimento máximo estrito imposto pelo hardware e atrasos de propagação conhecidos. A camada de rede não conhece esses parâmetros. Ela pode enviar um pacote grande subdividido em, digamos, 10 quadros, dos quais dois são perdidos, em média. O tempo necessário para efetivar a transmissão do pacote com sucesso poderá ser muito longo. Em vez disso, se quadros individuais forem confirmados e retransmitidos, então os erros podem ser corrigidos mais diretamente e mais rapidamente. Em canais confiáveis, como a fibra óptica, o overhead de um protocolo de enlace de dados muito sofisticado talvez seja desnecessário, mas, em canais sem fio (com sua inerente falta de confiabilidade), o custo geralmente compensa.

Voltando aos nossos serviços, o serviço mais sofisticado que a camada de enlace de dados é capaz de oferecer à camada de rede é o orientado a conexões. Com ele, as máquinas de origem e de destino estabelecem uma conexão antes de qualquer dado ser transferido. Cada quadro enviado pela conexão é numerado, e a camada de enlace de dados garante que cada quadro seja, de fato, recebido. Além disso, essa camada garante que todos os quadros sejam recebidos uma única vez e na ordem correta. Assim, os serviços orientados a conexões fornecem aos processos da camada de rede o equivalente a um fluxo de bits confiável. Isso é apropriado para enlaces longos, não confiáveis, como um canal de satélite ou um circuito telefônico interurbano. Se o serviço não orientado a conexões com confirmação fosse usado, é possível imaginar que as confirmações perdidas poderiam fazer um quadro ser enviado e recebido várias vezes, desperdiçando largura de banda.

Quando o serviço orientado a conexões é usado, as transferências passam por três fases distintas. Na primeira, a conexão é estabelecida, fazendo ambos os lados inicializar as variáveis e os contadores necessários para controlar os quadros que são recebidos e os que não são. Na segunda, um ou mais quadros são realmente transmitidos. Na terceira e última fase, a conexão é desfeita, liberando-se as variáveis, os buffers e os outros recursos usados para mantê-la.

3.1.2 Enquadramento

Para oferecer serviços à camada de rede, a camada de enlace de dados deve usar o serviço fornecido a ela pela camada física. O que a camada física faz é aceitar um fluxo de bits brutos e tentar entregá-lo ao destino. Se o canal tiver ruído, como acontece na maioria dos enlaces sem fio e em alguns enlaces com fio, a camada física acrescentará alguma redundância aos seus sinais, para reduzir a taxa de erros de bits para um nível tolerável. Contudo, o fluxo de bits recebido pela camada de enlace de dados não tem garantia de estar livre de erros. Alguns bits podem ter valores diferentes e o número de bits recebidos pode ser menor, igual ou maior que o transmitido. A camada de enlace de dados é responsável por detectar e, se necessário, corrigir erros.

Em geral, a estratégia adotada pela camada de enlace de dados é dividir o fluxo de bits em quadros distintos, calcular um pequeno valor (um token), chamado de checksum (soma de verificação), para cada quadro e incluir essa soma de verificação no quadro quando ele for transmitido. (Os algoritmos de checksum serão discutidos mais adiante neste capítulo.) Quando um quadro chega a seu destino, o checksum é recalculado com base no que foi recebido. Se o valor recém-calculado for diferente do que está contido no quadro, a camada de enlace de dados saberá que houve um erro e tomará providências para lidar com ele (p. ex., descartando o quadro defeituoso e possivelmente também enviando de volta um relatório de erros).

A divisão do fluxo de bits em quadros é mais difícil do que parece à primeira vista. Um bom projeto deve tornar fácil para um receptor encontrar o início de novos quadros usando pouca largura de banda do canal. Nesta seção, examinaremos quatro métodos:

1. Contagem de caracteres.
2. Bytes de flag com inserção de bytes.
3. Bits de flag com inserção de bits.
4. Violações de codificação da camada física.

O primeiro método de enquadramento utiliza um campo no cabeçalho para especificar o número de bytes no quadro. Quando vê a contagem de caracteres, a camada de enlace de dados de destino sabe quantos bytes devem vir em seguida e, consequentemente, onde está o final do quadro. Essa técnica é mostrada na Figura 3.3(a) para quatro pequenos quadros, como exemplo, de tamanhos 5, 5, 8 e 8 bytes, respectivamente.

O problema com esse algoritmo é que a contagem pode ser adulterada por um erro de transmissão. Por exemplo, se a contagem 5 no segundo quadro da Figura 3.3(b) se tornar 7, em virtude da inversão de um único bit, o destino perderá a sincronização e não será capaz de localizar o início do quadro seguinte. Mesmo que o checksum esteja incorreto, de modo que o destino saiba que o quadro está defeituoso, ele ainda não terá informações suficientes para saber onde começa o quadro seguinte. Enviar um quadro de volta à origem solicitando retransmissão também não ajuda, pois o destino não sabe quantos bytes deverão ser ignorados para chegar ao início da retransmissão. Por essa razão, o método de contagem de caracteres quase não é mais usado.

O segundo método de enquadramento contorna o problema de ressincronização após um erro, fazendo cada quadro começar e terminar com bytes especiais. Normalmente o mesmo byte, chamado **byte de flag**, é usado como delimitador de início e de final, como mostra a Figura 3.4(a), na qual ele é representado por FLAG. Dois bytes de flag consecutivos indicam o final de um quadro e o início do próximo. Assim, se o receptor perder a sincronização, ele poderá simplesmente procurar dois bytes de flag para encontrar o final do quadro atual e o início do seguinte.

Entretanto, ainda existe um problema. É bem possível que o byte de flag ocorra nos dados, especialmente quando são transmitidos dados binários, como fotografias ou músicas. Essa situação poderia interferir no enquadramento. Uma forma de solucionar esse problema é fazer a camada de enlace de dados do transmissor incluir um caractere de escape especial (ESC) imediatamente antes de cada byte de flag "acidental" nos dados. Assim, o byte de flag de enquadramento pode ser distinguido daquele nos dados pela ausência ou presença de um byte de escape antes dele. A camada de enlace de dados na extremidade receptora remove o byte de escape antes de entregar os dados à camada de rede. Essa técnica é chamada de **inserção de bytes** (**byte stuffing**).

É claro que a próxima pergunta é: o que acontecerá se um byte de escape ocorrer em uma posição dentro dos dados? Nesse caso, ele também será preenchido com um byte de escape. No receptor, o primeiro byte de escape é removido, deixando o byte de dados seguinte (que poderia ser outro byte de escape ou o byte de flag). Alguns exemplos são mostrados na Figura 3.4(b). Em todos os casos, a sequência de bytes entregue após a remoção dos bytes inseridos é exatamente igual à sequência de bytes original. Ainda podemos procurar por um limite de quadro buscando dois bytes de flag em sequência, sem nos preocuparmos em desfazer os escapes.

O esquema de inserção de bytes representado na Figura 3.4 é uma ligeira simplificação do que é utilizado no protocolo ponto a ponto, ou **PPP** (**Point-to-Point Protocol**), comum na Internet para carregar pacotes por enlaces de comunicação. Descreveremos o PPP na Seção 3.5.1.

O terceiro método de delimitação do fluxo de bits contorna uma desvantagem da inserção de bytes, ou seja, o fato de ela estar ligada ao uso de bytes de 8 bits. O enquadramento também pode ser feito em nível de bit, de modo que os quadros podem conter um número qualquer de bits, compostos de unidades de qualquer tamanho. Ele foi desenvolvido para o então muito popular protocolo de controle de enlace de dados de alto nível, ou **HDLC** (**High-level Data Link Control**). Cada quadro começa e termina com um padrão de bits especial, 01111110, ou 0x7E em hexadecimal (um byte de flag). Sempre que encontra cinco valores

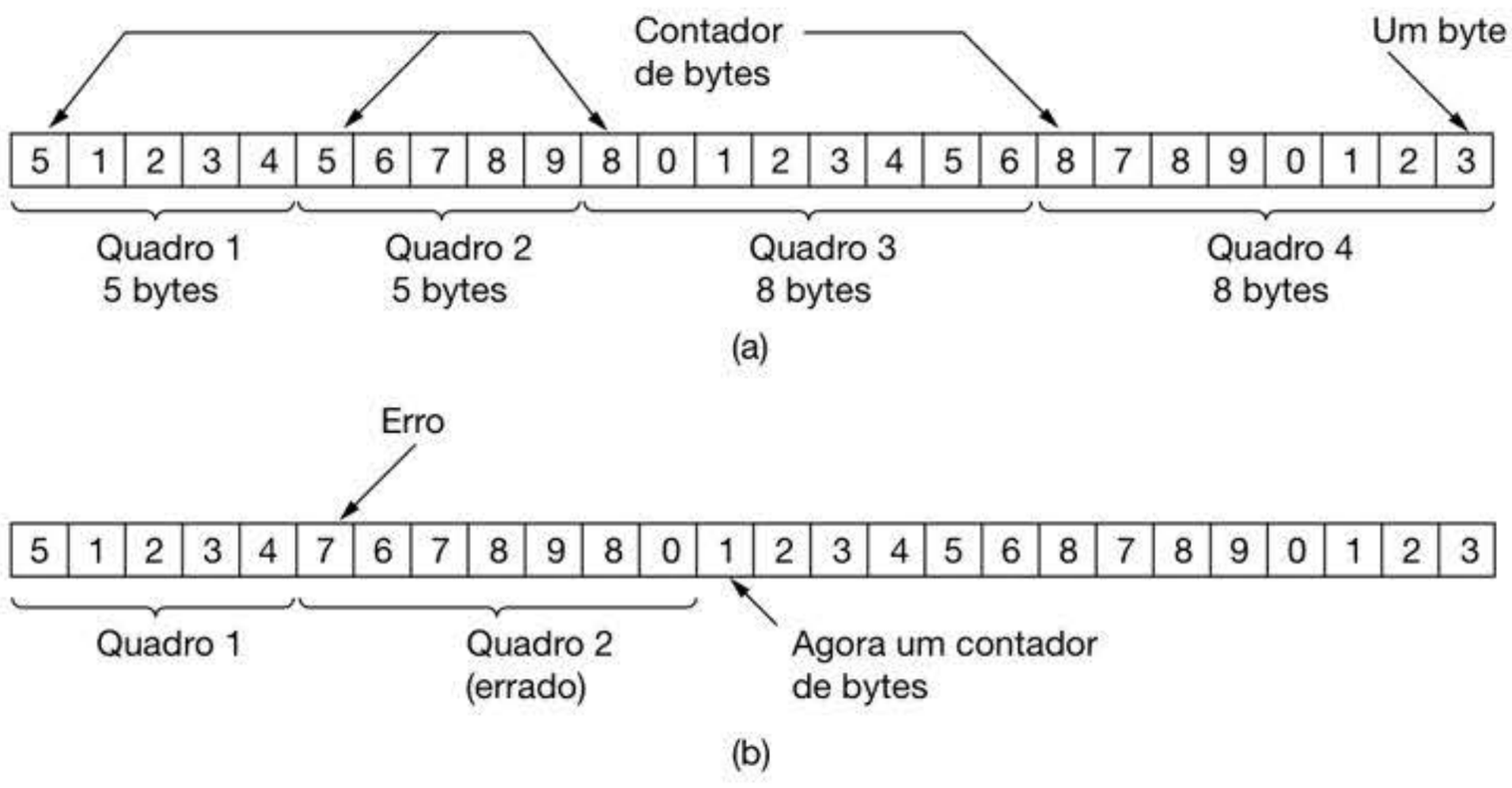


Figura 3.3 Um fluxo de caracteres. (a) Sem erros. (b) Com um erro.

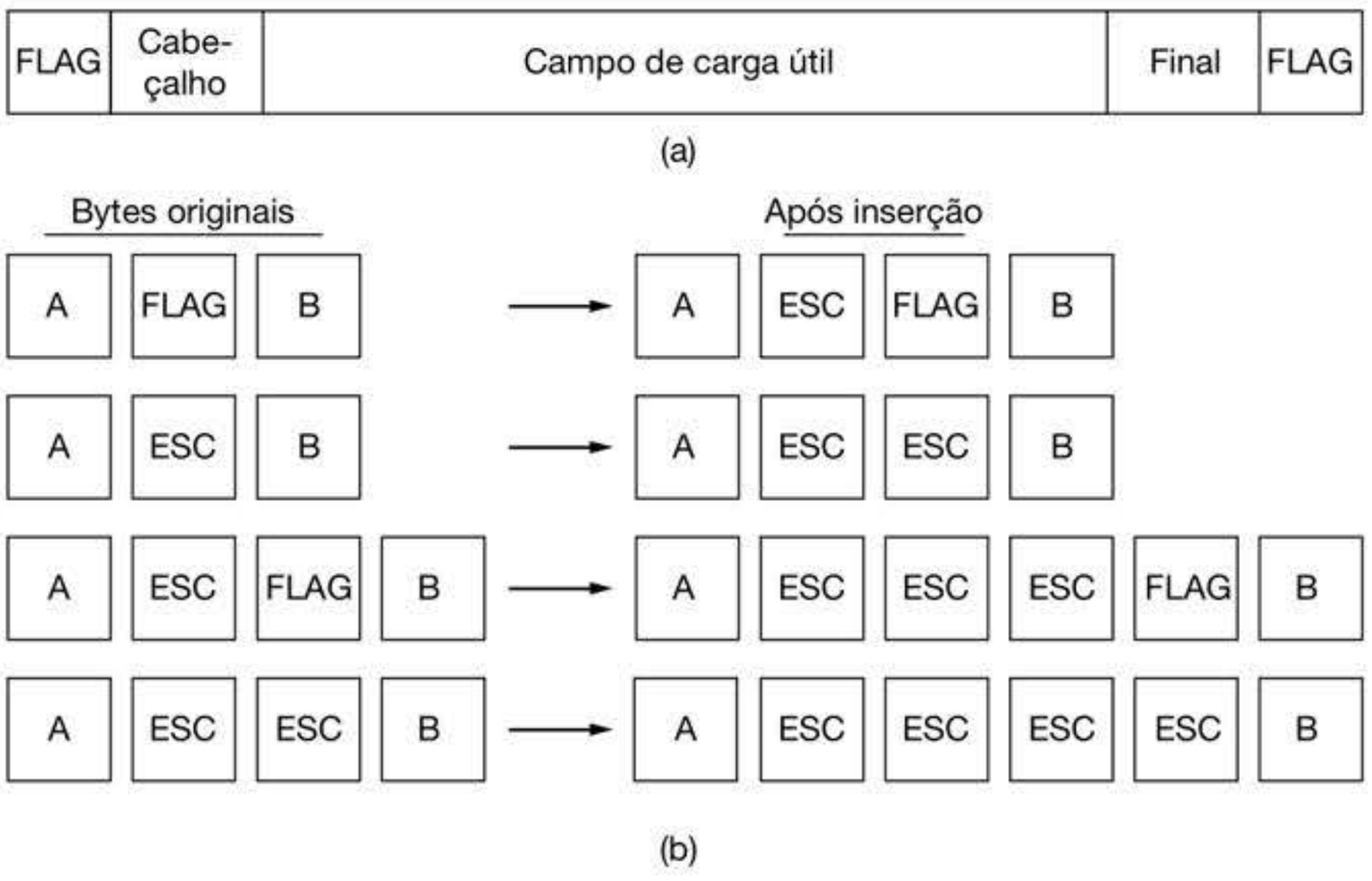


Figura 3.4 (a) Quadro delimitado por bytes de flag. (b) Quatro exemplos de sequências de bytes, antes e depois da inserção de bytes.

1 consecutivos nos dados, a camada de enlace de dados do transmissor automaticamente insere um bit 0 no fluxo de bits que está sendo enviado. Essa **inserção de bits** é semelhante à inserção de bytes, na qual um byte de escape é inserido no fluxo de caracteres enviado antes de ocorrer um byte de flag nos dados. Isso também garante uma densidade mínima de transições, o que ajuda a camada física a manter a sincronização. O USB (Universal Serial Bus) utiliza a inserção de bits por esse motivo.

Ao ver cinco bits 1 consecutivos sendo recebidos, seguidos por um bit 0, o receptor automaticamente remove o bit 0. A inserção de bits, assim como a inserção de bytes, é completamente transparente para a camada de rede de ambos os computadores. Se os dados do usuário contiverem o padrão de flag 01111110, esse flag será transmitido como 011111010, mas será armazenado na memória do receptor como 01111110. As camadas superiores não tomam conhecimento dessa inserção de bits. A Figura 3.5 mostra um exemplo de inserção de bits.

Com a inserção de bits, o limite entre dois quadros pode ser reconhecido sem qualquer tipo de ambiguidade pelo padrão de flags. Desse modo, se o receptor perder o controle de onde estão os dados, bastará varrer a entrada em busca de sequências de flags, uma vez que elas nunca ocorrem dentro dos dados, apenas nos limites dos quadros.

Com a inserção de bits e de bytes, um efeito colateral é que o comprimento de um quadro agora depende do conteúdo dos dados que ele transporta. Por exemplo, se não houver bytes de flag nos dados, 100 bytes podem ser transportados em um quadro de aproximadamente 100 bytes. Todavia, se os dados consistirem unicamente de bytes de flag, cada um terá um bit de escape associado e o quadro terá aproximadamente 200 bytes de comprimento. Com a inserção de bits, o aumento seria de aproximadamente 12,5%, pois 1 bit é inserido a cada byte.

O último método de enquadramento é usar um atalho da camada física. No Capítulo 2, vimos que a codificação de bits como sinais normalmente inclui redundância para ajudar o receptor. Essa redundância significa que alguns sinais não ocorrerão em dados regulares. Por exemplo, no código de linha 4B/5B, 4 bits de dados são mapeados para 5 bits de sinal, para garantir transições de bits suficientes. Isso significa que 16 das 32 possibilidades de sinal não são usadas. Podemos usar alguns sinais reservados para indicar o início e o final dos quadros. Com efeito, estamos usando "violações de código" (caracteres inválidos) para delimitar

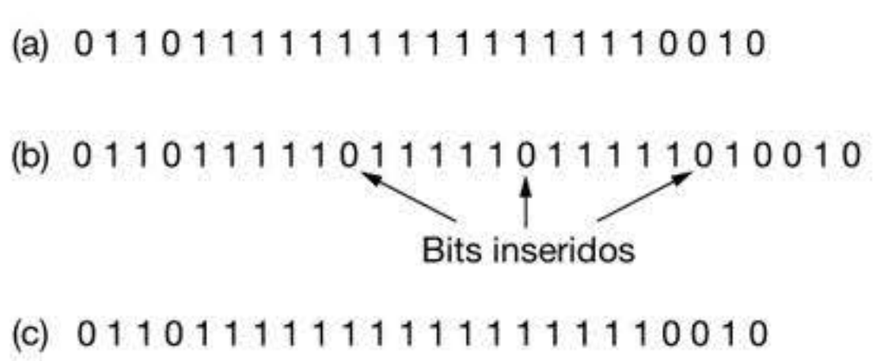


Figura 3.5 Inserção de bits. (a) Os dados originais. (b) Como os dados são exibidos na linha. (c) Como os dados são armazenados na memória do receptor após a remoção de bits.

os quadros. A beleza desse esquema é que, por serem sinais reservados, é fácil encontrar o início e o final dos quadros, e não é preciso inserir bits nos dados.

Muitos protocolos de enlace de dados, por segurança, usam uma combinação desses métodos. Um padrão comum usado para Ethernet e para 802.11 é fazer um quadro começar com um padrão bem definido, chamado **preâmbulo**. Esse padrão pode ser muito longo (72 bits é típico para redes 802.11), a fim de permitir que o receptor se prepare para um pacote que está chegando. O preâmbulo é, então, seguido por um campo de comprimento (ou seja, um contador) no cabeçalho, que é usado para localizar o final do quadro.

3.1.3 Controle de erros

Após resolvermos o problema da delimitação do início e do final de cada quadro, vamos ao problema seguinte: como ter certeza de que todos os quadros serão entregues na camada de rede de destino e na ordem apropriada? Suponha, para o momento, que o receptor consegue saber se um quadro que ele recebe contém informações corretas ou defeituosas (veremos os códigos usados para detectar e corrigir erros de transmissão na Seção 3.2). Para serviços não orientados a conexões, sem confirmação, pode ser suficiente que o emissor apenas continue enviando quadros sem se importar se eles chegaram corretamente, mas sem dúvida essa não seria uma boa opção para serviços orientados a conexões confiáveis.

A forma mais comum de garantir uma entrega confiável é dar ao transmissor algum tipo de feedback sobre o que está acontecendo no outro extremo da linha. Normalmente, o protocolo solicita que o receptor retorne quadros de controle especiais com confirmações positivas ou negativas sobre os quadros recebidos. Se receber uma confirmação positiva sobre um quadro, o transmissor saberá que ele chegou em segurança ao destino. Em contrapartida, uma confirmação negativa significa que algo saiu errado e que o quadro deve ser retransmitido.

Uma complicação adicional decorre da possibilidade de problemas de hardware fazerem um quadro desaparecer completamente (p. ex., em uma rajada de ruídos). Nesse caso, o receptor não reagirá de forma alguma, pois não há motivo para isso. De modo semelhante, se o quadro de confirmação se perder, o emissor não saberá como prosseguir. Deve ficar claro que um protocolo no qual o transmissor envia um quadro e depois espera por uma confirmação, positiva ou negativa, permanecerá suspenso para sempre caso um quadro tenha sido completamente perdido – por exemplo, em consequência de mau funcionamento do hardware ou canal de comunicação deficiente.

Essa possibilidade é tratada com a introdução de timers na camada de enlace de dados. Quando o transmissor envia um quadro, em geral ele também inicializa um timer. Este é ajustado para ser desativado após um intervalo suficientemente longo para o quadro chegar ao destino, ser processado e ter sua confirmação enviada de volta ao transmissor. Em geral, o quadro será recebido de forma correta e a confirmação voltará antes que se alcance o tempo-limite do timer e, nesse caso, ele será cancelado.

No entanto, se a confirmação ou o quadro se perder, o timer será desativado, alertando o transmissor para um problema potencial. A solução óbvia é simplesmente transmitir o quadro outra vez. Entretanto, quando os quadros podem ser transmitidos várias vezes, existe o perigo de o receptor aceitar o mesmo quadro duas ou mais vezes e de repassá-lo à camada de rede mais de uma vez. Para evitar que isso aconteça, geralmente é preciso atribuir números de sequência aos quadros transmitidos, de modo que o receptor possa distinguir as retransmissões dos originais.

A questão do gerenciamento dos timers e dos números de sequência para garantir que cada quadro seja realmente passado para a camada de rede no destino exatamente uma vez, nem mais nem menos, é uma parte importante das atribuições da camada de enlace de dados (e das camadas mais altas). Mais adiante neste capítulo, veremos uma série de exemplos cada vez mais sofisticados, para entender como é feito esse gerenciamento.

3.1.4 Controle de fluxo

Outro importante aspecto de projeto que ocorre na camada de enlace de dados (e também nas camadas mais altas) é o que fazer com um transmissor que sistematicamente deseja transmitir quadros mais rápido do que o receptor pode aceitá-los. Essa situação pode ocorrer quando o transmissor está rodando em um computador rápido e poderoso e o receptor está rodando em uma máquina lenta e inferior. Uma situação comum é quando um smartphone solicita uma página Web de um servidor muito mais poderoso, que abre a mangueira de incêndio e jorra os dados para o pobre e infeliz telefone, até que esteja completamente inundado. Mesmo que a transmissão não tenha erros, o receptor pode não conseguir lidar com os quadros com a rapidez com que chegam, e perderá alguns deles.

Sem dúvida, algo deve ser feito para impedir que essa situação ocorra. Em geral duas abordagens são usadas. Na primeira, chamada de **controle de fluxo baseado em feedback**, o receptor envia de volta ao transmissor informações que permitam a ele enviar mais dados, ou que pelo menos mostrem ao transmissor a sua situação real. Na segunda, chamada de **controle de fluxo baseado na velocidade**, o protocolo tem um mecanismo interno que limita a velocidade de envio de dados pelos transmissores, sem usar o feedback do receptor.

Neste capítulo, estudaremos os esquemas de controle de fluxo baseados em feedback, principalmente porque os esquemas baseados na velocidade são vistos apenas como parte da camada de transporte (Capítulo 5). Os esquemas baseados em feedback são vistos na camada de enlace e em camadas superiores. O último caso é mais comum hoje em dia, em que o hardware da camada de enlace é projetado

para trabalhar com rapidez suficiente para não causar perda. Às vezes, por exemplo, considera-se que as implementações de hardware da camada de enlace, como as **placas de interface de rede**, ou **NICs (Network Interface Cards)**, atuam na "velocidade do fio", o que significa que podem tratar de quadros tão rapidamente quanto eles chegam ao enlace. Portanto, qualquer overrun não será um problema de enlace, já que é tratado por camadas mais altas.

Existem diversos esquemas de controle de fluxo baseados em feedback. No entanto, a maioria deles utiliza o mesmo princípio básico. O protocolo contém regras bem definidas sobre quando um transmissor pode enviar o quadro seguinte. Com frequência, essas regras impedem que os quadros sejam enviados até que o receptor tenha concedido permissão para a transmissão, implícita ou explicitamente. Por exemplo, quando uma conexão é estabelecida, o receptor pode informar: "Você pode me enviar *n* quadros agora, mas, depois que eles tiverem sido enviados, não envie mais nada até ser informado de que deve prosseguir". Examinaremos os detalhes em breve.

3.2 DETECÇÃO E CORREÇÃO DE ERROS

Vimos no Capítulo 2 que os canais de comunicação têm diversas características. Alguns canais, como a fibra óptica nas redes de telecomunicações, têm taxas de erro muito pequenas, de modo que os erros de transmissão são uma ocorrência rara. Mas outros canais, especialmente enlaces sem fio e antigos circuitos terminais, têm taxas de erro muito maiores. Para esses enlaces, os erros de transmissão são a norma. Eles não podem ser evitados a um custo razoável em termos de desempenho. Ou seja, os erros de transmissão estão aqui para ficar e precisamos aprender a lidar com eles.

Os projetistas de redes desenvolveram duas estratégias básicas para lidar com os erros, e ambas acrescentam informações redundantes aos dados enviados. Uma estratégia é incluir informações redundantes suficientes para permitir que o receptor deduza quais foram os dados transmitidos. A outra é incluir apenas a redundância suficiente para permitir que o receptor deduza que houve um erro (mas não qual erro) e solicite uma retransmissão. A primeira estratégia usa **códigos de correção de erros**, enquanto a segunda usa **códigos de detecção de erros**. O uso de códigos de correção de erros normalmente é conhecido como **correção antecipada de erros**, ou **FEC (Forward Error Correction)**.

Cada uma dessas técnicas ocupa um nicho em ambientes específicos. Em canais altamente confiáveis, como os de fibra, é mais econômico utilizar um código de detecção de erros e simplesmente retransmitir o bloco defeituoso ocasional. Contudo, em canais como enlaces sem fio, que geram muitos erros, é melhor adicionar redundância suficiente a cada bloco para que o receptor seja capaz de descobrir qual era o bloco transmitido originalmente. A FEC é usada em canais com ruído porque as retransmissões podem ter erros tanto quanto a primeira transmissão.

Uma consideração importante para esses códigos é o tipo de erro que provavelmente ocorrerá. Nem os códigos de correção nem os de detecção de erros podem lidar com todos os erros possíveis, pois os bits redundantes que ofe-
recem proteção provavelmente serão recebidos com erro, como os bits de dados (o que pode comprometer sua proteção). Seria bom se o canal tratasse os bits redundantes de forma diferente da que trata os bits de dados, mas isso não acontece. Todos eles são apenas bits para o canal. Isso significa que, para evitar erros não detectados, o código precisa ser forte o suficiente para lidar com os erros esperados.

Um modelo é aquele em que os erros são causados por valores extremos de ruído térmico, que abafa o sinal rápida e ocasionalmente, fazendo surgir erros de único bit isolados. Outro modelo consiste em erros que tendem a vir em rajadas, em vez de isolados, oriundos dos processos físicos que os geram – como uma atenuação profunda em um canal sem fio ou interferência elétrica transitória em um canal com fio.

Os dois modelos importam na prática e enfrentam diferentes dilemas. O fato de os erros acontecerem em rajadas tem vantagens e desvantagens em relação aos erros isolados de único bit. Uma vantagem é que os dados de computadores sempre são enviados em blocos de bits. Suponha que o tamanho do bloco seja 1.000 bits e que a taxa de erros seja 0,001 por bit. Se os erros fossem independentes, a maioria dos blocos teria um erro. Todavia, se os erros surgirem em rajadas de 100, apenas um bloco em 100 será afetado, em média. A desvantagem dos erros em rajada é que, quando ocorrem, são muito mais difíceis de corrigir que os erros isolados.

Também existem outros tipos de erros. Às vezes, o local de um erro será conhecido, talvez porque a camada física tenha recebido um sinal analógico que foi longe do valor esperado para 0 ou 1 e declarado que o bit foi perdido. Essa situação é chamada de **canal de cancelamento**. É mais fácil corrigir erros em canais de cancelamento do que em canais que invertem bits, pois, mesmo que o valor do bit tenha sido perdido, pelo menos sabemos qual deles tem erro. Contudo, normalmente não temos o benefício dos cancelamentos.

Em seguida, examinaremos os códigos de correção de erros e os códigos de detecção de erros. Contudo, tenha dois pontos em mente. Primeiro, cobrimos esses códigos na camada de enlace porque esse é o primeiro lugar em que nos deparamos com o problema de transmitir grupos de bits de modo confiável. Contudo, os códigos são muito usados porque a confiabilidade é uma preocupação geral. Os códigos de correção de erros também são vistos na camada

física, principalmente para canais com ruído e em camadas superiores, particularmente para mídia em tempo real e distribuição de conteúdo. Os códigos de detecção de erros normalmente são usados nas camadas de enlace, de rede e de transporte.

O segundo ponto que se deve ter em mente é que os códigos de detecção e/ou correção de erros são matemática aplicada. A menos que você seja particularmente adepto aos campos de Galois ou às propriedades das matrizes esparsas, deverá obter códigos com boas propriedades a partir de uma fonte confiável, em vez de criar os seus próprios. De fato, é isso que muitos padrões de protocolo fazem, com os mesmos códigos aparecendo repetidas vezes. A seguir, estudaremos um código simples com detalhes e depois descreveremos rapidamente os códigos avançados. Desse modo, podemos entender os dilemas a partir do código simples e falar sobre os códigos usados na prática por meio dos códigos avançados.

3.2.1 Códigos de correção de erros

Vamos examinar quatro tipos de código de correção de erros:

1. Códigos de Hamming.
2. Códigos de convolução binários.
3. Códigos de Reed-Solomon.
4. Códigos de verificação de paridade de baixa densidade.

Todos esses códigos acrescentam redundância às informações enviadas. Um quadro consiste em m bits de dados (ou seja, mensagem) e r bits redundantes (ou seja, verificação). Em um **bloco de código**, os r bits de verificação são calculados unicamente como uma função dos m bits de dados com os quais eles são associados, como se os m bits fossem pesquisados em uma grande tabela para encontrar seus r bits de verificação correspondentes. Em um **código sistemático**, os m bits de dados são enviados diretamente, com os bits de verificação, em vez de ser codificados eles mesmos antes de ser enviados. Em um **código linear**, os r bits de verificação são calculados como uma função linear dos m bits de dados. A operação OU exclusivo (XOR), ou adição de módulo 2, é uma escolha popular. Isso significa que a codificação pode ser feita com operações como multiplicações de matrizes ou circuitos lógicos simples. Os códigos que veremos nesta seção são blocos de código lineares, sistemáticos, a menos que indiquemos de outra forma.

Considere que o tamanho total de um bloco seja n (ou seja, $n = m + r$). Descreveremos isso como um código (n,m). Uma unidade de n bits contendo bits de dados e verificação é conhecida como uma **palavra de código** de n bits. A **taxa de código**, ou simplesmente taxa, é a fração da palavra de código que transporta informações não redundantes, ou m/n. As taxas usadas na prática variam bastante. Elas poderiam ser 1/2 para um canal com ruído, em que metade da informação recebida é redundante, ou perto de 1 para um canal de alta qualidade, com apenas um pequeno número de bits de verificação acrescentados a uma mensagem grande.

Para entender como os erros podem ser tratados, primeiro é necessário examinar de perto o que realmente é um erro. Dadas duas palavras de código que podem ser transmitidas ou recebidas – digamos, 10001001 e 10110001 –, é possível determinar quantos bits correspondentes diferem. Nesse caso, são 3 os bits divergentes. Para determinar quantos bits apresentam diferenças, basta efetuar uma operação XOR entre as duas palavras de código e contar o número de bits 1 no resultado. Por exemplo:

$$\begin{array}{r} 10001001 \\ \underline{10110001} \\ 00111000 \end{array}$$

O número de posições de bits em que duas palavras de código diferem entre si é chamado **distância de Hamming**, em homenagem a Richard Hamming (Hamming, 1950). Isso significa que, se duas palavras de código estiverem a uma distância de Hamming uma da outra igual a d, será necessário corrigir d erros de bits isolados para converter uma palavra na outra.

Dado o algoritmo para calcular os bits de verificação, é possível construir uma lista completa das palavras de código válidas e, a partir dela, encontrar as duas palavras de código com a menor distância de Hamming. Essa é a distância de Hamming do código completo.

Na maioria das aplicações de transmissão de dados, todas as 2^m mensagens de dados possíveis são válidas; no entanto, em virtude da forma como os bits de verificação são calculados, nem todas as 2^n palavras de código possíveis são usadas. De fato, quando existem r bits de verificação, somente a pequena fração de $2^m/2^n$ ou $1/2^r$ das mensagens possíveis será uma palavra de código válida. É a dispersão com que a mensagem é embutida no espaço das palavras de código que permite que o receptor detecte e corrija erros.

As propriedades de detecção e de correção de erros de um código dependem de sua distância de Hamming. Para detectar d erros de modo confiável, você precisa de um código de distância $d + 1$, pois com tal código não há como d erros de bits transformarem uma palavra de código válida em outra. Quando o receptor descobre uma palavra de código inválida, isso pode indicar que houve um erro de transmissão. Da mesma forma, para corrigir d erros, você precisa de um código de distância $2d + 1$ porque, dessa forma, as palavras de código válidas estarão tão distantes que, mesmo com d alterações, a palavra de código original continuará mais próxima do que qualquer outra. Isso significa que a palavra de código original pode ser determinada exclusivamente com base na suposição de que um número maior de erros é menos provável.

Como um exemplo simples de código de correção de erros, considere um código que contenha apenas quatro palavras de código válidas:

0000000000, 0000011111, 1111100000 e 1111111111

Esse código tem uma distância igual a 5, o que significa que ele pode corrigir erros duplos ou detectar erros quádruplos. Se a palavra de código 0000000111 for detectada e esperarmos apenas erros de 1 ou 2 bits, o receptor saberá que a original deve ter sido 0000011111. No entanto, se um erro triplo transformar 0000000000 em 0000000111, o erro não será corrigido da maneira adequada. De maneira alternativa, se esperarmos todos esses erros, podemos detectá-los. Como nenhuma das palavras de código recebidas é uma palavra de código válida, um erro deve ter ocorrido. Deve ficar evidente que, nesse exemplo, não podemos corrigir erros duplos e detectar erros quádruplos, pois isso exigiria que interpretássemos uma palavra de código recebida de duas maneiras.

Em nosso exemplo, a tarefa de decodificar encontrando a palavra de código válida mais próxima da palavra de código recebida pode ser feita por inspeção. Infelizmente, no caso mais geral, quando todas as palavras de código precisam ser avaliadas como candidatas, essa tarefa pode ser uma busca demorada. Em vez disso, os códigos práticos são criados de modo que admitam atalhos para encontrar o que provavelmente foi a palavra de código original.

Suponha que desejemos criar um código com m bits de mensagem e r bits de verificação que permitirão a correção de todos os erros simples. Cada uma das 2^m mensagens válidas tem n palavras de código inválidas a uma distância da mensagem igual a 1. Essas palavras inválidas são formadas pela inversão sistemática de cada um dos n bits da palavra de código de n bits formada a partir dela. Portanto, cada uma das 2^m mensagens válidas exige $n + 1$ padrões de bits dedicados a ela. Como o número total de padrões de bits é 2^n, devemos ter $(n + 1)2^m \leq 2^n$. Utilizando $n = m + r$, esse requisito passa a ser

$$(m + r + 1) \leq 2^r \tag{3.1}$$

Se m for determinado, o limite para o número de bits de verificação necessários para corrigir erros isolados será mais baixo.

Esse limite teórico mais baixo pode, na verdade, ser alcançado pela utilização de um método criado por Hamming (1950). Nos **códigos de Hamming**, os bits da palavra de código são numerados consecutivamente, começando com o bit 1 da extremidade esquerda, o bit 2 imediatamente à sua direita, e assim por diante. Os bits que são potências de 2 (1, 2, 4, 8, 16, etc.) são de verificação. Os outros (3, 5, 6, 7, 9, etc.) são preenchidos com os m bits de dados. Esse padrão pode ser visto para um código de Hamming (11,7) com 7 bits de dados e 4 bits de verificação na Figura 3.6. Cada bit de verificação força a paridade de algum conjunto de bits, incluindo seu próprio conjunto, a ser par (ou ímpar). Um bit pode ser incluído em vários cálculos de verificação de bits. Se quiser ver para quais bits de verificação o bit de dados na posição k contribui, reescreva k como uma soma de potências de 2. Por exemplo, $11 = 1 + 2 + 8$ e $29 = 1 + 4 + 8 + 16$. Um bit é verificado apenas por aqueles bits de verificação que ocorrem em sua expansão (p. ex., o bit 11 é verificado pelos bits 1, 2 e 8). No exemplo, os bits de verificação são calculados para somas de paridade par para uma mensagem que é o código ASCII letra "A".

Essa construção resulta em um código com uma distância de Hamming igual a 3, o que significa que ela pode corrigir erros simples (ou detectar erros duplos). O motivo para a numeração muito cuidadosa dos bits de mensagem e de verificação torna-se aparente no processo de decodificação. Quando uma palavra de código chega, o receptor refaz os cálculos do bit de verificação, incluindo os valores dos bits de verificação recebidos. Chamamos estes de resultados de verificação. Se os bits de verificação forem corretos, então, para somas de paridade par, cada resultado de verificação deve ser zero. Nesse caso, a palavra de código é aceita como válida.

Entretanto, se os resultados da verificação não forem todos zero, um erro foi detectado. O conjunto de resultados de verificação forma a **síndrome de erro** que é usada para localizar e corrigir o erro. Na Figura 3.6, um erro de único bit ocorreu no canal, de modo que os resultados de verificação são 0, 1, 0 e 1 para $k = 8, 4, 2$ e 1, respectivamente. Isso gera uma síndrome de 0101 ou $4 + 1 = 5$. Pelo projeto do esquema, isso significa que o quinto bit está com erro. A inversão do bit incorreto (que pode ser um bit de verificação

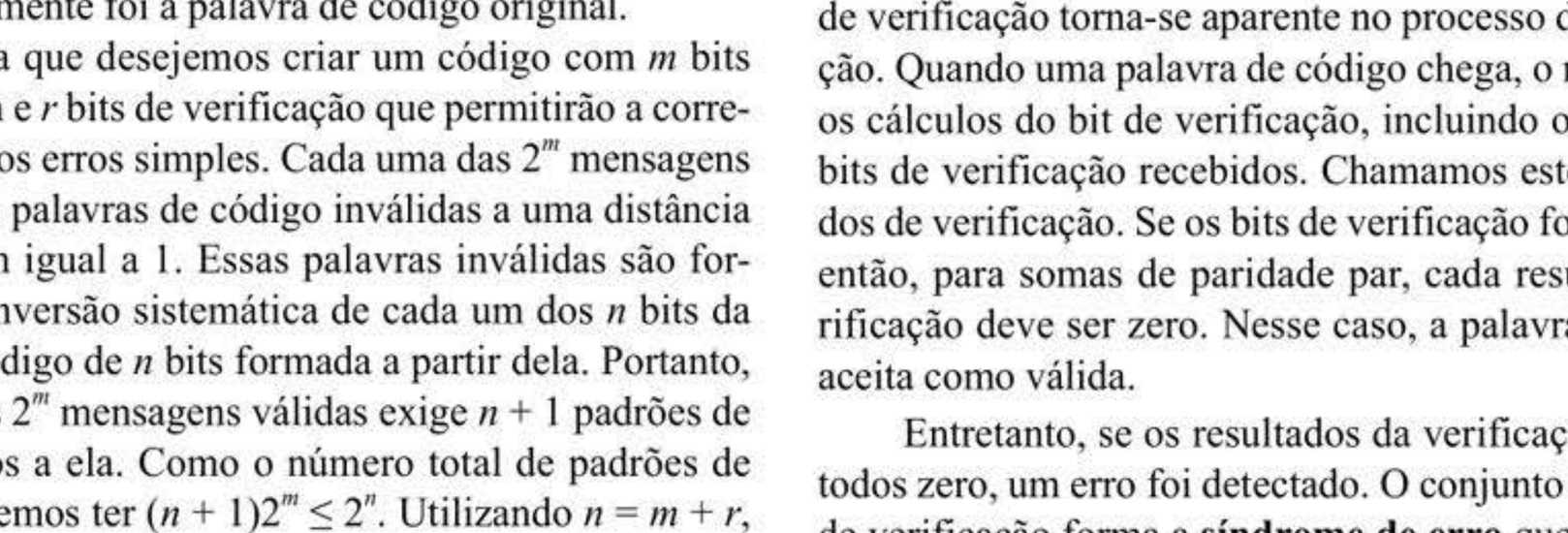


Figura 3.6 Exemplo de um código de Hamming (11,7) corrigindo um erro de único bit.

ou de dados) e o descarte dos bits de verificação geram a mensagem correta de um ASCII "A".

As distâncias de Hamming são valiosas para entender os blocos de códigos, e os códigos de Hamming são usados na memória de correção de erros. Contudo, a maioria das redes usa códigos mais fortes, como o **código de convolução**. Este é o único que veremos que não é um bloco de código. Em um código de convolução, um codificador processa uma sequência de bits de entrada e gera uma sequência de bits de saída. Não existe tamanho de mensagem ou limite de codificação natural, como em um bloco de código. A saída depende dos bits de entrada atual e anterior. Ou seja, o codificador tem memória. O número de bits anteriores do qual a saída depende é chamado **comprimento restritivo** do código. Os códigos de convolução são especificados em termos de sua taxa e comprimento restritivo.

Os códigos de convolução são muito usados em redes implantadas, por exemplo, como parte do sistema de telefonia móvel GSM, em comunicações por satélite e nas redes 802.11. Como exemplo, um código de convolução popular aparece na Figura 3.7. Esse código é conhecido como código de convolução da NASA, com $r = 1/2$ e $k = 7$, pois ele foi usado inicialmente para as missões espaciais Voyager, a partir de 1977. Desde então, ele tem sido bastante reutilizado, por exemplo, como parte do padrão 802.11.

Na Figura 3.7, cada bit de entrada no lado esquerdo produz dois bits de saída no lado direito, os quais são somas de operações XOR entre a entrada e o estado interno. Por lidar com bits e realizar operações lineares, esse é um código de convolução binário, linear. Como 1 bit de entrada produz 2 bits de saída, o código é 1/2. Ele não é sistemático, pois nenhum dos bits de saída é simplesmente o bit de entrada.

O estado interno é mantido em seis registradores de memória. Toda vez que outro bit é inserido, os valores nos registradores são deslocados para a direita. Por exemplo, se 111 for inserido e o estado inicial contiver zeros, o estado interno, escrito da esquerda para a direita, se tornará 100000, 110000 e 111000 após o primeiro, o segundo e o terceiro bits serem inseridos. Os bits de saída serão 11, seguidos por 10 e, depois, 01. São necessários sete deslocamentos para esvaziar uma entrada, de modo que ela não afete a saída. O comprimento da restrição desse código é, portanto, $k = 7$.

Um código de convolução é decodificado encontrando-se a sequência de bits de entrada que tem maior probabilidade de ter produzido a sequência observada de bits de saída (que inclui quaisquer erros). Para valores pequenos de k, isso é feito com um algoritmo bastante usado, desenvolvido por Viterbi (Forney, 1973). O algoritmo percorre a sequência observada, mantendo para cada etapa e para cada estado interno possível a sequência de entrada que teria produzido a sequência observada com o mínimo de erros. A sequência de entrada que exige o mínimo de erros no final é a mensagem mais provável.

Os códigos de convolução têm sido populares na prática porque é fácil fatorar a incerteza de um bit sendo 0 ou 1 na decodificação. Por exemplo, supondo que –1V seja o nível lógico 0 e +1V seja o nível lógico 1, poderíamos receber 0,9V e –0,1V para 2 bits. Em vez de mapear esses sinais como 1 e 0 imediatamente, gostaríamos de tratar 0,9V como "provavelmente 1" e –0,1V como "talvez 0" e corrigir a sequência como um todo. As extensões do algoritmo de Viterbi podem trabalhar com essas incertezas para oferecer uma correção de erros mais forte. Essa técnica de trabalho com a incerteza de um bit é chamada de **decodificação de decisão soft**. Por sua vez, a tarefa de decidir se cada bit é 0 ou 1 antes da correção de erros subsequente é chamada **decodificação de decisão hard**.

O terceiro tipo de código de correção de erros que descreveremos é o **código de Reed-Solomon**. Assim como os códigos de Hamming, os de Reed-Solomon são blocos de código lineares, e eles normalmente também são sistemáticos. Diferentemente dos códigos de Hamming, que operam sobre bits individuais, os códigos de Reed-Solomon operam sobre m símbolos de bit. Naturalmente, a matemática é mais complicada, de modo que descreveremos sua operação por analogia.

Os códigos de Reed-Solomon são baseados no fato de que cada polinômio de grau n é determinado unicamente por $n + 1$ pontos. Por exemplo, uma linha que tem a forma $ax + b$ é determinada por dois pontos. Os pontos extras na mesma linha são redundantes, o que é útil para a correção de erros. Imagine que temos dois pontos de dados que representam uma linha e enviamos esses dois pontos de dados mais dois pontos de verificação escolhidos para que se encontrem na mesma linha. Se um dos pontos for recebido com erro, ainda podemos recuperar os pontos de

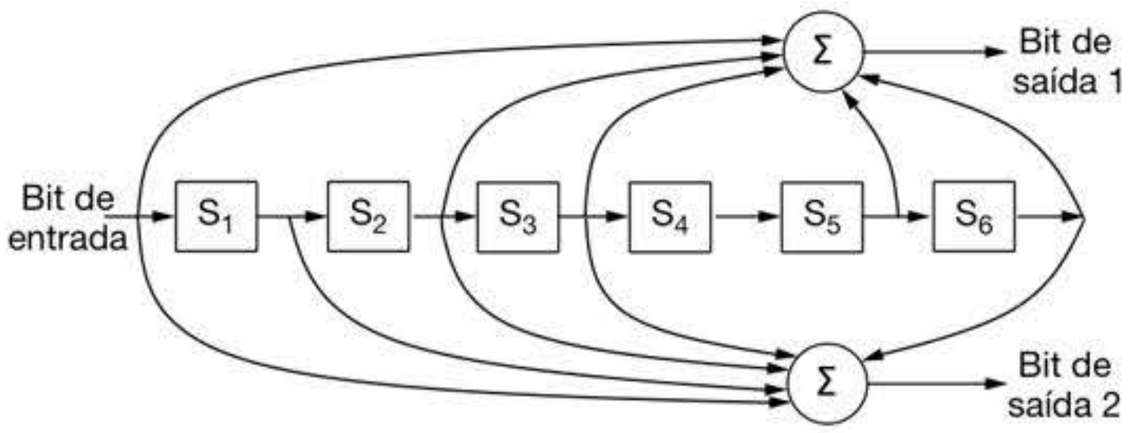


Figura 3.7 O código de convolução binário da NASA usado no padrão 802.11.

dados passando uma linha pelos pontos recebidos. Três dos pontos estarão na linha e um ponto, aquele com erro, não estará. Encontrando a linha, corrigiremos o erro.

Os códigos de Reed-Solomon, na realidade, são definidos como polinômios que operam por campos finitos, mas que funcionam de maneira semelhante. Para símbolos de m bits, as palavras de código têm $2^m - 1$ símbolos de comprimento. Uma escolha popular é tornar $m = 8$, de modo que os símbolos são bytes. Uma palavra de código tem, então, 255 bytes de comprimento. O código (255, 233) é bastante utilizado; ele acrescenta 22 símbolos redundantes a 233 símbolos de dados. A decodificação com correção de erros é feita com um algoritmo desenvolvido por Berlekamp e Massey, que pode realizar com eficiência a tarefa de ajuste para códigos de tamanho moderado (Massey, 1969).

Os códigos de Reed-Solomon são bastante utilizados na prática, em virtude de suas fortes propriedades de correção de erro, particularmente para erros em rajada. Eles são usados para DSL, dados sobre cabo, comunicações por satélite e talvez de forma mais intensa em CDs, DVDs e discos Blu-ray. Por serem baseados em símbolos de m bits, um erro de único bit e um erro em rajada de m bits são tratados simplesmente como um erro de símbolo. Quando $2t$ símbolos redundantes são somados, um código de Reed-Solomon é capaz de corrigir até t erros em qualquer um dos símbolos transmitidos. Isso significa, por exemplo, que o código (255, 233), que tem 22 símbolos redundantes, pode corrigir até 16 erros de símbolo. Como os símbolos podem ser consecutivos e ter 8 bits cada um, uma rajada de erros de até 128 bits pode ser corrigida. A situação é ainda melhor se o modelo de erro for de cancelamento (p. ex., um arranhão em um CD que destrói alguns símbolos). Nesse caso, até $2t$ erros podem ser corrigidos.

Os códigos de Reed-Solomon normalmente são usados em combinação com outros códigos, como um código de convolução. O pensamento é o seguinte: os códigos de convolução são eficazes no tratamento de erros de bit isolados, mas eles falharão, provavelmente com uma rajada de erros, se houver muitos erros no fluxo de bits recebido. Acrescentando um código Reed-Solomon dentro do código de convolução, a decodificação Reed-Solomon pode liquidar as rajadas de erros, uma tarefa na qual é muito bom. O código completo, então, oferece boa proteção contra erros simples e em rajada.

O último código de correção de erros que analisaremos é o de **verificação de paridade com baixa densidade**, ou **LDPC (Low-Density Parity Check)**. Os códigos LDPC são blocos de código lineares que foram inventados por Robert Gallagher em sua tese de doutorado (Gallagher, 1962). Assim como na maioria das teses, eles foram prontamente esquecidos, para serem reinventados apenas em 1995, quando os avanços na computação os tornaram viáveis.

Em um código LDPC, cada bit de saída é formado a partir de apenas uma fração dos bits de entrada. Isso leva a uma representação matricial do código, que tem uma baixa densidade de 1s, daí o nome para o código. As palavras de código recebidas são decodificadas com um algoritmo de aproximação que melhora interativamente com um melhor ajuste dos dados recebidos a uma palavra de código válida. Isso corrige erros.

Códigos LDPC são úteis para grandes tamanhos de bloco e têm excelente capacidade de correção de erros que, na prática, superam muitos outros códigos (incluindo aqueles que já examinamos). Por esse motivo, eles estão sendo rapidamente incluídos em novos protocolos e fazem parte do padrão para difusão de vídeo digital, 10 Gbps Ethernet, redes de linha de energia e a versão mais recente do 802.11. Você deverá ver muito mais deles nas redes do futuro.

3.2.2 Códigos de detecção de erros

Os códigos de correção de erros são muito utilizados em enlaces sem fio, conhecidos por serem ruidosos e propensos a erros em comparação à fiação de cobre ou à fibra óptica. Sem códigos de correção de erros, seria difícil conseguir algo. Contudo, usando-se fio de cobre ou fibra de alta qualidade, a taxa de erros é muito mais baixa e, assim, a detecção de erros e a retransmissão em geral são mais eficientes para lidar com o erro ocasional.

Examinaremos três códigos de detecção de erros. Todos eles são blocos de código lineares e sistemáticos:

1. Paridade.
2. Checksums.
3. Verificações de redundância cíclica (CRCs).

Para ver como eles podem ser mais eficientes do que os códigos de correção de erros, considere o primeiro código de detecção de erros, em que um único **bit de paridade** é acrescentado aos dados. O bit de paridade é escolhido de modo que o número de bits 1 na palavra de código seja par (ou ímpar). Fazer isso é equivalente a calcular o bit de paridade (par) como a soma de módulo 2 ou a operação XOR dos bits de dados. Por exemplo, quando 1011010 é enviado na paridade par, um bit é acrescentado ao final, para torná-lo 10110100. Com a paridade ímpar, 1011010 torna-se 10110101. Um código com um único bit de paridade tem uma distância de 2, pois qualquer erro de único bit produz uma palavra de código com a paridade errada. Isso significa que ele pode detectar erros de um único bit.

Considere um canal no qual os erros são isolados e a taxa de erros é de 10^{-6} por bit. Essa pode parecer uma taxa de erros pequena, mas é no mínimo uma taxa justa para um cabo longo que esteja desafiando a detecção de erros. Os enlaces de LAN comuns oferecem taxas de erro de bit de 10^{-10}. Defina o tamanho do bloco como 1.000 bits. Para proporcionar a correção de erros de blocos de 1.000 bits, sabemos, pela Equação 3.1, que são necessários 10 bits de

verificação. Assim, um megabit de dados necessitaria de 10.000 bits de verificação. Para simplesmente detectar um bloco com um único erro de 1 bit, um bit de paridade por bloco seria suficiente. A cada 1.000 blocos, será descoberto que um bloco possui erro e um bloco extra (1.001 bits) terá de ser transmitido para repará-lo. O overhead total para o método de detecção de erros e retransmissão é de apenas 2.001 bits por megabit de dados, contra 10.000 bits para um código de Hamming.

Uma dificuldade com esse esquema é que um único bit de paridade só pode detectar, de maneira confiável, um erro de único bit no bloco. Se o bloco tiver um erro em rajada longo, a probabilidade de ele ser detectado é de apenas 0,5, o que não é muito aceitável. As disparidades poderão ser consideravelmente melhoradas se cada bloco for enviado como uma matriz retangular com n bits de largura e k bits de altura. Agora, se calcularmos e enviarmos um bit de paridade para cada linha, até k erros de bit serão confiantemente detectados, desde que haja no máximo um erro por linha.

Contudo, há outra coisa que podemos fazer que oferece melhor proteção contra erros em rajada: calcular os bits de paridade sobre os dados em uma ordem diferente daquela em que os bits de dados são transmitidos pelo canal de comunicação. Isso é chamado de **entrelaçamento**. Nesse caso, calcularemos um bit de paridade para cada uma das n colunas e enviaremos todos os bits de dados como k linhas, enviando as linhas de cima para baixo e os bits em cada linha da esquerda para a direita, da maneira normal. Na última linha, enviamos os n bits de paridade. Essa ordem de transmissão pode ser vista na Figura 3.8 para $n = 7$ e $k = 7$.

O entrelaçamento é uma técnica geral para converter um código que detecta (ou corrige) erros isolados em um código que detecta (ou corrige) erros em rajada. Na Figura 3.8, quando ocorre um erro em rajada de tamanho $n = 7$, os bits que contêm erro estão espalhados por diferentes colunas. (Um erro em rajada não implica que todos os bits estejam errados, mas sim que pelo menos o primeiro e o último estejam. Na Figura 3.8, 4 bits foram invertidos por uma faixa de 7 bits.) No máximo 1 bit em cada uma das n colunas será afetado, de modo que os bits de paridade nessas colunas detectarão o erro. Esse método usa n bits de paridade sobre blocos de kn bits de dados para detectar um único erro em rajada de comprimento n ou menor.

Contudo, uma rajada de comprimento $n + 1$ passará sem ser detectada se o primeiro e o último bits forem invertidos e todos os outros bits estiverem corretos. Se o bloco estiver bastante alterado por uma extensa rajada ou por várias rajadas mais curtas, a probabilidade de que qualquer uma das n colunas tenha a paridade correta por acidente é 0,5, de modo que a probabilidade de um bloco com problema ser aceito quando não deveria é de 2^{-n}.

O segundo tipo de código de correção de erros, o **checksum**, está bastante relacionado aos grupos de bits de paridade. O termo "checksum" normalmente é usado para indicar um grupo de bits de verificação associados a uma mensagem, independentemente de como são calculados. Um grupo de bits de paridade é um exemplo de checksum. Contudo, existem outros checksums, mais robustos, baseados na soma acumulada dos bits de dados da mensagem. O checksum normalmente é colocado no final da mensagem, como complemento da função soma e não de soma. Desse modo, os erros podem ser detectados somando a palavra inteira de código recebida, tanto bits de dados quanto o checksum. Se o resultado for zero, nenhum erro foi detectado.

Um exemplo de checksum é a soma de verificação de 16 bits da Internet usada em todos os pacotes da rede como parte do protocolo IP (Braden et al., 1988). Esse checksum é uma soma dos bits da mensagem dividida por palavras de 16 bits. Como esse método opera sobre palavras, em vez de bits, assim como na paridade, os erros que deixam a paridade inalterada ainda podem alterar a soma e ser detectados. Por exemplo, se o bit de mais baixa ordem em duas palavras diferentes for invertido de 0 para 1, uma verificação de paridade entre esses bits deixaria de detectar um erro. Contudo, dois 1s serão acrescentados ao checksum de 16 bits para produzir um resultado diferente. O erro pode, então, ser detectado.

O checksum da Internet é calculado com a aritmética de complemento de um, em vez da soma de módulo 2^{16}.

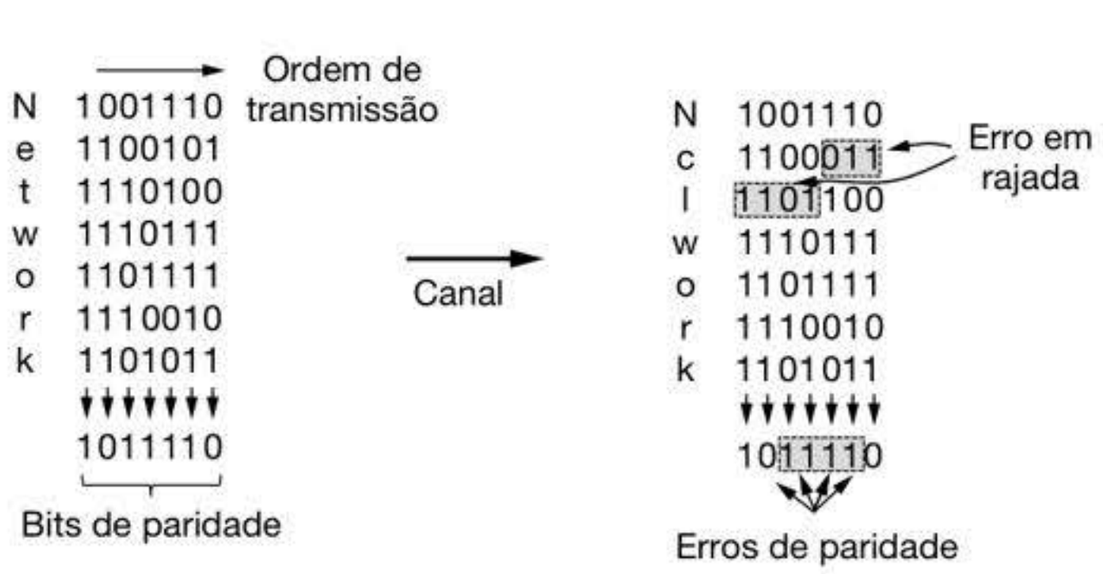


Figura 3.8 Entrelaçamento de bits de paridade para detectar um erro em rajada.

Na aritmética de complemento de um, um número negativo é o complemento bit a bit de seu correspondente positivo. Os computadores modernos trabalham na aritmética de complemento de dois, em que um número negativo é o complemento de um mais um. Em um computador com complemento de dois, a soma no complemento de um é equivalente a apanhar o total em módulo 2^{16} e somar qualquer overflow dos bits de alta ordem aos de baixa ordem. Esse algoritmo oferece uma cobertura mais uniforme dos dados pelos bits do checksum. De outra forma, dois bits de alta ordem podem ser somados, gerar overflow e serem perdidos sem alterar o total. Também existe outro benefício. O complemento de um tem duas representações de zero, todos os bits 0 e todos os bits 1. Isso permite um valor (p. ex., todos os bits 0) para indicar que não há um checksum, sem a necessidade de outro campo.

Ao longo de décadas, sempre foi considerado que os quadros a serem somados para verificação contêm bits aleatórios. Todas as análises dos algoritmos de checksum têm sido feitas sob essa hipótese. A inspeção de dados reais por Partridge et al. (1995) mostrou que essa suposição é bastante errada. Como consequência, os erros não detectados são, em alguns casos, muito mais comuns do que se havia imaginado.

O checksum da Internet em particular é eficiente e simples, mas oferece uma proteção fraca em alguns casos, exatamente porque essa é uma soma simples. Ele não detecta a exclusão ou o acréscimo de dados zero, nem a troca de partes da mensagem, e oferece pouca proteção contra pedaços da mensagem em que partes de dois pacotes são reunidas. A ocorrência desses erros pode parecer improvável por processos aleatórios, mas eles são simplesmente o tipo de erro que pode ocorrer com um hardware defeituoso.

Uma escolha melhor é o **checksum de Fletcher** (Fletcher, 1982). Ele inclui um componente posicional, somando o produto dos dados e sua posição à soma acumulada. Isso oferece melhor detecção das mudanças na posição dos dados.

Embora os dois esquemas anteriores às vezes possam ser adequados em camadas superiores, na prática, um terceiro tipo mais forte de código de detecção de erros tem o uso generalizado na camada de enlace: é o **código de redundância cíclica**, ou **CRC** (**Cyclic Redundancy Check**), também conhecido como **código polinomial**. Os códigos polinomiais são baseados no tratamento de sequências de bits como representações de polinômios com coeficientes de 0 e 1 apenas. Um quadro de k bits é considerado como a lista de coeficientes para um polinômio com k termos, variando de x^{k-1} a x^0. Dizemos que tal polinômio é de grau $k-1$. O bit de alta ordem (mais à esquerda) é o coeficiente de x^{k-1}, o próximo bit é o coeficiente de x^{k-2}, e assim por diante. Por exemplo, 110001 tem 6 bits e, portanto, representa um polinômio de seis termos com coeficientes 1, 1, 0, 0, 0 e 1: $1x^5 + 1x^4 + 0x^3 + 0x^2 + 0x^1 + 1x^0$.

A aritmética de polinômios é feita em módulo 2, de acordo com as regras da teoria algébrica. Ela não tem "vai uns" para a adição ou empréstimos para a subtração. Tanto a adição quanto a subtração são idênticos ao XOR. Por exemplo:

10011011	00110011	11110000	01010101
+ 11001010	+ 11001101	− 10100110	− 10101111
01010001	11111110	01010110	11111010

A divisão longa é efetuada do mesmo modo que em binário, exceto pelo fato de a subtração novamente ser de módulo 2. Diz-se que um divisor "cabe em" um dividendo se o dividendo tem a mesma quantidade de bits do divisor.

Quando o método do código polinomial é empregado, o transmissor e o receptor devem concordar em relação a um **polinômio gerador**, $G(x)$, antecipadamente. Tanto o bit de mais alta ordem quanto o de mais baixa ordem do polinômio gerador devem ser iguais a 1. Para calcular o CRC de um quadro com m bits, que corresponde ao polinômio $M(x)$, o quadro deve ter mais bits do que o polinômio gerador. A ideia é acrescentar um CRC ao final do quadro, de forma que o polinômio representado pelo quadro verificado pela soma seja divisível por $G(x)$. Quando obtiver o quadro verificado, o receptor tentará dividi-lo por $G(x)$. A existência de resto indica que houve um erro de transmissão.

O algoritmo para calcular o CRC é o seguinte:

1. Seja r o grau de $G(x)$. Acrescente r bits zero à extremidade de baixa ordem do quadro, de modo que ele passe a conter $m + r$ bits e corresponda ao polinômio $x^r M(x)$.
2. Divida a sequência de bits correspondente a $G(x)$ pela sequência de bits correspondente a $x^r M(x)$ utilizando a divisão de módulo 2.
3. Subtraia o resto (que tem sempre r ou menos bits) da sequência de bits correspondente a $x^r M(x)$ utilizando a subtração de módulo 2. O resultado é o quadro, com o checksum, que deverá ser transmitido. Chame o polinômio de $T(x)$.

A Figura 3.9 ilustra o cálculo referente a um quadro 1101011111, usando o gerador $G(x) = x^4 + x + 1$.

Deve ficar claro que $T(x)$ é divisível (em módulo 2) por $G(x)$. Em qualquer problema de divisão, se você subtrair o resto do dividendo, o resultado será divisível pelo divisor. Por exemplo, na base 10, se você dividir 210.278 por 10.941, o resto será 2.399. Subtraindo-se 2.399 de 210.278, o resultado final (207.879) será divisível por 10.941.

Agora vamos analisar o poder desse método. Que tipos de erro serão detectados? Imagine que ocorra um erro de transmissão de forma que, em lugar de chegar a sequência de bits correspondente a $T(x)$, seja recebida a soma $T(x) + E(x)$. Cada bit 1 em $E(x)$ corresponde a um bit que foi invertido. Se houver k bits 1 em $E(x)$, isso significa que ocorreram k erros de único bit. Um único erro em rajada é

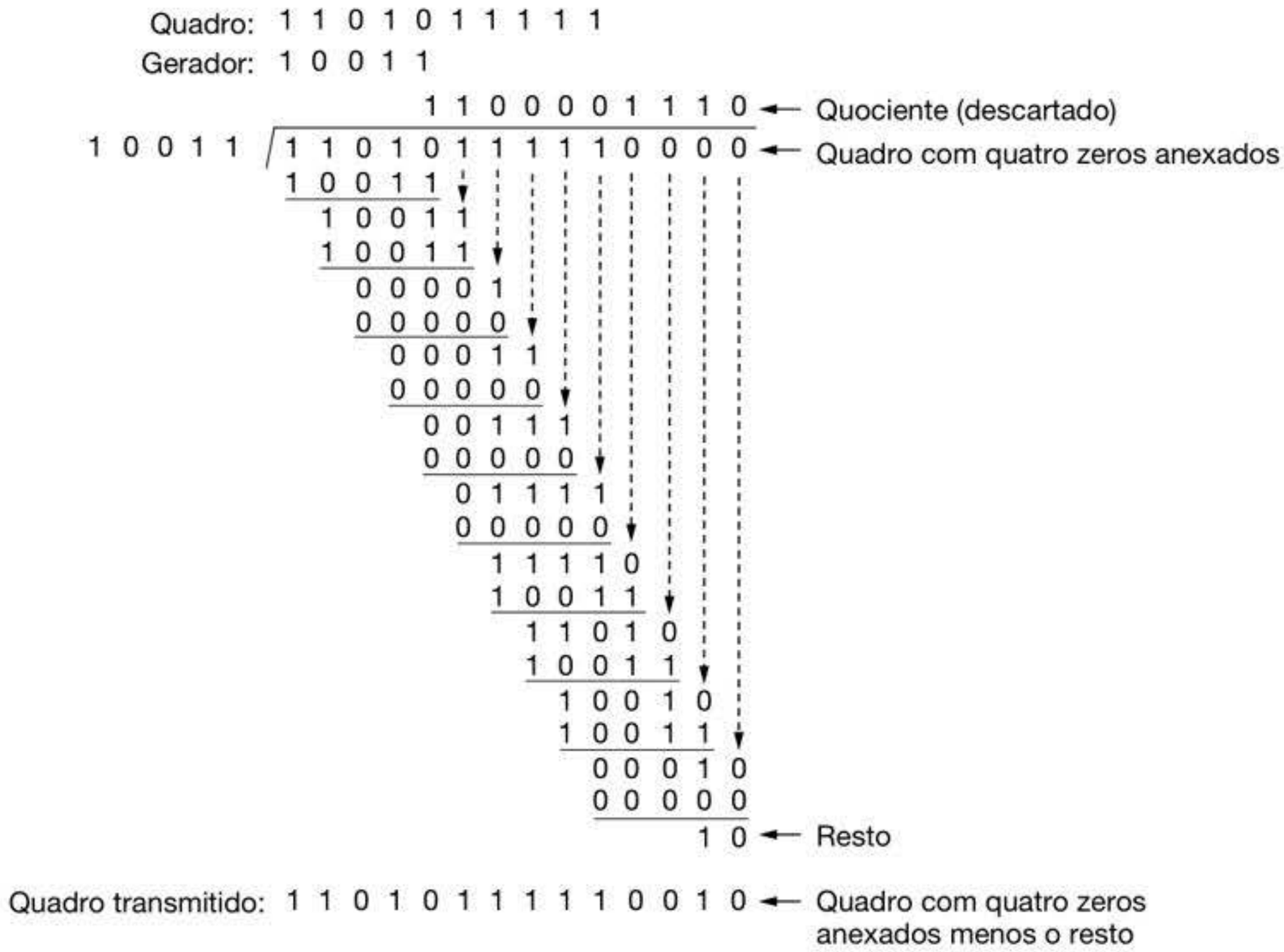


Figura 3.9 Exemplo de cálculo do CRC.

caracterizado por um bit 1 inicial, uma mistura de bits 0 e 1 e um bit 1 final, sendo todos os outros bits iguais a 0.

Ao receber o quadro com o checksum, o receptor o divide por $G(x)$; ou seja, ele calcula $[T(x) + E(x)]/G(x)$. $T(x)/G(x)$ é igual a 0; portanto, o resultado do cálculo é simplesmente $E(x)/G(x)$. Os erros que corresponderem a polinômios contendo $G(x)$ como fator serão simplesmente ignorados; todos os outros serão descobertos.

Se houver ocorrido um erro de único bit, $E(x) = x^i$, onde i determina o bit incorreto. Se contiver dois ou mais termos, $G(x)$ nunca dividirá $E(x)$; portanto, todos os erros de único bit serão detectados.

Se tiverem ocorrido dois erros isolados de único bit, $E(x) = x^i + x^j$, onde $i > j$. Como alternativa, esse cálculo pode ser representado como $E(x) = x^j(x^{i-j} + 1)$. Se considerarmos que $G(x)$ não é divisível por x, uma condição suficiente para todos os erros duplos serem detectados é que $G(x)$ não divida $x^k + 1$ para qualquer k até o valor máximo de $i - j$ (i.e., até o comprimento máximo do quadro). São conhecidos polinômios simples, de grau baixo, que protegem quadros longos. Por exemplo, $x^{15} + x^{14} + 1$ não dividirá $x^k + 1$ para qualquer valor de k abaixo de 32.768.

Se houver um número ímpar de bits com erros, $E(x)$ conterá um número ímpar de termos (p. ex., $x^5 + x^2 + 1$, mas não $x^2 + 1$). É interessante observar que nenhum polinômio com um número ímpar de termos tem $x + 1$ como fator no sistema de módulo 2. Ao tornar $x + 1$ um fator de $G(x)$, podemos detectar todos os erros que consistem em um número ímpar de bits invertidos. Estatisticamente, apenas isso já compreende metade dos casos.

Por último, e muito importante, um código polinomial com r bits de verificação detectará todos os erros em rajada que tiverem um tamanho $\leq r$. Um erro em rajada de tamanho k pode ser representado por $x^i(x^{k-1} + ... + 1)$, onde i determina a distância entre a rajada e a extremidade direita do quadro recebido. Se contiver um termo x^0, $G(x)$ não terá x^i como fator; portanto, se o grau da expressão entre parênteses for menor que o grau de $G(x)$, o resto nunca poderá ser igual a zero.

Se o tamanho da rajada for $r + 1$, o resto da divisão por $G(x)$ será zero se, e somente se, a rajada for idêntica a $G(x)$. Por definição de rajada, o primeiro e o último bits devem ser iguais a 1; assim, a correspondência entre os valores dependerá dos $r - 1$ bits intermediários. Se todas as combinações forem consideradas igualmente prováveis, a probabilidade de esse quadro incorreto ser aceito como válido será de $\frac{1}{2}^{r-1}$.

Também podemos mostrar que, ao ocorrer um erro em rajada com mais de $r + 1$ bits ou forem registradas várias rajadas mais curtas, a probabilidade de um quadro defeituoso passar despercebido poderá ser igual a $\frac{1}{2}^r$, supondo-se que todos os padrões de bits sejam igualmente prováveis.

Certos polinômios se tornaram padrões internacionais. O que é utilizado no IEEE 802 acompanhou o exemplo da Ethernet e é:

$$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + 1$$

Entre outras características interessantes, ele tem a propriedade de detectar todas as rajadas de comprimento 32 ou menores e todas as rajadas que afetam um número ímpar de bits. Tem sido muito usado desde a década de 1980, mas isso não significa que seja a melhor escolha. Usando uma busca computacional completa, Castagnoli et al. (1993) e Koopman (2002) encontraram os melhores CRCs. Esses CRCs têm uma distância de Hamming de 6 para os tamanhos de mensagem típicos, enquanto o padrão do IEEE CRC-32 tem uma distância de Hamming de apenas 4.

Apesar de o cálculo necessário para computar o checksum parecer complicado, Peterson e Brown (1961) mostraram que é possível criar um simples circuito shift register (registrador de deslocamento) para calcular e conferir os CRCs no hardware. Implementações mais recentes e mais rápidas são criadas regularmente (Mitra e Nyack, 2017). Na prática, esse hardware quase sempre é utilizado. Dezenas de padrões de rede compreendem diversos CRCs, incluindo praticamente todas as LANs (p. ex., Ethernet, 802.11) e enlaces ponto a ponto (p. ex., pacotes sobre SONET).

3.3 PROTOCOLOS BÁSICOS DE ENLACE DE DADOS

Como uma introdução ao estudo dos protocolos, vamos começar examinando três protocolos com graus de complexidade crescentes. Antes disso, é útil esclarecer algumas das suposições nas quais se baseia o modelo de comunicação.

3.3.1 Premissas básicas para simplificação

Processos independentes. Para começar, supomos que, na camada física, na camada de enlace de dados e na camada de rede existem processos independentes que se comunicam pelo envio de mensagens de um lado para outro. Uma implementação comum aparece na Figura 3.10. O processo da camada física e parte do processo da camada de enlace de dados funcionam em hardware dedicado, chamado **placa de interface de rede**, ou **NIC** (**Network Interface Card**). O restante do processo da camada de enlace e o processo da camada de rede atuam sobre a CPU principal como parte do sistema operacional, com o software para o processo da camada de enlace normalmente tomando a forma de um **driver de dispositivo**. No entanto, outras implementações também são possíveis (p. ex., três processos transferidos para um hardware dedicado, chamado **acelerador de rede**, ou três processos rodando na CPU principal a uma razão definida pelo software). Na realidade, a implementação preferida muda de uma década para a outra, com as mudanças de tecnologia. De qualquer forma, tratar as

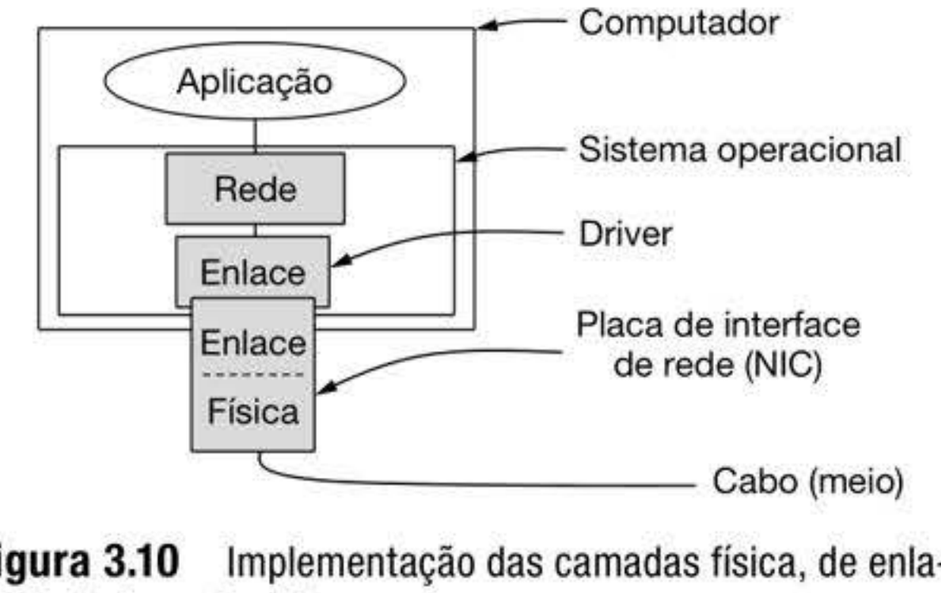


Figura 3.10 Implementação das camadas física, de enlace de dados e de rede.

três camadas como processos separados torna a discussão conceitualmente mais clara e também enfatiza a independência delas.

Comunicação unidirecional. Outra suposição de extrema importância é de que a máquina *A* deseja enviar um longo fluxo de dados à máquina *B* utilizando um serviço confiável e orientado a conexões. Mais adiante, consideraremos a situação em que *B* também deseja enviar dados a *A* simultaneamente. Supõe-se que *A* tenha um suprimento infinito de dados prontos para ser enviados, e nunca terá de esperar até que eles sejam produzidos. Quando a camada de dados de *A* solicitar dados, a camada de rede sempre será capaz de obedecer de imediato. (Mais adiante essa restrição também será superada.)

Máquinas e processos confiáveis. Também supomos que as máquinas não sofrerão panes. Isto é, esses protocolos lidam com erros de comunicação, mas não com os problemas causados por computadores que sofrem panes e são reiniciados.

No que se refere à camada de enlace de dados, o pacote repassado a ela pela camada de rede através da interface consiste em dados puros, em que cada bit deve ser entregue à camada de rede de destino. O fato de a camada de rede de destino interpretar parte do pacote como um cabeçalho não tem nenhum interesse para a camada de enlace de dados.

3.3.2 Noções básicas de transmissão e recepção

Quando a camada de enlace de dados aceita um pacote da camada de rede do transmissor, ela o encapsula em um quadro, acrescentando-lhe um cabeçalho e um final de enlace de dados (veja a Figura 3.1). Portanto, um quadro consiste em um pacote incorporado, algumas informações de controle (no cabeçalho) e um checksum (no final). Em seguida, o quadro é transmitido à camada de enlace de dados da outra máquina. Presumiremos que existem funções de biblioteca adequadas, *to_physical_layer* para enviar um quadro e *from_physical_layer* para receber um quadro. Essas

funções calculam ou acrescentam o checksum (o que normalmente é feito no hardware), de forma que os protocolos que desenvolvemos nesta seção não precisam se preocupar com isso. Por exemplo, eles poderiam usar o algoritmo de CRC discutido na seção anterior.

Inicialmente, o receptor nada tem a fazer. Ele apenas fica à espera de que algo aconteça. Nos exemplos de protocolos apresentados neste capítulo, indicaremos que a camada de enlace de dados está esperando que algo aconteça por meio da chamada função *wait_for_event*(&*event*). Essa função só retorna quando acontece algo (p. ex., quando chega um quadro). Ao retornar, a variável *event* informa o que aconteceu. O conjunto de eventos possíveis é diferente para os diversos protocolos a serem descritos e será definido separadamente para cada protocolo. Observe que, em uma situação mais realista, a camada de enlace de dados não ficará em um loop estrito à espera de um evento, como sugerimos, mas receberá uma interrupção, o que a fará interromper o que quer que esteja fazendo para manipular o quadro recebido. Apesar disso, por simplicidade, ignoraremos todos os detalhes de atividades paralelas na camada de enlace de dados, e presumiremos que ela se dedica em tempo integral apenas ao tratamento do nosso canal.

Quando um quadro chega ao receptor, este recalcula o checksum. Se este estiver incorreto (ou seja, se houve um erro de transmissão), a camada de enlace de dados será informada (*event* = *cksum_err*). Se o quadro recebido tiver chegado intacto, a camada de enlace de dados também será informada (*event* = *frame_arrival*), para que ela possa receber o quadro para inspeção usando *from_physical_layer*. Assim que recebe um quadro sem danos, a camada de enlace de dados verifica as informações de controle contidas no cabeçalho e, se tudo estiver correto, repassa a porção do pacote à camada de rede. Em nenhuma circunstância o cabeçalho do quadro será entregue à camada de rede.

Há uma boa razão para que a camada de rede nunca receba nenhuma parte do cabeçalho do quadro: manter os protocolos de rede e de enlace de dados completamente separados. Desde que a camada de rede não saiba absolutamente nada sobre o protocolo de enlace de dados ou sobre o formato do quadro, esses itens poderão ser alterados sem exigir mudanças no software da camada de rede. Isso acontece sempre que uma nova NIC é instalada em um computador. A utilização de uma interface rígida entre a camada de rede e a de enlace de dados simplifica bastante o projeto do software, pois os protocolos de comunicação das diferentes camadas podem evoluir de forma independente.

A Figura 3.11 mostra algumas declarações (na linguagem C) comuns a muitos dos protocolos que serão discutidos mais adiante. Cinco estruturas de dados são definidas no código: *boolean*, *seq_nr*, *packet*, *frame_kind* e *frame*. Um *boolean* é do tipo enumerado e pode assumir os valores verdadeiro (*true*) e falso (*false*). Um *seq_nr* é um inteiro pequeno usado para numerar os quadros, para facilitar sua distinção. Esses números de sequência variam de 0 até *MAX_SEQ* (inclusive), que é definido em cada protocolo que tenha necessidade. Um *packet* é a unidade de informação trocada entre as camadas de rede e de enlace de dados da mesma máquina, ou entre pares da camada de rede. Em nosso modelo, ele sempre contém *MAX_PKT* bytes; no entanto, de modo mais realista, ele teria comprimento variável.

A estrutura *frame* é composta por quatro campos: *kind*, *seq*, *ack* e *info*; os três primeiros contêm informações de controle, e o último pode conter os dados reais a serem transferidos. Esses campos de controle são chamados coletivamente de **cabeçalho do quadro**.

O campo *kind* indica se há dados no quadro, pois alguns protocolos distinguem quadros que contêm exclusivamente informações de controle daqueles que armazenam dados além dessas informações. Os campos *seq* e *ack* são usados para números de sequência e confirmações, respectivamente (seu uso será descrito detalhadamente mais adiante). O campo *info* de um quadro de dados contém um único pacote; o campo *info* de um quadro de controle não é usado. Uma implementação mais realista utilizaria um campo *info* de comprimento variável; nos quadros de controle, esse campo seria completamente omitido.

Novamente, é importante compreender o relacionamento entre um pacote e um quadro (veja a Figura 3.1). A camada de rede monta um pacote tomando uma mensagem da camada de transporte e acrescentando a ela o cabeçalho da camada de rede. Esse pacote é repassado à camada de enlace de dados para inclusão no campo *info* de um quadro que esteja sendo enviado. Quando o quadro chega ao destino, a camada de enlace de dados extrai o pacote do quadro e o envia à camada de rede. Dessa forma, a camada de rede pode atuar como se as máquinas pudessem trocar pacotes diretamente.

Na Figura 3.11 também estão listadas diversas funções, que são rotinas de biblioteca cujos detalhes são dependentes da implementação e cujo funcionamento interno não será discutido aqui. A função *wait_for_event* permanece à espera de que algo aconteça, como mencionamos anteriormente. As funções *to_network_layer* e *from_network _layer* são usadas pela camada de enlace de dados para enviar pacotes à camada de rede e aceitar pacotes dela, respectivamente. Observe que *from_physical_layer* e *to_physical_layer* repassam quadros entre a camada de enlace de dados e a camada física. Em outras palavras, *to_network_layer* e *from_network_layer* lidam com a interface entre as camadas 2 e 3, enquanto *from_physical_layer* e *to_physical_layer* lidam com a interface entre as camadas 1 e 2.

Na maioria dos protocolos, supomos o uso de um canal não confiável que perde quadros inteiros ocasionalmente. Para se recuperar dessas calamidades, sempre que envia um quadro, a camada de enlace de dados transmissora tem de inicializar um timer interno. Se nenhuma confirmação tiver sido recebida dentro de um intervalo predeterminado, o timer expirará por timeout e a camada de enlace de dados receberá um sinal de interrupção.

```
#define MAX_PKT 1024                                        /* determina tamanho do pacote em bytes */
typedef enum {false, true} boolean;                         /* tipo boolean */
typedef unsigned int seq_nr;                                /* números de sequência ou ack */
typedef struct {unsigned char data[MAX_PKT];}packet;        /* definição do pacote */
typedef enum {data, ack, nak} frame_kind;                   /* definição de frame_kind */

typedef struct {                                            /* quadros são transportados nesta camada */
   frame_kind kind;                                         /* que tipo de quadro é este? */
   seq_nr seq;                                              /* número de sequência */
   seq_nr ack;                                              /* número de confirmação */
   packet info;                                             /* o pacote da camada de rede */
} frame;

/* Espera que um evento aconteça; retorna o tipo de evento em event. */
void wait_for_event(event_type *event);

/* Busca um pacote da camada de rede para transmissão pelo canal. */
void from_network_layer(packet *p);

/* Entrega informação de um quadro que chega à camada de rede. */
void to_network_layer(packet *p);

/* Recebe um quadro de entrada da camada física e o copia para r. */
void from_physical_layer(frame *r);

/* Passa o quadro à camada física para transmissão. */
void to_physical_layer(frame *s);

/* Inicia o timer e habilita o evento timeout. */
void start_timer(seq_nr k);

/* Termina o timer e desativa o evento timeout. */
void stop_timer(seq_nr k);

/* Inicia um timer auxiliar e habilita o ack_timeout event. */
void start_ack_timer(void);

/* Encerra o timer auxiliar e desabilita o evento ack_timeout. */
void stop_ack_timer(void);

/* Permite que a camada de rede gere um evento network_layer_ready. */
void enable_network_layer(void);

/* Proíbe a camada de rede de um evento network_layer_ready. */
void disable_network_layer(void);

/* Macro inc é expandido em linha: incrementa k de modo circular. */
#define inc(k) if (k < MAX_SEQ)k = k + 1; else k = 0
```

Figura 3.11 Algumas definições utilizadas nos protocolos apresentados a seguir. Essas definições estão armazenadas no arquivo *protocol.h*.

Em nossos protocolos, isso é tratado permitindo-se à função *wait_for_event* retornar *event = timeout*. As funções *start_timer* e *stop_timer* ativam e desativam o timer, respectivamente. Os eventos de timeout só são possíveis quando o timer está funcionando e antes que *stop_timer* seja chamado. É explicitamente permitido chamar *start_timer* enquanto o timer está funcionando; esse tipo de chamada simplesmente reinicializa o timer para provocar o próximo timeout, depois de decorrer um intervalo do timer (a menos que ele seja reiniciado ou desativado).

As funções *start_ack_timer* e *stop_ack_timer* controlam um timer auxiliar cuja finalidade é gerar confirmações sob determinadas condições.

As funções *enable_network_layer* e *disable_network_layer* são usadas nos protocolos mais sofisticados, para os quais não mais supomos que a camada de rede sempre terá pacotes a serem enviados. Quando a camada de enlace de dados habilita a camada de rede, esta passa a ter permissão para causar uma interrupção sempre que tiver um pacote para enviar. Isso é indicado por *event = network_layer_ready*. Quando a camada de rede está inativa, ela não pode causar tais eventos. Definindo com cuidado os momentos em que ativa e desativa a camada de rede, a camada de enlace de dados pode impedir que a camada de rede fique sobrecarregada com pacotes para os quais não dispõe de espaço no buffer.

Os números de sequência dos quadros estão sempre na faixa de 0 a *MAX_SEQ* (inclusive), onde *MAX_SEQ* tem um valor diferente para os diversos protocolos. Com frequência, é necessário aumentar um número de sequência em uma unidade, de forma circular (i.e., *MAX_SEQ* é seguido por 0). A macro *inc* cuida desse incremento. Ela é definida como uma macro porque é usada em linha no caminho crítico. Como veremos adiante, com frequência o processamento de protocolos é o fator que limita o desempenho da rede; portanto, a definição de operações simples como macros (em vez de funções) não afeta a legibilidade do código, mas melhora o desempenho.

As declarações da Figura 3.11 fazem parte de cada um dos protocolos brevemente apresentados a seguir. Para economizar espaço e facilitar a consulta, essas declarações foram extraídas dos protocolos e são apresentadas juntas, mas conceitualmente elas devem estar integradas aos protocolos. Na linguagem C, essa integração é feita inserindo-se as definições em um arquivo de cabeçalho especial, neste caso *protocol.h*, e utilizando-se o recurso #include do pré-processador C, que inclui essas definições nos arquivos de protocolo.

3.3.3 Protocolo simplex da camada de enlace de dados

Nesta seção, examinaremos três protocolos simples, cada um capaz de lidar com uma situação mais realista que a anterior.

Utopia: sem controle de fluxo ou correção de erros

Como primeiro exemplo, consideraremos o protocolo mais simples possível, pois não se preocupa com a possibilidade de algo sair errado. Os dados são transmitidos em apenas um sentido. As camadas de rede do transmissor e do receptor estão sempre prontas. O tempo de processamento pode ser ignorado. O espaço disponível em buffer é infinito. E o melhor de tudo é que o canal de comunicação entre as camadas de enlace de dados nunca é danificado nem perde quadros. Esse protocolo absolutamente imaginário, que denominaremos "utopia", é simplesmente para mostrar a estrutura básica com a qual trabalharemos. Sua implementação aparece na Figura 3.12.

O protocolo consiste em dois procedimentos distintos, um que envia informações e outro que as recebe. O procedimento no transmissor é executado na camada de enlace de dados da máquina de origem, e no receptor é executado na camada de enlace de dados da máquina de destino. Não são usados números de sequência ou de confirmação; portanto, *MAX_SEQ* não é necessário. O único tipo de evento possível é *frame_arrival* (ou seja, a chegada de um quadro não danificado).

No transmissor há um loop while infinito que envia os dados o mais rápido possível. O corpo do loop é formado por três ações: buscar um pacote da (sempre prestativa) camada de rede, criar um quadro de saída utilizando a variável *s* e transmitir o quadro ao destino. Apenas o campo *info* do quadro é usado por esse protocolo, pois os outros campos se referem ao controle de fluxo e de erros e, nesse caso, não há erros nem restrições de controle de fluxo.

O receptor é igualmente simples. No início, ele espera que algo aconteça, e a única possibilidade é a chegada de um quadro não danificado. Finalmente, o quadro chega e a função *wait_for_event* retorna, com *event* definido como *frame_arrival* (o que, de qualquer forma, é ignorado). A chamada *from_physical_layer* remove o quadro recém-chegado do buffer do hardware e o coloca na variável *r*, onde o código receptor poderá buscá-lo quando necessário. Por fim, a parte referente aos dados é repassada à camada de rede, e a camada de enlace de dados volta a esperar pelo próximo quadro, ficando efetivamente em suspenso até a sua chegada.

O protocolo utópico (sem restrições) é imaginário porque não trata nem do controle de fluxo nem da correção de erros. Seu processamento é próximo ao de um serviço não confirmado não orientado a conexões, que conta com as camadas mais altas para resolver esses problemas, embora até mesmo um serviço desse tipo realize alguma detecção de erros.

Acrescentando controle de fluxo: stop-and-wait

Agora, trataremos do problema de impedir que o transmissor sobrecarregue o receptor com quadros mais rapidamente do que ele consegue processá-los. Essa situação pode facilmente acontecer na prática, de modo que é muito importante poder impedi-la. No entanto, continuamos supondo que o canal de comunicação não apresenta erros e que o tráfego de dados ainda é do tipo simplex.

Uma solução é montar o receptor para que seja poderoso o bastante para processar um fluxo contínuo de quadros de ponta a ponta (ou, de modo equivalente, definir a camada de enlace para que seja lenta o bastante para que o receptor possa acompanhar). Deve ter buffer e capacidade de processamento suficientes para atuar na velocidade da linha e deve ser capaz de passar os quadros recebidos à camada de rede com rapidez suficiente. Contudo, essa é uma solução no pior dos casos. Ela exige hardware dedicado e pode desperdiçar recursos se a utilização do enlace for quase sempre baixa. Além do mais, apenas passa o problema de lidar com um emissor muito rápido para outro lugar; nesse caso, para a camada de rede.

Uma solução mais geral para esse problema é fazer o receptor oferecer feedback ao transmissor. Depois de enviar um pacote à sua camada de rede, o receptor envia um pequeno quadro fictício de volta ao transmissor, permitindo a transmissão do próximo quadro. Após o envio de um quadro, o protocolo exige que o transmissor espere sua vez,

```
/* O protocolo 1 (utopia) oferece transmissão de dados em um único sentido, do transmissor
   para o receptor. Pressupõe-se que o canal de comunicação é livre de erros e que
   o receptor é capaz de processar toda a entrada de uma forma infinitamente rápida.
   Consequentemente, o transmissor permanece em um loop enviando os dados com a maior
   rapidez possível. */
typedef enum {frame_arrival} event_type;
#include "protocol.h"
void sender1(void)
{
   frame s;                          /* buffer para um quadro de saída */
   packet buffer;                    /* buffer para um pacote de saída */
   while (true) {
       from_network_layer(&buffer);  /* pega algo para enviar */
       s.info = buffer;              /* copia para s, para transmissão */
       to_physical_layer(&s);        /* envia-o pelo caminho */
   }                                 /* O amanhã, o amanhã, o amanhã,
                                        avança em pequenos passos, dia após dia,
                                        até a última sílaba da recordação.
                                           – Macbeth, V, v */
}
void receiver1(void)
{
   frame r;
   event_type event;                 /* preenchido ao se esperar ou aguardar, mas não usado aqui */
   while (true) {
       wait_for_event(&event);       /* única possibilidade é frame_arrival */
       from_physical_layer(&r);      /* recebe o quadro que chega */
       to_network_layer(&r.info);    /* passa os dados à camada de rede */
   }
}
```

Figura 3.12 Um protocolo simplex utópico.

até a chegada de um pequeno quadro fictício (i.e., de confirmação). Esse atraso é um exemplo simples de protocolo de controle de fluxo.

Os protocolos nos quais o transmissor envia um quadro e em seguida espera por uma confirmação antes de continuar sua operação são chamados **stop-and-wait** (**pare e espere**). A Figura 3.13 mostra um exemplo de protocolo simplex stop-and-wait.

Apesar de o tráfego de dados nesse exemplo ser simplex, indo apenas do transmissor ao receptor, os quadros são enviados em ambas as direções. Consequentemente, o canal de comunicação entre as duas camadas de enlace de dados deve ser capaz de realizar a transferência bidirecional de informações. No entanto, esse protocolo acarreta uma rígida alternância de fluxo: primeiro o transmissor envia um quadro, depois o receptor envia outro; em seguida, o transmissor envia mais um quadro, e então o receptor envia outro, e assim por diante. Um canal fisico half-duplex seria suficiente nesse caso.

A exemplo do protocolo 1, o transmissor começa extraindo um pacote da camada de rede, utilizando-o para criar um quadro que em seguida é transmitido a seu destino. Todavia, agora, ao contrário do que ocorre no protocolo 1, o transmissor deve aguardar a chegada de um quadro de confirmação antes de tornar a entrar em loop e buscar o próximo pacote da camada de rede. A camada de enlace de dados do transmissor não precisa sequer inspecionar o quadro recebido, pois só há uma possibilidade: ele é sempre uma confirmação.

A única diferença entre *receiver1* e *receiver2* é que, após entregar um pacote à camada de rede, o *receiver2* envia um quadro de confirmação de volta ao transmissor, antes de entrar mais uma vez no loop de espera. Como apenas a chegada do quadro de volta ao transmissor é importante, e não seu conteúdo, o receptor não precisa incluir qualquer informação específica no quadro.

Acrescentando correção de erros: números de sequência e ARQ

Agora, vamos considerar a situação normal de um canal de comunicação no qual ocorrem erros. Os quadros podem ser danificados ou completamente perdidos. No entanto, supomos que, se um quadro for danificado em trânsito, o hardware receptor detectará essa ocorrência ao calcular o checksum. Se o quadro for danificado de tal forma que o checksum nunca esteja correto – uma possibilidade muito improvável –, o protocolo em questão (e todos os outros protocolos) poderá apresentar falhas (i.e., poderá entregar um pacote incorreto à camada de rede).

```
/* O protocolo 2 (stop-and-wait) também implementa um fluxo de dados unidirecional entre
   o transmissor e o receptor. Presume-se mais uma vez que o canal de comunicação é
   totalmente livre de erros, como no protocolo 1. No entanto, dessa vez, o receptor tem buffer
   e velocidade de processamento finitos; portanto, o protocolo deverá impedir explicitamente
   que o transmissor sobrecarregue o receptor enviando dados mais rapidamente do que ele é
   capaz de processar. */
typedef enum {frame_arrival} event_type;
#include "protocol.h"
void sender2(void)
{
   frame s;                              /* buffer para um quadro de saída */
   packet buffer;                        /* buffer para um pacote de saída */
   event_type event;                     /* frame_arrival é a única possibilidade */
   while (true) {
       from_network_layer(&buffer);      /* apanha algo para enviar */
       s.info = buffer;                  /* copia para s, para transmissão */
       to_physical_layer(&s);            /* pequeno quadro de adeus */
       wait_for_event(&event);           /* não avança até um sinal verde */
   }
}
void receiver2(void)
{
   frame r, s;                           /* buffers para quadros */
   event_type event;                     /* frame_arrival é a única possibilidade */
   while (true) {
       wait_for_event(&event);           /* a única possibilidade é frame_arrival */
       from_physical_layer(&r);          /* apanha o quadro de entrada */
       to_network_layer(&r.info);        /* passa os dados para a camada de rede */
       to_physical_layer(&s);            /* envia quadro fictício para acordar o transmissor */
   }
}
```

Figura 3.13 Um protocolo simplex stop-and-wait.

À primeira vista, pode parecer que uma variação do protocolo 2 seria viável com a inclusão de um timer. O transmissor poderia enviar um quadro, mas o receptor só enviaria um quadro de confirmação se os dados fossem recebidos corretamente. Se um quadro danificado chegasse ao receptor, ele seria descartado. Após certo tempo, o transmissor alcançaria seu timeout e enviaria o quadro mais uma vez. Esse processo seria repetido até que o quadro finalmente chegasse intacto.

Esse esquema tem uma falha fatal. Pense no problema e tente descobrir o que poderia estar errado antes de continuar a leitura.

Para verificar o que poderia estar errado, lembre-se de que a função dos processos da camada de enlace de dados é oferecer comunicações transparentes e livres de erros entre os processos da camada de rede. A camada de rede da máquina *A* envia uma série de pacotes à camada de enlace de dados da mesma máquina. Esta, por sua vez, deve se certificar de que a camada de enlace de dados da máquina *B* enviará uma série idêntica de pacotes à camada de rede da mesma máquina. Em particular, a camada de rede da máquina *B* não tem como saber se um pacote foi perdido ou duplicado; portanto, a camada de enlace de dados deve garantir que nenhuma combinação de erros de transmissão, mesmo improvável, possa fazer um pacote duplicado ser entregue à camada de rede.

Considere a seguinte situação:

1. A camada de rede de *A* envia o pacote 1 à sua camada de enlace de dados. O pacote é corretamente recebido em *B* e repassado à camada de rede de *B*. *B* envia um quadro de confirmação de volta a *A*.
2. O quadro de confirmação se perde por completo. Ele simplesmente nunca chega ao destino. Tudo seria muito mais simples se o canal tivesse adulterado e perdido apenas quadros de dados, não quadros de controle. No entanto, para nossa tristeza, o canal não faz distinção entre quadros.
3. Finalmente, a camada de enlace de dados de *A* tem seu limite de tempo esgotado. Como não recebeu uma confirmação, ela presume (incorretamente) que seu quadro de dados se perdeu ou foi danificado e envia mais uma vez o quadro contendo o pacote 1.
4. O quadro duplicado também chega perfeitamente à camada de enlace de dados de *B* e é repassado de imediato, sem maiores problemas, à sua camada de rede. Caso *A* esteja enviando um arquivo a *B*, uma parte do

arquivo será duplicada (i.e., a cópia do arquivo criado por *B* estará incorreta e o erro não será detectado). Em outras palavras, o protocolo falhará.

Certamente, precisamos proporcionar ao receptor alguma forma de distinguir entre um quadro que ele está recebendo pela primeira vez e uma retransmissão. A maneira mais óbvia de conseguir isso é fazer o transmissor incluir um número de sequência no cabeçalho de cada quadro enviado. Dessa forma, o receptor poderá verificar o número de sequência de cada quadro recebido para confirmar se esse é um novo quadro ou se é uma cópia a ser descartada.

Como o protocolo deve ser correto e o campo de número de sequência no cabeçalho provavelmente é pequeno para usar o enlace de modo eficiente, surge a seguinte pergunta: qual é a quantidade mínima de bits necessária para o número de sequência? O cabeçalho poderia oferecer 1 bit, alguns bits, um byte ou múltiplos bytes para um número de sequência, dependendo do protocolo. O importante é que ele deve transportar números de sequência grandes o suficientes para que o protocolo funcione corretamente, ou o protocolo não terá valor.

A única ambiguidade nesse protocolo ocorre entre um quadro, m, e seu sucessor direto, $m + 1$. Se o quadro m tiver sido perdido ou danificado, o receptor não o confirmará; portanto, o transmissor continuará tentando enviá-lo. Uma vez que o quadro tenha sido corretamente recebido, o receptor enviará uma confirmação de volta ao transmissor. E aqui surge um problema potencial. Dependendo do fato de o quadro de confirmação voltar ao transmissor corretamente ou não, o transmissor poderá tentar enviar m ou $m + 1$.

No transmissor, o evento que dispara a transmissão do quadro $m + 1$ é a chegada de uma confirmação para o quadro m. Mas essa situação implica que $m - 1$ foi recebido corretamente e, além disso, que sua confirmação também foi recebida corretamente pelo transmissor. Caso contrário, o transmissor não teria iniciado com m, muito menos estaria considerando $m + 1$. Por conseguinte, a única ambiguidade é entre um quadro e seu predecessor ou sucessor imediato, não entre os próprios predecessores ou sucessores.

Um número de sequência de 1 bit (0 ou 1) é, portanto, suficiente. A cada instante, o receptor espera o próximo número de sequência. Quando chega um quadro contendo um número de sequência correto, ele é aceito e repassado à camada de rede, e depois confirmado. Em seguida, o número de sequência esperado é incrementado na base 2 (ou seja, 0 passa a ser 1 e 1 passa a ser 0). Qualquer quadro recebido que contenha o número de sequência errado será rejeitado por ser considerado uma cópia. Contudo, a última confirmação válida é repetida, de forma que o transmissor finalmente pode descobrir que o quadro foi recebido.

Um exemplo desse tipo de protocolo é mostrado na Figura 3.14. Os protocolos nos quais o transmissor espera por uma confirmação positiva antes de passar para o próximo item de dados frequentemente são chamados de **solicitação de repetição automática**, ou **ARQ** (**Automatic Repeat reQuest**), ou **confirmação positiva com retransmissão**, ou **PAR** (**Positive Acknowledgement with Retransmission**). A exemplo do protocolo 2, ele também transmite dados em apenas um sentido.

O protocolo 3 difere de seus predecessores pelo fato de tanto o transmissor quanto o receptor terem uma variável cujo valor é memorizado enquanto a camada de enlace de dados se encontra em estado de espera. Em *next_frame_to_send*, o transmissor memoriza o número de sequência do próximo quadro a ser enviado; em *frame_expected*, o receptor memoriza o número de sequência do próximo quadro esperado. Cada protocolo tem uma breve fase de inicialização antes de entrar no loop infinito.

Após enviar um quadro, o transmissor ativa o timer. Caso já esteja ativado, ele será reiniciado para permitir a contagem de outro intervalo, o qual deve ser escolhido de forma que haja tempo suficiente para o quadro chegar ao receptor, para o receptor processá-lo na pior das hipóteses e para o quadro de confirmação ser enviado de volta ao transmissor. Somente quando o intervalo tiver se esgotado, poderemos supor com segurança que o quadro transmitido ou sua confirmação se perdeu, e que será necessário enviar uma cópia. Se o intervalo de timeout for definido com um valor curto demais, o transmissor enviará quadros desnecessários. Embora não afetem a exatidão do protocolo, esses quadros extras prejudicarão o desempenho.

Depois de transmitir um quadro e ativar o timer, o transmissor espera que algo interessante aconteça. Existem apenas três possibilidades: o quadro de confirmação chegar sem danos, o quadro de confirmação chegar com erro ou o timer expirar. Se uma confirmação válida for recebida, o transmissor buscará o próximo pacote em sua camada de rede e o colocará no buffer, substituindo o pacote anterior. Ele também aumentará o número de sequência. Se for recebido um quadro com erro ou se o timer expirar, o buffer e o número de sequência permanecerão inalterados, de modo que uma cópia do quadro poderá ser enviada. De qualquer forma, o conteúdo do buffer (tanto o próximo pacote como uma cópia) é enviado em seguida.

Quando um quadro válido chega ao receptor, seu número de sequência é conferido, para verificar se ele é uma cópia. Se não for, o quadro será aceito, enviado à camada de rede, e uma confirmação será gerada. Cópias e quadros danificados não serão repassados à camada de rede, mas eles fazem o último quadro recebido corretamente ser confirmado para sinalizar ao transmissor para avançar ao próximo quadro ou retransmitir um quadro danificado.

3.4 MELHORANDO A EFICIÊNCIA

Nos protocolos apresentados anteriormente, os quadros de dados eram transmitidos em apenas um sentido. Em situações

```
/* O protocolo 3 (PAR) permite que dados unidirecionais fluam por um canal não confiável. */

#define MAX_SEQ 1                                   /* deve ser 1 para o protocolo 3 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void sender3(void)
{
  seq_nr next_frame_to_send;                        /* número seq do próximo quadro de saída */
  frame s;                                          /* variável auxiliar */
  packet buffer;                                    /* buffer para pacote de saída */
  event_type event;

  next_frame_to_send = 0;                           /* inicia números de sequência de saída */
  from_network_layer(&buffer);                      /* busca primeiro pacote */
  while (true) {
    s.info = buffer;                                /* monta um quadro para transmissão */
    s.seq = next_frame_to_send;                     /* insere número de sequência no quadro */
    to_physical_layer(&s);                          /* envia o quadro */
    start_timer(s.seq);                             /* se a resposta levar muito tempo, timeout */
    wait_for_event(&event);                         /* frame_arrival, cksum_err, timeout */
    if (event == frame_arrival) {
      from_physical_layer(&s);                      /* obtém a confirmação */
      if (s.ack == next_frame_to_send) {
        stop_timer(s.ack);                          /* desliga o timer */
        from_network_layer(&buffer);                /* pega o próximo quadro a enviar */
        inc(next_frame_to_send);                    /* inverte next_frame_to_send */
      }
    }
  }
}
void receiver3(void)
{
  seq_nr frame_expected;
  frame r, s;
  event_type event;

  frame_expected = 0;
  while (true) {
    wait_for_event(&event);                         /* possibilidades: frame_arrival, cksum_err */
    if (event == frame_arrival) {                   /* chegou um quadro válido */
      from_physical_layer(&r);                      /* pega quadro recém-chegado */
      if (r.seq == frame_expected) {                /* é isso que estávamos esperando */
        to_network_layer(&r.info);                  /* passa os dados para a camada de rede */
        inc(frame_expected);                        /* da próxima vez, espera outro número de sequência nr */
      }
      s.ack = 1 – frame_expected;                   /* diz qual quadro está sendo confirmado */
      to_physical_layer(&s);                        /* envia confirmação */
    }
  }
}
```

Figura 3.14 Uma confirmação positiva com protocolo de retransmissão.

mais práticas, há necessidade de transmitir dados em ambos os sentidos. Além disso, a camada de enlace pode ser mais eficiente se puder enviar vários quadros ao mesmo tempo antes de receber uma confirmação. Vamos explorar esses dois conceitos em seguida, para depois oferecer vários exemplos de protocolos que alcançam esses objetivos.

3.4.1 Objetivo: transmissão bidirecional, múltiplos quadros em andamento

Em seguida, vamos explicar um conceito chamado piggybacking, que pode ajudar um protocolo da camada de enlace a alcançar transmissão bidirecional, e um conceito

chamado janela deslizante, que pode melhorar a eficiência da transmissão ao permitir que o transmissor tenha vários bytes em andamento.

Transmissão bidirecional: piggybacking

Você pode obter uma transmissão de dados full-duplex definindo dois canais de comunicação distintos e cada um deles usando um enlace separado para um tráfego de dados simplex (em diferentes sentidos). Cada enlace é composto de um canal "direto" (para dados) e de um canal "reverso" (para confirmações). Em ambos os casos, a capacidade do canal reverso é quase totalmente desperdiçada.

Uma ideia melhor é usar o mesmo circuito para dados em ambos os sentidos. Afinal, nos protocolos 2 e 3 ele já estava sendo usado para transmitir quadros em ambos os sentidos, e o canal reverso normalmente tem a mesma capacidade do canal direto. Nesse modelo, os quadros de dados enviados de *A* para *B* são misturados com os quadros de confirmação enviados de *A* para *B*. Ao verificar o campo *kind* no cabeçalho de um quadro recebido, o receptor pode identificar se o quadro é de dados ou de confirmação.

Apesar de o entrelaçamento de quadros de dados e de controle no mesmo circuito representar um grande avanço em relação ao uso de dois enlaces físicos separados, ainda é possível introduzir outra melhoria. Quando um quadro de dados chega a seu destino, em vez de enviar imediatamente um quadro de controle separado, o receptor se contém e espera até a camada de rede enviar o próximo pacote. A confirmação é acrescentada ao quadro de dados que está sendo enviado (por meio do campo *ack* do cabeçalho do quadro). Na verdade, a confirmação pega carona no próximo quadro de dados que estiver sendo enviado. A técnica de retardar temporariamente as confirmações e enviá-las com o próximo quadro de dados é conhecida pelo nome de **piggybacking** (**pegar carona**).

A principal vantagem do piggybacking em relação ao envio de quadros de confirmação distintos é a melhor utilização da largura de banda disponível para o canal. O campo *ack* do cabeçalho do quadro precisa de apenas alguns bits, enquanto um quadro separado precisaria do cabeçalho, da confirmação e do checksum. Além disso, um número menor de quadros enviados significa uma carga de processamento menor no receptor. No próximo protocolo a ser examinado, o campo de piggyback necessita apenas de um bit no cabeçalho do quadro. Em geral, ele raramente precisa de mais que alguns bits.

No entanto, o piggybacking introduz uma complicação que não existe em confirmações separadas. Quanto tempo a camada de enlace de dados deve esperar por um pacote ao qual deverá acrescentar a confirmação? Se a camada de enlace de dados esperar durante um intervalo maior que o permitido pelo timeout do transmissor, o quadro será retransmitido, o que invalidará todo o processo de confirmação. Se a camada de enlace de dados fosse um oráculo e pudesse prever o futuro, ela saberia quando o próximo pacote da camada de rede estivesse chegando e poderia decidir entre esperar por ele e enviar imediatamente uma confirmação separada, dependendo da duração prevista do tempo de espera. É óbvio que a camada de enlace de dados não é capaz de prever o futuro; portanto, ela deve recorrer a algum esquema ad hoc, como esperar durante um número fixo de milissegundos. Se um novo pacote chegar rapidamente, a confirmação será acrescentada a ele; caso contrário, se nenhum pacote tiver chegado até o final desse intervalo, a camada de enlace de dados simplesmente enviará um quadro de confirmação separado.

Janelas deslizantes

Os três protocolos seguintes são bidirecionais e pertencem a uma classe identificada como protocolos de **janela deslizante**. Eles apresentam diferenças em termos de eficiência, complexidade e requisitos de buffer, como discutiremos adiante. Neles, assim como em todos os protocolos de janela deslizante, cada quadro enviado contém um número de sequência, variando de 0 até algum valor máximo. Em geral, o valor máximo é $2^n - 1$, de forma que o número de sequência caiba exatamente em um campo de n bits. O protocolo de janela deslizante stop-and-wait utiliza $n = 1$, restringindo os números de sequência a 0 e 1; no entanto, versões mais sofisticadas podem usar um valor arbitrário de n.

A essência de todos os protocolos de janela deslizante é o fato de que, em qualquer instante, o transmissor mantém um conjunto de números de sequência correspondentes a quadros que ele pode enviar. Dizemos que esses quadros estão reunidos na **janela de transmissão**. Da mesma forma, o receptor mantém uma **janela de recepção** correspondente ao conjunto de quadros que está apto a aceitar. As janelas do transmissor e do receptor não precisam ter os mesmos limites superior e inferior ou o mesmo tamanho. Em alguns protocolos, essas janelas têm tamanho fixo, mas em outros elas podem aumentar ou diminuir à medida que os quadros são enviados e recebidos.

Apesar de esses protocolos permitirem que a camada de enlace de dados tenha mais liberdade em relação à ordem em que pode enviar e receber quadros, definitivamente não descartamos a exigência de o protocolo entregar os pacotes à camada de rede na mesma ordem em que eles foram repassados à camada de enlace de dados na máquina transmissora. Outra exigência que não mudou é que o canal de comunicação física seja "como nos fios", ou seja, que entregue todos os quadros na ordem em que eles são enviados.

Os números de sequência contidos na janela do transmissor representam quadros que foram ou que podem ser enviados, mas que ainda não foram confirmados. Sempre que chega um novo pacote da camada de rede, ele recebe o

próximo número de sequência mais alto, e o limite superior da janela é incrementado em uma unidade. Quando uma confirmação é recebida, o limite inferior é incrementado em uma unidade. Dessa forma, a janela mantém continuamente uma lista de quadros não confirmados. A Figura 3.15 mostra um exemplo.

Tendo em vista que os quadros presentes atualmente na janela do transmissor podem ser perdidos ou danificados em trânsito, o transmissor deve mantê-los em sua memória para que a retransmissão seja possível. Assim, se o tamanho máximo da janela for *n*, o transmissor precisará de *n* buffers para armazenar os quadros não confirmados. Se a janela chegar a seu tamanho máximo, a camada de enlace de dados do transmissor será obrigada a desativar a camada de rede até que outro buffer esteja livre.

O tamanho da janela da camada de enlace de dados receptora corresponde aos quadros que ela é capaz de aceitar. Qualquer quadro que ficar dentro da janela é colocado no buffer do receptor. Quando for recebido um quadro cujo número de sequência for igual ao limite inferior da janela, ele será repassado à camada de rede, será gerada uma confirmação e a janela será incrementada em uma unidade. Qualquer quadro fora da janela é descartado. Em todos esses casos, uma confirmação subsequente é gerada para que o transmissor descubra como proceder. Observe que um tamanho de janela igual a 1 significa que a camada de enlace de dados só aceita quadros em ordem, mas para janelas maiores isso não é verdade. A camada de rede, ao contrário, sempre recebe dados na ordem adequada, independentemente do tamanho da janela da camada de enlace de dados.

A Figura 3.15 mostra um exemplo com um tamanho máximo de janela igual a 1. Inicialmente, não há quadros pendentes; portanto, os limites inferior e superior da janela do transmissor são iguais, mas, à medida que o tempo passa, a situação se desenvolve da maneira mostrada. Diferentemente da janela do transmissor, a janela do receptor sempre permanece em seu tamanho inicial, incrementando-se à medida que o próximo quadro é aceito e entregue à camada de rede.

3.4.2 Exemplos de protocolos full-duplex de janela deslizante

Agora, vejamos alguns exemplos de um protocolo de janela deslizante simples, de um único bit, além de protocolos que podem tratar da retransmissão de quadros com erro quando vários quadros estão a caminho.

Janela deslizante de um bit

Antes de abordarmos o caso geral, vamos examinar primeiro um protocolo de janela deslizante com um tamanho máximo de janela igual a 1. Esse tipo de protocolo utiliza o stop-and-wait, pois o transmissor envia um quadro e aguarda sua confirmação antes de enviar o quadro seguinte.

A Figura 3.16 representa esse tipo de protocolo. Assim como os demais, ele começa definindo algumas variáveis: *next_frame_to_send* informa qual quadro o transmissor está tentando enviar. De modo semelhante, *frame_expected* informa que quadro o receptor está esperando. Nos dois casos, 0 e 1 são as únicas possibilidades.

Normalmente, uma das duas camadas de enlace de dados, do transmissor ou do receptor, inicia e envia o primeiro quadro. Em outras palavras, apenas um dos programas da camada de enlace de dados pode conter as chamadas das

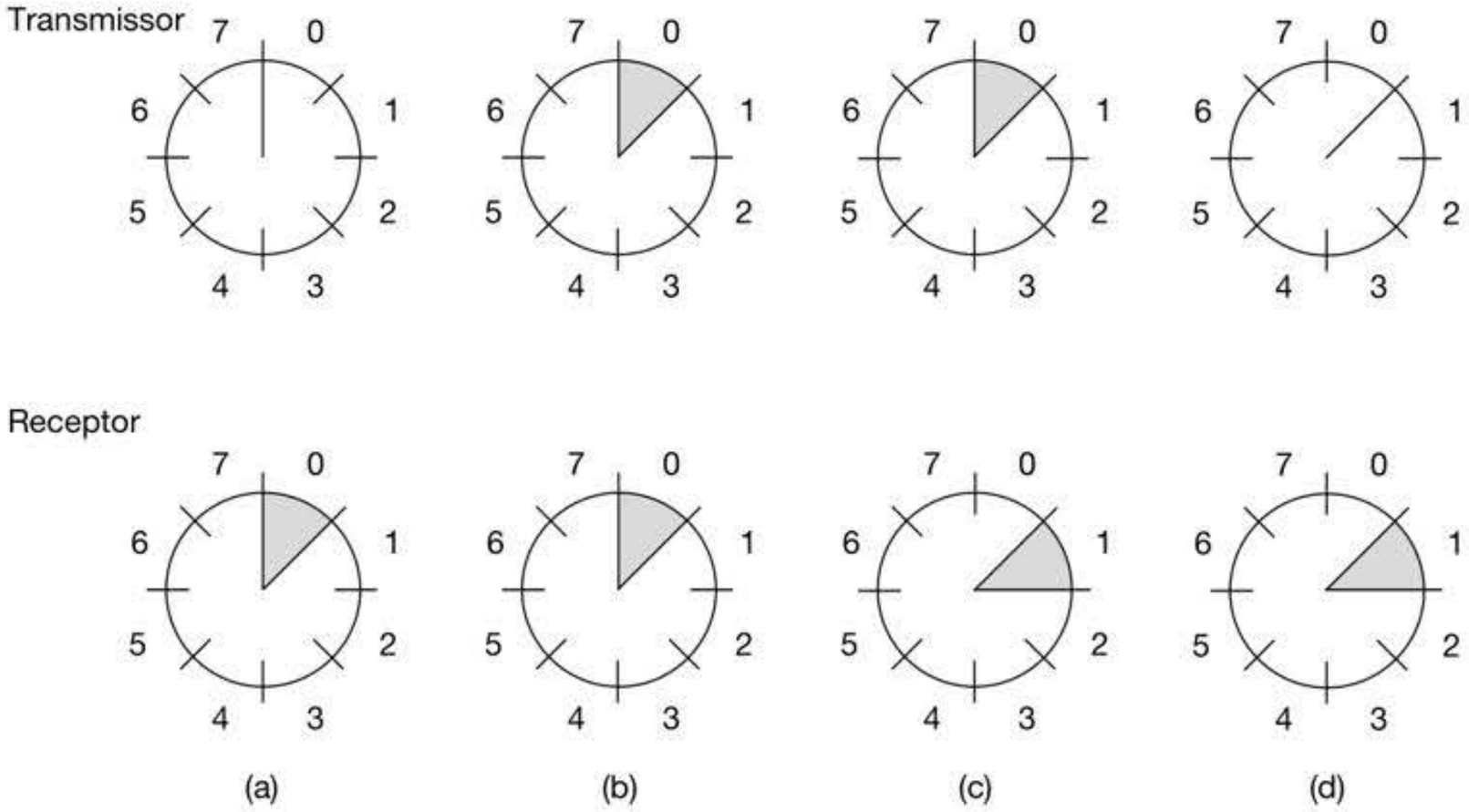


Figura 3.15 Uma janela deslizante de tamanho 1, com um número de sequência de 3 bits. (a) Inicialmente. (b) Depois que o primeiro quadro é enviado. (c) Depois que o primeiro quadro é recebido. (d) Depois que a primeira confirmação é recebida.

```
/* O protocolo 4 (janela deslizante) é bidirecional. */

#define MAX_SEQ 1                          /* deve ser 1 para protocolo 4 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"
void protocol4 (void)
{
   seq_nr next_frame_to_send;              /* 0 ou 1 apenas */
   seq_nr frame_expected                   /* 0 ou 1 apenas */
   frame r, s;                             /* variáveis auxiliares */
   packet buffer;                          /* pacote atual sendo enviado */
   event_type event;
   next_frame_to_send = 0;                 /* próximo quadro no fluxo de saída */
   frame_expected = 0;                     /* quadro esperado em seguida */
   from_network_layer(&buffer);            /* busca um pacote da camada de rede */
   s.info = buffer;                        /* prepara para enviar quadro inicial */
   s.seq = next_frame_to_send;             /* insere número de sequência no quadro */
   s.ack = 1 – frame_expected;             /* confirmação acrescentada */
   to_physical_layer(&s);                  /* transmite o quadro */
   start_timer(s.seq);                     /* inicia a execução do timer */
   while (true) {
      wait_for_event(&event);              /* frame_arrival, cksum_err ou timeout */
      if (event == frame_arrival) {        /* um quadro chegou intacto */
         from_physical_layer(&r);          /* vai pegá-lo */
         if (r.seq == frame_expected) {    /* trata do fluxo de quadros que chega */
               to_network_layer(&r.info);  /* passa pacote à camada de rede */
               inc(frame_expected);        /* inverte número de sequência esperado em seguida */
         }
         if (r.ack == next_frame_to_send) {  /* trata do fluxo de quadros que sai */
               stop_timer(r.ack);          /* desliga o timer */
               from_network_layer(&buffer);  /* busca novo pac. da camada de rede */
               inc(next_frame_to_send);    /* inverte número de sequência do transmissor */
         }
      }
      s.info = buffer;                     /* constrói quadro de saída */
      s.seq = next_frame_to_send;          /* insere número de sequência nele */
      s.ack = 1 – frame_expected;          /* número de sequência do último quadro recebido */
      to_physical_layer(&s);               /* transmite um quadro */
      start_timer(s.seq);                  /* inicia a execução do timer */
   }
}
```

Figura 3.16 Um protocolo de janela deslizante de um bit.

funções *to_physical_layer* e *start_timer* fora do loop principal. A máquina que inicia busca o primeiro pacote em sua camada de rede, constrói um quadro a partir dele e o envia. Quando esse (ou qualquer) quadro chega ao destino, a camada de enlace de dados receptora verifica se ele é uma cópia, como ocorreu no protocolo 3. Se o quadro for o esperado, ele será repassado à camada de rede e a janela do receptor será deslocada para cima.

O campo de confirmação contém o número do último quadro recebido sem erro. Se esse número estiver de acordo com o número de sequência do quadro que o transmissor está tentando enviar, o transmissor saberá que já cuidou do quadro armazenado em *buffer* e poderá buscar o pacote seguinte em sua camada de rede. Se o número de sequência for discordante, o transmissor deverá continuar tentando enviar o mesmo quadro. Sempre que um quadro é recebido, outro quadro também é enviado de volta.

Agora, vamos examinar o protocolo 4 para ver quanto ele é flexível em relação a situações patológicas. Suponha que o computador *A* esteja tentando enviar seu quadro 0 ao computador *B* e que *B* esteja tentando enviar seu quadro 0 ao computador *A*. Imagine que *A* envia um quadro a *B*, mas o intervalo de timeout de *A* é curto demais. Consequentemente, *A* pode completar o timeout repetidas vezes, enviando uma série de quadros idênticos, todos com *seq* = 0 e *ack* = 1.

Quando o primeiro quadro válido chegar a *B*, ele será aceito e *frame_expected* será definido como 1. Todos os quadros subsequentes serão rejeitados, porque *B* agora está esperando quadros com número de sequência 1, e não 0.

Além disso, como todas as cópias têm *ack* = 1 e *B* ainda está aguardando uma confirmação de 0, *B* não buscará um novo pacote em sua camada de rede.

Após a chegada de todas as cópias rejeitadas, *B* enviará um quadro para *A* contendo *seq* = 0 e *ack* = 0. Por fim, um desses quadros chegará sem erros à máquina *A*, fazendo-a começar a enviar o próximo pacote. Nenhuma combinação de quadros perdidos ou timeouts prematuros pode fazer o protocolo entregar pacotes duplicados à camada de rede, ignorar um pacote ou chegar a um impasse. O protocolo está correto.

Entretanto, para mostrar quão sutis as interações entre protocolos podem ser, surgirá uma situação peculiar se os dois lados enviarem simultaneamente um pacote inicial. Essa dificuldade de sincronização está ilustrada na Figura 3.17. Na parte (a), é exibida a operação normal do protocolo. Na parte (b), observamos a peculiaridade. Se *B* esperar pelo primeiro quadro de *A* antes de enviar um de seus quadros, a sequência será a da parte (a) e todos os quadros serão aceitos.

Contudo, se *A* e *B* iniciarem a comunicação ao mesmo tempo, seus primeiros quadros se cruzarão e as camadas de enlace de dados entrarão na situação (b). Em (a), cada quadro recebido traz um novo pacote para a camada de rede; não há cópias. Em (b), metade dos quadros contém cópias, embora não haja erros de transmissão. Situações similares podem ocorrer como resultado de timeouts prematuros, mesmo quando está claro que um lado começa primeiro. Na verdade, se ocorrerem vários timeouts prematuros, os quadros poderão ser enviados três ou mais vezes, desperdiçando uma largura de banda valiosa.

Go-back-n

Até agora estávamos supondo implicitamente que era insignificante o tempo de transmissão necessário para a chegada de um quadro até o receptor, somado ao tempo de transmissão para o retorno da confirmação. Às vezes, essa suposição é nitidamente falsa. Nessas situações, o longo tempo de viagem de ida e volta pode ter implicações importantes para a eficiência da utilização da largura de banda. Como exemplo, considere um canal de satélite de 50 kbps com um atraso de propagação de ida e volta de 500 ms. Vamos imaginar uma tentativa de usar o protocolo 4 para enviar quadros de 1.000 bits pelo satélite. Em $t = 0$, o transmissor começa a enviar o primeiro quadro. Em $t = 20$ ms, o quadro já foi completamente enviado. Até $t = 270$ ms, o quadro ainda não chegou completamente ao receptor, e até $t = 520$ ms, na melhor das hipóteses, a confirmação não terá voltado ao transmissor (sem nenhum tempo de espera no receptor e com um quadro de confirmação curto). Isso significa que o transmissor esteve bloqueado durante 500/520 ou 96% (i.e., apenas 4% da largura de banda disponível foi utilizada). É claro que a combinação de um longo tempo de trânsito, alta largura de banda e pequeno comprimento de quadro é desastrosa em termos de eficiência.

O problema descrito pode ser visto como uma consequência da regra que exige que um transmissor espere por uma confirmação antes de enviar outro quadro. Se essa restrição não for rigorosa, podemos obter uma eficiência muito melhor. Basicamente, a solução está em permitir que o transmissor envie até *w* quadros antes do bloqueio, e não apenas 1. Com uma escolha apropriada de *w*, o transmissor será capaz de transmitir quadros continuamente, pois as confirmações chegarão aos quadros anteriores antes que a janela se complete, impedindo o bloqueio do transmissor.

Para achar um valor apropriado para *w*, precisamos saber quantos quadros cabem dentro do canal à medida que se propagam do transmissor ao receptor. Essa capacidade é

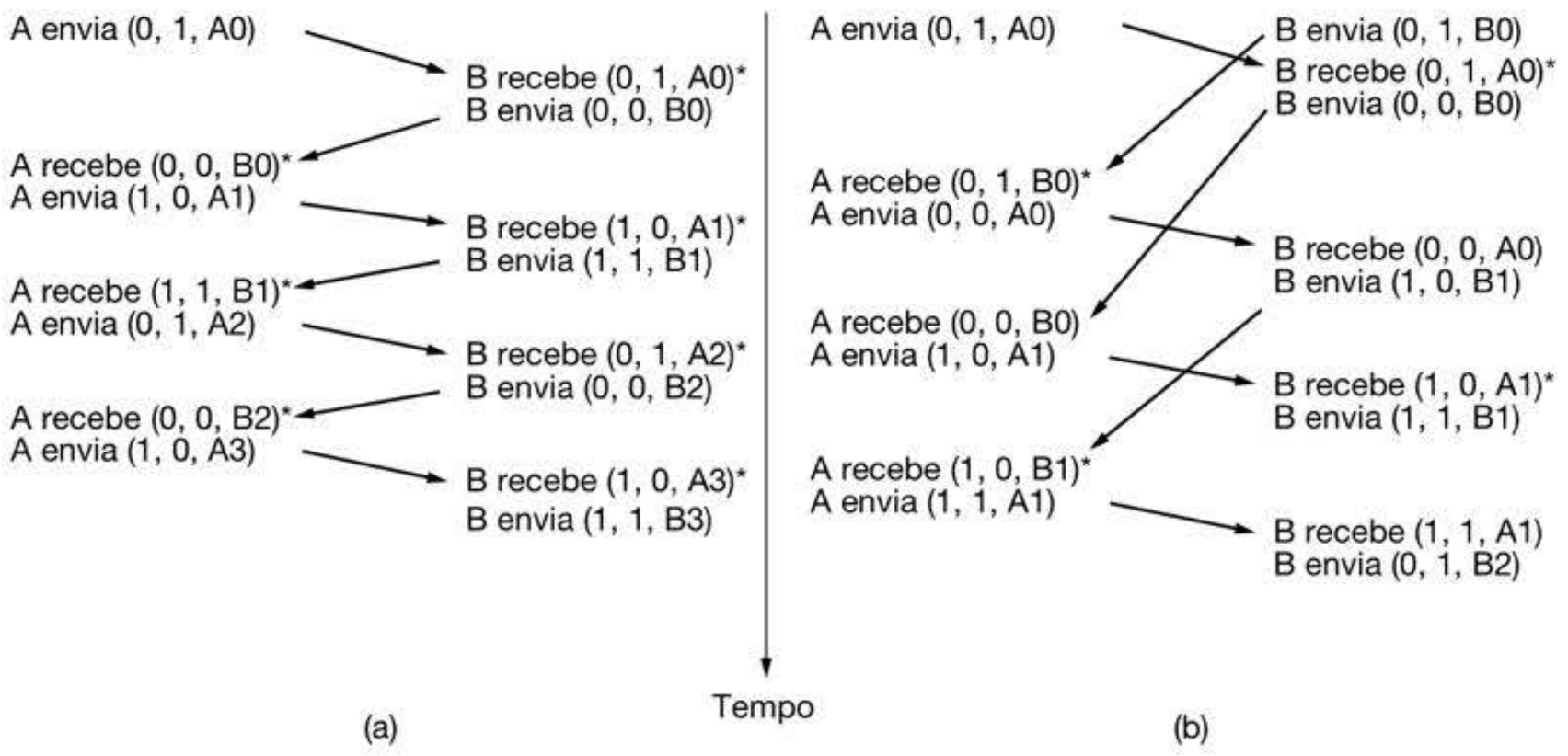


Figura 3.17 Dois cenários referentes ao protocolo 4. (a) Caso normal. (b) Caso anormal. A notação é sequência, confirmação, número do pacote. Um asterisco indica onde uma camada de rede aceita um pacote.

determinada pela largura de banda em bits/s, multiplicada pelo tempo de trânsito em mão única, ou **produto largura de banda-atraso** do enlace. Podemos dividir essa quantidade pelo número de bits em um quadro para expressá-lo como um número de quadros. Chame essa quantidade de *BD*. Então, *w* deve ser definido como $2BD + 1$. O dobro da largura de banda-atraso é o número de quadros que podem estar pendentes se o transmissor enviar quadros continuamente quando o tempo de ida e volta para receber uma confirmação for considerado. O "+1" é porque um quadro de confirmação não será enviado antes que um quadro completo seja recebido.

Para o enlace do exemplo com uma largura de banda de 50 kbps e um tempo de trânsito unidirecional de 250 ms, o produto largura de banda-atraso é de 12,5 kbits ou 12,5 quadros de 1.000 bits cada um. $2BD + 1$ significa, então, 26 quadros. Suponha que o transmissor comece enviando o quadro 0, como antes, e transmita um novo quadro a cada 20 ms. Quando ele tiver terminado de enviar 26 quadros, em $t = 520$ ms, a confirmação para o quadro 0 terá acabado de chegar. Depois disso, as confirmações chegarão a cada 20 ms, de modo que o transmissor sempre terá permissão para continuar assim que precisar. A partir desse ponto, 25 ou 26 quadros não confirmados sempre estarão pendentes. Em outras palavras, o tamanho de janela máxima do transmissor é 26.

Para tamanhos de janela menores, a utilização do enlace será menor que 100%, pois o transmissor às vezes será bloqueado. Podemos escrever a utilização como a fração de tempo em que o transmissor não está bloqueado:

$$\text{utilização do enlace} \leq \frac{w}{1 + 2BD}$$

Esse valor é um limite superior, pois não leva em consideração nenhum tempo de processamento de quadro e trata o quadro de confirmação como tendo tamanho zero, pois ele normalmente é curto. A equação mostra a necessidade de ter uma janela *w* grande sempre que o produto de largura de banda-atraso for grande. Se o atraso for alto, o transmissor rapidamente esgotará sua janela, mesmo para uma largura de banda moderada, como no exemplo do satélite. Se a largura de banda for alta, mesmo para um atraso moderado, o transmissor esgotará sua janela rapidamente, a menos que tenha uma janela grande (p. ex., um enlace de 1 Gbps com atraso de 1 ms mantém 1 megabit). Com o stop-and-wait, para o qual $w = 1$, se houver um atraso de propagação de até mesmo um quadro, a eficiência será menor que 50%.

Essa técnica de manter vários quadros pendentes é um exemplo de **pipelining**. O pipelining de quadros em um canal de comunicação não confiável faz surgir algumas questões muito sérias. Primeiro, o que acontecerá se um quadro em meio a um longo fluxo for danificado ou perdido? Um grande número de quadros sucessivos chegará ao receptor antes mesmo de o transmissor descobrir que algo está errado. Quando um quadro danificado chega ao receptor, sem dúvida ele deve ser descartado. No entanto, o que o receptor deve fazer com todos os quadros corretos que o seguem? Lembre-se de que a camada de enlace de dados receptora é obrigada a entregar pacotes à camada de rede em sequência.

Há duas estratégias básicas para lidar com erros na presença do pipelining, ambas mostradas na Figura 3.18.

Em uma opção denominada **go-back-n**, o receptor simplesmente descarta todos os quadros subsequentes e não envia nenhuma confirmação referente a eles. Essa estratégia corresponde a uma janela de recepção de tamanho 1. Em outras palavras, a camada de enlace de dados se recusa a aceitar qualquer quadro, exceto o próximo, que ela tem de entregar à camada de rede. Se a janela do transmissor for totalmente preenchida antes de o timer encerrar a contagem, o pipeline começará a se esvaziar. Consequentemente, o transmissor interromperá a transmissão e retransmitirá todos os quadros não confirmados em ordem, começando pelo quadro danificado ou perdido. Essa abordagem poderá desperdiçar uma grande quantidade de largura de banda se a taxa de erros for alta.

Na Figura 3.18(a), vemos o go-back-n para o caso em que a janela do receptor é 1. Os quadros 0 e 1 são corretamente recebidos e confirmados. Contudo, o quadro 2 está danificado ou perdido. O transmissor, desavisado desse problema, continua a enviar quadros até expirar o timer correspondente ao quadro 2. Em seguida, ele volta até o quadro 2 e começa tudo de novo a partir dele, enviando, mais uma vez, os quadros 2, 3, 4, e assim por diante.

Retransmissão seletiva

O protocolo go-back-n funciona bem se os erros forem raros, mas, se a linha for ruim, isso desperdiçará muita largura de banda nos quadros retransmitidos. Precisamos fazer algo melhor – e isso é possível. Uma alternativa é o protocolo de **retransmissão seletiva**, que permite que o receptor aceite e mantenha os quadros corretos em um buffer após um quadro danificado ou perdido.

Quando ela é utilizada, um quadro incorreto recebido é descartado, mas os quadros sem defeitos posteriores são aceitos e mantidos no buffer. Quando o transmissor chega ao timeout, apenas o quadro não confirmado mais antigo é retransmitido. Se ele chegar corretamente, o receptor poderá entregar à camada de rede, em sequência, todos os quadros que armazenou no buffer. A estratégia de retransmissão seletiva corresponde a uma janela receptora maior que 1. Caso a janela seja muito grande, essa abordagem poderá exigir um volume de memória muito grande da camada de enlace de dados.

Com frequência, a retransmissão seletiva é combinada com a ação de fazer o receptor enviar uma confirmação negativa, ou NAK (negative acknowledgement), ao detectar

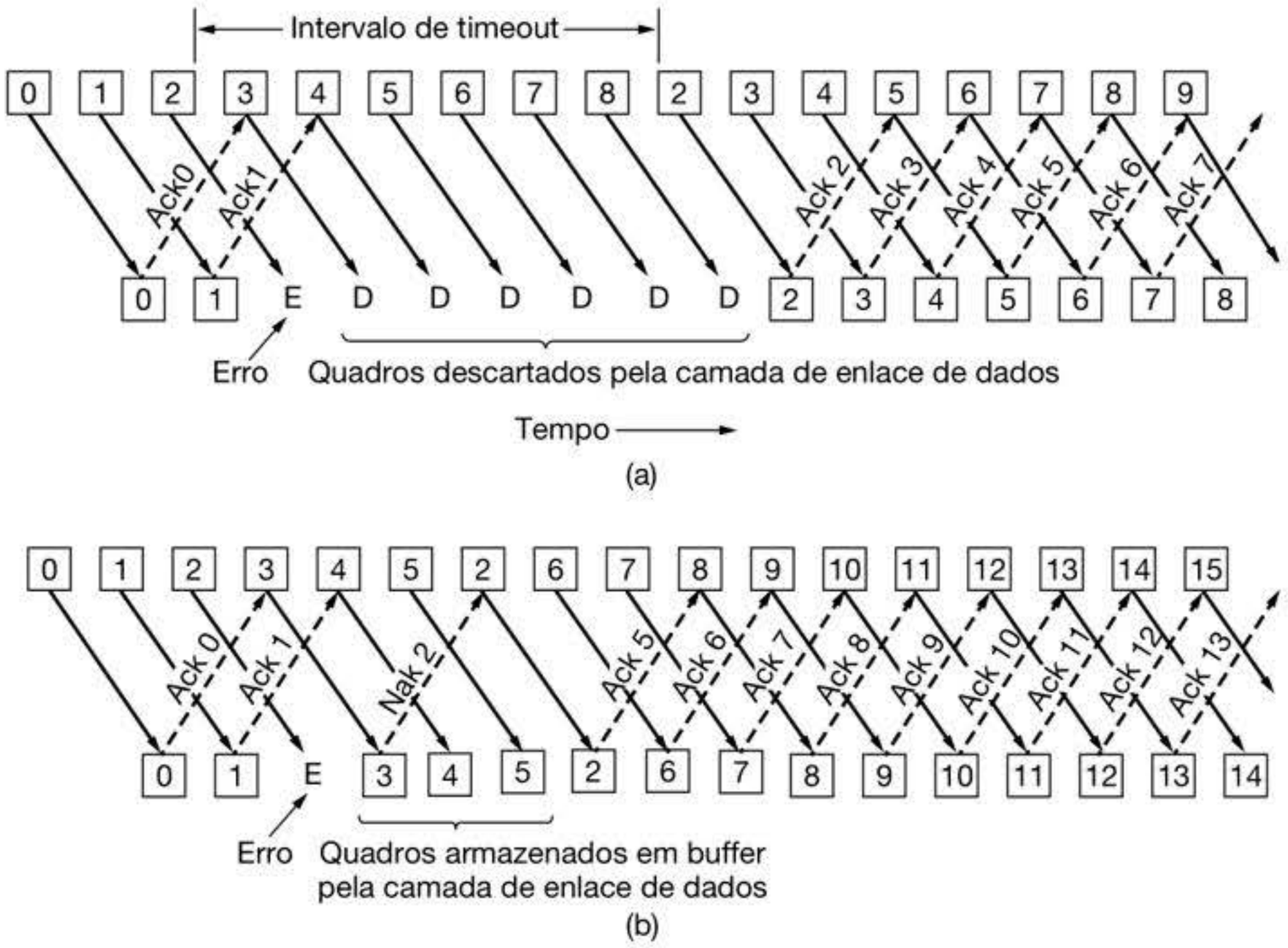


Figura 3.18 Pipelining e recuperação de erros. Efeito de um erro quando (a) o tamanho da janela receptora é igual a 1 e (b) quando o tamanho da janela receptora é grande.

um erro, por exemplo, quando receber um erro de checksum ou um quadro fora de sequência. As NAKs estimulam a retransmissão antes de expirar o timer correspondente e, desse modo, melhoram o desempenho.

Na Figura 3.18(b), os quadros 0 e 1 são mais uma vez recebidos e confirmados corretamente, e o quadro 2 é perdido. Quando o quadro 3 chega ao receptor, a camada de enlace de dados do receptor percebe que perdeu um quadro e, assim, envia de volta uma NAK correspondente ao quadro 2, mas armazena o quadro 3 no buffer. Quando os quadros 4 e 5 chegam, eles também são inseridos no buffer pela camada de enlace de dados, em vez de serem repassados à camada de rede. Por fim, a NAK do quadro 2 volta ao transmissor, que o retransmite de imediato. Quando esse quadro chega, a camada de enlace de dados fica com os quadros 2, 3, 4 e 5, e pode repassar todos eles à camada de rede na ordem correta. Ela também pode confirmar todos os quadros, inclusive até o 5, como mostra a figura. Se a NAK se perder, o transmissor chegará ao timeout correspondente ao quadro 2 e o enviará (e apenas esse quadro) por sua própria iniciativa, mas isso pode acontecer um pouco mais tarde.

Esses dois enfoques alternativos são dilemas entre uso eficiente de largura de banda e espaço no buffer da camada de enlace de dados. Dependendo de qual recurso é mais escasso, um ou outro poderá ser usado. A Figura 3.19 mostra um protocolo go-back-n no qual a camada de enlace de dados receptora aceita apenas quadros em ordem; os quadros que vierem depois de um quadro com erro serão descartados. Nesse protocolo, pela primeira vez abandonamos a suposição de que a camada de rede sempre tem um suprimento infinito de pacotes a enviar. Quando a camada de rede tem um pacote que deseja enviar, ela pode provocar a ocorrência de um evento *network_layer_ready*. Entretanto, para reforçar o limite de controle de fluxo sobre a janela do transmissor ou o número de quadros não confirmados pendentes em qualquer instante, a camada de enlace de dados deve ser capaz de proibir a camada de rede de sobrecarregá-la com mais trabalho. As funções de biblioteca *enable_network_layer* e *disable_network_layer* executam essa funcionalidade.

Observe que o número máximo de quadros que podem estar pendentes em qualquer instante não é o mesmo que o tamanho do espaço do número de sequência. Para go-back-n, *MAX_SEQ* quadros podem estar pendentes em qualquer instante, mesmo que haja *MAX_SEQ* + 1 números de sequência distintos (que são 0, 1, 2,..., *MAX_SEQ*). Veremos uma restrição ainda maior para o protocolo seguinte, a retransmissão seletiva. Para saber por que essa restrição é necessária, considere a situação a seguir, com *MAX_SEQ* = 7:

1. O transmissor envia quadros de 0 a 7.
2. Uma confirmação para o quadro 7 volta de carona (piggyback) ao transmissor.
3. O transmissor envia mais oito quadros, novamente com números de sequência de 0 a 7.
4. Agora chega outra confirmação de carona correspondente ao quadro 7.

```
/* O protocolo 5 (go-back-n) permite a existência de muitos quadros pendentes. O transmissor poderá transmitir até
   MAX_SEQ quadros sem a necessidade de esperar por uma confirmação. Além disso, ao contrário dos protocolos
   anteriores, não presumimos que a camada de rede esteja sempre recebendo um novo pacote. Em vez disso, a camada
   de rede provoca o evento network_layer_ready quando há um pacote a ser enviado. */

#define MAX_SEQ 7

typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready} event_type;

#include "protocol.h"

static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
/* Retorna true se a <= b < c de modo circular; caso contrário, false. */
   if (((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a)))
        return(true);
      else
        return(false);
}

static void send_data(seq_nr frame_nr,seq_nr frame_expected,packet buffer[ ])
{
/* Constrói e envia um quadro de dados. */
   frame s;                                    /* variável auxiliar */
   s.info = buffer[frame_nr];                  /* insere pacote no quadro */
   s.seq = frame_nr;                           /* insere número de sequência no quadro */
   s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1);   /* confirma piggyback */
   to_physical_layer(&s);                      /* transmite o quadro */
   start_timer(frame_nr);                      /* inicia o timer */
}

void protocol5(void)
{
   seq_nr next_frame_to_send;                  /* MAX_SEQ > 1; usado para fluxo de saída */
   seq_nr ack_expected;                        /* quadro mais antigo como ainda não confirmado */
   seq_nr frame_expected;                      /* próximo quadro esperado no fluxo de entrada */
   frame r;                                    /* variável auxiliar */
   packet buffer[MAX_SEQ + 1];                 /* buffers para o fluxo de saída */
   seq_nr nbuffered;                           /* número de buffers de saída atualmente em uso */
   seq_nr i;                                   /* usado para indexar array no buffer */
   event_type event;
   enable_network_layer();                     /* permite network_layer_ready events */
   ack_expected = 0;                           /* próximo ack esperado na entrada */
   next_frame_to_send = 0;                     /* próximo quadro saindo */
   frame_expected= 0;                          /* número do quadro esperado na entrada */
   nbuffered = 0;                              /* inicialmente, nenhum pacote no buffer */
   while (true) {
      wait_for_event(&event);                  /* quatro possibilidades: ver event_type acima */
```

Figura 3.19 Um protocolo de janela deslizante que utiliza go-back-n. *(Continua)*

```
switch(event) {
  case network_layer_ready:                    /* a camada de rede tem um pacote a enviar */
    /* Aceita, salva e transmite um novo quadro. */
    from_network_layer(&buffer[next_frame_to_send]); /* busca novo pacote */
    nbuffered = nbuffered + 1;                 /* expande a janela do transmissor */
    send_data(next_frame_to_send, frame_expected, buffer); /* transmite o quadro */
    inc(next_frame_to_send);                   /* avança o limite da janela superior do transmissor */
    break;
  case frame_arrival:                          /* um quadro de dados ou controle chegou */
    from_physical_layer(&r);                   /* pega quadro que chegou da camada física */
    if (r.seq == frame_expected) {
    /* Quadros são aceitos apenas em ordem. */
      to_network_layer(&r.info);               /* passa pacote à camada de rede */
      inc(frame_expected);                     /* avança o limite inferior da janela do receptor */
    }
    /* Ack n implica n – 1, n – 2, etc. Verifica isso. */
    while (between(ack_expected, r.ack, next_frame_to_send)) {
      /* Trata o ack piggyback. */
      nbuffered = nbuffered – 1;               /* um quadro a menos no buffer */
      stop_timer(ack_expected);                /* quadro chegou intacto; interrompe timer */
      inc(ack_expected);                       /* ajusta a janela do transmissor */
    }
    break;
case cksum_err: break;                         /* apenas ignora quadros ruins */
case timeout:                                  /* problema; retransmite todos os quadros pendentes */
  next_frame_to_send = ack_expected;           /* inicia retransmissão aqui */
        for (i = 1; i <= nbuffered; i++) {
            send_data(next_frame_to_send, frame_expected, buffer);   /* reenvia quadro */
            inc(next_frame_to_send);                    /* prepara para enviar o quadro seguinte */
        }
    }
    if (nbuffered < MAX_SEQ)
            enable_network_layer();
    else
            disable_network_layer();
  }
}
```

Figura 3.19 *(Continuação)* Um protocolo de janela deslizante que utiliza go-back-n.

A questão é: os oito quadros pertencentes ao segundo lote chegaram com sucesso, ou todos eles se perderam (a contagem descarta os quadros posteriores a um erro, considerando-os perdidos)? Nos dois casos, o receptor enviaria o quadro 7 como confirmação. O transmissor não tem como saber disso. Por essa razão, o número máximo de quadros pendentes deve ser restrito à *MAX_SEQ* (e não *MAX_SEQ* + 1).

Apesar de não armazenar no buffer os quadros recebidos após um quadro com erro, o protocolo 5 não evita totalmente o problema do armazenamento em buffer. Tendo em vista que um transmissor talvez seja obrigado a retransmitir todos os quadros não confirmados em determinado momento no futuro, ele deverá reter todos os quadros transmitidos até ter certeza de que eles foram aceitos pelo receptor.

Quando uma confirmação chega para o quadro n, os quadros $n - 1$, $n - 2$, e assim por diante, também são confirmados de forma automática. Esse tipo de confirmação é chamado de **confirmação cumulativa**. Essa propriedade é especialmente importante nos casos em que alguns dos quadros anteriores que representavam confirmações se perderam ou foram adulterados. Sempre que chega uma confirmação, a camada de enlace de dados verifica se algum buffer pode ser liberado. Se os buffers puderem ser liberados (i.e., se houver espaço disponível na janela), uma camada de rede bloqueada anteriormente poderá ter permissão para provocar mais eventos *network_layer_ready*.

Para esse protocolo, supomos que sempre existe tráfego no sentido inverso, para que as confirmações possam

ser transportadas por piggyback. O protocolo 4 não precisa dessa suposição, pois ele envia um quadro de volta toda vez que recebe um quadro, mesmo que já o tenha enviado. No próximo protocolo, resolveremos de modo elegante o problema do tráfego de mão única.

Por ter vários quadros pendentes, é claro que o protocolo 5 necessita de vários timers, um para cada quadro pendente. Cada quadro tem um timeout independente de todos os demais. Contudo, todos esses timers podem ser facilmente simulados por software, usando-se um único timer do hardware para provocar interrupções periódicas. Os timeouts pendentes formam uma lista encadeada, com cada nó da lista contendo a quantidade de pulsos do timer até que ele expire, o quadro sendo sincronizado e um ponteiro para o nó seguinte.

Para ilustrar como os timers poderiam ser implementados, considere o exemplo da Figura 3.20(a). Suponha que o timer pulse uma vez a cada 1 ms. Inicialmente, o tempo real é 10:00:00.000 e há três timeouts pendentes, em 10:00:00.005, 10:00:00.013 e 10:00:00.019. Toda vez que o timer do hardware pulsar, o tempo real será atualizado e o contador de pulsos no início da lista será decrementado. Quando o contador de pulsos for igual a zero, ocorrerá um timeout e o nó será removido da lista, como mostra a Figura 3.20(b). Embora essa organização exija que a lista seja examinada quando *start_timer* ou *stop_timer* são chamadas, ela não requer muito trabalho por pulso. No protocolo 5, essas duas funções receberam um parâmetro, que indica o quadro a ser sincronizado.

Nesse protocolo, tanto o transmissor quanto o receptor mantêm uma janela de números de sequência pendentes e aceitáveis, respectivamente. O tamanho da janela do transmissor é medido a partir de 0 e atinge um número máximo predefinido. Por sua vez, a janela do receptor tem sempre um tamanho fixo e igual ao máximo predeterminado. O receptor tem um buffer reservado para cada número de sequência dentro de sua janela fixa. Associado a cada buffer há um bit (*arrived*) que informa se o buffer está cheio ou vazio. Sempre que um quadro chega, seu número de sequência é verificado pela função *between*, para confirmar se ele se enquadra na janela. Se isso ocorrer e o quadro ainda não tiver sido recebido, ele será aceito e armazenado. Essa ação é executada sem levar em conta se o quadro contém ou não o próximo pacote esperado pela camada de rede. É obvio que ele deve ser mantido dentro da camada de enlace de dados e não deve ser repassado à camada de rede até que todos os quadros de números mais baixos já tenham sido entregues a ela na ordem correta. Um protocolo que utiliza esse algoritmo é apresentado na Figura 3.21.

A recepção não sequencial introduz outras restrições sobre os números de sequência em comparação a protocolos nos quais os quadros só são aceitos em ordem. Podemos ilustrar melhor o problema com um exemplo. Imagine que haja um número de sequência de 3 bits, de modo que o transmissor tenha permissão para transmitir até sete quadros antes de ser obrigado a esperar por uma confirmação.

Inicialmente, as janelas do transmissor e do receptor são semelhantes às da Figura 3.22(a). No momento, o transmissor envia os quadros de 0 a 6. A janela do receptor permite que ele aceite qualquer quadro com número de sequência entre 0 e 6, ambos inclusive. Todos os sete quadros chegam corretamente; assim, o receptor os confirma e avança a janela para permitir a recepção de 7, 0, 1, 2, 3, 4 ou 5, conforme mostra a Figura 3.22(b). Todos os sete buffers são marcados como vazios.

Nesse ponto ocorre um desastre, na forma de um raio que atinge a central telefônica e apaga todas as confirmações. O protocolo deve operar corretamente apesar disso. Mais tarde, o transmissor entra em timeout e retransmite o quadro 0. Quando esse quadro chega ao receptor, é feita uma conferência para ver se ele se ajusta à janela do receptor. Infelizmente, na Figura 3.22(b), o quadro 0 está dentro da nova janela e, assim, ele será aceito como um novo quadro. O receptor também envia uma confirmação com piggyback para o quadro 6, pois os quadros de 0 a 6 foram recebidos.

O transmissor fica feliz em saber que todos os quadros transmitidos realmente chegaram de forma correta,

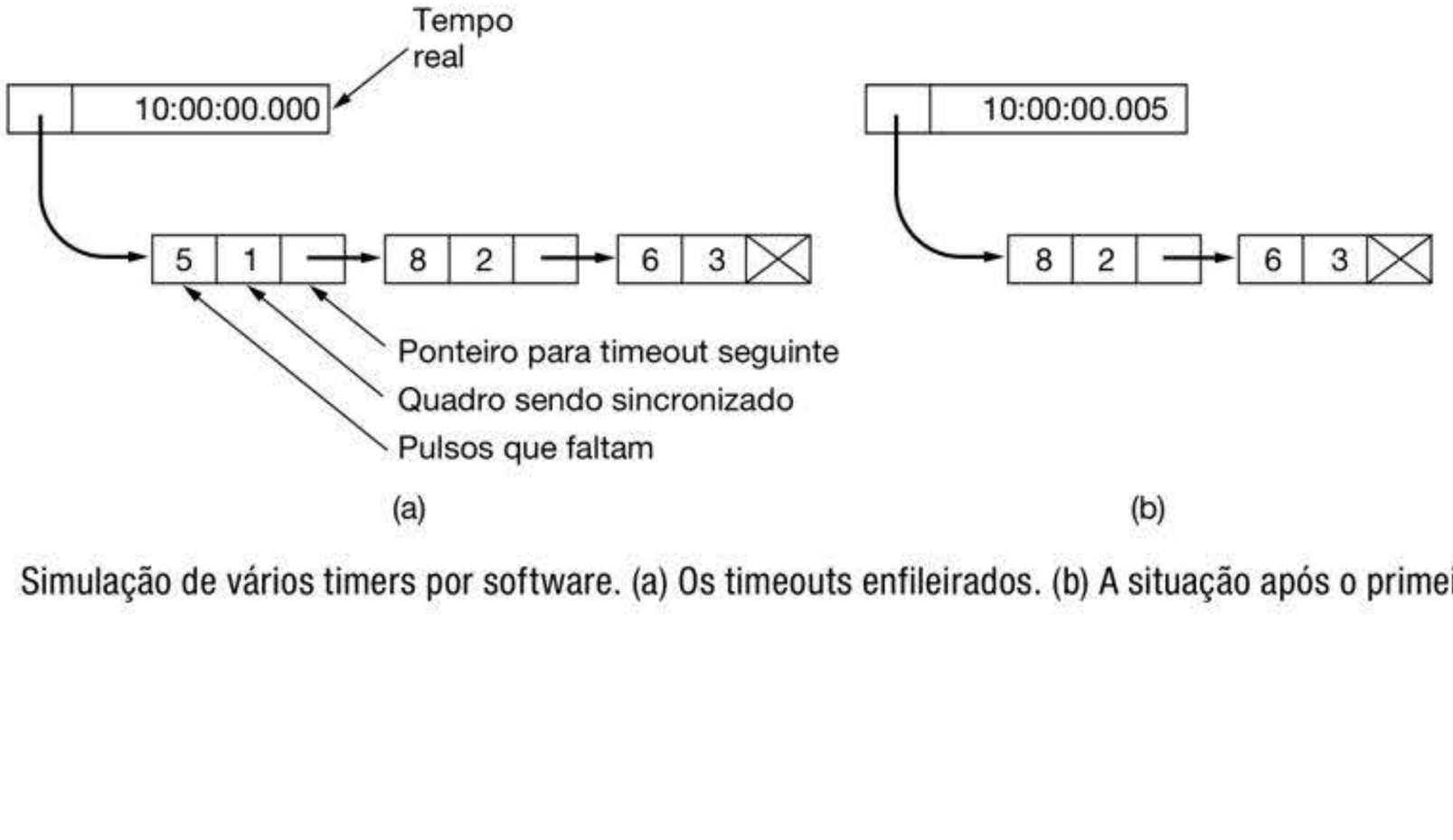


Figura 3.20 Simulação de vários timers por software. (a) Os timeouts enfileirados. (b) A situação após o primeiro timeout expirar.

```
/* O protocolo 6 (retransmissão seletiva) aceita quadros fora de ordem, mas repassa pacotes para a camada
   de rede obedecendo à ordem de transmissão. Há um timer associado a cada quadro pendente. Quando
   o timer expira, apenas o quadro que o contém é retransmitido, e não todos os quadros pendentes, como
   ocorria no protocolo 5. */
#define MAX_SEQ 7                                        /* deve ser 2^n – 1 */
#define NR_BUFS ((MAX_SEQ + 1)/2)
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready, ack_timeout} event_type;
#include "protocol.h"
boolean no_nak = true;                                   /* nenhuma nak foi enviada */
seq_nr oldest_frame = MAX_SEQ + 1;                       /* valor inicial é apenas para o simulador */

static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
/* O mesmo que no protocolo 5, porém mais curto e mais confuso. */
   return ((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a));
}

static void send_frame(frame_kind fk, seq_nr frame_nr, seq_nr frame_expected, packet buffer[ ])
{
/* Constrói e envia um quadro de dados, ack ou nak. */
   frame s;                                              /* variável auxiliar */
   s.kind = fk;                                          /* kind == data, ack, ou nak */
   if (fk == data) s.info = buffer[frame_nr % NR_BUFS];
   s.seq = frame_nr;                                     /* significativo apenas para quadros de dados */
   s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1);
   if (fk == nak) no_nak = false;                        /* um nak por quadro, por favor */
   to_physical_layer(&s);                                /* transmite o quadro */
   if (fk == data) start_timer(frame_nr % NR_BUFS);
   stop_ack_timer();                                     /* não necessário para quadro ack separado */
}
void protocol6(void)
{
   seq_nr ack_expected;                                  /* limite inferior da janela do transmissor */
   seq_nr next_frame_to_send;                            /* limite superior da janela do transmissor + 1 */
   seq_nr frame_expected;                                /* limite inferior da janela do receptor */
   seq_nr too_far;                                       /* limite superior da janela do receptor + 1 */
   int i;                                                /* índice para os buffers de entrada/saída */
   frame r;                                              /* variável auxiliar */
   packet out_buf[NR_BUFS];                              /* buffers para o fluxo de saída */
   packet in_buf[NR_BUFS];                               /* buffers para o fluxo de entrada */
   boolean arrived[NR_BUFS];                             /* mapa de bits de entrada */
   seq_nr nbuffered;                                     /* quantidade de buffers de saída usados atualmente */
   event_type event;
   enable_network_layer();                               /* inicializa */
   ack_expected = 0;                                     /* próximo ack esperado no fluxo de entrada */
   next_frame_to_send = 0;                               /* número do próximo quadro de saída */
   frame_expected = 0;
   too_far = NR_BUFS;
   nbuffered = 0;                                        /* inicialmente nenhum pacote em buffer */
   for (i = 0; i < NR_BUFS; i++) arrived[i] = false;
   while (true) {
      wait_for_event(&event);                            /* cinco possibilidades: ver event_type acima */
```

Figura 3.21 Um protocolo de janela deslizante que utiliza a retransmissão seletiva. *(Continua)*

```
switch(event) {
  case network_layer_ready:                                   /* aceita, salva e transmite um novo quadro */
    nbuffered = nbuffered + 1;                                /* estende a janela */
    from_network_layer(&out_buf[next_frame_to_send % NR_BUFS]); /* busca novo pacote */
    send_frame(data, next_frame_to_send, frame_expected, out_buf); /* transmite o quadro */
    inc(next_frame_to_send);                                  /* avança o limite da janela superior */
    break;

  case frame_arrival:                                         /* um quadro de dados ou de controle chegou */
    from_physical_layer(&r);                                  /* busca quadro que chega da camada física */
    if (r.kind == data) {
      /* Um quadro correto chegou. */
      if ((r.seq != frame_expected) && no_nak)
        send_frame(nak, 0, frame_expected, out_buf); else start_ack_timer();
      if (between(frame_expected,r.seq,too_far) && (arrived[r.seq%NR_BUFS]==false)) {
        /* Quadros podem ser aceitos em qualquer ordem. */
        arrived[r.seq % NR_BUFS] = true;                      /* marca o buffer como cheio */
        in_buf[r.seq % NR_BUFS] = r.info;                     /* insere dados no buffer */
        while (arrived[frame_expected % NR_BUFS]) {
          /* Passa quadros e avança a janela. */
          to_network_layer(&in_buf[frame_expected % NR_BUFS]);
          no_nak = true;
          arrived[frame_expected % NR_BUFS] = false;
          inc(frame_expected);                                /* avança o limite inferior da janela do receptor */
          inc(too_far);                                       /* avança o limite superior da janela do receptor */
          start_ack_timer();                                  /* para ver se precisa de ack separado */
        }
      }
    }
    if((r.kind==nak) && between(ack_expected,(r.ack+1)%(MAX_SEQ+1),next_frame_to_send))
      send_frame(data, (r.ack+1) % (MAX_SEQ + 1), frame_expected, out_buf);
    while (between(ack_expected, r.ack, next_frame_to_send)) {
      nbuffered = nbuffered – 1;                              /* trata do ack piggyback */
      stop_timer(ack_expected % NR_BUFS);                     /* quadro chegou intacto */
      inc(ack_expected);                                      /* avança o limite inferior da janela do transmissor */
    }
    break;
  case cksum_err:
    if (no_nak) send_frame(nak, 0, frame_expected, out_buf);  /* quadro danificado */
    break;
  case timeout:
    send_frame(data, oldest_frame, frame_expected, out_buf);  /* timeout */
    break;
  case ack_timeout:
    send_frame(ack,0,frame_expected, out_buf);                /* timer de ack expirou; envia ack */
  }
  if (nbuffered < NR_BUFS) enable_network_layer(); else disable_network_layer();
 }
}
```

Figura 3.21 *(Continuação)* Um protocolo de janela deslizante que utiliza a retransmissão seletiva.

portanto, ele avança sua janela e envia imediatamente os quadros 7, 0, 1, 2, 3, 4 e 5. O quadro 7 será aceito pelo receptor e seu pacote será repassado diretamente à camada de rede. Logo depois, a camada de enlace de dados receptora verifica se já tem um quadro 0 válido, descobre que sim e repassa o pacote incorporado à camada de rede como se fosse um novo. Consequentemente, a camada de rede recebe um pacote incorreto e o protocolo falha.

A essência do problema é que, depois que o receptor avançou a janela, a nova faixa de números de sequência válidos substituiu a antiga. Consequentemente, o próximo lote de quadros poderia ser formado por cópias (se todas as confirmações se perderam) ou por novos quadros (se todas as confirmações foram recebidas). O receptor não tem como distinguir esses dois casos.

A saída desse dilema reside em ter certeza de que, depois que o receptor avançou sua janela, não há sobreposição entre essa janela e a original. Para assegurar que não há sobreposição, o tamanho máximo da janela deve ser igual à metade do intervalo dos números de sequência, como ocorre na Figura 3.22(c) e (d). Com 3 bits, os números de sequência vão variar de 0 a 7. Apenas quatro quadros não confirmados devem estar pendentes em qualquer instante. Dessa forma, se o receptor só tiver aceito os quadros de 0 a 3 e avançado sua janela para aceitar os quadros de 4 a 7, ele poderá saber, sem nenhuma dúvida, se os quadros subsequentes são retransmissões (0 a 3) ou novos quadros (4 a 7). Em geral, o tamanho da janela para o protocolo 6 será (*MAX_SEQ* + 1)/2.

Uma pergunta interessante é: quantos buffers o receptor deverá ter? De maneira alguma ele aceitará quadros cujos números de sequência estejam abaixo do limite inferior da janela ou acima do limite superior. Consequentemente, o número de buffers necessário é igual ao tamanho da janela, e não ao intervalo dos números de sequência. No exemplo anterior de um número de sequência de 3 bits, são necessários quatro buffers, numerados de 0 a 3. Quando o quadro *i* chega, ele é colocado no buffer *i* mod 4. Observe que, apesar de *i* e (*i* + 4) mod 4 estarem "competindo" pelo mesmo buffer, eles nunca estão dentro da janela ao mesmo tempo, pois isso implicaria um tamanho de janela de no mínimo 5.

Pela mesma razão, o número de timers necessários é igual ao número de buffers, e não ao tamanho do espaço de sequência. Efetivamente, um timer é associado a cada buffer. Quando o timer chega a seu timeout, o conteúdo do buffer é retransmitido.

O protocolo 6 também alivia a suposição implícita de que o canal está muito carregado. Fizemos essa suposição no protocolo 5, quando contamos com quadros enviados na direção inversa das confirmações com piggyback. Se o tráfego inverso for leve, a confirmação será retida por um longo período, o que poderá causar problemas. No limite, se houver um tráfego intenso em um sentido e nenhum tráfego no outro, então o protocolo será bloqueado quando a janela transmissora atingir seu máximo.

Para aliviar essa suposição, após um quadro de dados sequencial ser recebido, um timer auxiliar é iniciado por *start_ack_timer*. Se nenhum tráfego inverso tiver se apresentado antes desse timer expirar, é enviado um quadro de confirmação separado. Uma interrupção provocada pelo timer auxiliar é chamada evento *ack_timeout*. Diante dessa organização, o fluxo de tráfego unidirecional passa a ser possível nesse momento, pois a falta de quadros de dados inversos, nos quais as confirmações possam ser transportadas, não representa mais um obstáculo. Existe apenas um timer auxiliar e, se *start_ack_timer* for chamada durante o intervalo em que o timer estiver funcionando, ela não terá efeito. O timer não é reiniciado ou estendido, pois sua finalidade é oferecer uma taxa mínima de confirmações.

É essencial que o timeout associado ao timer auxiliar seja ligeiramente mais curto que o timer utilizado para sincronizar quadros de dados. Essa condição é necessária para assegurar que a confirmação de um quadro corretamente recebido chegue antes de o timer de retransmissão do quadro expirar, de modo que o transmissor não tenha de retransmiti-lo.

O protocolo 6 utiliza uma estratégia mais eficiente que o protocolo 5 para tratamento de erros. Sempre que há motivos para suspeitar da ocorrência de um erro, o receptor envia um quadro de confirmação negativa (NAK) de volta ao transmissor. Ele é um pedido de retransmissão do quadro especificado na NAK. Existem dois casos que podem

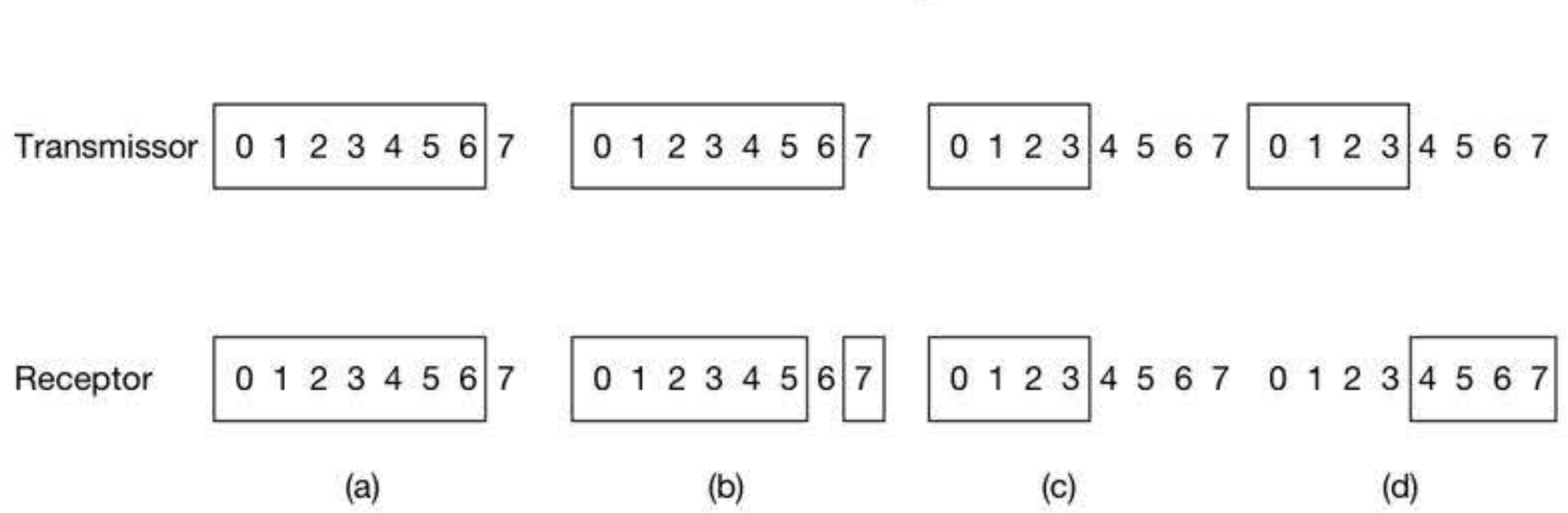


Figura 3.22 (a) Situação inicial com uma janela de tamanho 7. (b) Após sete quadros serem transmitidos e recebidos, mas não confirmados. (c) Situação inicial com uma janela de tamanho 4. (d) Após quatro quadros serem transmitidos e recebidos, mas não confirmados.

provocar a suspeita do receptor: a chegada de um quadro danificado ou de um quadro diferente do esperado (quadro potencialmente perdido). Para impedir que sejam feitas várias solicitações de retransmissão do mesmo quadro perdido, o receptor deve controlar se já foi enviada uma NAK correspondente a determinado quadro. A variável *no_nak* no protocolo 6 será verdadeira (*true*) se nenhuma NAK tiver sido enviada para *frame_expected*. Se a NAK for danificada ou perdida, não haverá qualquer prejuízo real, pois, com o término do intervalo de timeout, de qualquer forma, o transmissor retransmitirá o quadro ausente. Se o quadro errado chegar depois que uma NAK tiver sido enviada e perdida, *no_nak* será verdadeira e o timer auxiliar será inicializado. Quando o timer expirar, uma ACK será enviada para ressincronizar o transmissor com o status atual do receptor.

Em algumas situações, o tempo necessário para que um quadro se propague até o destino, seja processado lá e tenha a confirmação retornada é (praticamente) constante. Nessas situações, o transmissor pode ajustar seu timer para ser "estreito", apenas ligeiramente maior que o intervalo normal esperado entre o envio de um quadro e a recepção de sua confirmação. As NAKs não são úteis nesse caso.

Todavia, em outras situações o tempo deve ser bastante variável. Por exemplo, se o tráfego inverso for esporádico, o tempo antes da confirmação será mais curto quando houver tráfego inverso, e mais longo quando não houver. O transmissor enfrenta a escolha de definir o intervalo para um valor pequeno (arriscando retransmissões desnecessárias) ou defini-lo para um valor grande (e ficar ocioso por longo período após um erro). As duas opções desperdiçam largura de banda. Em geral, sempre que o desvio padrão do intervalo de confirmação é grande em comparação com o próprio intervalo, o timer pode ser ajustado "mais livremente" para ser conservador. As NAKs podem, então, acelerar bastante a retransmissão de quadros perdidos ou danificados.

Um problema intimamente relacionado com o uso de timeouts e NAKs é determinar o quadro que provocou um timeout. No protocolo 5, ele é sempre *ack_expected*, porque é sempre o mais antigo. No protocolo 6, não há uma forma trivial para determinar o quadro que chegou em timeout. Imagine que os quadros de 0 a 4 tenham sido transmitidos, significando que a lista de quadros pendentes é 01234, na ordem do mais antigo para o mais recente. Agora, imagine que o quadro 0 chegue após o timeout, que 5 (um novo quadro) seja transmitido, 1 e 2 cheguem após o timeout e 6 (outro quadro novo) seja transmitido. Nesse ponto, a lista de quadros pendentes será 3405126, do mais antigo para o mais recente. Se todo o tráfego de chegada (i.e., quadros que transportam confirmações) for perdido durante algum tempo, esses sete quadros pendentes chegarão em timeout nessa ordem.

Para evitar que o exemplo fique ainda mais complicado do que já está, não mostramos a administração do timer. Em vez disso, consideramos apenas que a variável *oldest_frame* é ativada no momento de timeout para indicar o quadro que chegou após o timeout.

3.5 EXEMPLOS DE PROTOCOLOS DE ENLACE DE DADOS

Dentro de um prédio, as LANs são muito utilizadas para interconexão, mas a maior parte da infraestrutura de rede a longa distância é montada a partir de linhas ponto a ponto. No Capítulo 4, examinaremos melhor as LANs. Aqui, em três situações comuns, veremos os protocolos de enlace de dados nas linhas ponto a ponto da Internet. A primeira situação é quando os pacotes são enviados por enlaces de fibra óptica SONET, nas redes a longas distâncias. Esses enlaces são muito utilizados, por exemplo, para conectar roteadores nos diferentes locais da rede de um ISP. A segunda situação é a dos enlaces ADSL que se localizam no circuito terminal da rede telefônica, na borda da Internet. A terceira situação é para enlaces DOCSIS no circuito terminal de uma rede a cabo. Tanto ADSL quanto DOCSIS conectam milhões de indivíduos e empresas à Internet.

A Internet precisa de enlaces ponto a ponto para esses fins, bem como modems de linha discada, linhas alugadas, modems a cabo, e assim por diante. Um protocolo padrão, chamado **protocolo ponto a ponto**, ou **PPP** (**Point-to-Point Protocol**), é usado para enviar pacotes por esses enlaces. O PPP é definido na RFC 1661 e mais elaborado na RFC 1662 e em outras RFCs (Simpson, 1994a, 1994b). Enlaces SONET, ADSL e DOCSIS aplicam PPP, mas de maneiras diferentes.

3.5.1 Pacotes sobre SONET

O SONET, que analisamos na Seção 2.5.3, é o protocolo da camada física mais utilizado pelos enlaces de fibra óptica que compõem o backbone das redes de comunicação, incluindo o sistema telefônico. Ele oferece um fluxo de bits que trabalha em uma taxa bem definida, por exemplo, 2,4 Gbps para um enlace OC-48. Esse fluxo de bits é organizado como cargas úteis de bytes, com tamanho fixo, que se repetem a cada 125 μs, havendo ou não dados do usuário para transmitir.

Para transportar pacotes por esses enlaces, é preciso que haja algum mecanismo de enquadramento para distinguir pacotes ocasionais do fluxo de bits contínuo no qual são transportados. O PPP atua sobre roteadores IP para fornecer esse mecanismo, como mostra a Figura 3.23.

O PPP é uma melhoria de um protocolo mais antigo e mais simples, chamado **SLIP** (**Serial Line Internet Protocol**), e é usado para lidar com a configuração do enlace de detecção de erros, dar suporte a múltiplos protocolos,

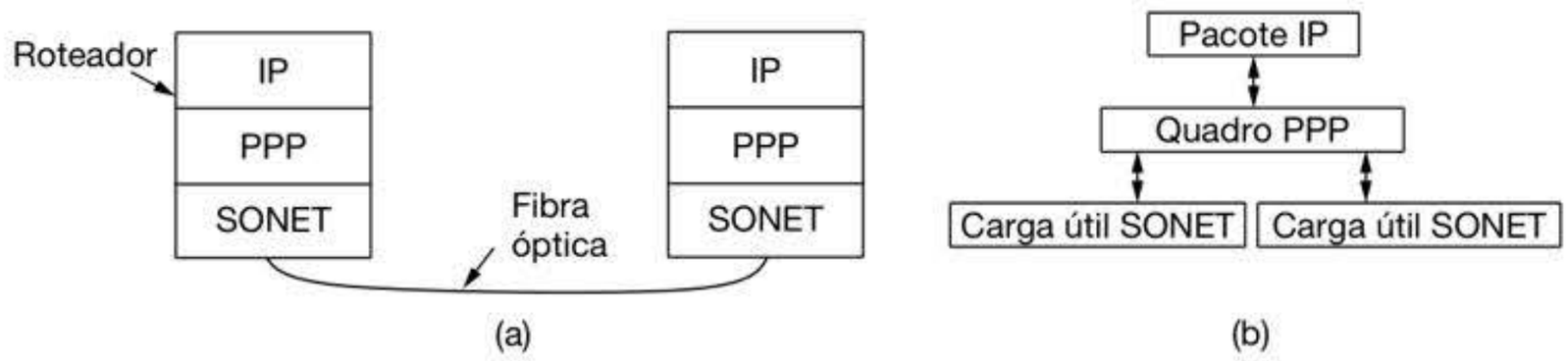


Figura 3.23 Pacotes sobre SONET. (a) Uma pilha de protocolos. (b) Relacionamentos de quadros.

permitir autenticação, entre outros. Com um grande conjunto de opções, o PPP dispõe de três recursos principais:

1. Um método de enquadramento que delineia de forma não ambígua o final de um quadro e o início do seguinte. O formato do quadro também lida com a detecção de erros.
2. Um protocolo de controle de enlace usado para ativar linhas, testá-las, negociar opções e desativá-las novamente de forma controlada quando não forem mais necessárias. Esse protocolo é denominado protocolo de controle de enlace, ou **LCP (Link Control Protocol)**.
3. Uma maneira de negociar as opções da camada de rede de modo independente do protocolo dessa camada a ser utilizado. O método escolhido deve ter um protocolo de controle de rede, ou **NCP (Network Control Protocol)**, diferente para cada camada de rede aceita.

O formato de quadro PPP foi definido de modo a ter uma aparência semelhante ao formato do quadro **HDLC (High-level Data Link Control)**, uma instância bastante utilizada de uma família de protocolos mais antiga, pois não há motivo algum para a definição de um novo padrão.

A principal diferença entre o PPP e o HDLC é que o primeiro é orientado a caracteres, e não a bits. Especificamente, o PPP utiliza a técnica de inserção de bytes e todos os quadros representam um número inteiro de bytes. Já o HDLC usa a inserção de bits, e permite quadros de, digamos, 30,25 bytes.

Contudo, na prática, há uma segunda diferença importante. O HDLC oferece transmissão confiável com uma janela deslizante, confirmações e timeouts da forma como estudamos. O PPP também pode oferecer transmissão confiável em ambientes com ruído, como redes sem fio; os detalhes exatos são definidos na RFC 1663. Entretanto, isso raramente é feito na prática. Em vez disso, um padrão adotado "ainda sem número" quase sempre é usado na Internet para oferecer serviço não orientado a conexão sem confirmação.

A Figura 3.24 mostra o formato do quadro PPP. Todos os quadros PPP começam com o byte de flag padrão do HDLC, 0x7E (01111110). O byte de flag é inserido se ocorrer dentro do campo de *Carga útil* usando o byte de escape 0x7D. O byte seguinte é o XOR entre o byte de escape e 0x20, que inverte o quinto bit. Por exemplo, 0x7D 0x5E é a sequência de escape para o byte de flag 0x7E. Isso significa que o início e o final dos quadros podem ser pesquisados simplesmente procurando-se pelo byte 0x7E, pois ele não ocorrerá em nenhum outro lugar. A regra de retirada na recepção de um quadro é procurar por 0x7D, removê-lo e realizar o XOR do byte seguinte com 0x20. Além disso, apenas um byte de flag é necessário entre os quadros. Vários bytes de flag podem ser usados para preencher o enlace quando não existem quadros para transmitir.

Após o byte de flag de início do quadro, temos o campo *Endereço*, que sempre é definido como o valor binário 11111111, indicando que todas as estações devem aceitar o quadro. A utilização desse valor evita o problema da necessidade de atribuição de endereços de enlace de dados.

O campo *Controle* vem depois do campo *Endereço* e seu valor padrão é 00000011. Esse valor indica um quadro não numerado.

Como os campos *Endereço* e *Controle* sempre são constantes na configuração padrão, o LCP oferece o mecanismo necessário para as duas partes negociarem uma opção que os omita completamente e economize 2 bytes por quadro.

O quarto campo do quadro PPP é o *Protocolo*. Sua tarefa é informar o tipo de pacote que se encontra no campo *Carga útil*. Os códigos que começam com o bit 0 são definidos para o IP versão 4, IP versão 6 e outros protocolos da camada de rede que poderiam ser usados, como IPX e AppleTalk. Os códigos que começam com o bit 1 são utilizados para protocolos de configuração do PPP, incluindo

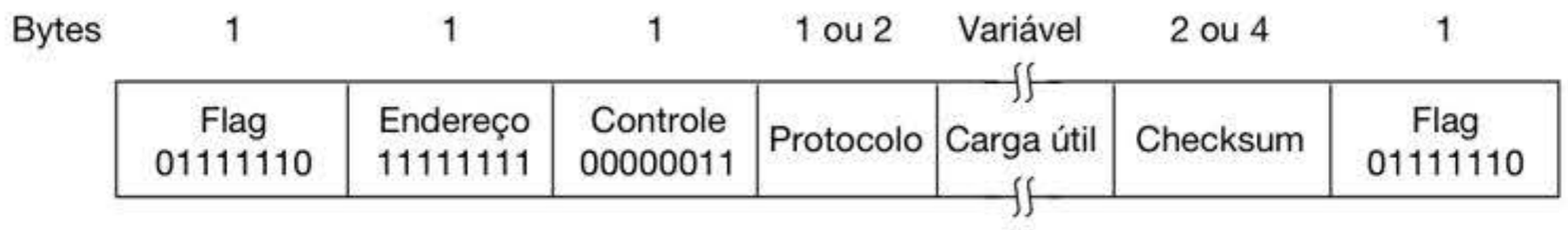


Figura 3.24 O formato completo do quadro PPP para a operação no modo não numerado.

LCP e um NCP diferente para cada protocolo da camada de rede admitido. O tamanho padrão do campo *Protocolo* é 2 bytes, mas é possível negociar uma redução para 1 byte, utilizando-se o LCP. Os projetistas talvez tenham sido demasiadamente cuidadosos, achando que algum dia poderia haver mais de 256 protocolos em uso.

O campo *Carga útil* tem comprimento variável, podendo se estender até o tamanho máximo negociado. Se o comprimento não for negociado com o uso do LCP durante a configuração da linha, será empregado um comprimento padrão de 1.500 bytes. Poderá haver um preenchimento logo após a carga útil, caso seja necessário.

Depois do campo *Carga útil*, temos o campo *Checksum*, que normalmente tem 2 bytes, embora seja possível negociar um checksum de 4 bytes, o qual, na verdade, é o mesmo CRC de 32 bits cujo polinômio gerador é mostrado no final da Seção 3.2.2. O checksum de 2 bytes também é um CRC padrão.

O PPP é um mecanismo de enquadramento que pode transportar os pacotes de vários protocolos por muitos tipos de camadas físicas. Para usá-lo sobre SONET, as escolhas a fazer estão descritas na RFC 2615 (Malis e Simpson, 1999). É utilizado um checksum de 4 bytes, pois esse é o meio principal de detectar erros de transmissão pelas camadas física, de enlace e de rede. Recomenda-se que os campos *Endereço*, *Controle* e *Protocolo* não sejam compactados, pois os enlaces SONET já trabalham com velocidades relativamente altas.

Também há um recurso incomum. A carga útil PPP é misturada (conforme descrevemos na Seção 2.4.3) antes de ser inserida na carga útil SONET. A mistura realiza a operação XOR da carga útil com uma longa sequência pseudoaleatória antes de ser transmitida. O problema é que o fluxo de bits SONET precisa de transições de bits frequentes para realizar a sincronização. Essas transições vêm naturalmente com a variação em sinais de voz, mas na comunicação de dados o usuário escolhe a informação enviada e poderia enviar um pacote com uma longa sequência de 0s. Com a mistura, a probabilidade de um usuário causar problemas enviando uma longa sequência de 0s se torna extremamente baixa.

Antes que os quadros PPP possam ser transportados por linhas SONET, o enlace PPP precisa ser estabelecido e configurado. As fases pelas quais o enlace passa quando é criado, usado e removido novamente aparecem na Figura 3.25.

O enlace começa com a linha no estado *DEAD*, o que significa que não há nenhuma conexão na camada física. Depois de estabelecida a conexão na camada física, o enlace passa para a fase *ESTABLISH*. Nesse ponto, as partes do PPP trocam uma série de pacotes LCP, cada um transportado no campo *Carga útil* de um quadro PPP, para selecionar as opções do PPP para o enlace a partir das possibilidades mencionadas anteriormente. A parte que inicia propõe opções e a parte que responde as aceita ou rejeita, total ou

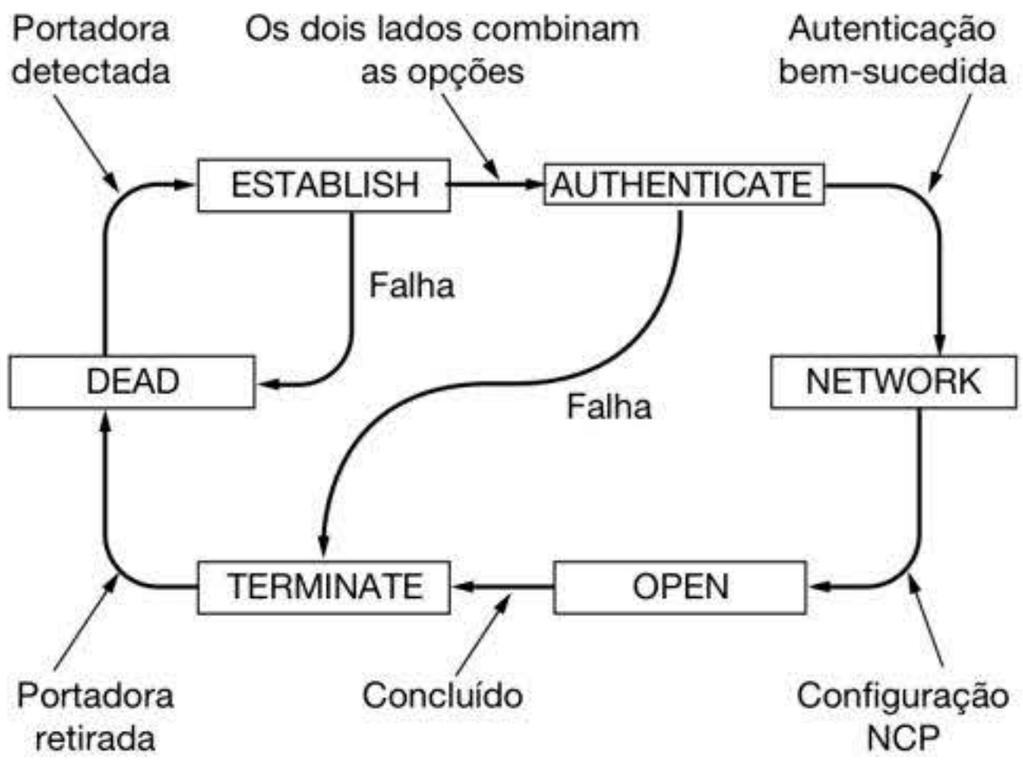


Figura 3.25 Diagrama de estado para ativar e desativar um enlace PPP.

parcialmente. A parte que responde também pode fazer propostas alternativas.

Se a negociação de opções do LCP for bem-sucedida, o enlace chega à fase *AUTHENTICATE*. Agora, as duas partes poderão verificar suas identidades mutuamente, se desejarem. Se a autenticação tiver sucesso, o estado *NETWORK* é alcançado e diversos pacotes NCP são enviados para configurar a camada de rede. É difícil generalizar a respeito dos protocolos NCP, pois cada um é específico a algum protocolo da camada de rede e permite que sejam feitas solicitações de configuração específicas a esse protocolo. Para o IP, por exemplo, a atribuição de endereços IP às duas extremidades do enlace é a possibilidade mais importante.

Quando a fase *OPEN* é alcançada, o transporte de dados pode ser feito. É nesse estado que os pacotes IP são transportados em quadros PPP pela linha SONET. Quando o transporte de dados é concluído, o enlace entra na fase *TERMINATE* e, de lá, volta ao estado *DEAD* quando a conexão da camada física é desativada.

3.5.2 ADSL (Asymmetric Digital Subscriber Line)

A ADSL conecta milhões de assinantes domésticos à Internet em taxas de Mbits/s pelo mesmo circuito terminal de telefone usado para o serviço telefônico tradicional. Na Seção 2.5.2, descrevemos como um dispositivo chamado modem DSL é colocado em uma residência. Ele envia bits do circuito terminal para um dispositivo chamado DSLAM (DSL Access Multiplexer), na estação local da companhia telefônica. Agora, vamos examinar com mais detalhes como os pacotes são transportados por enlaces ADSL.

A imagem geral para os protocolos e dispositivos usados com ADSL aparece na Figura 3.26. Diferentes

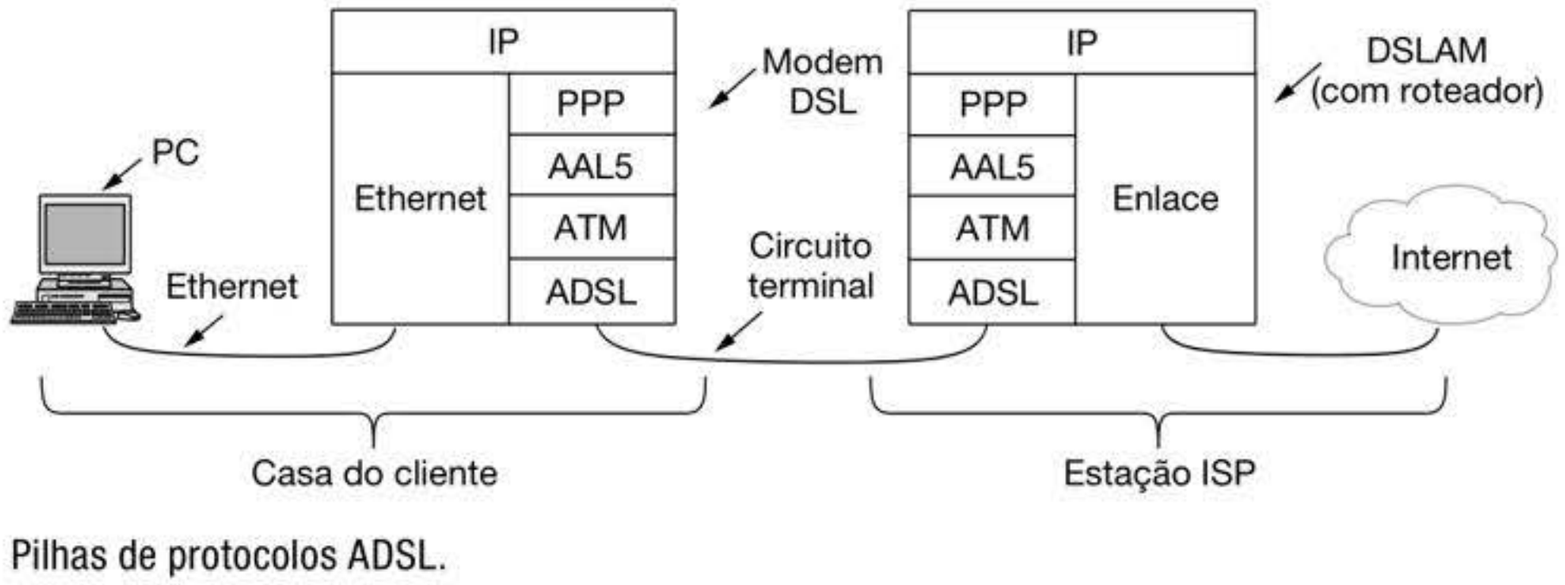


Figura 3.26 Pilhas de protocolos ADSL.

protocolos são empregados em diferentes redes, de modo que escolhemos mostrar o caso mais popular. Dentro da casa, um computador envia pacotes IP ao modem DSL usando uma camada de enlace como o padrão Ethernet. O modem DSL, então, envia os pacotes IP do circuito terminal para o DSLAM usando os protocolos que estamos prestes a estudar. No DSLAM (ou um roteador conectado a ele, dependendo da implementação), os pacotes IP são extraídos e entram em uma rede do ISP, de modo que possam alcançar qualquer destino na Internet.

Os protocolos mostrados pelo enlace ADSL na Figura 3.26 começam de baixo, com a camada física ADSL. Eles são baseados em um esquema de modulação digital, chamado multiplexação por divisão de frequência ortogonal (também conhecido como multitom discreto), como vimos na Seção 2.5.2. Perto do topo da pilha, logo abaixo da camada de rede IP, está o PPP. Esse protocolo é o mesmo PPP que estudamos para transportes de pacote sobre SONET. Ele funciona da mesma maneira para estabelecer e configurar o enlace e transportar pacotes IP.

Entre ADSL e PPP estão ATM e AAL5. Estes são protocolos novos, que ainda não estudamos. O **modo de transferência assíncrono**, ou **ATM** (**Asynchronous Transfer Mode**), foi elaborado no início da década de 1990 e lançado com incrível entusiasmo. Ele prometia uma tecnologia de rede que resolveria os problemas de telecomunicações do mundo, reunindo voz, dados, televisão a cabo, telégrafo, pombo-correio, latinhas conectadas por barbante, tambores e tudo o mais em um sistema integrado, que poderia fazer tudo para todos. Isso não aconteceu. Em grande parte, os problemas do ATM eram semelhantes aos que já descrevemos com relação aos protocolos OSI, ou seja, má sincronização, tecnologia, implementação e política. Apesar disso, o ATM teve muito mais sucesso do que o padrão OSI. Embora não tenha ganho o mundo, ele continua sendo muito usado em nichos, incluindo linhas de acesso de banda larga, como DSL, e enlaces de WAN dentro das redes telefônicas.

O ATM é uma camada de enlace baseada na transmissão de **células** de informação de tamanho fixo. O "assíncrono" em seu nome significa que as células nem sempre precisam ser enviadas da maneira como os bits são continuamente enviados por linhas síncronas, como SONET. As células só precisam ser enviadas quando existe informação para transportar. O ATM é uma tecnologia orientada à conexão. Cada célula transporta um identificador do **circuito virtual** em seu cabeçalho, e os dispositivos usam esse identificador para encaminhar células ao longo dos caminhos das conexões estabelecidas.

Cada célula tem 53 bytes de extensão, consistindo em uma carga útil de 48 bytes mais um cabeçalho de 5 bytes. Usando células pequenas, o ATM pode dividir a largura de banda de um enlace de camada física entre diferentes usuários, de forma flexível, em fatias finas. Esse recurso é útil quando, por exemplo, são enviados voz e dados por um enlace sem ter longos pacotes de dados, que causariam enormes atrasos nas amostras de voz. A escolha incomum para o tamanho da célula (p. ex., em comparação com a escolha mais natural de uma potência de 2) é uma indicação de como o projeto do ATM foi político. O tamanho de 48 bytes para a carga útil foi um meio-termo para resolver um impasse entre a Europa, que queria células de 32 bytes, e os Estados Unidos, que queriam células de 64 bytes. Uma breve visão geral do ATM pode ser encontrada em Siu e Jain (1995).

Para enviar dados por uma rede ATM, eles precisam ser mapeados para uma sequência de células. Esse mapeamento é feito com uma camada de adaptação ATM em um processo chamado segmentação e remontagem. Várias camadas de adaptação foram definidas para diferentes serviços, variando de amostras periódicas de voz até pacotes de dados. A principal usada para os pacotes de dados é a **camada de adaptação ATM 5**, ou **AAL5** (**ATM Adaptation Layer 5**).

Um quadro AAL5 aparece na Figura 3.27. Em vez de um cabeçalho, ele tem um final que oferece o comprimento e um CRC de 4 bytes para detecção de erros. Naturalmente, o CRC é o mesmo usado para redes locais PPP e IEEE 802, como Ethernet. Wang e Crowcroft (1992) mostraram que ele é forte o bastante para detectar erros não tradicionais, como reordenação de célula. Assim como uma carga útil, o quadro AAL5 tem preenchimento. Isso encerra o tamanho geral como um múltiplo de 48 bytes, de modo que o quadro pode ser dividido igualmente em células. Não são necessários endereços no quadro, pois o identificador de

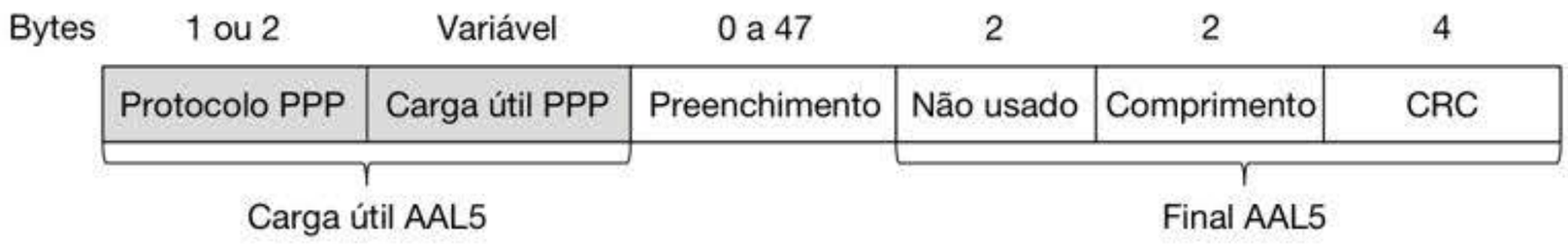


Figura 3.27 Quadro AAL5 transportando dados PPP.

circuito virtual transportado em cada célula o levará ao destino correto.

Agora que já descrevemos o ATM, só precisamos descrever como o PPP o utiliza no caso da ADSL. Isso é feito com outro padrão, chamado **PPPoA** (**PPP over ATM**). Esse padrão não é realmente um protocolo (de modo que não aparece na Figura 3.26), e sim mais uma especificação de como trabalhar com quadros PPP e AAL5. Ele é descrito na RFC 2364 (Gross et al., 1998).

Somente os campos de protocolo e carga útil do PPP são colocados na carga útil do AAL5, como vemos na Figura 3.27. O campo de protocolo indica ao DSLAM no outro extremo se a carga útil é um pacote IP ou um pacote de outro protocolo, como LCP. O canto extremo sabe que as células contêm informações do PPP porque um circuito virtual ATM é estabelecido para essa finalidade.

Dentro do quadro AAL5, o enquadramento PPP não é necessário, pois não teria nenhuma finalidade; ATM e AAL5 já oferecem o enquadramento, e mais seria inútil. O CRC do PPP também não é necessário, pois o AAL5 já inclui o mesmo CRC. Esse mecanismo de detecção de erros suplementa a codificação da camada física da ADSL de um código de Reed-Solomon para correção de erros e um CRC de 1 byte para a detecção de quaisquer erros restantes, não descobertos de outra maneira. Esse esquema tem um mecanismo de recuperação de erros muito mais sofisticado do que quando os pacotes são enviados por uma linha SONET, pois a ADSL é um canal com muito mais ruído.

3.5.3 DOCSIS (Data Over Cable Service Interface Specification)

O protocolo **DOCSIS** (**Data Over Cable Service Interface Specification**) geralmente é descrito como tendo dois componentes: a camada física (PHY), conforme descrito no capítulo anterior (às vezes chamada de PMD ou subcamada dependente do meio), e a camada de controle de acesso ao meio (MAC), que abordaremos com mais detalhes no Capítulo 4. Acima da camada física, o DOCSIS deve lidar com uma série de tarefas para a camada de rede, incluindo a alocação de largura de banda nas direções upstream e downstream (controle de fluxo), enquadramento e correção de erros (é claro que, às vezes, a correção de erros é vista como uma construção da camada física). Descrevemos cada um desses conceitos anteriormente neste capítulo. Nesta seção, exploraremos como o DOCSIS trata de cada um desses problemas.

Um quadro DOCSIS contém várias informações, incluindo indicadores de qualidade de serviço e suporte para fragmentação ou concatenação de quadros. Cada sequência unidirecional de quadros é chamada de **fluxo de serviço**. Os fluxos de serviço principais permitem que o CMTS (sistema de terminação de modem a cabo, na central da empresa de cabo) comunique mensagens de gerenciamento a cada modem a cabo. Cada fluxo de serviço tem um identificador único e normalmente está associado a uma classe de serviço, que pode ser o melhor esforço, pesquisa (em que um modem a cabo faz solicitações explícitas de largura de banda) e serviço de concessão (em que um modem a cabo transmite rajadas de dados a um taxa de dados garantida). Um fluxo de serviço primário é o fluxo de serviço padrão que transporta todos os quadros que não são classificados para outro serviço. Em muitos arranjos de serviço de banda larga, há apenas um fluxo de serviço upstream e downstream padrão entre o CM e o CMTS, que transporta todo o tráfego do usuário, bem como todas as mensagens de gerenciamento. As redes DOCSIS foram historicamente projetadas assumindo que a maioria dos dados é transmitida no sentido downstream. Certos aplicativos, como videoconferência, vão contra essas tendências, embora serviços de jogos em nuvem anunciados recentemente (p. ex., Stadia, GeForce Now, xCloud) possam resultar em uma utilização ainda mais downstream, já que essas aplicações têm como objetivo taxas de streaming contínuo de 30 a 35 Mbps.

Depois que um modem a cabo é ligado, ele estabelece uma conexão com o CMTS, o que normalmente permite que ele se conecte ao resto da rede. Ao se registrar no CMTS, ele adquire canais de comunicação upstream e downstream para usar, bem como chaves de criptografia do CMTS. As operadoras upstream e downstream oferecem dois canais compartilhados para todos os modems a cabo. Na direção downstream, todos os modems a cabo conectados ao CMTS recebem todos os pacotes transmitidos. Na direção upstream, muitos modems a cabo transmitem e o CMTS é o único receptor. Pode haver vários caminhos físicos entre o CMTS e cada modem a cabo.

Antes do DOCSIS 3.1, os pacotes no sentido downstream eram divididos em quadros MPEG de 188 bytes, cada um com um cabeçalho de 4 bytes e uma carga útil de 184 bytes (a chamada camada de convergência de transmissão MPEG). Além dos dados em si, o CMTS envia periodicamente informações de gerenciamento para o modem a cabo, incluindo informações sobre alcance, atribuição de

canal e outras tarefas relacionadas à alocação de canal que são realizadas pela camada MAC (que abordaremos com mais detalhes no Capítulo 4). Embora o DOCSIS 3.1 ainda tenha suporte para essa camada de convergência para propósitos legados, ele não depende mais dela para a comunicação downstream.

A camada de enlace DOCSIS organiza a transmissão de acordo com **perfis de modulação**. Um perfil de modulação é uma lista de ordens de modulação (i.e., carregamentos de bits) que correspondem às subportadoras OFDM. No sentido downstream, o CMTS pode usar perfis diferentes para modems a cabo diferentes, mas normalmente um grupo de modems a cabo que têm o mesmo desempenho ou desempenho semelhante será agrupado no mesmo perfil. Com base na identificação do fluxo de serviço e nos parâmetros de QoS, a camada de enlace (no DOCSIS 3.1), agora chamada **camada de convergência**, agrupa pacotes que têm o mesmo perfil no mesmo buffer de envio; normalmente, há um buffer de envio por perfil, cada um dos quais é superficial, para evitar latência significativa. O construtor de palavras-código então mapeia cada quadro DOCSIS para as palavras-código FEC correspondentes, puxando pacotes dos buffers apenas perfis diferentes a cada limite de palavra-código. A codificação FEC vê o quadro DOCSIS como um fluxo de bits, não como uma sequência de bytes. DOCSIS depende de uma palavra-código LDPC. No sentido downstream, uma palavra-código completa tem até 2027 bytes, dos quais até 1799 bytes são dados e 225 são paridade. Dentro de cada byte de um quadro DOCSIS, o bit menos significativo é transferido primeiro; quando um valor superior a um byte é transmitido, os bytes são ordenados do mais significativo para o menos significativo, uma sequência às vezes chamada de **ordem de rede**. O CMTS também adota o preenchimento de bytes: se não houver um quadro DOCSIS no sentido downstream, o CMTS insere subportadoras preenchidas com bits zero nos símbolos OFDM ou simplesmente armazena sequências de 1s em palavras-código, como mostra a Figura 3.28.

Desde a versão 3.0, o DOCSIS oferece suporte a uma tecnologia chamada **ligação de canal**, que permite a um único assinante usar vários canais upstream e downstream simultaneamente. Essa tecnologia é uma forma de **agregação de enlace**, que pode combinar vários enlaces físicos, ou portas, para criar uma única conexão lógica. O DOCSIS 3.0 permite que até 32 canais downstream e 8 canais upstream sejam ligados, onde cada canal pode ter de 6 a 8 MHz de largura. A ligação de canal no DOCSIS 3.1 é a mesma do DOCSIS 3.0, embora o DOCSIS 3.1 suporte canais mais largos para upstream e downstream: a diferença é que esses canais podem ser muito mais largos (até 192 MHz em downstream, 96 MHz em upstream, em comparação com 6 ou 8 MHz downstream e até 6,4 MHz upstream, no caso do DOCSIS 3.0). Por outro lado, um modem DOCSIS 3.1 pode ligar canais de vários tipos (p. ex., um modem DOCSIS 3.1 pode ligar um canal OFDM de 192 MHz e quatro canais SC-QAM de 6 MHZ).

3.6 RESUMO

A tarefa da camada de enlace de dados é converter o fluxo de dados brutos fornecido pela camada física em um fluxo de quadros a ser utilizado pela camada de rede. A camada de enlace pode apresentar esse fluxo com níveis de confiabilidade variados, desde o serviço não orientado à conexão sem confirmação até o serviço confiável, orientado à conexão.

Diversos métodos de enquadramento são utilizados, inclusive a contagem e a inserção de bytes e de bits. Os protocolos de enlace de dados podem oferecer recursos de controle de erros para a detecção ou correção de quadros danificados e a retransmissão de quadros perdidos. Para evitar que um transmissor rápido sobrecarregue um receptor lento, o protocolo de enlace de dados também pode fornecer controle de fluxo. O mecanismo de janela deslizante é bastante utilizado para integrar os controles de erros e de fluxo de maneira simples. Quando o tamanho da janela é de 1 pacote, o protocolo é stop-and-wait.

Os códigos para correção e detecção de erros acrescentam informações redundantes às mensagens usando diversas técnicas matemáticas. Os códigos de convolução e de Reed-Solomon são muito utilizados para correção de

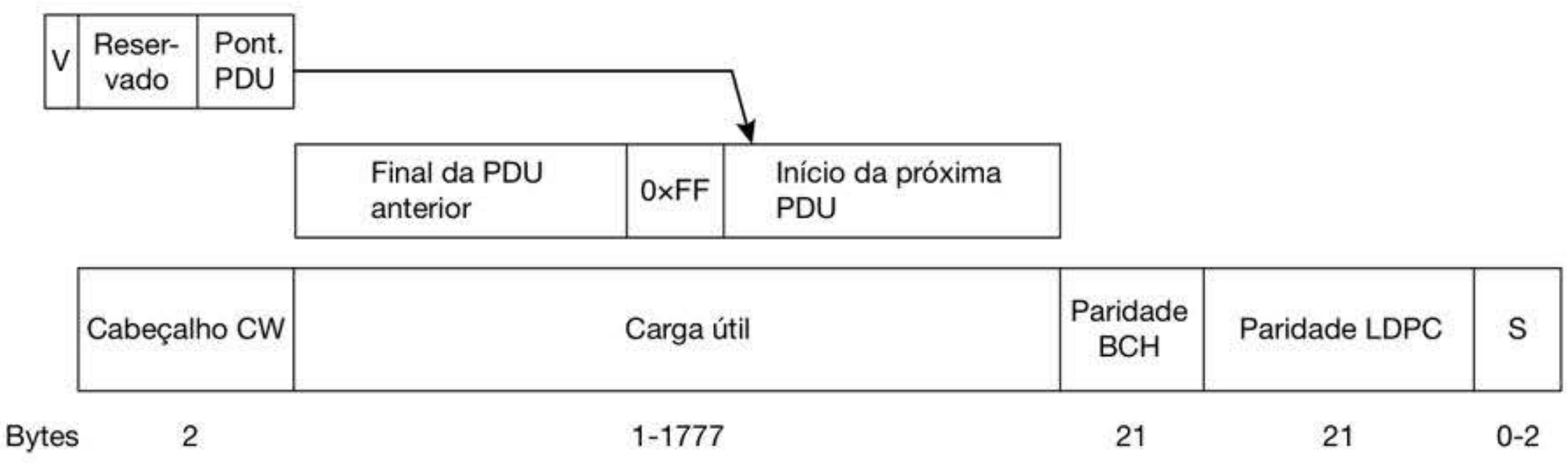


Figura 3.28 Quadro DOCSIS para mapeamento da palavra-código.